

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Primer Barber – Gestão de Barbearia</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,500;9..144,600;9..144,700&family=Inter:wght@400;500" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script src="https://cdn.jsdelivr.net/npm/xlsx@0.18.5/dist/xlsx.full.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/qrcodejs@1.0.0/qrcode.min.js"></script>
<!-- Meta Pixel Code -->
<script>
!function(f,b,e,v,n,t,s)
{if(f.fbq)return;n=f.fbq=function(){n.callMethod?
n.callMethod.apply(n,arguments):n.queue.push(arguments)};
if(!f._fbq)f._fbq=n;n.push=n;n.loaded=!0;n.version='2.0';
n.queue=[];t=b.createElement(e);t.async=!0;
t.src=v;s=b.getElementsByTagName(e)[0];
s.parentNode.insertBefore(t,s)}(window, document,'script',
'https://connect.facebook.net/en_US/fbevents.js');
fbq('init', '1606280897548168');
fbq('track', 'PageView');
</script>
<noscript></noscript>
<!-- End Meta Pixel Code -->
<!-- Deixa o site instalável na tela de início do celular -->
<meta name="theme-color" content="#0a0b0d">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<meta name="apple-mobile-web-app-title" content="Primer Barber">
<meta name="mobile-web-app-capable" content="yes">
<style>
/* ===== Tema ESCURO – preto neutro, branco e dourado ===== */
:root[data-theme="dark"]{
--bg:#0c0d0f;
--panel:#15171a;
--panel-2:#1e2126;
--border:#2a2e35;

```

```

--gold:#c9a24b;
--gold-bright:#e3c078;
--ink:#f4f5f6;
--muted:#a2a8b0;
--muted-2:#737a84;
--ok:#5fa96f;
--ok-bg:#16241a;
--bad:#cf6b5c;
--bad-bg:#2a1a18;
--warn:#d0a94f;
/* painel lateral - preto mais fundo que o conteúdo */
--side-bg:#0a0b0d;
--side-ink:#f4f5f6;
--side-muted:#a2a8b0;
--side-muted-2:#737a84;
--side-border:#23272d;
--side-hover:#1a1d22;
}
/* ===== Tema CLARO - padrão do sistema ===== */
:root, :root[data-theme="light"]{
  --bg:#f4f4f5;
  --panel:#ffffff;
  --panel-2:#f0f0f1;
  --border:#e0e0e2;
  --gold:#b8912f;
  --gold-bright:#8f6f19;
  --ink:#14161a;
  --muted:#5c6067;
  --muted-2:#8b9098;
  --ok:#2f7a3f;
  --ok-bg:#e8f2ea;
  --bad:#b03328;
  --bad-bg:#fbe9e7;
  --warn:#8a6a10;
  /* painel lateral segue preto mesmo no tema claro */
  --side-bg:#101215;
  --side-ink:#f5f5f6;
  --side-muted:#a3a8b0;
  --side-muted-2:#767c86;
  --side-border:#262a30;
  --side-hover:#1c2026;
}
*{box-sizing:border-box;}
body{margin:0;background:var(--bg);color:var(--ink);font-family:'Inter',sans-serif;}
.font-display{font-family:'Fraunces',serif;}
.font-mono{font-family:'IBM Plex Mono',monospace;}

```

```

::selection{background:var(--gold);color:#1a1610;}
::-webkit-scrollbar{width:8px;height:8px;}
::-webkit-scrollbar-thumb{background:var(--border);border-radius:8px;}
::-webkit-scrollbar-track{background:transparent;}

a,button{font-family:inherit;}
button{cursor:pointer;}
input,select,textarea{font-family:inherit;background:var(--panel-2);border:1px
solid var(--border);color:var(--ink);border-radius:8px;padding:9px 12px;font-
size:14px;outline:none;transition:border-color .15s;}
input:focus,select:focus,textarea:focus{border-color:var(--gold);}
input::placeholder{color:var(--muted-2);}
label{font-size:12.5px;color:var(--muted);display:block;margin-
bottom:5px;letter-spacing:.02em;}

.card{background:var(--panel);border:1px solid var(--border);border-
radius:14px;}
.btn{border-radius:9px;padding:10px 16px;font-weight:600;font-
size:14px;border:1px solid transparent;transition:all .15s;display:inline-
flex;align-items:center;gap:7px;}
.btn-gold{background:var(--gold);color:#1a1610;}
.btn-gold:hover{background:var(--gold-bright);}
.btn-ghost{background:transparent;border-color:var(--border);color:var(--ink);}
.btn-ghost:hover{border-color:var(--gold);color:var(--gold-bright);}
.btn-danger{background:transparent;border-color:var(--bad);color:var(--bad);}
.btn-danger:hover{background:var(--bad-bg);}
.btn-disabled{opacity:.4;cursor:not-allowed;}

.divider-blade{height:2px;border-radius:2px;background:linear-gradient(90deg,
transparent, var(--gold) 15%, var(--gold-bright) 50%, var(--gold) 85%,
transparent);opacity:.55;}

.nav-item{display:flex;align-items:center;gap:11px;padding:10px 14px;border-
radius:10px;color:var(--side-muted);font-size:14px;font-
weight:500;cursor:pointer;transition:all .15s;border:1px solid transparent;}
.nav-item:hover{color:var(--side-ink);background:var(--side-hover);}
.nav-item.active{color:#e3c078;background:var(--side-hover);border-color:var(--
side-border);}
/* dentro do painel lateral, o texto segue a paleta escura */
.sidebar{color:var(--side-ink);}
.sidebar .btn-ghost{border-color:var(--side-border);color:var(--side-muted);}
.sidebar .btn-ghost:hover{border-color:#c9a24b;color:#e3c078;}
.nav-item svg{flex-shrink:0;}
.grid-slot:hover{background:rgba(201,162,75,.16);box-shadow:inset 0 0 0 1px
var(--gold);}

.pill{font-size:11.5px;font-weight:600;padding:3px 10px;border-

```

```

radius:99px;letter-spacing:.02em;}
.pill-ok{background:var(--ok-bg);color:var(--ok);}
.pill-bad{background:var(--bad-bg);color:var(--bad);}
.pill-warn{background:#2c2515;color:var(--warn);}
.pill-muted{background:var(--panel-2);color:var(--muted);}

.ticket{position:relative;background:var(--panel-2);border:1px solid var(--
border);border-radius:10px 10px 0 0;padding:14px 16px 18px;}
.ticket::after{

content:'';display:block;position:absolute;left:0;right:0;bottom:-9px;height:10px;
background-image:
    linear-gradient(135deg, var(--panel-2) 50%, transparent 50%),
    linear-gradient(-135deg, var(--panel-2) 50%, transparent 50%);
background-size:14px 14px;background-position:left bottom;background-
repeat:repeat-x;
}

.modal-overlay{position:fixed;inset:0;background:rgba(10,8,6,.65);backdrop-
filter:blur(2px);display:flex;align-items:center;justify-content:center;z-
index:50;padding:16px;}
.modal{background:var(--panel);border:1px solid var(--border);border-
radius:16px;max-width:480px;width:100%;max-height:88vh;overflow-y:auto;}

.toast{position:fixed;bottom:22px;left:50%;transform:translateX(-50%);background:va
-panel-2);border:1px solid var(--gold);color:var(--ink);padding:11px 20px;border-
radius:10px;font-size:13.5px;z-index:100;box-shadow:0 8px 24px rgba(0,0,0,.4);}

/* ===== Tela de acesso (login / cadastro / paywall) ===== */
.auth-bg{
    min-height:100vh; width:100%; display:flex; align-items:center; justify-
content:center;
    padding:28px 20px; position:relative; overflow:hidden;
    background:
        radial-gradient(1100px 560px at 50% -8%, rgba(201,162,75,.16), transparent
62%),
        radial-gradient(700px 420px at 100% 100%, rgba(201,162,75,.06), transparent
55%),
        repeating-linear-gradient(128deg, rgba(201,162,75,.035) 0px,
        rgba(201,162,75,.035) 1.5px, transparent 1.5px, transparent 30px),
        #0a0b0d;
}
.auth-card{
    width:100%; max-width:408px; position:relative; z-index:1;
    background:var(--panel); border:1px solid var(--border); border-radius:22px;
    box-shadow:0 40px 90px -30px rgba(0,0,0,.7), 0 0 0 1px rgba(201,162,75,.05);
    overflow:hidden;
}

```

```

.auth-stripe{
  height:5px; width:100%;
  background:repeating-linear-gradient(52deg,
    #c9a24b 0px, #c9a24b 9px,
    #e3c078 9px, #e3c078 18px,
    #0a0b0d 18px, #0a0b0d 22px,
    #e3c078 22px, #e3c078 31px);
  background-size:44px 100%;
  flex-shrink:0;
}
.auth-inner{padding:36px 34px 32px;}
.auth-logo-wrap{
  width:60px; height:60px; border-radius:16px; margin:0 auto 18px;
  background:linear-gradient(160deg, var(--panel-2), var(--panel));
  border:1px solid var(--border);
  display:flex; align-items:center; justify-content:center; padding:11px;
  box-shadow:0 10px 26px -10px rgba(201,162,75,.35), inset 0 0 0 1px
  rgba(201,162,75,.08);
}
.auth-eyebrow{
  text-align:center; font-size:10.5px; letter-spacing:.18em; text-
transform:uppercase;
  color:var(--muted-2); margin-bottom:6px; font-weight:600;
}
.auth-title{
  text-align:center; font-size:26px; line-height:1.15; margin-bottom:8px;
}
.auth-subtitle{
  text-align:center; font-size:13px; color:var(--muted); margin-bottom:26px;
line-height:1.55;
}
.auth-trial-pill{
  display:inline-flex; align-items:center; gap:6px; margin:0 auto 22px;
padding:6px 14px;
  border-radius:999px; background:rgba(201,162,75,.1); border:1px solid
  rgba(201,162,75,.28);
  color:var(--gold-bright); font-size:11.5px; font-weight:600; letter-
spacing:.02em;
}
.auth-trial-row{display:flex; justify-content:center;}
.auth-field label{font-size:11px; letter-spacing:.03em; text-
transform:uppercase; color:var(--muted-2); font-weight:600; margin-bottom:6px;
display:block;}
.auth-field input{
  width:100%; background:var(--bg); border:1px solid var(--border); border-
radius:11px;
  padding:12px 14px; font-size:14px; color:var(--ink); transition:border-color

```

```

.15s, box-shadow .15s;
}

.auth-field input:focus{border-color:var(--gold); box-shadow:0 0 0 3px
rgba(201,162,75,.12); outline:none;}

.auth-submit{
    width:100%; justify-content:center; padding:13px; font-size:14px; font-
weight:600;
    border-radius:11px; margin-top:4px; box-shadow:0 12px 26px -12px
rgba(201,162,75,.5);
}

.auth-switch{width:100%; justify-content:center; padding:11px; font-size:13px;
border-radius:11px;}

.auth-error{
    background:var(--bad-bg); color:var(--bad); border-radius:10px; padding:11px
13px;
    font-size:12.5px; margin-bottom:16px; border:1px solid rgba(193,123,106,.25);
line-height:1.5;
}

@media (max-width:480px){ .auth-inner{padding:28px 22px 26px;} .auth-title{font-
size:22px;} }

.legal-content h2{font-size:16px;font-weight:600;margin:26px 0 10px;color:var(--
gold-bright);}

.legal-content h2:first-child{margin-top:0;}

.legal-content p{margin-bottom:12px;color:var(--muted);}

.legal-content ul{margin:0 0 12px 20px;color:var(--muted);}

.legal-content li{margin-bottom:6px;}

.legal-content b{color:var(--ink);}

.tab-scroll{scrollbar-width:thin;}

table{width:100%;border-collapse:collapse;}

th{text-align:left;font-size:11.5px;text-transform:uppercase;letter-
spacing:.05em;color:var(--muted-2);font-weight:600;padding:0 12px 10px;}

td{padding:12px;font-size:14px;border-top:1px solid var(--border);}

tr:hover td{background:rgba(255,255,255,.015);}

/* ===== Adaptação para celular ===== */

.mobile-topbar{display:none;}

.mobile-drawer{display:none;}

@media (max-width: 860px){
    /* menu lateral vira gaveta que abre por cima */

    .sidebar{
        /* !important é necessário: o elemento tem position:sticky inline,
        que venceria esta regra e deixaria a barra ocupando espaço. */
        position:fixed !important; top:0 !important; left:0 !important; bottom:0
!important;
        z-index:120;
        transform:translateX(-100%); transition:transform .22s ease;

```

```

    width:270px !important; height:100vh !important; max-width:85vw;
    box-shadow:14px 0 40px -12px rgba(0,0,0,.6);
}
/* trava qualquer rolagem lateral indesejada */
html, body{max-width:100%; overflow-x:hidden;}
body.drawer-open .sidebar{transform:translateX(0);}
.mobile-drawer{
    display:block; position:fixed; inset:0; z-index:110;
    background:rgba(0,0,0,.55); opacity:0; pointer-events:none;
transition:opacity .22s;
}
body.drawer-open .mobile-drawer{opacity:1; pointer-events:auto;}
.mobile-topbar{
    display:flex; align-items:center; gap:12px;
    position:sticky; top:0; z-index:90;
    background:var(--side-bg); border-bottom:1px solid var(--side-border);
    padding:11px 14px; color:var(--side-ink);
}
main{padding:18px 15px 90px !important; max-width:100% !important;}
/* cartões e tabelas respiram melhor */
.card{border-radius:12px;}
table{display:block; overflow-x:auto; white-space:nowrap;}
/* alvos de toque maiores */
.btn{padding:11px 15px; font-size:14px;}
input, select, textarea{font-size:16px;} /* evita o zoom automático do iPhone
*/
.dash-grid{grid-template-columns:1fr !important;}
.desk-bell{display:none !important;}
/* grade da agenda: colunas mais estreitas e rolagem lateral suave */
.card{scrollbar-width:thin;}
/* páginas públicas ocupam a largura toda */
.auth-card{max-width:100% !important;}
/* cabeçalhos de página não quebram feio */
.pill{font-size:11px;}
/* modais viram uma folha que sobe de baixo */
.modal-overlay{align-items:flex-end !important; padding:0 !important;}
.modal{
    max-height:92vh; overflow-y:auto;
    border-radius:20px 20px 0 0 !important;
    width:100% !important; max-width:100% !important;
    padding-bottom:env(safe-area-inset-bottom);
}
}
@media (max-width: 520px){
    h1, .font-display{letter-spacing:-.01em;}
    main{padding:14px 12px 90px !important;}
}

```

```

/* Quando instalado como app (tela cheia), respeita o entalhe do celular */
@media (display-mode: standalone){
  .mobile-topbar{padding-top:calc(11px + env(safe-area-inset-top));}
}
</style>
</head>
<body>
<div id="app"></div>

<script>
/* ===== ICONS (minimal inline SVG set) ===== */
const ICON = {
  dashboard:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><rect x="3" y="3" width="7" height="9"
rx="1.5"/><rect x="14" y="3" width="7" height="5" rx="1.5"/><rect x="14" y="12"
width="7" height="9" rx="1.5"/><rect x="3" y="16" width="7" height="5" rx="1.5"/>
</svg>`,
  agenda:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><rect x="3" y="5" width="18" height="16"
rx="2"/><path d="M3 10h18M8 3v4M16 3v4"/></svg>`,
  team:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><circle cx="9" cy="8" r="3.2"/><path
d="M3.5 20c0-3.5 2.7-6 5.5-6s5.5 2.5 5.5 6"/><circle cx="17.5" cy="8.5" r="2.4"/>
<path d="M15.7 14.3c2.4.3 4.3 2.4 4.3 5.7"/></svg>`,
  scissors:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><circle cx="6" cy="6" r="2.6"/><circle
cx="6" cy="18" r="2.6"/><path d="M20 4L8.5 15.5M20 20L8.5 8.5"/></svg>`,
  cash:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><rect x="2.5" y="6" width="19"
height="13" rx="2.2"/><circle cx="12" cy="12.5" r="3"/><path d="M6 6V4.5A1.5 1.5 0
0 1 7.5 3h9A1.5 1.5 0 0 1 18 4.5V6"/></svg>`,
  percent:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><circle cx="6.5" cy="6.5" r="2.5"/>
<circle cx="17.5" cy="17.5" r="2.5"/><path d="M19 5L5 19"/></svg>`,
  club:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M12 3l2.6 5.5 6 .7-4.4 4.1 1.2
5.9L12 16.4l-5.4 2.8 1.2-5.9-4.4-4.1 6-.7z"/></svg>`,
  gear:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><circle cx="12" cy="12" r="3"/><path
d="M19.4 13.5a1.7 1.7 0 000-3l1-1.7-1.7-1.7-1.7 1a1.7 1.7 0 00-3 0l-1-1.7H10l-1
1.7a1.7 1.7 0 00-3 0l-1-1.7-1L2.6 8.8l1 1.7a1.7 1.7 0 000 3l1-1 1.7 1.7 1.7 1.7-1a1.7
1.7 0 003 0l1 1.7h2l1-1.7a1.7 1.7 0 003 0l1.7 1 1.7-1.7z"/></svg>`,
  plus:`<svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.2"><path d="M12 5v14M5 12h14"/></svg>`,
  trash:`<svg width="15" height="15" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M3 6h18M8 6V4.5A1.5 1.5 0 019.5
3h5A1.5 1.5 0 0116 4.5V6m2 0l-.8 13.4A2 2 0 0115.2 21H8.8a2 2 0 01-2-1.6L6 6"/>

```

```

</svg>`,
  edit: `
```

```
8.5 8.5 0 01-7.6 4.7 8.38 8.38 0 01-3.8-.9L3 2111.9-5.7a8.38 8.38 0 01-.9-3.8 8.5
8.5 0 014.7-7.6 8.38 8.38 0 013.8-.9h.5a8.48 8.48 0 018 8v.5z"/></svg>`,
  send:`<svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M22 2L11 13M22 21-7 20-4-9-9-4
20-7z"/></svg>`,
  star:`<svg width="14" height="14" viewBox="0 0 24 24" fill="currentColor"
stroke="currentColor" stroke-width="1"><path d="M12 213.1 6.3 6.9 1-5 4.9 1.2 6.9-
6.2-3.3-6.2 3.3 1.2-6.9-5-4.9 6.9-1z"/></svg>`,
  starOutline:`<svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.6"><path d="M12 213.1 6.3 6.9 1-5 4.9 1.2
6.9-6.2-3.3-6.2 3.3 1.2-6.9-5-4.9 6.9-1z"/></svg>`,
  shield:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M12 317 3v6c0 4.5-3 8-7 9-4-1-7-
4.5-7-9V6z"/><path d="M9 1212 2 4-4"/></svg>`,
  receipt:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M6 3h12v18l-2.5-1.5L13 211-2.5-
1.5L8 211-2-1.5z"/><path d="M9 8h6M9 12h6"/></svg>`,
  gift:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><rect x="3" y="8" width="18" height="13"
rx="1"/><path d="M12 8v13M3 12h18"/><path d="M12 8c-1.5 0-4-.5-4-3a2 2 0 014-.5A2
2 0 0116 5c0 2.5-2.5 3-4 3z"/></svg>`,
  image:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><rect x="3" y="4" width="18" height="16"
rx="2"/><circle cx="8.5" cy="9.5" r="1.7"/><path d="M21 161-5.5-5.5a2 2 0 00-2.8
0L4 19"/></svg>`,
  instagram:`<svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><rect x="3" y="3" width="18" height="18"
rx="5"/><circle cx="12" cy="12" r="4"/><circle cx="17.2" cy="6.8" r="1"/></svg>`,
  facebook:`<svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M15 4h-2a4 4 0 00-4
4v3H7v4h2v7h4v-7h3l1-4h-4V8a1 1 0 011-1h3z"/></svg>`,
  tiktok:`<svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M14 3v11.5a3.5 3.5 0 11-3.5-
3.5c.2 0 .3 0 .5.03"/><path d="M14 3a5 5 0 005 5"/></svg>`,
  upload:`<svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M12 16V4m0 0L7 9m5-5l5 5"/><path
d="M4 16v3a2 2 0 002 2h12a2 2 0 002-2v-3"/></svg>`,
  timer:`<svg width="13" height="13" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><circle cx="12" cy="13" r="8"/><path
d="M12 9v4l2.5 1.5M9 2h6"/></svg>`,
  listPlus:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M4 6h11M4 12h7M4 18h7"/><path
d="M18 14v6M15 17h6"/></svg>`,
  bell:`<svg width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="1.8"><path d="M6 9a6 6 0 112 0c0 5 2 6 2
6H4s2-1 2-6z"/><path d="M10 21a2 2 0 004 0"/></svg>`,
  help:`<svg width="18" height="18" viewBox="0 0 24 24" fill="none"
```

```

stroke="currentColor" stroke-width="1.8"><circle cx="12" cy="12" r="9"/><path
d="M9.5 9a2.5 2.5 0 015 .5c0 1.5-2 1.7-2.4 3.1M12 17h.01"/></svg>`,
};

/* ===== CONFIGURAÇÃO - PREENCHA ANTES DE PUBLICAR ===== */
// 1) Crie um projeto gr\u00e9tis em https://supabase.com
// 2) V\u00e1 em Project Settings > API e copie a "Project URL" e a chave "anon
public"
// 3) Cole os dois valores abaixo
const SUPABASE_URL = 'https://pjaorieeirdgumdwuwew.supabase.co';
const SUPABASE_ANON_KEY =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InBqYW9yaWVla
';

// 4) Crie DOIS produtos de assinatura no Stripe (Basic e Pro) e um
// "Payment Link" para cada um. Cole os links abaixo:
const STRIPE_PAYMENT_LINK_BASIC =
'https://buy.stripe.com/eVq3cv6tMh062E39Wm0Jq00';
const STRIPE_PAYMENT_LINK_PRO = 'https://buy.stripe.com/00waEXdWe6ls6Ujd8y0Jq01';

// Dias de teste grátis para barbearias novas
const TRIAL_DAYS = 14;

// Logo do app (marca "Primer Barber"), embutida como imagem
const APP_LOGO_ICON =
'
';

// Logo completa (monograma + PRIMER BARBER), para a tela de login
const APP_LOGO_FULL =
'
';

// Limites de cada plano
const PLAN_LIMITS = {
  basic: { maxProfessionals: 4, club: false, label: 'Basic' },
  pro: { maxProfessionals: Infinity, club: true, label: 'Pro' },
};

function planOf(shop){
  // Durante o período de teste grátis, libera tudo (nível Pro) – a pessoa
  // testando decide o plano de verdade só quando o teste acabar. Depois
  // que vencer, a tela de assinatura (renderPaywallScreen) já tranca o
  // acesso inteiro até escolher um plano, então essa liberação nunca
  // "vaza" pra depois do prazo.
  if(shop?.subscription_status==='trialing' && shop?.trial_ends_at && new
Date(shop.trial_ends_at).getTime() > Date.now()){
    return 'pro';
  }
  return PLAN_LIMITS[shop?.plan] ? shop.plan : 'basic';
}

```

```

function planLimits(shop){ return PLAN_LIMITS[planOf(shop)]; }

const supa = (SUPABASE_URL.startsWith('http'))
  ? window.supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY)
  : null;

/* ===== STATE ===== */
const uid = () => Date.now().toString(36) + Math.random().toString(36).slice(2,7);
// Escapa texto antes de inserir na tela, para impedir que nomes/telefones
// digitados por qualquer pessoa (inclusive visitantes anônimos no link
// público de agendamento) rodem como código na tela de quem for ver.
function esc(str){
  if(str==null) return '';
  return String(str)
    .replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
    .replace(/"/g, '&quot;').replace(/'/g, '&#39;');
}
const todayISO = () => new Date().toISOString().slice(0,10);
const BRL = n => (Number(n)||0).toLocaleString('pt-BR',
{style:'currency',currency:'BRL'});
const STORAGE_KEY = 'barberpro-state-v1';
let currentShop = null; // linha da tabela `shops` do usuário logado
let notifState = { items: [], open: false, channel: null };

function seedState(){
  const profs = [
    {id:uid(), name:'Carlos Menezes', phone:'(11) 98888-1010', commission:45,
pin:'1010', workDays:[1,2,3,4,5,6], workStart:'09:00', workEnd:'19:00', timeOff:
[], serviceOverrides:{}},
    {id:uid(), name:'Fernando Alves', phone:'(11) 98888-2020', commission:40,
pin:'2020', workDays:[2,3,4,5,6], workStart:'10:00', workEnd:'20:00', timeOff:[],
serviceOverrides:{}},
    {id:uid(), name:'Bruno Ferraz', phone:'(11) 98888-3030', commission:50,
pin:'3030', workDays:[1,2,3,4,5,6], workStart:'09:00', workEnd:'18:00', timeOff:
[], serviceOverrides:{}},
    {id:uid(), name:'Diego Rocha', phone:'(11) 98888-4040', commission:40,
pin:'4040', workDays:[1,2,3,4,5], workStart:'09:00', workEnd:'19:00', timeOff:[],
serviceOverrides:{}},
    {id:uid(), name:'Rafael Souza', phone:'(11) 98888-5050', commission:45,
pin:'5050', workDays:[3,4,5,6], workStart:'11:00', workEnd:'20:00', timeOff:[],
serviceOverrides:{}},
  ];
  const services = [
    {id:uid(), name:'Corte Masculino', price:45, duration:30,
requireDeposit:false, depositType:'percentual', depositValue:30},
    {id:uid(), name:'Barba', price:35, duration:20, requireDeposit:false,
depositType:'percentual', depositValue:30},
  ];

```

```

    {id:uid(), name:'Corte + Barba', price:70, duration:50, requireDeposit:true,
depositType:'fixo', depositValue:20},
    {id:uid(), name:'Sobrancelha', price:15, duration:10, requireDeposit:false,
depositType:'percentual', depositValue:30},
    {id:uid(), name:'Pigmentação de Barba', price:60, duration:40,
requireDeposit:true, depositType:'percentual', depositValue:30},
];
const t = todayISO();
const appts = [
    {id:uid(), date:t, time:'09:00', professionalId:profs[0].id,
serviceId:services[2].id, client:'Jonas Prado', status:'concluido',
confirmed:true, depositStatus:'pago', depositAmount:20, createdAt:new
Date().toISOString(), rating:{stars:5, comment:'Sempre impecável.'}},
    {id:uid(), date:t, time:'09:15', professionalId:profs[0].id,
serviceId:services[0].id, client:'Ícaro Nunes', status:'concluido',
confirmed:true, depositStatus:'nao_aplicavel', depositAmount:0, createdAt:new
Date().toISOString(), rating:null},
    {id:uid(), date:t, time:'09:30', professionalId:profs[1].id,
serviceId:services[0].id, client:'Marcos Lins', status:'concluido',
confirmed:true, depositStatus:'nao_aplicavel', depositAmount:0, createdAt:new
Date().toISOString(), rating:{stars:4, comment:''}},
    {id:uid(), date:t, time:'10:15', professionalId:profs[2].id,
serviceId:services[1].id, client:'Eduardo Reis', status:'concluido',
confirmed:true, depositStatus:'nao_aplicavel', depositAmount:0, createdAt:new
Date().toISOString(), rating:null},
    {id:uid(), date:t, time:'11:00', professionalId:profs[0].id,
serviceId:services[0].id, client:'Vitor Hugo', status:'agendado', confirmed:false,
depositStatus:'nao_aplicavel', depositAmount:0, createdAt:new
Date().toISOString(), rating:null},
    {id:uid(), date:t, time:'14:00', professionalId:profs[3].id,
serviceId:services[4].id, client:'Anderson Melo', status:'agendado',
confirmed:false, depositStatus:'pendente', depositAmount:18, createdAt:new
Date().toISOString(), rating:null},
    {id:uid(), date:t, time:'15:30', professionalId:profs[4].id,
serviceId:services[2].id, client:'Paulo Cesar', status:'faltou', confirmed:true,
depositStatus:'nao_aplicavel', depositAmount:0, createdAt:new
Date().toISOString(), rating:null},
    {id:uid(), date:t, time:'16:00', professionalId:profs[1].id,
serviceId:services[3].id, client:'Ícaro Nunes', status:'concluido',
confirmed:true, depositStatus:'nao_aplicavel', depositAmount:0, createdAt:new
Date().toISOString(), rating:null},
];
const cashSession = {
    id:uid(), date:t, status:'aberto', openingAmount:200, openedAt:new
Date().toISOString(), closedAt:null, closingCounted:null,
    entries:[
        {id:uid(), type:'entrada', desc:'Corte + Barba - Jonas Prado', amount:70,

```

```

time:'09:32', auto:true},
    {id:uid(), type:'entrada', desc:'Corte Masculino – Marcos Lins', amount:45,
time:'10:01', auto:true},
    {id:uid(), type:'entrada', desc:'Barba – Eduardo Reis', amount:35,
time:'10:38', auto:true},
    {id:uid(), type:'saida', desc:'Compra de toalhas', amount:32, time:'11:10',
auto:false},
    {id:uid(), type:'entrada', desc:'Venda de pomada modeladora', amount:28,
time:'11:40', auto:false},
  ],
};

const plans = [
  {id:uid(), name:'Plano Bronze', price:79.9, benefits:'1 corte por mês'},
  {id:uid(), name:'Plano Prata', price:129.9, benefits:'2 cortes por mês + 1
barba grátis'},
  {id:uid(), name:'Plano Ouro', price:199.9, benefits:'Cortes ilimitados + 15%
off em produtos'},
];

const members = [
  {id:uid(), name:'Ícaro Nunes', phone:'(11) 97777-1111', planId:plans[1].id,
paymentMethod:'asaas', status:'pago', dueDate:t, asaasChargeId:'pay_9f3a21',
lastPaymentDate:t, lastPaymentAmount:129.9},
  {id:uid(), name:'Renato Vidal', phone:'(11) 97777-2222', planId:plans[0].id,
paymentMethod:'asaas', status:'atrasado', dueDate:'2026-08-05',
asaasChargeId:'pay_7b10dd', lastPaymentDate:null, lastPaymentAmount:0},
  {id:uid(), name:'Otávio Prates', phone:'(11) 97777-3333', planId:plans[2].id,
paymentMethod:'dinheiro', status:'pago', dueDate:'2026-08-02', asaasChargeId:'',
lastPaymentDate:'2026-08-02', lastPaymentAmount:199.9},
  {id:uid(), name:'Bruno Castro', phone:'(11) 97777-4444', planId:plans[0].id,
paymentMethod:'asaas', status:'pendente', dueDate:t, asaasChargeId:'pay_4c88e0',
lastPaymentDate:null, lastPaymentAmount:0},
];

const clients = [
  {id:uid(), name:'Jonas Prado', phone:'(11) 96666-1111', birthday:'08-24',
points:70, notes:'', referredBy:'', referralBonusGiven:false},
  {id:uid(), name:'Marcos Lins', phone:'(11) 96666-2222', birthday:'03-12',
points:45, notes:'', referredBy:'', referralBonusGiven:false},
  {id:uid(), name:'Eduardo Reis', phone:'(11) 96666-3333', birthday:'08-29',
points:35, notes:'Prefere máquina 1 nas laterais', referredBy:'',
referralBonusGiven:false},
  {id:uid(), name:'Vitor Hugo', phone:'(11) 96666-4444', birthday:'01-15',
points:10, notes:'', referredBy:'Jonas Prado', referralBonusGiven:false},
  {id:uid(), name:'Ícaro Nunes', phone:'(11) 97777-1111', birthday:'08-22',
points:120, notes:'Membro do Plano Prata', referredBy:'',
referralBonusGiven:false},
];

const products = [

```

```

    {id:uid(), name:'Pomada modeladora', stock:14, minStock:5, cost:12, price:28},
    {id:uid(), name:'Óleo para barba', stock:3, minStock:5, cost:9, price:24},
    {id:uid(), name:'Shampoo anticaspa', stock:9, minStock:4, cost:10, price:26},
    {id:uid(), name:'Navalha descartável (cx)', stock:2, minStock:3, cost:18,
price:0},
    {id:uid(), name:'Toalha de barbearia', stock:22, minStock:10, cost:6,
price:0},
  ];
  const waitlist = [
    {id:uid(), clientName:'Otávio Prates', phone:'(11) 97777-3333',
serviceId:services[2].id, professionalId:profs[0].id, preferredDate:t,
note:'Prefere fim de tarde', createdAt:new Date().toISOString()},
  ];
  return {
    professionals: profs, services, appointments: appts,
    cashSessions:[cashSession], commissionPayments:[],
    subscriptionPlans: plans, subscriptionMembers: members,
    clients, products, waitlist,
    loyalty:{pointsPerReal:1, redeemThreshold:100, redeemValue:20,
referralBonusPoints:30},
    onlineBooking:{slug:'primer-barber', allowCancel:true, allowReschedule:true,
active:true},
    shopProfile:{
      name:'Primer Barber', logoDataUrl:APP_LOGO_ICON, bannerDataUrl:'',
      bio:'Barbearia de bairro com atendimento premium – cortes, barba e cuidados
masculinos.',
      address:'', phone:'',
      social:{instagram:'', facebook:'', tiktok:'', whatsapp:'', website:''},
    },
    asaas:{apiKey:'', walletId:'', env:'sandbox', enabled:false},
    messageSettings:{reminderHours:3, winbackDays:23, depositExpireMinutes:30},
    messageTemplates:{
      confirmation:'Olá {cliente}! Seu horário na Primer Barber está marcado para
{data} às {hora} – {servico} com {profissional}. Pode confirmar sua presença? 🗓️',
      reminder:'Oi {cliente}, passando pra lembrar: seu horário é hoje às {hora} –
{servico} com {profissional}. Te esperamos na Primer Barber! 🗓️',
      winback:'Sentimos sua falta, {cliente}! Já faz {dias} dias desde seu último
corte. Que tal marcar um horário essa semana? Responda aqui que já te encaixamos.
✂️',
      upsellServices:'Oi {cliente}! Notamos que você ainda não experimentou
{sugestao} aqui na Primer Barber. Quer aproveitar no seu próximo horário?',
      upsellProducts:'{cliente}, temos {sugestao} disponíveis na barbearia –
combinam bem com o seu estilo de corte. Quer que a gente separe um pra você na
próxima visita?',
      waitlistOpen:'Boa notícia, {cliente}! Abriu um horário em {data} às {hora}
com {profissional}. Quer confirmar? Responda rápido que é por ordem de chegada.',
      reviewRequest:'Oi {cliente}! Como foi seu atendimento hoje na Primer Barber?

```

```

Dá uma nota de 1 a 5 pra gente por aqui, sua opinião ajuda muito. 🌟',
    },
    messageLog:[],
    commissionSettings:{clubStaffPercent:60},
    nfe:{enabled:false, provider:'', apiKey:'', cnpj:'', autoEmit:false},
    nfeLog:[],
  };
}

function blankShopData(){
  return {
    professionals: [], services: [], appointments: [],
    cashSessions:[], commissionPayments:[], withdrawalRequests:[],
    subscriptionPlans: [], subscriptionMembers: [],
    clients: [], products: [], waitlist: [],
    loyalty:{pointsPerReal:1, redeemThreshold:100, redeemValue:20,
referralBonusPoints:30},
    onlineBooking:{slug:'', allowCancel:true, allowReschedule:true, active:true},
    themePreference:'light',
    alertSound:true, alertVibrate:true, alertPush:true,
    ownerName:'',
    shopProfile:{
      name:'Minha Barbearia', logoDataUrl:APP_LOGO_ICON, bannerDataUrl:'',
      bio:'', address:'', phone:'', hours:'', clubAbout:'',
      social:{instagram:'', facebook:'', tiktok:'', whatsapp:'', website:''},
    },
    asaas:{apiKey:'', walletId:'', env:'sandbox', enabled:false},
    messageSettings:{reminderHours:3, winbackDays:23, depositExpireMinutes:30},
    messageTemplates:{
      confirmation:'Olá {cliente}! Seu horário está marcado para {data} às {hora}
- {servico} com {profissional}. Pode confirmar sua presença? 🗓️',
      reminder:'Oi {cliente}, passando pra lembrar: seu horário é hoje às {hora} -
{servico} com {profissional}. Te esperamos! 🗓️',
      winback:'Sentimos sua falta, {cliente}! Já faz {dias} dias desde seu último
corte. Que tal marcar um horário essa semana? Responda aqui que já te encaixamos.
✂️',
      upsellServices:'Oi {cliente}! Notamos que você ainda não experimentou
{sugestao}. Quer aproveitar no seu próximo horário?',
      upsellProducts:'{cliente}, temos {sugestao} disponíveis na barbearia -
combinam bem com o seu estilo de corte. Quer que a gente separe um pra você na
próxima visita?',
      waitlistOpen:'Boa notícia, {cliente}! Abriu um horário em {data} às {hora}
com {profissional}. Quer confirmar? Responda rápido que é por ordem de chegada.',
      reviewRequest:'Oi {cliente}! Como foi seu atendimento hoje? Dá uma nota de 1
a 5 pra gente por aqui, sua opinião ajuda muito. 🌟',
    },
    messageLog:[],
  }
}

```

```

    commissionSettings:{clubStaffPercent:60},
    nfe:{enabled:false, provider:'', apiKey:'', cnpj:'', autoEmit:false},
    nfeLog:[],
  };
}

let state = null;
let route = 'dashboard';
let toastTimer = null;
let userSession = {role:'dono', professionalId:null}; // não persistido - apenas
separa a visão por login
function isOwnerLevel(){ return userSession.role==='dono' ||
userSession.role==='admin'; }
// Só quem "atende clientes" aparece como barbeiro na agenda, no
// agendamento online, nas comissões e no pote do clube.
// Ordem alfabética, respeitando acentos do português
function byName(a, b){ return (a.name||'').localeCompare((b.name||'), 'pt-BR',
{sensitivity:'base'})); }
function servingProfessionals(){
  return (state.professionals||
[]).filter(p=>p.servesClients!==false).sort(byName);
}
function roleLabel(r){ return r==='admin' ? 'Admin' : r==='receptionista' ?
'Recepcionista' : 'Profissional'; }

async function loadState(){
  if(currentShop && currentShop.data && Object.keys(currentShop.data).length){
    state = currentShop.data;
  } else {
    state = blankShopData();
  }
  migrateState();
  render();
}
function applyTheme(theme){
  document.documentElement.dataset.theme = theme==='dark' ? 'dark' : 'light';
}
function migrateState(){
  // Garante que sessões salvas antes destas funcionalidades recebam valores
padrão.
  state.themePreference = state.themePreference || 'light';
  if(state.alertSound===null) state.alertSound = true;
  if(state.alertVibrate===null) state.alertVibrate = true;
  if(state.alertPush===null) state.alertPush = true;
  applyTheme(state.themePreference);
  if(state.ownerName===null) state.ownerName = '';
  state.withdrawalRequests = state.withdrawalRequests || [];
}

```

```

    state.onlineBooking = state.onlineBooking || {slug:'', allowCancel:true,
allowReschedule:true, active:true, autoAccept:false};
    if(state.onlineBooking.autoAccept==null) state.onlineBooking.autoAccept = false;
    if(state.onlineBooking.autoSubscribe==null) state.onlineBooking.autoSubscribe =
false;
    state.waitlist = state.waitlist || [];
    state.commissionSettings = state.commissionSettings || {clubStaffPercent:60};
    if(state.commissionSettings.clubStaffPercent==null)
state.commissionSettings.clubStaffPercent = 60;
    state.nfe = state.nfe || {enabled:false, provider:'', apiKey:'', cnpj:'',
autoEmit:false};
    state.nfeLog = state.nfeLog || [];
    state.shopProfile = state.shopProfile || {
        name:'Primer Barber', logoDataUrl:APP_LOGO_ICON, bannerDataUrl:'', bio:'',
address:'', phone:'',
        hours:'', clubAbout:'',
        social:{instagram:'', facebook:'', tiktok:'', whatsapp:'', website:''},
    };
    if(state.shopProfile.hours==null) state.shopProfile.hours = '';
    if(state.shopProfile.clubAbout==null) state.shopProfile.clubAbout = '';
    // profissionais já cadastrados: recepcionista não atende, o resto sim
    (state.professionals||[]).forEach(p=>{
        if(p.servesClients==null) p.servesClients = (p.accessRole!=='recepcionista');
    });
    (state.subscriptionPlans||[]).forEach(p=>{
        if(!Array.isArray(p.benefitsList)) p.benefitsList = [];
        if(p.featured==null) p.featured = false;
        if(p.hidden==null) p.hidden = false;
        // planos antigos passam a valer todos os dias por padrão
        if(!Array.isArray(p.validDays)) p.validDays = [0,1,2,3,4,5,6];
    });
    if(!state.shopProfile.social) state.shopProfile.social = {instagram:'',
facebook:'', tiktok:'', whatsapp:'', website:''};
    state.loyalty = state.loyalty || {pointsPerReal:1, redeemThreshold:100,
redeemValue:20};
    if(state.loyalty.referralBonusPoints==null) state.loyalty.referralBonusPoints =
30;
    state.messageSettings = state.messageSettings || {reminderHours:3,
winbackDays:23};
    if(state.messageSettings.depositExpireMinutes==null)
state.messageSettings.depositExpireMinutes = 30;
    state.messageTemplates = state.messageTemplates || {};
    state.messageTemplates.waitlistOpen = state.messageTemplates.waitlistOpen ||
'Boa notícia, {cliente}! Abriu um horário em {data} às {hora} com {profissional}.
Quer confirmar? Responda rápido que é por ordem de chegada.';
    state.messageTemplates.reviewRequest = state.messageTemplates.reviewRequest ||
'Oi {cliente}! Como foi seu atendimento hoje na Primer Barber? Dá uma nota de 1 a

```

```

5 pra gente por aqui, sua opinião ajuda muito. ☆';
(state.professionals||[]).forEach(p=>{
  if(!p.pin) p.pin = String(1000+Math.floor(Math.random()*9000));
  if(!p.workDays) p.workDays = [1,2,3,4,5,6];
  if(!p.workStart) p.workStart = '09:00';
  if(!p.workEnd) p.workEnd = '19:00';
  if(!p.timeOff) p.timeOff = [];
});
state.timeBlocks = state.timeBlocks || [];
(state.professionals||[]).forEach(p=>{
  if(!p.serviceOverrides) p.serviceOverrides = {};
});
(state.services||[]).forEach(s=>{
  if(s.requireDeposit===null) s.requireDeposit = false;
  if(!s.depositType) s.depositType = 'percentual';
  if(s.depositValue===null) s.depositValue = 30;
  if(s.clubFichas===null) s.clubFichas = 0;
  if(s.hideOnline===null) s.hideOnline = false;
});
(state.appointments||[]).forEach(a=>{
  if(!a.depositStatus) a.depositStatus = 'nao_aplicavel';
  if(a.depositAmount===null) a.depositAmount = 0;
  if(!a.createdAt) a.createdAt = new Date().toISOString();
  if(a.rating===undefined) a.rating = null;
  if(a.confirmed===null) a.confirmed = false;
});
(state.clients||[]).forEach(c=>{
  if(c.referredBy===null) c.referredBy = '';
  if(c.referralBonusGiven===null) c.referralBonusGiven = false;
});
}
let ultimoSalvamentoOk = true;
function avisarFalhaAoSalvar(detalle){
  if(ultimoSalvamentoOk === false) return; // não repete o aviso a cada tentativa
  ultimoSalvamentoOk = false;
  console.error('Falha ao salvar', detalle);
  showToast('⚠ NÃO FOI POSSÍVEL SALVAR. Confira sua internet e recarregue a página
- o que você fez agora pode se perder.');
```

// barra fixa, para não passar despercebido

```

  if(!document.getElementById('save-error-bar')){
    const bar = document.createElement('div');
    bar.id = 'save-error-bar';
    bar.style.cssText = 'position:fixed;top:0;left:0;right:0;z-
index:9999;background:var(--bad);color:#fff;padding:10px 14px;font-size:13px;text-
align:center;font-weight:600;';
    bar.innerHTML = '⚠ As alterações não estão sendo salvas. Recarregue a página
antes de continuar. <span style="text-decoration:underline;cursor:pointer;"
```

```

id="save-error-reload">Recarregar agora</span>';
    document.body.appendChild(bar);
    document.getElementById('save-error-reload').addEventListener('click',
    ()=>location.reload());
  }
}

function limparAvisoDeFalha(){
  if(ultimoSalvamentoOk) return;
  ultimoSalvamentoOk = true;
  document.getElementById('save-error-bar')?.remove();
}

let salvandoAgora = false;
async function persist(){
  if(!supa || !currentShop) return;
  salvandoAgora = true;
  setTimeout(()=>{ salvandoAgora = false; }, 1500);
  try{
    if(teamSession){
      // Equipe logada por PIN - grava por uma função que revalida o PIN
      const { data: res, error } = await supa.rpc('team_save', {
        p_shop_id: teamSession.shopId, p_prof_id: teamSession.profId,
        p_pin: teamSession.pin, p_data: state
      });
      if(error || res?.error){ avisarFalhaAoSalvar(error || res.error); }
      else { limparAvisoDeFalha(); }
      return;
    }
    const slug = state?.onlineBooking?.slug || null;
    const { error } = await supa.from('shops').update({ data: state, booking_slug:
slug || null }).eq('id', currentShop.id);
    if(error){ avisarFalhaAoSalvar(error); }
    else { limparAvisoDeFalha(); }
  }catch(e){ avisarFalhaAoSalvar(e); }
}

/* Aplica a alteração sobre o dado MAIS RECENTE do servidor, não sobre o que
    está na memória deste navegador. Sem isso, dois aparelhos usando ao mesmo
    tempo se sobrescreviam: o último a salvar apagava o trabalho do outro. */
let filaDeSalvamento = Promise.resolve();

function update(mutator){
  // 1) aplica na tela na hora, para a resposta ser instantânea
  mutator(state);
  render();
  // 2) e enfileira a gravação segura (uma de cada vez, na ordem)
  filaDeSalvamento = filaDeSalvamento.then(()=>salvarComMerge(mutator)).catch(e=>{
    console.error(e);
  });
}

```

```

    });
}

async function buscarEstadoRemoto(){
    if(!supa || !currentShop) return null;
    try{
        if(teamSession){
            const { data: res, error } = await supa.rpc('team_login', {
                p_shop_id: teamSession.shopId, p_prof_id: teamSession.profId, p_pin:
teamSession.pin
            });
            if(error || !res?.ok) return null;
            return res.data;
        }
        const { data, error } = await supa.from('shops').select('data').eq('id',
currentShop.id).maybeSingle();
        if(error || !data) return null;
        return data.data;
    }catch(e){ return null; }
}

async function salvarComMerge(mutator){
    const remoto = await buscarEstadoRemoto();
    if(remoto){
        // aplica a MESMA alteração sobre os dados frescos do servidor
        const anterior = state;
        state = remoto;
        try{
            migrateState();
            mutator(state);
        }catch(e){
            // se a alteração não fizer sentido no dado novo, mantém o que tínhamos
            console.error('Não foi possível aplicar sobre os dados novos', e);
            state = anterior;
        }
    }
    await persist();
    render();
}

function showToast(msg){
    clearTimeout(toastTimer);
    const old = document.querySelector('.toast');
    if(old) old.remove();
    const el = document.createElement('div');
    el.className='toast';
    el.textContent = msg;
    document.body.appendChild(el);
}

```

```

    toastTimer = setTimeout(()=>el.remove(), 2600);
}

/* ===== DERIVED HELPERS ===== */
function profName(id){ return state.professionals.find(p=>p.id===id)?.name || '-';
}
// Valor que realmente entrou por um atendimento: usa o que foi cobrado no
// fechamento; para atendimentos antigos, recalcula pela regra do plano.
function serviceById(id){ return state.services.find(s=>s.id===id); }

/* ===== SUPORTE A MAIS DE UM SERVIÇO POR AGENDAMENTO =====
Guardamos os serviços em appt.serviceIds (array). appt.serviceId
continua existindo, apontando para o primeiro – é o que todo o
código antigo (relatórios, lembretes, etc.) ainda lê sozinho. */
function appointmentServiceIds(appt){
    if(!appt) return [];
    if(Array.isArray(appt.serviceIds) && appt.serviceIds.length) return
appt.serviceIds;
    return appt.serviceId ? [appt.serviceId] : [];
}
function appointmentServices(appt){
    return appointmentServiceIds(appt).map(id=>serviceById(id)).filter(Boolean);
}
function appointmentDuration(appt){
    const svcs = appointmentServices(appt);
    if(svcs.length===0) return 30;
    return svcs.reduce((t,s)=>t+(Number(s.duration)||30),0);
}
function appointmentServiceNames(appt){
    const svcs = appointmentServices(appt);
    return svcs.length ? svcs.map(s=>s.name).join(' + ') : 'Serviço removido';
}
// Preço de cada serviço do agendamento, já considerando o clube – soma o
// total e guarda o detalhe de cada item (necessário pra comissão e fichas
// funcionarem certo quando um dos serviços está coberto e outro não).
function precoParaCobrarMultiplo(appt){
    const svcs = appointmentServices(appt);
    if(svcs.length===0) return { total:0, itens:[], motivoCheio:'', membro:null,
todosCobertos:false };
    let total = 0, motivoCheio = '', membro = null;
    const itens = svcs.map(svc=>{
        const calc = precoParaCobrar(appt, svc);
        total += calc.basePrice;
        if(calc.member && !membro) membro = calc.member;
        if(calc.motivoCheio && !motivoCheio) motivoCheio = calc.motivoCheio;
        return { service: svc, calc };
    });

```

```

    const todosCobertos = itens.every(i=>i.calc.planoAtivo);
    return { total, itens, motivoCheio, membro, todosCobertos };
}

function valorRealAtendimento(ap){
  if(!ap) return 0;
  if(Array.isArray(ap.valorPorServico) && ap.valorPorServico.length){
    return ap.valorPorServico.reduce((t,i)=>t+Number(i.valor||0),0);
  }
  if(ap.valorCobrado != null) return Number(ap.valorCobrado);
  return precoParaCobrarMultiplo(ap).total;
}

// Comissão de SERVIÇO gerada por um atendimento, para um profissional -
// usa o detalhe salvo por serviço quando existe (atendimento com mais de
// um serviço, cada um podendo ter % diferente), senão cai no cálculo
// simples de antes (atendimentos de um serviço só).
function comissaoDoAtendimento(ap, prof){
  if(Array.isArray(ap.valorPorServico) && ap.valorPorServico.length){
    return ap.valorPorServico.reduce((t,item)=>{
      if(item.valor<=0) return t;
      const pct = effectiveCommissionPercent(prof, item.serviceId);
      return t + item.valor*(pct/100);
    },0);
  }
  const svc = serviceById(ap.serviceId);
  if(!svc) return 0;
  const valorCobrado = (ap.valorCobrado != null) ? Number(ap.valorCobrado) :
precoParaCobrar(ap, svc).basePrice;
  if(valorCobrado<=0) return 0;
  return valorCobrado * (effectiveCommissionPercent(prof, svc.id)/100);
}

function openSession(){ return state.cashSessions.find(c=>c.status==='aberto') ||
null; }
const PAYMENT_METHODS = [
  {id:'dinheiro', label:'Dinheiro', inDrawer:true},
  {id:'pix', label:'PIX', inDrawer:false},
  {id:'debito', label:'Cartão de débito', inDrawer:false},
  {id:'credito', label:'Cartão de crédito', inDrawer:false},
];
function methodLabelPay(id){ return (PAYMENT_METHODS.find(m=>m.id===id)||{}).label
|| 'Dinheiro'; }
function sessionTotals(session){
  if(!session) return {in:0,out:0,balance:0,byMethod:{},cashIn:0,cashOut:0};
  const entries = session.entries;
  const inn =
entries.filter(e=>e.type==='entrada').reduce((a,e)=>a+Number(e.amount),0);

```

```

    const out =
entries.filter(e=>e.type==='saida').reduce((a,e)=>a+Number(e.amount),0);
    // Detalhamento por forma de pagamento (entradas)
    const byMethod = {};
    PAYMENT_METHODS.forEach(m=>{ byMethod[m.id] = 0; });
    entries.filter(e=>e.type==='entrada').forEach(e=>{
        const m = e.method || 'dinheiro';
        byMethod[m] = (byMethod[m]||0) + Number(e.amount);
    });
    // O caixa físico (gaveta) só conta o que entrou/saiu em dinheiro
    const cashIn = entries.filter(e=>e.type==='entrada' &&
(e.method||'dinheiro')==='dinheiro').reduce((a,e)=>a+Number(e.amount),0);
    const cashOut = entries.filter(e=>e.type==='saida' &&
(e.method||'dinheiro')==='dinheiro').reduce((a,e)=>a+Number(e.amount),0);
    return {in:inn, out, byMethod, cashIn, cashOut, balance: session.openingAmount +
cashIn - cashOut};
}
function todaysAppointments(){ return
state.appointments.filter(a=>a.date===todayISO()).sort((a,b)=>a.time.localeCompare(
)
// Compara nomes ignorando acentos, maiúsculas e espaços repetidos
function normNome(s){
    return String(s||'').toLowerCase()
        .normalize('NFD').replace(/[\u0300-\u036f]/g,'')
        .replace(/\s+/g,' ').trim();
}
function soDigitos(s){ return String(s||'').replace(/\D/g,''); }

// Assinante realmente em atraso: além do status, precisa ter um plano
// que ainda exista. Sem isso, quem ficou órfão de um plano excluído
// continuava aparecendo no aviso de cobrança para sempre.
function assinantesEmAtraso(){
    return (state.subscriptionMembers||[]).filter(m=>{
        if(m.status !== 'atrasado') return false;
        const plano = (state.subscriptionPlans||[]).find(p=>p.id===m.planId);
        return !!plano;
    });
}
function assinantesOrfaos(){
    return (state.subscriptionMembers||[]).filter(m=>
        m.status!=='cancelado' && !(state.subscriptionPlans||
[]).some(p=>p.id===m.planId)
    );
}
function clubMemberForClient(clientName){
    if(!clientName) return null;
    const alvo = normNome(clientName);

```

```

    const ativos = (state.subscriptionMembers||
[])].filter(m=>m.status!=='cancelado');

    // 0) vínculo manual: se o dono ligou este assinante a um cliente,
    // essa ligação vale acima de qualquer tentativa de adivinhar pelo nome.
    const vinculado = ativos.find(m=>m.linkedClientName &&
normNome(m.linkedClientName)===alvo);
    if(vinculado) return vinculado;

    // 1) nome igual (já ignorando acento e espaços)
    let achado = ativos.find(m=>normNome(m.name)===alvo);
    if(achado) return achado;

    // 2) pelo telefone do cliente – mais confiável que o nome
    const cliente = (state.clients||[]).find(cl=>normNome(cl.name)===alvo);
    const tel = soDigitos(cliente?.phone);
    if(tel.length>=8){
        achado = ativos.find(m=>{
            const t = soDigitos(m.phone);
            return t.length>=8 && t.slice(-8)===tel.slice(-8);
        });
        if(achado) return achado;
    }

    // 3) um nome contém o outro (ex: "João Silva" x "João Silva Costa")
    achado = ativos.find(m=>{
        const n = normNome(m.name);
        if(n.length<5 || alvo.length<5) return false;
        return n.startsWith(alvo+' ') || alvo.startsWith(n+' ');
    });
    if(achado) return achado;

    // 4) mesmo primeiro nome + algum sobrenome em comum.
    // Resolve "Arthur Siqueira" (Asaas) x "Arthur da Silva Siqueira" (cadastro),
    // que é o caso mais comum quando os dois vieram de origens diferentes.
    const ignorar = new Set(['da','de','do','dos','das','e']);
    const partes = t => t.split(' ').filter(x=>x.length>1 && !ignorar.has(x));
    const alvoP = partes(alvo);
    if(alvoP.length >= 2){
        achado = ativos.find(m=>{
            const mP = partes(normNome(m.name));
            if(mP.length < 2) return false;
            if(mP[0] !== alvoP[0]) return false;
            // primeiro nome tem que
bater
            const sobrenomesAlvo = new Set(alvoP.slice(1));
            return mP.slice(1).some(s=>sobrenomesAlvo.has(s)); // e ao menos um
sobrenome

```

```

    });
  }
  return achado || null;
}

/* Igual à função acima, mas devolve TODOS os planos ativos que o cliente
tem – não só o primeiro. É comum um cliente ter mais de uma assinatura
ao mesmo tempo (ex: plano de cabelo seg-sáb + plano de barba seg-qui),
e cada serviço precisa ser cobrado pelo plano certo, não pelo primeiro
que o sistema encontrar. Usa a mesma ordem de confiança de antes, mas
pega TODOS os que baterem no primeiro nível que encontrar algo. */
function clubMembersForClient(clientName) {
  if(!clientName) return [];
  const alvo = normNome(clientName);
  const ativos = (state.subscriptionMembers||
[]).filter(m=>m.status!=='cancelado');

  const vinculados = ativos.filter(m=>m.linkedClientName &&
normNome(m.linkedClientName)===alvo);
  if(vinculados.length) return vinculados;

  const porNome = ativos.filter(m=>normNome(m.name)===alvo);
  if(porNome.length) return porNome;

  const cliente = (state.clients||[]).find(cl=>normNome(cl.name)===alvo);
  const tel = soDigitos(cliente?.phone);
  if(tel.length>=8){
    const porTel = ativos.filter(m=>{
      const t = soDigitos(m.phone);
      return t.length>=8 && t.slice(-8)===tel.slice(-8);
    });
    if(porTel.length) return porTel;
  }

  const porParcial = ativos.filter(m=>{
    const n = normNome(m.name);
    if(n.length<5 || alvo.length<5) return false;
    return n.startsWith(alvo+' ') || alvo.startsWith(n+' ');
  });
  if(porParcial.length) return porParcial;

  const ignorar = new Set(['da','de','do','dos','das','e']);
  const partes = t => t.split(' ').filter(x=>x.length>1 && !ignorar.has(x));
  const alvoP = partes(alvo);
  if(alvoP.length >= 2){
    const porSobrenome = ativos.filter(m=>{
      const mP = partes(normNome(m.name));

```

```

    if(mP.length < 2) return false;
    if(mP[0] !== alvoP[0]) return false;
    const sobrenomesAlvo = new Set(alvoP.slice(1));
    return mP.slice(1).some(s=>sobrenomesAlvo.has(s));
  });
  if(porSobrenome.length) return porSobrenome;
}
return [];
}

```

```

function clubStarBadge(clientName){
  const membros = clubMembersForClient(clientName);
  if(membros.length === 0) return '';

  const vencidoM = m => m.dueDate && String(m.dueDate) < todayISO();
  const emDiaM = m => m.status === 'pago' || (m.status === 'pendente' &&
!vencidoM(m));
  const emDiaCount = membros.filter(emDiaM).length;

  let cor, txt;
  if(membros.length > 1){
    // mais de um plano: mostra a situação combinada
    if(emDiaCount === membros.length){
      cor = 'var(--gold-bright)';
      txt = `Assinante do clube - ${membros.length} planos, todos em dia`;
    } else if(emDiaCount === 0){
      cor = 'var(--bad)';
      txt = `Assinante do clube - ${membros.length} planos, todos em atraso`;
    } else {
      cor = 'var(--warn)';
      txt = `Assinante do clube - ${emDiaCount} de ${membros.length} planos em dia
(confira qual está atrasado)`;
    }
    return `<span title="${txt}" style="color:${cor};display:inline-
flex;">${ICON.star}</span>`;
  }
}

```

```

const m = membros[0];
const vencido = vencidoM(m);
if(m.status === 'atrasado' || vencido){
  cor = 'var(--bad)';
  txt = 'Assinante do clube - mensalidade atrasada (paga valor cheio)';
} else if(m.status === 'pago'){
  cor = 'var(--gold-bright)';
  txt = 'Assinante do clube - em dia';
} else if(m.status === 'pendente'){
  cor = 'var(--gold-bright)';

```

```

    txt = m.dueDate
    ? `Assinante do clube - em dia (vence em ${new
Date(String(m.dueDate)+'T12:00').toLocaleDateString('pt-BR')})`
    : 'Assinante do clube - em dia';
  } else {
    cor = 'var(--muted-2)';
    txt = `Assinante do clube - ${m.status}`;
  }
  return `${ICON.star}</span>`;
}
function clubOverdueWaButton(clientName, phone){
  const m = clubMemberForClient(clientName);
  if(!m || m.status!=='atrasado') return '';
  const plan = state.subscriptionPlans.find(p=>p.id===m.planId);
  let text = `Olá ${m.name}! Notamos que sua mensalidade do ${plan?
plan.name:'clube'} está em atraso.`;
  if(m.invoiceUrl) text += ` Você pode regularizar por aqui: ${m.invoiceUrl}`;
  else text += ` Pode regularizar quando puder? Qualquer dúvida estamos à
disposição.`;
  return messageChannelRow(phone||m.phone, text);
}
function weekDates(){
  const now = new Date();
  const day = (now.getDay()+6)%7; // 0 = segunda
  const monday = new Date(now); monday.setDate(now.getDate()-day);
  return Array.from({length:7}, (_,i)=>{
    const d = new Date(monday); d.setDate(monday.getDate()+i);
    return d.toISOString().slice(0,10);
  });
}
function clientsPerProfPerDay(profId, date){
  return state.appointments.filter(a=>a.professionalId===profId && a.date===date
&& a.status!=='cancelado').length;
}
function clientStats(name){
  const own =
state.appointments.filter(a=>a.client.trim().toLowerCase()===name.trim().toLowerCase
  const done = own.filter(a=>a.status==='concluido');
  const total = done.reduce((a,ap)=>a+valorRealAtendimento(ap),0);
  const last = done.sort((a,b)=>(b.date+b.time).localeCompare(a.date+a.time))[0];
  return {visits: done.length, total, lastVisit: last ? last.date : null};
}

/* ===== DETECÇÃO E MESCLAGEM DE CLIENTES DUPLICADOS =====
  O sistema liga tudo (agenda, assinatura, comissão) pelo NOME digitado.
  Por isso, mesclar não é só apagar - é trocar o nome em cada lugar que

```

```

    aponta pro registro removido, para o histórico não se perder. */
function encontrarClientesDuplicados(){
  const clientes = state.clients||[];
  const porTelefone = {};
  const porNome = {};
  clientes.forEach(c=>{
    const tel = soDigitos(c.phone).slice(-8);
    if(tel.length>=8){ (porTelefone[tel] = porTelefone[tel]||[]).push(c); }
    const n = normNome(c.name);
    if(n){ (porNome[n] = porNome[n]||[]).push(c); }
  });

  const grupos = [];
  const jaAgrupados = new Set();

  // confiança alta: mesmo telefone
  Object.values(porTelefone).forEach(lista=>{
    if(lista.length<2) return;
    grupos.push({tipo:'telefone', confianca:'alta', clientes:lista});
    lista.forEach(c=>jaAgrupados.add(c.id));
  });

  // confiança média: nome idêntico (ignorando acento/maiúscula), mas não
  // já resolvido pelo grupo de telefone
  Object.values(porNome).forEach(lista=>{
    const restantes = lista.filter(c=>!jaAgrupados.has(c.id));
    if(restantes.length<2) return;
    grupos.push({tipo:'nome', confianca:'media', clientes:restantes});
    restantes.forEach(c=>jaAgrupados.add(c.id));
  });

  return grupos;
}

/* Junta um grupo de clientes duplicados num só registro, e atualiza
   todos os lugares do sistema que referenciam os nomes removidos. */
function mesclarClientesDuplicados(grupo, idParaManter){
  const manter = grupo.find(c=>c.id===idParaManter);
  const outros = grupo.filter(c=>c.id!==idParaManter);
  const nomesAntigos = outros.map(c=>c.name);
  const nomeNovo = manter.name;

  update(s=>{
    const alvo = s.clients.find(c=>c.id===manter.id);
    // soma pontos, mantém o telefone/aniversário se o principal não tiver,
    // e junta as observações sem duplicar texto
    outros.forEach(o=>{

```

```

    alvo.points = (alvo.points||0) + (o.points||0);
    if(!alvo.phone && o.phone) alvo.phone = o.phone;
    if(!alvo.birthday && o.birthday) alvo.birthday = o.birthday;
    if(o.notes && !(alvo.notes||'').includes(o.notes)){
        alvo.notes = alvo.notes ? `${alvo.notes} | ${o.notes}` : o.notes;
    }
});

const bateAlumNomeAntigo = (valor) => {
    const n = normNome(valor);
    return nomesAntigos.some(x=>normNome(x)===n);
};

// renomeia em cada lugar que referencia os nomes removidos
s.appointments.forEach(a=>{ if(bateAlumNomeAntigo(a.client)) a.client =
nomeNovo; });
s.subscriptionMembers.forEach(m=>{
    if(bateAlumNomeAntigo(m.name)) m.name = nomeNovo;
    if(m.linkedClientName && bateAlumNomeAntigo(m.linkedClientName))
m.linkedClientName = nomeNovo;
});
s.clients.forEach(c=>{ if(c.referredBy && bateAlumNomeAntigo(c.referredBy))
c.referredBy = nomeNovo; });

// remove os registros de cliente duplicados
const idsRemover = new Set(outros.map(o=>o.id));
s.clients = s.clients.filter(c=>!idsRemover.has(c.id));
});
}

function monthDayLabel(md) {
    if(!md) return '-';
    const [m,d] = md.split('-');
    const meses =
['jan', 'fev', 'mar', 'abr', 'mai', 'jun', 'jul', 'ago', 'set', 'out', 'nov', 'dez'];
    return `${d} ${meses[parseInt(m)-1]}`;
}

/* ===== MENSAGENS AUTOMÁTICAS ===== */
function fillTemplate(tpl, vars) {
    return tpl.replace(/\{(\w+)\}/g, (_,k)=> (vars[k]!==null ? vars[k] : `${k}`));
}

function waLink(phone, text) {
    const digits = (phone||'').replace(/\D/g, '');
    const withCountry = digits.length<=11 ? '55'+digits : digits;
    return `https://wa.me/${withCountry}?text=${encodeURIComponent(text)}`;
}

```

```

function logMessage(type, clientName, phone, note){
  update(s=>{ s.messageLog.push({id:uid(), type, clientName, phone, note, date:new
Date().toISOString()}); });
}
function pendingConfirmations(){
  return state.appointments.filter(a=>a.status==='agendado' && !a.confirmed &&
a.date>=todayISO())
    .sort((a,b)=>(a.date+a.time).localeCompare(b.date+b.time));
}
function pendingReminders(){
  const now = new Date();
  return state.appointments.filter(a=>{
    if(a.status!=='agendado' || a.reminderSent || a.date!==todayISO()) return
false;
    const apptTime = new Date(a.date+'T'+a.time+':00');
    const diffH = (apptTime-now)/3600000;
    return diffH>0 && diffH<=state.messageSettings.reminderHours;
  }).sort((a,b)=>a.time.localeCompare(b.time));
}
function absentClients(){
  return state.clients.map(c=>({client:c, stats:clientStats(c.name)}))
    .filter(x=>x.stats.lastVisit)
    .map(x=>({...x, daysSince: Math.floor((new Date(todayISO())-new
Date(x.stats.lastVisit))/86400000)}))
    .filter(x=>x.daysSince>=state.messageSettings.winbackDays)
    .sort((a,b)=>b.daysSince-a.daysSince);
}
function suggestServicesFor(clientName){
  const done = new Set(state.appointments.filter(a=>a.status==='concluido' &&
a.client.trim().toLowerCase()===clientName.trim().toLowerCase()).map(a=>a.serviceId));
  return state.services.filter(s=>!done.has(s.id));
}
function phoneForClientName(name){
  return
state.clients.find(c=>c.name.trim().toLowerCase()===name.trim().toLowerCase())?.phone
|| '';
}
function currentPeriod(){ return todayISO().slice(0,7); }
function commissionOwed(profId){
  const prof = state.professionals.find(p=>p.id===profId);
  const period = currentPeriod();
  const paidService = state.commissionPayments.filter(c=>c.professionalId===profId
&& c.source==='servico').reduce((a,c)=>a+Number(c.amount),0);
  const paidPote = state.commissionPayments.filter(c=>c.professionalId===profId &&
c.source==='pote' && c.period===period).reduce((a,c)=>a+Number(c.amount),0);
  const paidSaque = state.commissionPayments.filter(c=>c.professionalId===profId
&& c.source==='saque').reduce((a,c)=>a+Number(c.amount),0);

```

```

    // vales (adiantamentos) também descontam do que o profissional tem a receber
    const paidVale = state.commissionPayments.filter(c=>c.professionalId===profId &&
c.source==='vale').reduce((a,c)=>a+Number(c.amount),0);
    const concludedAppts = state.appointments.filter(a=>a.professionalId===profId &&
a.status==='concluido').sort((a,b)=>(a.date+a.time).localeCompare(b.date+b.time));
    let serviceEarned = 0;
    concludedAppts.forEach(ap=>{
        // Comissão sobre o que REALMENTE foi cobrado. Atendimento coberto pelo
        // clube não gera comissão de serviço – o profissional é pago por ele
        // através das fichas do pote. Sem isso, ele receberia duas vezes.
        // comissaoDoAtendimento já lida com atendimentos de 1 ou vários
        // serviços, cada um com seu próprio percentual quando configurado.
        serviceEarned += comissaoDoAtendimento(ap, prof);
    });
    const potEarned = potCommissionForProf(profId);
    // O vale e o saque são adiantamentos: saem do que o profissional tem a
    // receber. Se o vale for MAIOR que o que ele já ganhou, o saldo fica
    // negativo de propósito – ele "deve" à barbearia, e isso é abatido
    // sozinho conforme novas comissões entram (basta o cálculo não
    // travar em zero: totalEarned sobe a cada atendimento novo).
    const serviceOwed = serviceEarned - paidService - paidSaque - paidVale;
    const potOwed = Math.max(0, potEarned - paidPote);
    const totalEarned = serviceEarned + potEarned;
    const totalPaid = paidService + paidPote + paidSaque + paidVale;
    return {
        earned: totalEarned, serviceEarned, potEarned,
        paid: totalPaid, paidService, paidPote, paidSaque, paidVale,
        owed: totalEarned - totalPaid, serviceOwed, potOwed,
        devendo: (totalEarned - totalPaid) < 0,
    };
}

function monthlyEarnedForProf(profId){
    const prof = state.professionals.find(p=>p.id===profId);
    if(!prof) return 0;
    const period = currentPeriod();
    const concludedThisMonth =
state.appointments.filter(a=>a.professionalId===profId && a.status==='concluido'
&& a.date.startsWith(period));
    let total = 0;
    concludedThisMonth.forEach(ap=>{
        // usa o valor REALMENTE cobrado (com desconto e clube aplicados),
        // não o preço de tabela – comissaoDoAtendimento já soma certo
        // quando o atendimento tem mais de um serviço
        total += comissaoDoAtendimento(ap, prof);
    });
    return total + potCommissionForProf(profId);
}

```

```

function effectiveCommissionPercent(prof, serviceId){
  return (prof.serviceOverrides && prof.serviceOverrides[serviceId]!==null) ?
prof.serviceOverrides[serviceId] : prof.commission;
}

/* ===== #4 HORÁRIOS E FOLGAS POR PROFISSIONAL ===== */
/* ===== BLOQUEIOS DE HORÁRIO (folga, almoço, compromisso) =====
  Cada bloqueio: {id, professionalId, date, start, end, motivo, repeteSemanal,
diaSemana}
  Se repeteSemanal = true, vale toda semana naquele dia da semana. */
function blocksForDay(profId, dateISO){
  const dow = new Date(dateISO+'T12:00').getDay();
  return (state.timeBlocks||[]).filter(b =>
    b.professionalId === profId &&
    (b.repeteSemanal ? b.diaSemana === dow : b.date === dateISO)
  );
}
function isBlocked(profId, dateISO, hhmm){
  return blocksForDay(profId, dateISO).some(b => hhmm >= b.start && hhmm < b.end);
}
function isProfessionalWorking(prof, dateStr){
  const weekday = new Date(dateStr+'T00:00').getDay();
  if(!prof.workDays.includes(weekday)) return false;
  if(prof.timeOff.includes(dateStr)) return false;
  return true;
}
function timesOverlap(startA,durA,startB,durB){
  const toMin = t=>{ const [hh,mm]=t.split(':').map(Number); return hh*60+mm; };
  const aStart=toMin(startA), aEnd=aStart+durA, bStart=toMin(startB),
bEnd=bStart+durB;
  return aStart<bEnd && bStart<aEnd;
}
function generateSlotsForProfessional(prof, dateStr, durationMin){
  if(!isProfessionalWorking(prof, dateStr)) return [];
  const slots = [];
  let [h,m] = prof.workStart.split(':').map(Number);
  const [eh,em] = prof.workEnd.split(':').map(Number);
  const workEndMinutes = eh*60+em;
  while(h*60+m < workEndMinutes){
    const timeStr = String(h).padStart(2,'0')+':'+String(m).padStart(2,'0');
    if(h*60+m+durationMin <= workEndMinutes){
      const conflict = state.appointments.some(a=>a.professionalId===prof.id &&
a.date===dateStr && a.status!=='cancelado' && a.status!=='faltou' &&
        timesOverlap(a.time, appointmentDuration(a), timeStr, durationMin));
      // horários bloqueados (folga, almoço) também saem da lista
      const blocked = blocksForDay(prof.id, dateStr).some(b=>{
        const [bsh,bsm] = b.start.split(':').map(Number);

```

```

        const [beh,bem] = b.end.split(':').map(Number);
        return timesOverlap(b.start, (beh*60+bem)-(bsh*60+bsm), timeStr,
durationMin);
    });
    if(!conflict && !blocked) slots.push(timeStr);
}
m+=15; if(m>=60){ m-=60; h+=1; }
}
return slots;
}

/* ===== #1 SINAL ANTECIPADO (DEPÓSITO) ===== */
function depositAmountFor(service){
    if(!service || !service.requireDeposit) return 0;
    return service.depositType==='fixo' ? service.depositValue : +(service.price*
(service.depositValue/100)).toFixed(2);
}
function pendingDeposits(){
    const now = Date.now();
    return state.appointments.filter(a=>a.status==='agendado' &&
a.depositStatus==='pendente');
}
function expiredDepositAppointments(){
    const now = Date.now();
    const limitMs = state.messageSettings.depositExpireMinutes*60000;
    return state.appointments.filter(a=>a.status==='agendado' &&
a.depositStatus==='pendente' && (now - new Date(a.createdAt).getTime()) >
limitMs);
}

/* ===== #2 LISTA DE ESPERA ===== */
function tryFillFromWaitlist(freedDate, freedTime, freedProfessionalId){
    if(!state.waitlist.length) return null;
    const idx = state.waitlist.findIndex(w=> (!w.preferredDate ||
w.preferredDate===freedDate) && (!w.professionalId ||
w.professionalId===freedProfessionalId));
    if(idx===-1) return null;
    return state.waitlist[idx];
}

/* ===== #9 NO-SHOW ===== */
function noShowStats(){
    const relevant = state.appointments.filter(a=>
['concluido','faltou','cancelado'].includes(a.status));
    const noShows = relevant.filter(a=>a.status==='faltou');
    const rate = relevant.length ? (noShows.length/relevant.length*100) : 0;
    const byProf = servingProfessionals().map(p=>{

```

```

        const total = relevant.filter(a=>a.professionalId===p.id).length;
        const faltas = relevant.filter(a=>a.professionalId===p.id &&
a.status==='faltou').length;
        return {prof:p, total, faltas, rate: total ? (faltas/total*100) : 0};
    });
    return {rate, total:noShows.length, byProf};
}

/* ===== #7 AVALIAÇÕES ===== */
function ratingStats(profId){
    const rated = state.appointments.filter(a=>a.professionalId===profId &&
a.rating);
    if(rated.length===0) return {avg:0, count:0};
    const avg = rated.reduce((a,r)=>a+r.rating.stars,0)/rated.length;
    return {avg, count:rated.length};
}

/* ===== #8 PROGRAMA DE INDICAÇÃO ===== */
function maybeAwardReferral(client){
    if(!client.referredBy || client.referralBonusGiven) return;
    const referrer =
state.clients.find(c=>c.name.trim().toLowerCase()===client.referredBy.trim().toLower
    if(!referrer) return;
    referrer.points = (referrer.points||0) + (state.loyalty.referralBonusPoints||0);
    client.referralBonusGiven = true;
}

/* ===== SISTEMA DE POTE (clube de assinatura) – por fichas ===== */
// Todo mês, a soma de todos os assinantes pagos forma UM pote geral.
// Desse pote, uma % fica pra barbearia e o restante ("bolo dos profissionais")
// é dividido proporcionalmente pela quantidade de fichas que cada profissional
// acumulou no mês. Cada serviço tem um valor de fichas configurável – quando
// um profissional atende um cliente assinante pago, ganha as fichas daquele
serviço.
// Pagou DENTRO do mês? É isso que faz o pote virar quando o mês muda.
// Antes olhávamos só o status "pago", que ficava marcado do mês anterior.
function pagouNoMes(m, ym){
    if(!m || !m.lastPaymentDate) return false;
    return String(m.lastPaymentDate).slice(0,7) === ym;
}

function clubMonthlyPot(ym){
    const mes = ym || todayISO().slice(0,7);
    return state.subscriptionMembers.filter(m=>pagouNoMes(m,
mes)).reduce((total,m)=>{
        const plan = state.subscriptionPlans.find(p=>p.id===m.planId);
        return total + (plan ? plan.price : 0);
    }, 0);
}

```

```

}
function clubFichaEntries(){
  const monthPrefix = todayISO().slice(0,7);
  // A ficha é do trabalho feito: se o profissional atendeu um assinante,
  // ela conta mesmo que a mensalidade ainda não tenha entrado. O que
  // depende do pagamento é o VALOR do pote, não a contagem de fichas.
  // Um agendamento com mais de um serviço soma a ficha de cada um.
  const entries = [];
  state.appointments
    .filter(a=>a.status==='concluido' && a.date.startsWith(monthPrefix))
    .forEach(a=>{
      const membro = clubMemberForClient(a.client);
      if(!membro) return;
      appointmentServices(a).forEach(service=>{
        const fichas = service?.clubFichas || 0;
        if(fichas>0) entries.push({appt:a, service, membro, fichas});
      });
    });
  return entries;
}
function clubFichasBreakdown(){
  const entries = clubFichaEntries();
  const totalFichas = entries.reduce((a,e)=>a+e.fichas,0);
  const potTotal = clubMonthlyPot();
  const staffPercent = state.commissionSettings.clubStaffPercent ?? 60;
  const potStaff = potTotal * (staffPercent/100);
  const potBarbearia = potTotal - potStaff;
  const byProf = {};
  entries.forEach(e=>{ byProf[e.appt.professionalId] =
  (byProf[e.appt.professionalId]||0) + e.fichas; });
  // usa TODOS os profissionais aqui (não só quem "atende clientes"), porque
  // alguém pode ter feito atendimentos no passado e depois ter sido
  // marcado como "não atende" - a ficha dele não pode sumir da conta.
  const rows = (state.professionals||[]).map(p=>{
    const fichas = byProf[p.id] || 0;
    const value = totalFichas>0 ? potStaff*(fichas/totalFichas) : 0;
    return {prof:p, fichas, value};
  }).filter(r=>r.fichas>0);
  return {potTotal, potStaff, potBarbearia, staffPercent, totalFichas, entries,
  rows};
}

// Diagnóstico: por que um atendimento de assinante não gerou ficha?
function diagnosticoFichas(){
  const monthPrefix = todayISO().slice(0,7);
  const problemas = [];
  state.appointments

```

```

        .filter(a=>a.status==='concluido' && a.date.startsWith(monthPrefix))
        .forEach(a=>{
            const membro = clubMemberForClient(a.client);
            if(!membro) return; // não é assinante, não deveria gerar ficha mesmo
            const svcs = appointmentServices(a);
            const prof = state.professionals.find(p=>p.id===a.professionalId);
            if(svcs.length===0){ problemas.push({a, tipo:'servico_excluido',
motivo:'Serviço foi excluído do sistema'}); return; }
            svcs.forEach(service=>{
                if(!service.clubFichas) problemas.push({a, tipo:'servico_sem_fichas',
refId:service.id, motivo:`Serviço "${service.name}" está com 0 fichas
configuradas`});
            });
            if(prof && prof.servesClients===false){ problemas.push({a,
tipo:'prof_ao_atende', refId:prof.id, motivo:`${prof.name} está marcado como "não
atende clientes" - a ficha conta, mas ele não aparece nas listas comuns`}); }
            });
            return problemas;
        }
    // Agrupa o diagnóstico por causa (mesmo serviço/profissional), para não
    // repetir o mesmo aviso várias vezes e para dar um botão de correção
    // que já sabe exatamente qual registro abrir.
    function diagnosticoFichasAgrupado(){
        const problemas = diagnosticoFichas();
        const grupos = {};
        problemas.forEach(p=>{
            const key = p.tipo + ':' + (p.refId || p.motivo);
            if(!grupos[key]) grupos[key] = { motivo:p.motivo, tipo:p.tipo, refId:p.refId,
apps:[] };
            grupos[key].apps.push(p.a);
        });
        return { total: problemas.length, grupos: Object.values(grupos) };
    }
    function potCommissionForProf(profId){
        const row = clubFichasBreakdown().rows.find(r=>r.prof.id===profId);
        return row ? row.value : 0;
    }
    function asaasSummary(){
        const inMonth = state.subscriptionMembers.filter(m=>m.paymentMethod==='asaas');
        const cash = state.subscriptionMembers.filter(m=>m.paymentMethod==='dinheiro');
        const planPrice = m => state.subscriptionPlans.find(p=>p.id===m.planId)?.price
        || 0;
        return {
            asaasPago: inMonth.filter(m=>m.status==='pago'),
            asaasPendente: inMonth.filter(m=>m.status==='pendente'),
            asaasAtrasado: inMonth.filter(m=>m.status==='atrasado'),
            cashPago: cash.filter(m=>m.status==='pago'),

```

```

        cashPendente: cash.filter(m=>m.status!=='pago' && m.status!=='cancelado'),
        // soma apenas o valor que de fato entrou (cobranças pagas), não saldo de
        conta
        totalRecebidoAsaas: inMonth.filter(m=>pagouNoMes(m,
todayISO().slice(0,7))).reduce((a,m)=>a+planPrice(m),0),
        totalRecebidoDinheiro: cash.filter(m=>pagouNoMes(m,
todayISO().slice(0,7))).reduce((a,m)=>a+planPrice(m),0),
    };
}

/* ===== MODAL SYSTEM ===== */
let modalRoot;
function openModal(html, onMount){
    closeModal();
    modalRoot = document.createElement('div');
    modalRoot.className = 'modal-overlay';
    modalRoot.innerHTML = `<div class="modal">${html}</div>`;
    modalRoot.addEventListener('click', e=>{ if(e.target===modalRoot) closeModal();
});
    document.body.appendChild(modalRoot);
    if(onMount) onMount(modalRoot);
}
function closeModal(){ if(modalRoot){ modalRoot.remove(); modalRoot=null; } }

/* ===== NAV CONFIG (por categoria) ===== */
const NAV_GROUPS = [
    {label:'Visão geral', items:[
        {id:'dashboard', label:'Painel', icon:ICON.dashboard},
    ]},
    {label:'Atendimento', items:[
        {id:'agenda', label:'Agenda', icon:ICON.agenda},
        {id:'espera', label:'Lista de Espera', icon:ICON.listPlus},
        {id:'clientes', label:'Clientes', icon:ICON.team},
        {id:'configuracoes', label:'Configurações da Página', icon:ICON.image},
        {id:'agendamento-online', label:'Agendamento Online', icon:ICON.globe},
    ]},
    {label:'Comunicação', items:[
        {id:'mensagens', label:'Mensagens Automáticas', icon:ICON.chat},
    ]},
    {label:'Financeiro', items:[
        {id:'caixa', label:'Caixa', icon:ICON.cash},
        {id:'comissoes', label:'Comissões', icon:ICON.percent},
        {id:'central', label:'Central de Inteligência', icon:ICON.chart},
        {id:'relatorios', label:'Relatórios', icon:ICON.chart},
    ]},
    {label:'Equipe & Catálogo', items:[
        {id:'profissionais', label:'Profissionais', icon:ICON.team},
    ]},

```

```

    {id:'servicos', label:'Serviços', icon:ICON.scissors},
    {id:'estoque', label:'Estoque', icon:ICON.box},
  ]},
  {label:'Crescimento', items:[
    {id:'clube', label:'Clube de Assinatura', icon:ICON.club},
    {id:'fidelidade', label:'Fidelidade', icon:ICON.heart},
  ]},
  {label:'Sistema', items:[
    {id:'meus-links', label:'Meus Links', icon:ICON.link},
    {id:'minha-assinatura', label:'Configurações da Conta', icon:ICON.shield},
    {id:'config', label:'Integração Asaas', icon:ICON.gear},
    {id:'notafiscal', label:'Notas Fiscais', icon:ICON.receipt},
  ]},
  {label:'Ajuda', items:[
    {id:'ajuda', label:'Central de Ajuda', icon:ICON.help},
  ]},
];

const NAV = NAV_GROUPS.flatMap(g=>g.items);
function navAllowedForSession(){
  if(isOwnerLevel()) return NAV.map(n=>n.id);
  if(userSession.role==='receptionista') return
['dashboard','agenda','espera','clientes','mensagens','caixa','agendamento-
online','servicos','estoque','ajuda'];
  // profissional: acesso restrito à própria agenda, comissões, mensagens e ajuda
  return ['dashboard','agenda','espera','comissoes','ajuda'];
}

/* ===== ROOT RENDER ===== */
function render(){
  const app = document.getElementById('app');
  const cashOpen = openSession();
  if(!navAllowedForSession().includes(route)) route = 'dashboard';
  const activeProf = userSession.professionalId ?
state.professionals.find(p=>p.id===userSession.professionalId) : null;
  app.innerHTML = `
    <div style="display:flex;min-height:100vh;">
      <aside class="sidebar" style="width:236px;flex-shrink:0;background:var(--
side-bg);border-right:1px solid var(--side-border);padding:22px
14px;position:sticky;top:0;height:100vh;overflow-y:auto;display:flex;flex-
direction:column;">
        <div style="display:flex;align-items:center;gap:9px;padding:0 8px 22px;">
          <div style="width:34px;height:34px;border-radius:9px;background:var(--
side-hover);display:flex;align-items:center;justify-content:center;padding:5px;">
            
          </div>
          <div>

```

```

        <div class="font-display" style="font-size:19px;font-weight:600;line-
height:1;">Primer Barber</div>
        <div style="font-size:10.5px;color:var(--side-muted-2);letter-
spacing:.08em;text-transform:uppercase;margin-top:2px;">Gestão de barbearia</div>
    </div>
</div>
<nav style="display:flex;flex-direction:column;gap:14px;flex:1;">
    ${NAV_GROUPS.map(g=>{
        const items =
g.items.filter(n=>navAllowedForSession().includes(n.id));
        if(items.length===0) return '';
        return `<div>
            <div style="font-size:10.5px;color:var(--side-muted-2);text-
transform:uppercase;letter-spacing:.08em;padding:0 10px;margin-
bottom:5px;">${g.label}</div>
            <div style="display:flex;flex-direction:column;gap:3px;">
                ${items.map(n=>{
                    // contador de saques pendentes no menu Comissões
                    const pend = (n.id==='comissoes' && isOwnerLevel())
                    ? (state?.withdrawalRequests||
[[]].filter(w=>w.status==='pendente')).length : 0;
                    return `<div class="nav-item ${route===n.id?'active':''}" data-
route="${n.id}">
                        ${n.icon}<span style="flex:1;">${n.label}</span>
                        ${pend>0?`<span style="background:var(--bad);color:#fff;font-
size:10px;font-weight:700;border-radius:999px;min-
width:17px;height:17px;display:flex;align-items:center;justify-
content:center;padding:0 5px;">${pend}</span>`:''}
                    </div>`;
                }).join('')}
            </div>
        </div>`;
    }).join('')}
</nav>
<div class="divider-blade" style="margin:16px 6px;"></div>
<div style="padding:0 8px;margin-bottom:12px;">
    <div style="font-size:11px;color:var(--side-muted-2);text-
transform:uppercase;letter-spacing:.06em;margin-bottom:8px;">Status do caixa</div>
    <div class="pill ${cashOpen?'pill-ok':'pill-bad'}">${cashOpen?'Caixa
aberto':'Caixa fechado'}</div>
</div>
<div style="padding:0 8px;margin-bottom:12px;display:flex;align-
items:center;justify-content:space-between;gap:8px;">
    <div style="font-size:11px;color:var(--side-muted-2);text-
transform:uppercase;letter-spacing:.06em;">Tema</div>
    <div style="display:flex;border:1px solid var(--side-border);border-
radius:8px;overflow:hidden;">

```

```

        <button class="theme-toggle-btn" data-theme-choice="dark"
style="padding:5px 10px;font-
size:11.5px;border:none;cursor:pointer;background:${state.themePreference=== 'dark'?
-side-hover) '}};color:${state.themePreference=== 'dark'? '#12140f': 'var(--side-
muted) '}};">Escuro</button>
        <button class="theme-toggle-btn" data-theme-choice="light"
style="padding:5px 10px;font-
size:11.5px;border:none;cursor:pointer;background:${state.themePreference!== 'dark'?
-side-hover) '}};color:${state.themePreference!== 'dark'? '#12140f': 'var(--side-
muted) '}};">Claro</button>
    </div>
</div>
    <div style="padding:10px 8px 0;border-top:1px solid var(--side-border);">
        <div style="font-size:11px;color:var(--side-muted-2);text-
transform:uppercase;letter-spacing:.06em;margin:10px 0 6px;">${ICON.shield} Perfil
ativo</div>
        <div style="font-size:13px;font-weight:600;margin-
bottom:8px;">${userSession.role=== 'dono' ? 'Dono / Gerente' : (activeProf?.name ||
'Profissional') + (userSession.role=== 'admin'? '
Admin':userSession.role=== 'recepcionista'? 'Recepcionista:')}</div>
        <button class="btn btn-ghost" id="btn-switch-profile"
style="width:100%;justify-content:center;padding:7px;font-size:12.5px;margin-
bottom:6px;">Trocar perfil</button>
        <button class="btn btn-ghost" id="btn-logout" style="width:100%;justify-
content:center;padding:7px;font-size:12.5px;">Sair</button>
    </div>
</aside>
<div class="mobile-drawer" id="drawer-back"></div>
<div style="flex:1;min-width:0;">
<div class="mobile-topbar">
    <button class="btn btn-ghost" id="btn-open-menu" style="padding:8px
11px;">
        <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M4 7h16M4 12h16M4 17h16"/></svg>
    </button>
    <div style="width:26px;height:26px;border-radius:7px;background:var(--
side-hover);display:flex;align-items:center;justify-
content:center;padding:3px;flex-shrink:0;">
        
    </div>
    <div class="font-display" style="font-size:15px;font-
weight:600;flex:1;">${esc(state.shopProfile?.name || 'Primer Barber')}</div>
    <button class="btn btn-ghost" id="btn-notif-bell-m"
style="position:relative;padding:8px 10px;">${ICON.bell}</button>
</div>
<main style="flex:1;padding:26px 30px 60px;max-width:1180px;">

```

```

        <div style="display:flex;justify-content:flex-end;margin-
bottom:10px;position:relative;" class="desk-bell">
            <button class="btn btn-ghost" id="btn-notif-bell"
style="position:relative;padding:9px 11px;">${ICON.bell}</button>
            <div id="notif-dropdown"
style="display:${notifState.open?'block':'none'};position:absolute;top:44px;right:(
height:420px;overflow-y:auto;background:var(--panel);border:1px solid var(--
border);border-radius:10px;box-shadow:0 10px 30px rgba(0,0,0,.35);z-
index:80;">${notifState.open?notifDropdownContent():''}</div>
        </div>
        <div id="view"></div>
    </main>
</div>
</div>
<button id="btn-feedback" title="O que podemos melhorar?" style="
position:fixed;right:18px;bottom:18px;z-index:80;
width:52px;height:52px;border-radius:50%;border:1px solid var(--gold);
background:var(--panel);color:var(--gold-bright);cursor:pointer;
display:flex;align-items:center;justify-content:center;
box-shadow:0 8px 24px -6px rgba(0,0,0,.5);transition:transform .15s;"
onmouseover="this.style.transform='scale(1.08)'"
onmouseout="this.style.transform='none'">
    ${ICON.chat}
</button>
`;
app.querySelectorAll('[data-route]').forEach(el=>{
    el.addEventListener('click', ()=>{ route = el.dataset.route; render(); });
});
const fecharGaveta = ()=>document.body.classList.remove('drawer-open');
app.querySelector('#btn-open-menu')?.addEventListener('click',
()=>document.body.classList.add('drawer-open'));
app.querySelector('#drawer-back')?.addEventListener('click', fecharGaveta);
// ao escolher uma tela no menu, a gaveta fecha sozinha
app.querySelectorAll('.nav-item').forEach(el=>el.addEventListener('click',
fecharGaveta));
app.querySelector('#btn-notif-bell-m')?.addEventListener('click', (e)=>{
    e.stopPropagation(); notifState.open = !notifState.open; render();
});
app.querySelector('#btn-feedback')?.addEventListener('click', feedbackModal);
app.querySelector('#btn-switch-profile').addEventListener('click',
openProfileSwitchModal);
app.querySelector('#btn-logout').addEventListener('click', async ()=>{
    if(notifState.channel){ supa.removeChannel(notifState.channel);
notifState.channel=null; }
    notifState.items = []; notifState.open = false;
    if(teamSession){
        const slug = state?.onlineBooking?.slug;

```

```

        teamSession = null; currentShop = null; state = null;
        if(slug){ renderTeamLogin(slug); return; }
    }
    await supa.auth.signOut(); currentShop=null; state=null; boot();
  });
  app.querySelectorAll('.theme-toggle-btn').forEach(b=>b.addEventListener('click',
  ()=>{
    update(s=>{ s.themePreference = b.dataset.themeChoice; });
    applyTheme(b.dataset.themeChoice);
    render();
  }));
  app.querySelector('#btn-notif-bell')?.addEventListener('click', (e)=>{
    e.stopPropagation();
    notifState.open = !notifState.open;
    render();
  });
  app.querySelector('#notif-dropdown')?.addEventListener('click',
  e=>e.stopPropagation());
  app.querySelectorAll('[data-ir-
  comissoes]').forEach(b=>b.addEventListener('click', ()=>{
    notifState.open = false; route = 'comissoes'; render();
  }));
  app.querySelectorAll('[data-notif-
  accept]').forEach(b=>b.addEventListener('click', async ()=>{
    const req = notifState.items.find(r=>r.id===b.dataset.notifAccept);
    if(req) await acceptBookingRequest(req);
    render();
  }));
  app.querySelectorAll('[data-notif-
  decline]').forEach(b=>b.addEventListener('click', async ()=>{
    await declineBookingRequest(b.dataset.notifDecline);
    render();
  }));
  if(notifState.open){
    setTimeout(()=>{
      document.addEventListener('click', function closeNotifOnce(){
        notifState.open = false;
        document.removeEventListener('click', closeNotifOnce);
        render();
      }, {once:true});
    }, 0);
  }
  renderView();
}
function openProfileSwitchModal(){
  openModal(`
    <div style="padding:22px;">

```

```

    <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:16px;">
      <div class="font-display" style="font-size:19px;font-weight:600;">${ICON.shield} Trocar perfil</div>
      <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}</button>
    </div>
    <div style="font-size:12.5px;color:var(--muted);margin-bottom:14px;line-height:1.6;">Cada profissional entra com um PIN e vê apenas sua própria agenda, clientes e comissões. O dono vê tudo. Este é um controle de visualização simples – para autenticação real e segura, é necessário um backend.</div>
    <div style="display:flex;flex-direction:column;gap:8px;margin-bottom:14px;">
      <button class="btn ${userSession.role==='dono'?'btn-gold':'btn-ghost'}" style="justify-content:center;" data-set-role="dono">Entrar como Dono / Gerente</button>
    </div>
    <div style="font-size:11px;color:var(--muted-2);text-transform:uppercase;letter-spacing:.06em;margin-bottom:8px;">Ou entrar como profissional</div>
    <div style="display:flex;flex-direction:column;gap:8px;">
      ${state.professionals.slice().sort(byName).map(p=>`
        <div style="display:flex;gap:8px;align-items:center;">
          <div style="flex:1;font-size:13.5px;">${esc(p.name)} <span style="font-size:11px;color:var(--muted-2);">- ${roleLabel(p.accessRole)}</span>
        </div>
        <input type="password" maxlength="4" placeholder="PIN" data-pin-for="${p.id}" style="width:70px;text-align:center;">
        <button class="btn btn-ghost" style="padding:8px 10px;" data-login-prof="${p.id}">Entrar</button>
      </div>
    `).join('')}
    </div>
  </div>
  `, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('[data-set-role="dono"]').addEventListener('click', ()=>{
      userSession = {role:'dono', professionalId:null};
      closeModal(); render();
    });
    root.querySelectorAll('[data-login-prof]').forEach(b=>b.addEventListener('click', ()=>{
      const profId = b.dataset.loginProf;
      const prof = state.professionals.find(p=>p.id===profId);
      const pin = root.querySelector(`[data-pin-for="${profId}"]`).value;
      if(pin !== prof.pin){ showToast('PIN incorreto.');
```

```

        route = 'dashboard';
        closeModal(); render();
        showToast(`Logado como ${prof.name}.`);
    }));
});
}

```

```

function renderView(){
    const view = document.getElementById('view');
    // Trava de segurança: se a rota atual não é permitida para este perfil,
    // volta pro painel (evita acesso direto a telas restritas).
    if(!navAllowedForSession().includes(route)){
        route = 'dashboard';
        return view.innerHTML = viewDashboard();
    }
    if(route==='dashboard') return view.innerHTML = viewDashboard();
    if(route==='agenda') return renderAgenda(view);
    if(route==='caixa') return renderCaixa(view);
    if(route==='comissoes') return renderComissoes(view);
    if(route==='profissionais') return renderProfissionais(view);
    if(route==='servicos') return renderServicos(view);
    if(route==='clubes') return renderClube(view);
    if(route==='config') return renderConfig(view);
    if(route==='minha-assinatura') return renderMinhaAssinatura(view);
    if(route==='meus-links') return renderMeusLinks(view);
    if(route==='clientes') return renderClientes(view);
    if(route==='estoque') return renderEstoque(view);
    if(route==='relatorios') return renderRelatorios(view);
    if(route==='central') return renderCentral(view);
    if(route==='fidelidade') return renderFidelidade(view);
    if(route==='agendamento-online') return renderOnlineBooking(view);
    if(route==='mensagens') return renderMensagens(view);
    if(route==='espera') return renderWaitlist(view);
    if(route==='notafiscal') return renderNotaFiscal(view);
    if(route==='ajuda') return renderAjuda(view);
    if(route==='configuracoes') return renderConfiguracoes(view);
}

```

```

/* ===== HEADER PARTIAL ===== */

```

```

function pageHeader(title, subtitle, actionHtml='') {
    return `
        <div style="display:flex;justify-content:space-between;align-items:flex-
end;flex-wrap:wrap;gap:14px;margin-bottom:22px;">
            <div>
                <div class="font-display" style="font-size:28px;font-weight:600;">${title}
            </div>
                <div style="color:var(--muted);font-size:14px;margin-top:3px;">${subtitle}

```

```

</div>
    </div>
    <div>${actionHtml}</div>
</div>
`;
}

/* ===== DASHBOARD ===== */
function viewDashboard(){
  const isProfView = userSession.role==='profissional';
  const myId = userSession.professionalId;
  const session = openSession();
  const totals = sessionTotals(session);
  // O profissional vê apenas os próprios atendimentos e a própria comissão
  const appts = isProfView
    ? todaysAppointments().filter(a=>a.professionalId===myId)
    : todaysAppointments();
  const done = appts.filter(a=>a.status==='concluido');
  // Faturamento real: usa o valor cobrado (já com desconto e clube aplicados),
  // não o preço de tabela. Antes um assinante que pagou R$ 0 entrava como R$ 35.
  const faturamentoServicos = done.reduce((a,ap)=>a+valorRealAtendimento(ap),0);
  const atendimentosClube = done.filter(ap=>valorRealAtendimento(ap) <= 0).length;
  const commissionsPending = isProfView
    ? commissionOwed(myId).owed
    : state.professionals.reduce((a,p)=>a+commissionOwed(p.id).owed,0);
  const activeMembers =
state.subscriptionMembers.filter(m=>m.status==='pago').length;
  const overdueMembers = assinantesEmAtraso().length;
  const mrr =
state.subscriptionMembers.filter(m=>m.status!=='cancelado').reduce((a,m)=>{
    const plan = state.subscriptionPlans.find(p=>p.id===m.planId); return a+(plan?
plan.price:0);
  },0);

  return `
    ${pageHeader('Painel', new Date().toLocaleDateString('pt-BR',{weekday:'long',
day:'2-digit', month:'long'}))}
    <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(210px,1fr));gap:14px;margin-bottom:22px;">
      ${statCard(
        isProfView?'Seu faturamento hoje':'Faturamento em serviços hoje',
        BRL(faturamentoServicos),
        `${done.length} atendimento(s)${atendimentosClube>0?`
${atendimentosClube} pelo clube (não entram no valor)`:''}`,
        'ok')}
      ${!isProfView ? statCard('Saldo do caixa', session?BRL(totals.balance):'-',
session?`Entradas ${BRL(totals.in)} · Saídas ${BRL(totals.out)}`: 'Nenhum caixa

```

```

aberto', session?'ok':'bad') : ''}
    ${statCard(isProfView?'Você tem a receber':'Comissões pendentes',
BRL(commissionsPending), isProfView?'em comissões':'a pagar aos profissionais',
commissionsPending>0?'warn':'ok')}
    ${!isProfView ? statCard('Clube de assinatura', activeMembers+' membros
ativos', 'MRR '+BRL(mrr), 'ok') : ''}
    ${!isProfView ? statCard('Assinantes em atraso', overdueMembers,
overdueMembers>0?'precisam de cobrança':'nenhum em atraso',
overdueMembers>0?'bad':'ok') : ''}
</div>

<div style="display:grid;grid-template-columns:${isProfView?'1fr':'1.4fr
1fr'};gap:16px;align-items:start;" class="dash-grid">
  <div class="card" style="padding:20px;">
    <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:14px;">
      <div class="font-display" style="font-size:17px;font-weight:600;">Agenda
de hoje</div>
      <span class="pill pill-muted">${appts.length} horários</span>
    </div>
    ${appts.length===0 ? emptyState('Nenhum agendamento para hoje.') : `
      <div style="display:flex;flex-direction:column;gap:8px;">
        ${appts.slice(0,6).map(a=>agendaRow(a)).join('')}
      </div>`}
    </div>
    ${!isProfView ? `
    <div class="card" style="padding:20px;">
      <div class="font-display" style="font-size:17px;font-weight:600;margin-
bottom:14px;">Comissões por profissional</div>
      <div style="display:flex;flex-direction:column;gap:10px;">
        ${servingProfessionals().map(p=>{
          const c = commissionOwed(p.id);
          return `<div style="display:flex;justify-content:space-between;align-
items:center;font-size:13.5px;">
            <span style="color:var(--ink);">${esc(p.name)}</span>
            <span class="font-mono" style="color:${c.owed<0?'var(--bad)':'
(c.owed>0?'var(--warn)':'var(--muted-2)')};">${BRL(c.owed)}${c.owed<0?'
(deve)':''}</span>
          </div>`;
        }).join('')}
      </div>
    </div>`} : ''}
  </div>

  ${overdueMembers>0 && !isProfView ? `
  <div class="card" style="padding:20px;margin-top:16px;border-color:var(--
bad);">

```

```

    <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:14px;">
      <div class="font-display" style="font-size:17px;font-weight:600;">${ICON.bell} Assinantes em atraso</div>
      <span class="pill pill-bad">${overdueMembers}</span>
    </div>
    <div style="display:flex;flex-direction:column;gap:8px;">
      ${assinantesEmAtraso().map(m=>{
        const plan = state.subscriptionPlans.find(p=>p.id===m.planId);
        const text = `Olá ${m.name}! Notamos que sua mensalidade do ${plan?plan.name:'clube'} está em atraso. Pode regularizar quando puder? Qualquer dúvida estamos à disposição.`;
        return `<div style="display:flex;justify-content:space-between;align-items:center;gap:10px;padding:8px 0;border-bottom:1px solid var(--border);">
          <div style="font-size:13.5px;">
            <b>${esc(m.name)}</b> <span style="color:var(--muted-2);font-size:11.5px;">— ${esc(plan?plan.name:'plano removido')} · venceu em ${m.dueDate?new Date(m.dueDate+'T00:00').toLocaleDateString('pt-BR')}:'-'}</span>
          </div>
          ${messageChannelRow(m.phone, text)}
        </div>`;
      })}.join('')}
    </div>` : ''}
  `;
}
function statCard(label, value, sub, tone){
  const color = tone==='ok'?`var(--ok)`:tone==='bad'?`var(--bad)`:`var(--warn)`;
  return `<div class="card" style="padding:16px 18px;">
    <div style="font-size:12px;color:var(--muted);margin-bottom:8px;">${label}</div>
    <div class="font-mono" style="font-size:22px;font-weight:600;color:var(--ink);">${value}</div>
    <div style="font-size:12px;margin-top:6px;color:${color};">${sub}</div>
  </div>`;
}
function emptyState(msg, hint=''){
  return `<div style="text-align:center;padding:34px 10px;color:var(--muted);">
    <div style="font-size:14px;">${msg}</div>
    ${hint?`<div style="font-size:12.5px;color:var(--muted-2);margin-top:5px;">${hint}</div>`:''}
  </div>`;
}
function agendaRow(a){
  const nomeServ = appointmentServiceNames(a);
  const statusPill = a.status==='concluido' ? `<span class="pill pill-ok">Concluído</span>`

```

```

    : a.status==='cancelado' ? '<span class="pill pill-bad">Cancelado</span>'
    : '<span class="pill pill-warn">Agendado</span>';
    return `<div style="display:flex;align-items:center;gap:12px;padding:9px
10px;border-radius:9px;background:var(--panel-2);">
    <div class="font-mono" style="font-size:13px;color:var(--gold-bright);min-
width:44px;">${a.time}</div>
    <div style="flex:1;min-width:0;">
    <div style="font-size:13.5px;font-weight:500;overflow:hidden;text-
overflow:ellipsis;white-space:nowrap;">${esc(a.client)} ${clubStarBadge(a.client)}
<span style="color:var(--muted-2);">— ${esc(nomeServ)}</span></div>
    <div style="font-size:12px;color:var(--
muted);">${esc(profName(a.professionalId))}</div>
    </div>
    ${statusPill}
</div>`;
}

/* ===== AGENDA ===== */
/* Agenda em grade: horas na lateral, profissionais em colunas — como um
quadro de horários de verdade. */
function agendaGridView(dateISO, profList){
    const appts = state.appointments.filter(a=>a.date===dateISO &&
a.status!=='cancelado');
    // Faixa de horários: cobre a jornada de todos os profissionais do dia
    let minH = 23, maxH = 1;
    profList.forEach(p=>{
        const [sh] = (p.workStart||'09:00').split(':').map(Number);
        const [eh] = (p.workEnd||'19:00').split(':').map(Number);
        minH = Math.min(minH, sh); maxH = Math.max(maxH, eh);
    });
    appts.forEach(a=>{
        const h = parseInt(a.time.split(':')[0]);
        minH = Math.min(minH, h); maxH = Math.max(maxH, h+1);
    });
    if(minH>maxH){ minH = 8; maxH = 20; }
    // guarda os limites do expediente antes de esticar
    const expedienteIni = minH, expedienteFim = maxH;
    if(agendaEstendida){ minH = Math.max(0, minH-3); maxH = Math.min(23, maxH+3); }
    const hours = [];
    for(let h=minH; h<=maxH; h++) hours.push(h);
    const foraDoExpediente = (h)=> h < expedienteIni || h >= expedienteFim;
    const SLOT_H = 56; // altura de 1 hora, em pixels

    // desenha os bloqueios (folga, almoço) como faixas hachuradas
    const blocksFor = (prof)=>{
        return blocksForDay(prof.id, dateISO).map(b=>{
            const [sh,sm] = b.start.split(':').map(Number);

```

```

    const [eh,em] = b.end.split(':').map(Number);
    const top = ((sh - minH) + sm/60) * SLOT_H;
    const height = Math.max(20, ((eh + em/60) - (sh + sm/60)) * SLOT_H - 2);
    return `
```

```

próprio cliente, pelo link">${ICON.globe}</span>`:'</div>
    <div style="font-size:12px;font-weight:600;white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;">${esc(a.client)}
${clubStarBadge(a.client)}</div>
    ${height>44 && svcs.length?`<div style="font-size:10.5px;color:var(--
muted);white-space:nowrap;overflow:hidden;text-
overflow:ellipsis;">${esc(nomesServicos)}</div>`:'</div>`;
    }).join('');
};

return `
    <div class="card" style="padding:0;overflow-x:auto;margin-bottom:20px;">
        <div style="min-width:${120 + profList.length*170}px;">
            <div style="display:flex;border-bottom:1px solid var(--
border);position:sticky;top:0;background:var(--panel);z-index:2;">
                <div style="width:60px;flex-shrink:0;"></div>
                ${profList.map(p=>`
                    <div style="flex:1;min-width:170px;padding:12px
10px;display:flex;align-items:center;gap:9px;border-left:1px solid var(--
border);">
                        <div style="width:34px;height:34px;border-
radius:50%;background:var(--panel-2);border:1px solid var(--
border);display:flex;align-items:center;justify-
content:center;overflow:hidden;flex-shrink:0;color:var(--muted-2);">
                            ${p.photoDataUrl ? `` : ICON.team}
                        </div>
                        <div style="min-width:0;">
                            <div style="font-size:13px;font-weight:600;white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;">${esc(p.name)}</div>
                            <div style="font-size:10.5px;color:var(--muted-
2);">${isProfessionalWorking(p, dateISO)?`${p.workStart}-${p.workEnd}`:'Folga'}
                        </div>
                    </div>
                </div>
            </div>
            <div style="display:flex;position:relative;">
                <div style="width:60px;flex-shrink:0;">
                    ${hours.map(h=>`<div style="height:${SLOT_H}px;border-bottom:1px solid
var(--border);padding:4px 8px;${foraDoExpediente(h)?'opacity:.5':'</div>`
                    <span style="font-size:11px;color:var(--muted-
2);">${String(h).padStart(2,'0')}:00</span>
                </div>`}).join('')}
            </div>
        </div>
    </div>
`

```

```

    ${profList.map(p=>`
      <div style="flex:1;min-width:170px;position:relative;border-left:1px
solid var(--border);background:${isProfessionalWorking(p,
dateISO)?'transparent':'repeating-linear-gradient(45deg, transparent, transparent
8px, var(--panel-2) 8px, var(--panel-2) 16px)'};">
        ${hours.map(h=>`
          <div style="height:${SLOT_H}px;border-bottom:1px solid var(--
border);display:flex;flex-direction:column;${foraDoExpediente(h)?'background:var(--
-panel-2);opacity:.55;':''}">
            <div class="grid-slot" data-prof="${p.id}" data-
time="${String(h).padStart(2,'0')}:00" ${foraDoExpediente(h)?'data-fora="1"':''}"
style="flex:1;cursor:pointer;"></div>
            <div class="grid-slot" data-prof="${p.id}" data-
time="${String(h).padStart(2,'0')}:30" ${foraDoExpediente(h)?'data-fora="1"':''}"
style="flex:1;cursor:pointer;border-top:1px dashed var(--border);"></div>
          </div>
        `).join('')}
        ${blocksFor(p)}
        ${colFor(p)}
      </div>
    `).join('')}
  </div>
</div>
</div>
`;
}

```

```

function weeklyByProfessionalTable() {
  const days = weekDates();
  const dayLabels = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb', 'Dom'];
  const todayStr = todayISO();
  return `
    <div class="card" style="padding:18px 20px;margin-bottom:22px;overflow-
x:auto;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:14px;flex-wrap:wrap;gap:8px;">
        <div class="font-display" style="font-size:16px;font-weight:600;">Clientes
por profiissional na semana</div>
        <div style="font-size:12px;color:var(--muted-2);">${new
Date(days[0]+'T00:00').toLocaleDateString('pt-BR')} - ${new
Date(days[6]+'T00:00').toLocaleDateString('pt-BR')}</div>
      </div>
      <table style="min-width:620px;">
        <thead>
          <tr>
            <th style="min-width:150px;">Profiissional</th>
            ${days.map((d,i)=>`<th style="text-

```

```

align:center;${d===todayStr?'color:var(--gold-bright);':''}">${dayLabels[i]}<div
style="font-weight:400;font-size:10.5px;color:var(--muted-2);">${new
Date(d+'T00:00').toLocaleDateString('pt-BR',{day:'2-digit',month:'2-digit'})}
</div></th>`).join('')
    <th style="text-align:center;">Total</th>
  </tr>
</thead>
<tbody>
    ${state.professionals.length===0 ? `<tr><td colspan="${days.length+2}"
style="text-align:center;color:var(--muted);padding:20px;">Nenhum profissional
cadastrado.</td></tr>` :
    servingProfessionals().map(p=>{
      const counts = days.map(d=>clientsPerProfPerDay(p.id, d));
      const total = counts.reduce((a,c)=>a+c,0);
      return `<tr>
        <td style="font-weight:500;">${esc(p.name)}</td>
        ${counts.map((c,i)=>`<td style="text-
align:center;${days[i]===todayStr?'background:rgba(201,162,75,.08);':''}">
          ${c>0?`<span class="pill pill-warn font-mono">${c}</span>`:`<span
style="color:var(--muted-2);">-</span>`}
        </td>`).join('')}
        <td style="text-align:center;font-weight:600;" class="font-
mono">${total}</td>
      </tr>`;
    }).join('')}
  </tbody>
  ${state.professionals.length>0?`<tfoot><tr>
    <td style="font-weight:600;color:var(--muted);">Total do dia</td>
    ${days.map(d=>{
      const t =
state.professionals.reduce((a,p)=>a+clientsPerProfPerDay(p.id,d),0);
      return `<td style="text-align:center;font-
weight:600;${d===todayStr?'color:var(--gold-bright);':'color:var(--muted);'}`
class="font-mono">${t}</td>`;
    }).join('')}
    <td style="text-align:center;font-weight:700;" class="font-
mono">${state.professionals.reduce((a,p)=>a+days.reduce((b,d)=>b+clientsPerProfPerI
</td>
  </tr></tfoot>`:''}
</table>
</div>
`;
}
function depositPill(a){
  if(a.depositStatus==='pago') return `<span class="pill pill-ok">${ICON.timer}
Sinal pago</span>`;
  if(a.depositStatus==='pendente') return `<span class="pill pill-

```

```

warn">${ICON.timer} Sinal pendente</span>`;
    return '';
}
function renderAgenda(view){
    autoExpireUnpaidDeposits();
    const isProfView = userSession.role==='profissional';
    const myId = userSession.professionalId;
    if(!agendaDate) agendaDate = todayISO();
    const viewDate = agendaDate;
    const isToday = viewDate === todayISO();
    let appts =
state.appointments.filter(a=>a.date===viewDate).sort((a,b)=>a.time.localeCompare(b.
    if(isProfView) appts = appts.filter(a=>a.professionalId===myId);

    // Dias da semana em que a data selecionada está
    const sel = new Date(viewDate+'T12:00');
    const dow = (sel.getDay()+6)%7; // 0 = segunda
    const monday = new Date(sel); monday.setDate(sel.getDate()-dow);
    const weekDays = Array.from({length:7}, (_,i)=>{
        const d = new Date(monday); d.setDate(monday.getDate()+i);
        return d.toISOString().slice(0,10);
    });
    const shiftWeek = (delta)=>{
        const d = new Date(viewDate+'T12:00'); d.setDate(d.getDate()+delta*7);
        return d.toISOString().slice(0,10);
    };
    const dayLabel = new Date(viewDate+'T12:00').toLocaleDateString('pt-BR',
{weekday:'long', day:'2-digit', month:'long'});
    const doneCount = appts.filter(a=>a.status==='concluido').length;
    const revenue =
appts.filter(a=>a.status==='concluido').reduce((t,a)=>t+valorRealAtendimento(a),0);

    view.innerHTML = `
        ${pageHeader('Agenda', dayLabel.charAt(0).toUpperCase()+dayLabel.slice(1), `
        <div style="display:flex;gap:8px;flex-wrap:wrap;">
            <button class="btn btn-ghost" id="btn-new-block">${ICON.lock} Bloquear
horário</button>
            <button class="btn btn-gold" id="btn-new-appt">${ICON.plus} Novo
agendamento</button>
        </div>
        `)}

        <div class="card" style="padding:14px 16px;margin-bottom:18px;">
            <div style="display:flex;align-items:center;gap:10px;margin-
bottom:12px;flex-wrap:wrap;">
                <button class="btn btn-ghost" id="wk-prev" style="padding:7px 11px;"><
Semana anterior</button>

```

```

        <button class="btn btn-ghost" id="wk-today" style="padding:7px
11px;${isToday?'border-color:var(--gold);color:var(--gold-
bright);':''}">Hoje</button>
        <button class="btn btn-ghost" id="wk-next" style="padding:7px
11px;">Próxima semana </button>
        <div style="flex:1;"></div>
        <input type="date" id="wk-picker" value="${viewDate}" style="padding:7px
10px;">
    </div>
    <div style="display:grid;grid-template-columns:repeat(7,1fr);gap:6px;">
        ${weekDays.map(d=>{
            const dd = new Date(d+'T12:00');
            const count = state.appointments.filter(a=>a.date===d && (!isProfView ||
a.professionalId===myId) && a.status!=='cancelado')).length;
            const selected = d===viewDate;
            const today = d===todayISO();
            return `<button class="btn btn-ghost wk-day" data-day="${d}"
style="flex-direction:column;gap:2px;padding:9px 4px;${selected?'background:var(--
gold);color:#1a1610;border-color:var(--gold);':today?'border-color:var(--
gold);':''}">
                <span style="font-size:10px;text-
transform:uppercase;opacity:.75;">${(['seg','ter','qua','qui','sex','sáb','dom']
[(dd.getDay()+6)%7])}</span>
                <span style="font-size:15px;font-
weight:600;">${String(dd.getDate()).padStart(2,'0')}</span>
                <span style="font-size:9.5px;opacity:.75;">${count>0?count+
(count===1?' hor.': ' hor.'): '-'}</span>
            </button>`;
        })).join('')}
    </div>
</div>

    ${appts.length>0 ? `
        <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(150px,1fr));gap:12px;margin-bottom:18px;">
            ${statCard('Agendamentos', appts.length, 'neste dia', 'ok')}
            ${statCard('Concluídos', doneCount, doneCount===appts.length?'todos
atendidos':'de '+appts.length, doneCount>0?'ok':'warn')}
            ${statCard('Faturamento', BRL(revenue), 'atendimentos concluídos', 'ok')}
        </div>
    ` : ''}

    ${state.professionals.length===0 ? emptyState('Cadastre profissionais antes de
agendar.') : `
        <div style="display:flex;align-items:center;gap:8px;margin-bottom:14px;">
            <div style="display:flex;border:1px solid var(--border);border-
radius:9px;overflow:hidden;">

```

```

<button class="agenda-mode" data-mode="grade" style="padding:7px
14px;font-
size:12.5px;border:none;cursor:pointer;background:${agendaMode==='grade'?`var(--
gold)`: 'transparent'};color:${agendaMode==='grade'?`#1a1610`: 'var(--
muted)'}; ">${ICON.clock} Grade</button>
<button class="agenda-mode" data-mode="lista" style="padding:7px
14px;font-
size:12.5px;border:none;cursor:pointer;background:${agendaMode==='lista'?`var(--
gold)`: 'transparent'};color:${agendaMode==='lista'?`#1a1610`: 'var(--
muted)'}; ">${ICON.listPlus} Lista</button>
${!isProfView?`<button class="agenda-mode" data-mode="semana"
style="padding:7px 14px;font-
size:12.5px;border:none;cursor:pointer;background:${agendaMode==='semana'?`var(--
gold)`: 'transparent'};color:${agendaMode==='semana'?`#1a1610`: 'var(--
muted)'}; ">${ICON.chart} Semana</button>`: ''}
</div>
${agendaMode==='grade'?`
<button class="btn btn-ghost" id="btn-estender" style="padding:7px
13px;font-size:12.5px;${agendaEstendida?'border-color:var(--gold);color:var(--
gold-bright)`: ''} ">
${agendaEstendida?'Ocultar horários extras':`+ Horários fora do
expediente`}
</button>
`: ''}
</div>
`}

${state.professionals.length>0 && agendaMode==='semana' && !isProfView ?
weeklyByProfessionalTable() : ''}

${state.professionals.length>0 && agendaMode==='grade'
? agendaGridView(viewDate, isProfView ?
state.professionals.filter(p=>p.id===myId) : servingProfessionals())
: ''}

${agendaMode==='lista' ? (appts.length===0 ? emptyState(isToday?'Nenhum
horário para hoje.': 'Nenhum horário neste dia.', isToday?'Toque em "Novo
agendamento" para começar.': 'Selecione outro dia no calendário acima.') : `
<div style="display:flex;flex-direction:column;gap:10px;">
${appts.map(a=>{
const svcsList = appointmentServices(a);
const nomeServ = appointmentServiceNames(a);
const durTotal = appointmentDuration(a);
return `<div class="card" style="padding:14px 16px;display:flex;align-
items:center;gap:16px;flex-wrap:wrap;">
<div class="font-mono" style="font-size:15px;color:var(--gold-
bright);min-width:50px;">${a.time}</div>

```

```

<div style="flex:1;min-width:180px;">
    <div style="font-weight:600;font-size:14.5px;">${esc(a.client)}
    ${clubStarBadge(a.client)} ${a.origem==='online'?`<span class="pill"
    style="background:rgba(74,159,216,.15);color:#4a9fd8;border-
    color:rgba(74,159,216,.35);font-size:10px;">${ICON.globe} agendou pelo
    link</span>`:''} ${a.confirmed?`<span style="color:var(--ok);font-size:11px;">✓
    confirmado</span>`:''}</div>
    <div style="font-size:12.5px;color:var(--muted);margin-
    top:2px;">${svcsList.length?esc(nomeServ):'Serviço removido'} ·
    ${esc(profName(a.professionalId))} ${svcsList.length?`· ${ICON.clock} ${durTotal}
    min`:''}</div>
    ${a.rating ? `<div style="font-size:12px;color:var(--gold-
    bright);margin-top:3px;">${ICON.star} ${a.rating.stars}/5 ${a.rating.comment?'-
    '+esc(a.rating.comment):''}</div>` : ''}
    ${clubMemberForClient(a.client)?.status==='atrasado' ? `<div
    style="margin-top:6px;display:flex;align-items:center;gap:8px;flex-wrap:wrap;">
    <span class="pill pill-bad" style="font-size:11px;">Clube em atraso – cobrança em
    valor cheio</span>${clubOverdueWaButton(a.client)}</div>` : ''}
    </div>
    ${depositPill(a)}
    <div class="font-mono" style="font-size:14px;color:var(--muted);">${s?
    BRL(s.price):'-'}</div>
    <div style="display:flex;gap:6px;align-items:center;flex-wrap:wrap;">
    ${a.status==='agendado' && a.depositStatus==='pendente' ? `<button
    class="btn btn-ghost" style="padding:7px 10px;color:var(--ok);border-color:var(--
    ok);" data-mark-deposit="${a.id}">Marcar sinal pago</button>` : ''}
    ${a.status==='agendado'?`<button class="btn btn-ghost"
    style="padding:7px 10px;color:var(--ok);border-color:var(--ok);" data-
    conclude="${a.id}">${ICON.check} Concluir</button>`:''}
    ${a.status==='concluido' && !a.rating ? `<button class="btn btn-
    ghost" style="padding:7px 10px;" data-request-review="${a.id}">${ICON.star} Pedir
    avaliação</button>` : ''}
    ${a.status==='concluido'?`<span class="pill pill-
    ok">Concluído</span>`:''}
    ${a.status==='cancelado'?`<span class="pill pill-
    bad">Cancelado</span>`:''}
    ${a.status==='faltou'?`<span class="pill pill-
    bad">Faltou</span>`:''}
    ${a.status==='agendado'?`<button class="btn btn-ghost"
    style="padding:7px 9px;color:var(--bad);border-color:var(--bad);" data-
    noshow="${a.id}" title="Cliente faltou">${ICON.bell}</button>`:''}
    ${a.status!=='concluido' && a.status!=='cancelado'?`<button
    class="btn btn-ghost" style="padding:7px 9px;" data-resched="${a.id}"
    title="Reagendar">${ICON.clock}</button>`:''}
    ${a.status==='concluido' && isOwnerLevel()?`<button class="btn btn-
    ghost" style="padding:7px 9px;color:var(--warn);border-color:var(--warn);" data-
    reabrir="${a.id}" title="Reabrir comanda">${ICON.clock}</button>`:''}

```

```

        ${a.status=== 'agendado'?`<button class="btn btn-ghost"
style="padding:7px 9px;color:var(--bad);border-color:var(--bad);" data-
cancel="${a.id}" title="Cancelar">${ICON.x}</button>`:''}
        <button class="btn btn-ghost" style="padding:7px 9px;" data-del-
appt="${a.id}" title="Excluir">${ICON.trash}</button>
    </div>
</div>`;
    }).join('')
</div>
`) : ''
`;
view.querySelector('#btn-new-appt')?.addEventListener('click', openApptModal);
view.querySelector('#btn-new-block')?.addEventListener('click',
()=>blockModal({date:viewDate}, view));
view.querySelectorAll('.btn-del-block').forEach(b=>b.addEventListener('click',
(e)>{
    e.stopPropagation();
    const bloco = (state.timeBlocks||[]).find(x=>x.id===b.dataset.block);
    if(!bloco) return;
    const msg = bloco.repeteSemanal
        ? 'Remover este bloqueio de TODAS as semanas?'
        : 'Remover este bloqueio?';
    if(!confirm(msg)) return;
    update(s=>{ s.timeBlocks = s.timeBlocks.filter(x=>x.id!==b.dataset.block); });
    showToast('Bloqueio removido.');
```

```

    view.querySelector('#wk-prev')?.addEventListener('click', ()=>{ agendaDate =
shiftWeek(-1); renderAgenda(view); });
    view.querySelector('#wk-next')?.addEventListener('click', ()=>{ agendaDate =
shiftWeek(1); renderAgenda(view); });
    view.querySelector('#wk-today')?.addEventListener('click', ()=>{ agendaDate =
todayISO(); renderAgenda(view); });
    view.querySelector('#wk-picker')?.addEventListener('change', e=>{
if(e.target.value){ agendaDate = e.target.value; renderAgenda(view); } });
    view.querySelectorAll('[data-conclude]').forEach(b=>b.addEventListener('click',
()=>concludeAppt(b.dataset.conclude)));
    view.querySelectorAll('[data-mark-
deposit]').forEach(b=>b.addEventListener('click', ()=>{
    update(s=>{
s.appointments.find(a=>a.id===b.dataset.markDeposit).depositStatus='pago'; });
    showToast('Sinal marcado como pago.');
    }));
    view.querySelectorAll('[data-resched]').forEach(b=>b.addEventListener('click',
()=>reagendarModal(b.dataset.resched, view)));
    view.querySelectorAll('[data-reabrir]').forEach(b=>b.addEventListener('click',
()=>reabrirComanda(b.dataset.reabrir, view)));
    view.querySelectorAll('[data-cancel]').forEach(b=>b.addEventListener('click',
()=>cancelAppointment(b.dataset.cancel, 'cancelado')));
    view.querySelectorAll('[data-noshow]').forEach(b=>b.addEventListener('click',
()=>cancelAppointment(b.dataset.noshow, 'faltou')));
    view.querySelectorAll('[data-del-appt]').forEach(b=>b.addEventListener('click',
()=>{
    update(s=>{ s.appointments =
s.appointments.filter(a=>a.id!==b.dataset.delAppt); });
    }));
    view.querySelectorAll('[data-request-
review]').forEach(b=>b.addEventListener('click',
()=>requestReviewModal(b.dataset.requestReview)));
}
function autoExpireUnpaidDeposits(){
    const expired = expiredDepositAppointments();
    if(expired.length===0) return;
    update(s=>{
        expired.forEach(a=>{
            const appt = s.appointments.find(x=>x.id===a.id);
            if(appt){ appt.status='cancelado'; appt.depositStatus='pendente'; }
        });
    });
}
function cancelAppointment(id, newStatus){
    let freed = null;
    update(s=>{
        const appt = s.appointments.find(a=>a.id===id);

```

```

    appt.status = newStatus;
    freed = {date:appt.date, time:appt.time, professionalId:appt.professionalId};
  });
  const match = tryFillFromWaitlist(freed.date, freed.time, freed.professionalId);
  if(match){
    showToast(`${newStatus==='faltou'? 'Falta registrada': 'Cancelado'}`).
    `${match.clientName} está na lista de espera para esse horário – avise-o em Lista
    de Espera.`);
  }else{
    showToast(newStatus==='faltou' ? 'Falta registrada.' : 'Agendamento
    cancelado.');
```

```

  }
}
function requestReviewModal(apptId){
  const appt = state.appointments.find(a=>a.id===apptId);
  const phone = phoneForClientName(appt.client);
  const text = fillTemplate(state.messageTemplates.reviewRequest,
  {cliente:appt.client});
  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
      items:center;margin-bottom:16px;">
        <div class="font-display" style="font-size:19px;font-
        weight:600;">Avaliação do atendimento</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
      </button>
      </div>
      <div style="font-size:12.5px;color:var(--muted);margin-
      bottom:12px;">Registre a nota que o cliente deu (por telefone/balcão) ou peça por
      WhatsApp.</div>
      <div style="display:flex;gap:6px;margin-bottom:12px;" id="star-picker">
        ${[1,2,3,4,5].map(n=>`<button type="button" class="btn btn-ghost" data-
        star="${n}" style="padding:8px 12px;">${ICON.starOutline}</button>`).join('')}
      </div>
      <label>Comentário (opcional)</label>
      <input id="f-comment" style="width:100%;margin-bottom:14px;" placeholder="O
      que o cliente achou">
      <div style="display:flex;gap:8px;">
        <button class="btn btn-gold" id="f-submit-rating" style="flex:1;justify-
        content:center;">Salvar avaliação</button>
        ${phone ? `<a class="btn btn-ghost" href="${waLink(phone, text)}"
        target="_blank" rel="noopener">${ICON.send} Pedir por WhatsApp</a>` : ''}
      </div>
    </div>
  `, root=>{
    let stars = 0;
    root.querySelector('#m-close').addEventListener('click', closeModal);
  });
}

```

```

    root.querySelectorAll('[data-
star]').forEach(btn=>btn.addEventListener('click', ()=>{
        stars = parseInt(btn.dataset.star);
        root.querySelectorAll('[data-star]').forEach(b2=>{ b2.innerHTML =
parseInt(b2.dataset.star)<=stars ? ICON.star : ICON.starOutline; b2.style.color =
parseInt(b2.dataset.star)<=stars?'var(--gold-bright)':'var(--muted)'; });
    }));
    root.querySelector('#f-submit-rating').addEventListener('click', ()=>{
        if(stars===0){ showToast('Selecione uma nota.');
```

```

        sessaoDoLancamento = s; lancamento = e;
    }
    });
});
}

const caixaFechado = sessaoDoLancamento && sessaoDoLancamento.status ===
'fechado';
const valor = lancamento ? Number(lancamento.amount) : 0;

openModal(`
    <div style="padding:22px;">
        <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
            <div class="font-display" style="font-size:19px;font-weight:600;">Reabrir
comanda</div>
            <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
        </div>
        <div style="font-size:13px;color:var(--muted);margin-bottom:16px;">
            ${esc(a.client)} · ${esc(appointmentServiceNames(a))} · ${new
Date(a.date+'T12:00').toLocaleDateString('pt-BR')} às ${a.time}
        </div>

        <div class="card" style="padding:14px 16px;background:var(--panel-2);margin-
bottom:16px;">
            <div style="font-size:12px;color:var(--muted-2);margin-bottom:5px;">O que
foi lançado no caixa</div>
            ${lancamento
                ? `<div class="font-mono" style="font-size:19px;color:var(--gold-
bright);">${BRL(valor)}</div>
                    <div style="font-size:11.5px;color:var(--muted-2);margin-top:3px;">
                        ${methodLabelPay(lancamento.method)} · caixa de ${new
Date((sessaoDoLancamento.date)+'T12:00').toLocaleDateString('pt-BR')}
                        ${caixaFechado?' <span style="color:var(--warn);">(já fechado)
</span>':''}
                    </div>`
                : `<div style="font-size:12.5px;color:var(--muted);">Nenhum lançamento
encontrado no caixa para este atendimento.</div>`}
            </div>

            ${caixaFechado ? `
                <div style="padding:12px 14px;border-radius:9px;background:var(--panel-
2);border:1px solid var(--warn);font-size:12px;color:var(--muted);line-
height:1.6;margin-bottom:16px;">
                    <b style="color:var(--warn);">O caixa desse dia já foi fechado.</b><br>
                    Não dá para apagar um lançamento de caixa fechado sem bagunçar a

```

conferência daquele dia.

Em vez disso, vou registrar uma saída de estorno de \${BRL(valor)} no caixa aberto de hoje.

</div>

` : ''}

<div style="font-size:12.5px;color:var(--muted);line-height:1.7;margin-bottom:18px;">

Ao reabrir:

- O atendimento volta para "agendado"

- O valor sai da comissão do profissional

- \${caixaFechado ? 'Um estorno é lançado no caixa de hoje' : lancamento ?

'O lançamento sai do caixa' : 'Nada muda no caixa'}

</div>

<button class="btn btn-gold" id="rc-confirmar" style="width:100%;justify-content:center;padding:13px;">Reabrir comanda</button>

<button class="btn btn-ghost" id="rc-cancelar" style="width:100%;justify-content:center;margin-top:9px;font-size:12.5px;">Deixar como está</button>

</div>

`, root=>{

const fechar = ()=>closeModal();

root.querySelector('#m-close').addEventListener('click', fechar);

root.querySelector('#rc-cancelar').addEventListener('click', fechar);

root.querySelector('#rc-confirmar').addEventListener('click', ()=>{

const aberto = openSession();

if(caixaFechado && valor>0 && !aberto){

showToast('Abra o caixa de hoje para registrar o estorno.');

return;

}

update(s=>{

const ap = s.appointments.find(x=>x.id===apptId);

ap.status = 'agendado';

ap.paymentMethod = null;

ap.reabertoEm = new Date().toISOString();

if(lancamento){

const sess = s.cashSessions.find(x=>x.id===sessaoDoLancamento.id);

if(sess && sess.status==='aberto'){

sess.entries = sess.entries.filter(e=>e.id!==lancamento.id);

} else if(sess){

// caixa fechado: não mexe no passado, lança o estorno hoje

const hoje = s.cashSessions.find(x=>x.status==='aberto');

if(hoje){

hoje.entries.push({

id:uid(), type:'saida', method:lancamento.method || 'dinheiro',

estorno:true, apptId,

```

        desc:`Estorno - ${appointmentServiceNames(ap)} de ${ap.client}
(comanda reaberta)` ,
        amount: valor, time:new Date().toString().slice(0,5),
auto:true,
    });
    }
    }
    });
    closeModal();
    showToast(caixaFechado && valor>0
        ? `Comanda reaberta. Estorno de ${BRL(valor)} lançado no caixa de hoje.`
        : 'Comanda reaberta. Agora é só corrigir ou cancelar o atendimento.');
```

```

function precoParaCobrar(appt, service){
    const svc = service || serviceById(appt.serviceId);
    const out = {
        member: null, basePrice: svc ? Number(svc.price) : 0,
        priceNote: '', motivoCheio: '', planoAtivo: false, plano: null,
    };
    if(!svc) return out;

    // Um cliente pode ter mais de uma assinatura ao mesmo tempo - por
    // exemplo, cabelo de segunda a sábado e barba só de segunda a quinta,
    // pra economizar. Cada atendimento precisa achar o plano CERTO pra
    // aquele serviço, não sempre o primeiro assinante encontrado - senão
    // um plano acaba sendo aplicado sobre um serviço que não é dele.
    const membros = clubMembersForClient(appt.client);
    if(membros.length === 0) return out;

    let melhor = null; // o plano que cobre este serviço com o menor
preço
    let algumEmDiaSemCobrir = null; // usado só para explicar o motivo, se nada
cobrir

    membros.forEach(member=>{
        const plan = state.subscriptionPlans.find(p=>p.id===member.planId);
        const vencido = member.dueDate && String(member.dueDate) < todayISO();
        const emDia = member.status === 'pago' || (member.status === 'pendente' &&
!vencido);
        if(!emDia || !plan || !planValidOnDate(plan, appt.date)) return;

        let candidato = null;
```

```

        if(plan.includedServiceIds && plan.includedServiceIds.includes(svc.id)){
            candidato = { member, plan, basePrice: 0, priceNote: ' (incluso no clube)',
planoAtivo: true };
        } else if(plan.discountPercent > 0){
            candidato = { member, plan, basePrice: svc.price * (1 -
plan.discountPercent/100), priceNote: ` (${plan.discountPercent}% desconto
clube)`, planoAtivo: false };
        } else {
            if(!algumEmDiaSemCobrir) algumEmDiaSemCobrir = { member, plan };
            return;
        }
        if(!melhor || candidato.basePrice < melhor.basePrice) melhor = candidato;
    });

    if(melhor){
        out.member = melhor.member;
        out.plano = melhor.plan;
        out.basePrice = melhor.basePrice;
        out.priceNote = melhor.priceNote;
        out.planoAtivo = melhor.planoAtivo;
        return out;
    }

    // Nenhum plano em dia cobre este serviço – explica o motivo usando o
    // plano mais relevante pra essa mensagem (o que estava em dia mas não
    // incluía o serviço, ou senão o principal do cliente).
    const principal = algumEmDiaSemCobrir?.member ||
clubMemberForClient(appt.client);
    const planoPrincipal = algumEmDiaSemCobrir?.plan || (principal ?
state.subscriptionPlans.find(p=>p.id===principal.planId) : null);
    out.member = principal;
    out.plano = planoPrincipal;
    if(!principal) return out;

    const vencido = principal.dueDate && String(principal.dueDate) < todayISO();
    const emDia = principal.status === 'pago' || (principal.status === 'pendente' &&
!vencido);
    const maisDeUm = membros.length > 1;

    if(emDia){
        if(!planoPrincipal){
            out.motivoCheio = 'O plano deste assinante foi removido do sistema.';
        } else if(!planValidOnDate(planoPrincipal, appt.date)){
            out.priceNote = ' (fora dos dias do plano)';
            out.motivoCheio = `${planoPrincipal.name}:
${planDaysLabel(planoPrincipal.validDays).toLowerCase()}. Hoje não está
incluído.`;

```

```

    } else if(!planoPrincipal.includedServiceIds ||
planoPrincipal.includedServiceIds.length === 0){
        out.motivoCheio = `O plano "${planoPrincipal.name}" está sem serviços
inclusos. Edite o plano em Clube de Assinatura e marque quais serviços entram.`;
    } else {
        out.motivoCheio = maisDeUm
            ? `"${svc.name}" não está incluído em nenhum dos ${membros.length} planos
ativos deste cliente. Se deveria estar, edite o plano certo e marque este
serviço.`
            : `"${svc.name}" não está incluído no plano "${planoPrincipal.name}". Se
deveria estar, edite o plano e marque este serviço.`;
    }
} else if(principal.status === 'atrasado' || vencido){
    out.priceNote = ` (clube em atraso – valor cheio)`;
    out.motivoCheio = principal.dueDate
        ? `Mensalidade venceu em ${new
Date(String(principal.dueDate)+'T12:00').toLocaleDateString('pt-BR')} e não foi
paga – o plano não vale enquanto estiver em atraso.`
        : `Assinante com mensalidade atrasada – o plano não vale enquanto estiver em
atraso.`;
    } else {
        out.priceNote = ` (assinatura ${esc(principal.status)})`;
        out.motivoCheio = `Assinante com status "${principal.status}" – o plano não
está valendo. Verifique a situação dele no Clube de Assinatura.`;
    }
    return out;
}

```

```

/* Quando um atendimento chega na hora de finalizar sem um serviço válido
(ex: o profissional esqueceu de escolher, ou o serviço foi excluído
depois), pede pra escolher aqui mesmo, antes de seguir pro fechamento. */
function escolherServicoAntesDeConcluir(appt, id){
    openModal(`
        <div style="padding:22px;">
            <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
                <div class="font-display" style="font-size:19px;font-weight:600;">Falta
escolher o serviço</div>
                <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
            </div>
            <div style="font-size:12.5px;color:var(--muted);margin-
bottom:16px;">${esc(appt.client)} · ${new
Date(appt.date+'T12:00').toLocaleDateString('pt-BR')} às ${appt.time}</div>
            <label>Qual serviço foi feito?</label>
            <select id="esc-service" style="width:100%;margin-bottom:16px;">
                <option value="">– selecione –</option>

```

```

        ${state.services.map(s=><option value="${s.id}">${esc(s.name)} -
        ${BRL(s.price)}</option>`).join('')}
    </select>
    <button class="btn btn-gold" id="esc-save" style="width:100%;justify-
content:center;">Continuar para o fechamento</button>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#esc-save').addEventListener('click', ()=>{
        const serviceId = root.querySelector('#esc-service').value;
        if(!serviceId){ showToast('Escolha um serviço. '); return; }
        update(s=>{ const ap = s.appointments.find(a=>a.id===id); if(ap)
ap.serviceId = serviceId; });
        closeModal();
        concludeAppt(id);
    });
});
}

function concludeAppt(id){
    const session = openSession();
    if(!session){ showToast('Abra o caixa antes de concluir atendimentos. ');
route='caixa'; render(); return; }
    const appt = state.appointments.find(a=>a.id===id);
    const svcs = appointmentServices(appt);
    if(svcs.length===0){
        // Não deixa finalizar sem um serviço de verdade – é exatamente o que
        // causava atendimento sem ficha e sem valor certo de comissão.
        showToast('Este agendamento está sem serviço definido. Escolha o serviço
correto antes de finalizar. ');
        escolherServicoAntesDeConcluir(appt, id);
        return;
    }

    // Calcula o valor de CADA serviço considerando o clube – um pode estar
    // incluso no plano e outro não, então cada um é avaliado por si.
    const multi = precoParaCobrarMultiplo(appt);
    const member = multi.membro;
    const basePrice = multi.total;
    const motivoCheio = multi.motivoCheio;
    const planoAtivo = multi.todosCobertos; // só mostra a telinha verde se TODOS os
serviços estiverem inclusos
    const nomesServicos = appointmentServiceNames(appt);
    // nota combinada para o lançamento do caixa (ex: "Corte (incluso no clube),
Barba (15% desconto)")
    let priceNote = multi.itens.length>1
        ? ' - ' +

```

```

multi.itens.map(i=>`${i.service.name}${i.calc.priceNote||''}`).join(', ')
  : (multi.itens[0]?calc.priceNote || '');

let netAmount = appt.depositStatus==='pago' ? Math.max(0, basePrice -
appt.depositAmount) : basePrice;

// pagamentos: lista de {method, amount}. Um só ou vários (dividido).
const finish = (method, pagamentos, desconto)=>{
  const lista = (pagamentos && pagamentos.length) ? pagamentos : [{method,
amount: netAmount}];
  const totalCobrado = lista.reduce((t,p)=>t+Number(p.amount),0);
  // Guarda o valor de CADA serviço separadamente – é isso que permite a
  // comissão e as fichas calcularem certo depois, mesmo quando um
  // serviço estava incluso no plano e outro não. Se o total final for
  // diferente da soma original (desconto, cortesia), distribui
  // proporcionalmente entre os serviços – sem isso, uma cortesia
  // continuaria gerando comissão sobre o preço cheio de cada serviço.
  const somaOriginal = multi.itens.reduce((t,i)=>t+i.calc.basePrice,0);
  let acumulado = 0;
  const valorPorServico = multi.itens.map((i,idx)=>{
    let valor;
    if(somaOriginal<=0 || totalCobrado<=0){
      valor = 0;
    } else if(idx === multi.itens.length-1){
      valor = Math.max(0, Math.round((totalCobrado - acumulado)*100)/100); // o
último absorve o arredondamento
    } else {
      valor = Math.round((i.calc.basePrice/somaOriginal)*totalCobrado*100)/100;
      acumulado += valor;
    }
    return { serviceId: i.service.id, valor, coberto: i.calc.planoAtivo };
  });
  update(s=>{
    const ap = s.appointments.find(a=>a.id===id);
    ap.status='concluido';
    ap.paymentMethod = lista.length===1 ? lista[0].method : 'misto';
    ap.pagamentos = lista;
    ap.desconto = Number(desconto)||0;
    // guarda quanto foi cobrado de fato – é a base da comissão.
    ap.valorCobrado = totalCobrado;
    ap.valorPorServico = valorPorServico;
    ap.cobertoPeloClube = planoAtivo;
    const sess = s.cashSessions.find(c=>c.status==='aberto');
    const obs = `${nomesServicos} –
${ap.client}${priceNote}${(Number(desconto)>0)?` (desconto
${BRL(desconto)})`:''}${ap.depositStatus==='pago'?` (saldo, sinal já
recebido)`:''}`;

```

```

// um lançamento por forma de pagamento, para o caixa somar certo
lista.forEach((p,i)=>{
  if(Number(p.amount) <= 0) return;
  sess.entries.push({
    id:uid(), type:'entrada', method:p.method, apptId:id,
    desc: obs + (lista.length>1 ? ` [${i+1}/${lista.length}]` : ''),
    amount: Number(p.amount), time:new Date().toString().slice(0,5),
    auto:true,
  });
});

const client =
s.clients.find(c=>c.name.trim().toLowerCase()===ap.client.trim().toLowerCase());
  if(client) maybeAwardReferral(client);
});
closeModal();
const total = lista.reduce((t,p)=>t+Number(p.amount),0);
showToast(lista.length>1
  ? `Atendimento concluído - ${BRL(total)} em ${lista.length} formas de
pagamento.`
  : `Atendimento concluído - ${BRL(total)} em
${methodLabelPay(lista[0].method)}.`);
});

const planoNome = member ?
(state.subscriptionPlans.find(p=>p.id===member.planId)?.name || '') : '';

// Todos os serviços cobertos pelo plano: tela própria, deixando claro que não
se cobra
if(netAmount<=0 && planoAtivo){
  openModal(`
    <div style="padding:26px 22px;text-align:center;">
      <div style="width:62px;height:62px;border-radius:50%;background:var(--ok-
bg);border:2px solid var(--ok);
        display:flex;align-items:center;justify-content:center;margin:0 auto
16px;color:var(--ok);">
        <svg width="30" height="30" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2.5"><path d="M20 6 9 17l-5-5"/></svg>
      </div>
      <div class="pill pill-ok" style="font-size:12px;padding:6px 16px;letter-
spacing:.05em;">★ PLANO ATIVO</div>
      <div class="font-display" style="font-size:20px;font-
weight:600;margin:14px 0 4px;">${esc(appt.client)}</div>
      <div style="font-size:13px;color:var(--muted);margin-
bottom:16px;">${esc(nomesServicos)}${planoNome?` · ${esc(planoNome)} `:''}</div>
      <div class="font-mono" style="font-size:38px;font-weight:600;color:var(--
ok);line-height:1;">R$ 0,00</div>
      <div style="font-size:13px;color:var(--ok);margin-top:8px;font-

```

```

weight:600; ">NADA A COBRAR</div>
    <div style="font-size:12px;color:var(--muted-2);margin-top:12px;line-
height:1.6;">
        ${svcs.length>1?'Todos os serviços já estão incluídos na
assinatura.': 'Este serviço já está incluído na assinatura.'}<br>
        As fichas entram no pote do mês normalmente.
    </div>
    <button class="btn btn-gold" id="pa-ok" style="width:100%;justify-
content:center;margin-top:20px;padding:14px;font-size:14.5px;">Confirmar
atendimento</button>
    <button class="btn btn-ghost" id="m-close" style="width:100%;justify-
content:center;margin-top:9px;font-size:12.5px;">Cancelar</button>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#pa-ok').addEventListener('click',
    ()=>finish('dinheiro'));
    });
    return;
}

if(netAmount<=0){ finish('dinheiro'); return; }

const valorOriginal = netAmount;
let desconto = 0;
let pagamentos = [];    // [{method, amount}]

openModal(`
    <div style="padding:22px;">
        <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
            <div class="font-display" style="font-size:19px;font-
weight:600;">Finalizar atendimento</div>
            <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
        </div>
        <div style="font-size:13px;color:var(--muted);margin-bottom:10px;">
            ${esc(appt.client)} ${clubStarBadge(appt.client)}
        </div>
        ${multi.itens.length>1 ? `
            <div style="display:flex;flex-direction:column;gap:4px;margin-
bottom:14px;">
                ${multi.itens.map(i=>`
                    <div style="display:flex;justify-content:space-between;font-
size:12.5px;color:var(--muted);">
                        <span>${esc(i.service.name)}${i.calc.priceNote?
esc(i.calc.priceNote): ''}</span>

```

```

        <span class="font-mono">${BRL(i.calc.basePrice)}</span>
    </div>
    `).join('')}
</div>
` : `<div style="font-size:13px;color:var(--muted);margin-
bottom:14px;">${esc(multi.itens[0]?.service.name||'')}${esc(multi.itens[0]?.calc.pr
</div>`}

    ${motivoCheio ? `<div style="padding:10px 12px;border-
radius:9px;background:var(--panel-2);border:1px solid var(--warn);
    font-size:11.5px;color:var(--muted);line-height:1.6;margin-bottom:14px;">
    <b style="color:var(--warn);">Por que está cobrando?</b>
<br>${esc(motivoCheio)}
    </div>` : ''}

    <div class="card" style="padding:15px 17px;background:var(--panel-2);margin-
bottom:16px;">
        <div style="display:flex;justify-content:space-between;font-
size:13px;margin-bottom:9px;">
            <span style="color:var(--muted);">Valor ${multi.itens.length>1?'dos
serviços':'do serviço'}</span>
            <span class="font-mono">${BRL(valorOriginal)}</span>
        </div>
        <div style="display:flex;justify-content:space-between;align-
items:center;gap:10px;font-size:13px;">
            <span style="color:var(--muted);">Desconto</span>
            <input id="ck-desc" type="number" step="0.01" min="0"
max="${valorOriginal}" value="0"
                style="width:110px;text-align:right;padding:6px 9px;font-size:13px;">
        </div>
        <div style="display:flex;justify-content:space-between;align-
items:center;margin-top:11px;padding-top:11px;border-top:1px solid var(--
border);">
            <span style="font-size:13.5px;font-weight:600;">Total a pagar</span>
            <span class="font-mono" id="ck-total" style="font-size:23px;color:var(--
gold-bright);">${BRL(valorOriginal)}</span>
        </div>
    </div>

    <label>Como o cliente pagou</label>
    <div id="ck-pagamentos" style="display:flex;flex-
direction:column;gap:8px;margin:8px 0 10px;"></div>
    <button class="btn btn-ghost" id="ck-add" style="width:100%;justify-
content:center;font-size:12.5px;padding:9px;">
        ${ICON.plus} Dividir em outra forma de pagamento
    </button>

```

```

<div id="ck-resumo" style="margin-top:14px;"></div>
<button class="btn btn-gold" id="ck-finalizar" style="width:100%;justify-
content:center;padding:14px;font-size:14.5px;margin-top:12px;">
  Finalizar
</button>
${member ? `<button class="btn btn-ghost" id="pm-zerar"
style="width:100%;justify-content:center;margin-top:9px;font-size:12.5px;">
  Cortesia – registrar sem cobrar
</button>` : ''}
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);

  const totalAPagar = ()=> Math.max(0, valorOriginal - desconto);

  const desenharPagamentos = ()=>{
    const wrap = root.querySelector('#ck-pagamentos');
    wrap.innerHTML = pagamentos.map((p,i)=>`
      <div style="display:flex;gap:8px;align-items:center;">
        <select data-pm="${i}" style="flex:1;font-size:13px;">
          ${PAYMENT_METHODS.map(m=>`<option value="${m.id}"
${p.method===m.id?'selected':''}>${m.label}</option>`).join('')}
        </select>
        <input type="number" step="0.01" min="0" data-pv="${i}"
value="${p.amount.toFixed(2)}"
          style="width:110px;text-align:right;font-size:13px;">
          ${pagamentos.length>1?`<button class="btn btn-ghost" data-pr="${i}"
style="padding:8px 10px;color:var(--bad);border-color:var(--bad);">${ICON.x}
</button>`:`<div style="width:38px;"></div>`}
        </div>
      `).join('');
    wrap.querySelectorAll('[data-pm]').forEach(el=>el.addEventListener('change',
    ()=>{
      pagamentos[+el.dataset.pm].method = el.value; atualizar();
    }));
    wrap.querySelectorAll('[data-pv]').forEach(el=>el.addEventListener('input',
    ()=>{
      pagamentos[+el.dataset.pv].amount = parseFloat(el.value)||0;
atualizarResumo();
    }));
    wrap.querySelectorAll('[data-pr]').forEach(el=>el.addEventListener('click',
    ()=>{
      pagamentos.splice(+el.dataset.pr,1); atualizar();
    }));
  };

  const atualizarResumo = ()=>{

```

```

const alvo = totalAPagar();
const pago = pagamentos.reduce((t,p)=>t+(Number(p.amount)||0),0);
const dif = pago - alvo;
const temDinheiro = pagamentos.some(p=>p.method==='dinheiro' &&
Number(p.amount)>0);
const el = root.querySelector('#ck-resumo');
const btn = root.querySelector('#ck-finalizar');

if(Math.abs(dif) < 0.01){
  el.innerHTML = `<div style="text-align:center;font-size:12.5px;color:var(--ok);">✓ Valor fechado certinho.</div>`;
  btn.disabled = false; btn.style.opacity = '1';
  btn.textContent = `Finalizar - ${BRL(alvo)}`;
} else if(dif < 0){
  el.innerHTML = `<div style="padding:11px 13px;border-radius:9px;background:var(--panel-2);text-align:center;font-size:13px;color:var(--warn);">
    Faltava <b class="font-mono">${BRL(Math.abs(dif))}</b>
    <div style="margin-top:9px;">
      <button class="btn btn-ghost" id="ck-vira-desconto" style="font-size:12px;padding:7px 12px;">
        Cliente não tem o restante - dar ${BRL(Math.abs(dif))} de desconto
      </button>
    </div>
  </div>`;
  btn.disabled = true; btn.style.opacity = '.5';
  btn.textContent = 'Complete o valor para finalizar';
  el.querySelector('#ck-vira-desconto')?.addEventListener('click', ()=>{
    // transforma o que falta em desconto, mantendo o que já foi pago
    desconto = Math.min(valorOriginal, desconto + Math.abs(dif));
    root.querySelector('#ck-desc').value = desconto.toFixed(2);
    root.querySelector('#ck-total').textContent = BRL(totalAPagar());
    desenharPagamentos();
    atualizarResumo();
    showToast(`Desconto de ${BRL(Math.abs(dif))} aplicado.`);
  });
} else if(temDinheiro){
  el.innerHTML = `<div style="padding:10px 12px;border-radius:9px;background:var(--ok-bg);text-align:center;font-size:13px;color:var(--ok);">
    Troco: <b class="font-mono">${BRL(dif)}</b>
  </div>`;
  btn.disabled = false; btn.style.opacity = '1';
  btn.textContent = `Finalizar - ${BRL(alvo)}`;
} else {
  el.innerHTML = `<div style="padding:10px 12px;border-radius:9px;background:var(--bad-bg);text-align:center;font-size:12.5px;color:var(--

```

```

-bad);line-height:1.5;">
    Passou <b class="font-mono">${BRL(dif)}</b> do total.<br>
    <span style="font-size:11.5px;">Sem dinheiro em espécie não há como dar
troco - ajuste os valores.</span>
</div>`;
    btn.disabled = true; btn.style.opacity = '.5';
    btn.textContent = 'Ajuste os valores';
  }
};

const atualizar = ()=>{
  root.querySelector('#ck-total').textContent = BRL(totalAPagar());
  // com uma só forma de pagamento, acompanha o total automaticamente
  if(pagamentos.length===1) pagamentos[0].amount = totalAPagar();
  desenharPagamentos();
  atualizarResumo();
};

root.querySelector('#ck-desc').addEventListener('input', (e)=>{
  let v = parseFloat(e.target.value)||0;
  if(v < 0) v = 0;
  if(v > valorOriginal){ v = valorOriginal; e.target.value = v.toFixed(2); }
  desconto = v;
  atualizar();
});

root.querySelector('#ck-add').addEventListener('click', ()=>{
  const jaPago = pagamentos.reduce((t,p)=>t+(Number(p.amount)||0),0);
  const restante = Math.max(0, totalAPagar() - jaPago);
  const usados = new Set(pagamentos.map(p=>p.method));
  const proximo = (PAYMENT_METHODS.find(m=>!usados.has(m.id)) ||
PAYMENT_METHODS[0]).id;
  pagamentos.push({ method: proximo, amount: restante });
  desenharPagamentos();
  atualizarResumo();
});

root.querySelector('#ck-finalizar').addEventListener('click', ()=>{
  const alvo = totalAPagar();
  const pago = pagamentos.reduce((t,p)=>t+(Number(p.amount)||0),0);
  if(pago + 0.01 < alvo){ showToast('O valor pago não cobre o total.');
```

return; }

```

  // troco não é receita: registra só o que corresponde ao total
  let restante = alvo;
  const registrar = pagamentos
    .filter(p=>Number(p.amount)>0)
    .map(p=>{

```

```

        const v = Math.min(Number(p.amount), restante);
        restante = Math.max(0, restante - v);
        return { method:p.method, amount:Number(v.toFixed(2)) };
    })
    .filter(p=>p.amount>0);
netAmount = alvo;
// se o total virou zero (desconto integral), ainda precisa registrar
if(registrar.length === 0){
    finish('dinheiro', [{ method:'dinheiro', amount:0 }], desconto);
    return;
}
finish(registrar[0].method, registrar, desconto);
});

root.querySelector('#pm-zerar')?.addEventListener('click', ()=>{
    if(!confirm('Registrar este atendimento sem cobrar nada?')) return;
    netAmount = 0; priceNote = ' (cortesia)';
    finish('dinheiro', [{method:'dinheiro', amount:0}], valorOriginal);
});

// começa com uma forma de pagamento cobrindo o total
pagamentos = [{ method:'dinheiro', amount: valorOriginal }];
atualizar();
});
}

function servingProfissionaisOk(){
    if(servingProfessionals().length===0 || state.services.length===0){
        showToast('Cadastre ao menos 1 profissional que atende e 1 serviço primeiro.');
```

return false;

```

    }
    return true;
}

/* Menu que abre ao clicar num agendamento da grade */
function apptActionsModal(apptId, view){
    const a = state.appointments.find(x=>x.id===apptId);
    if(!a) return;
    const svcs = appointmentServices(a);
    const nomesServicos = appointmentServiceNames(a);
    const duracaoTotal = appointmentDuration(a);
    const membro = clubMemberForClient(a.client);
    const podeMexer = isOwnerLevel() || userSession.role==='repcionista' ||
a.professionalId===userSession.professionalId;
    // valor real considerando o plano do cliente, somando todos os serviços
    const multi = svcs.length ? precoParaCobrarMultiplo(a) : null;
    const precoTabela = svcs.reduce((t,s)=>t+Number(s.price),0);
    const valorReal = multi ? (a.depositStatus==='pago' ? Math.max(0, multi.total -

```

```

a.depositAmount) : multi.total) : 0;
const statusMap = {
  agendado:{l:'Agendado',t:'muted'}, confirmado:{l:'Confirmado',t:'ok'},
  concluido:{l:'Concluído',t:'ok'}, cancelado:{l:'Cancelado',t:'bad'}, faltou:
{l:'Faltou',t:'bad'},
};
const st = statusMap[a.status] || {l:a.status,t:'muted'};

openModal(`
  <div style="padding:22px;">
    <div style="display:flex;justify-content:space-between;align-items:flex-
start;gap:10px;margin-bottom:14px;">
      <div style="min-width:0;">
        <div class="font-display" style="font-size:19px;font-
weight:600;display:flex;align-items:center;gap:7px;flex-wrap:wrap;">
          ${esc(a.client)} ${clubStarBadge(a.client)}
        </div>
        <div style="font-size:13px;color:var(--muted);margin-top:4px;">
          ${esc(nomesServicos)} · ${esc(profName(a.professionalId))}
        </div>
        <div class="font-mono" style="font-size:12.5px;color:var(--muted-
2);margin-top:3px;">
          ${new Date(a.date+'T12:00').toLocaleDateString('pt-BR',
{weekday:'short', day:'2-digit', month:'2-digit'})} às ${a.time}${svcs.length?`
${duracaoTotal} min`:''}
        </div>
      </div>
      <button class="btn btn-ghost" style="padding:6px;flex-shrink:0;" id="m-
close">${ICON.x}</button>
    </div>

    <div style="display:flex;gap:7px;flex-wrap:wrap;margin-bottom:16px;">
      <span class="pill pill-${st.t}">${st.l}</span>
      ${a.origem==='online'?`<span class="pill"
style="background:rgba(74,159,216,.15);color:#4a9fd8;border-
color:rgba(74,159,216,.35);">${ICON.globe} Agendado pelo cliente
(link)${a.clientPhone?' · '+esc(a.clientPhone):''}</span>`:`<span class="pill
pill-muted">${ICON.shield} Marcado pela barbearia${a.criadoPor?' ·
'+esc(a.criadoPor):''}</span>`}
      ${a.seriesId?`<span class="pill pill-muted">${ICON.clock} recorrente
(${a.seriesFreqDias||7} em ${a.seriesFreqDias||7} dias)</span>`:''}
      ${svcs.length ? (multi.todosCobertos
? `<span class="pill pill-ok" style="font-weight:600;">★ PLANO ATIVO ·
R$ 0,00</span>`
: `<span class="pill pill-muted font-mono">${BRL(valorReal)}${valorReal
< precoTabela ? `<span style="text-decoration:line-
through;opacity:.6;">${BRL(precoTabela)}</span>` : ''}</span>`) : `<span

```

```

class="pill pill-bad">Sem serviço definido</span>` }
    ${membro && membro.status=== 'atrasado'?`<span class="pill pill-bad">Clube
em atraso</span>`: ''}
    ${a.depositStatus=== 'pago'?`<span class="pill pill-ok">Sinal pago
${BRL(a.depositAmount)}</span>`: ''}
</div>

    ${multi?.motivoCheio ? `<div style="padding:10px 12px;border-
radius:9px;background:var(--panel-2);border:1px solid var(--warn);font-
size:11.5px;color:var(--muted);line-height:1.6;margin-bottom:14px;">
    <b style="color:var(--warn);">Vai cobrar valor cheio${svcs.length>1?' em
algum item': ''}</b><br>${esc(multi.motivoCheio)}
    </div>` : ''}

    ${!podeMexer ? `<div style="font-size:12.5px;color:var(--muted-2);">Este
agendamento é de outro profissional.</div>` : `
    <div style="display:flex;flex-direction:column;gap:9px;">
        ${a.status!=='concluido' && a.status!=='cancelado' ? `
            <button class="btn btn-gold" id="ac-concluir" style="justify-
content:center;padding:14px;font-size:14.5px;">
                ${multi?.todosCobertos ? `${ICON.check} Finalizar – nada a cobrar` :
`${ICON.cash} Finalizar e receber ${valorReal>0?BRL(valorReal): ''}}`
            </button>
            <div style="display:flex;gap:9px;flex-wrap:wrap;">
                <button class="btn btn-ghost" id="ac-editar-servico"
style="flex:1;justify-content:center;min-width:130px;">${ICON.edit|| ''} Editar
serviço</button>
                <button class="btn btn-ghost" id="ac-reagendar"
style="flex:1;justify-content:center;min-width:130px;">${ICON.clock}
Reagendar</button>
                ${!a.confirmed?`<button class="btn btn-ghost" id="ac-confirmar"
style="flex:1;justify-content:center;min-width:130px;">${ICON.check}
Confirmar</button>`: ''}
            </div>
            <div style="display:flex;gap:9px;flex-wrap:wrap;">
                <button class="btn btn-ghost" id="ac-faltou" style="flex:1;justify-
content:center;min-width:130px;">Marcar falta</button>
                <button class="btn btn-ghost" id="ac-cancelar"
style="flex:1;justify-content:center;min-width:130px;color:var(--bad);border-
color:var(--bad);">Cancelar</button>
            </div>
            ${a.seriesId?`<button class="btn btn-ghost" id="ac-cancelar-serie"
style="justify-content:center;color:var(--bad);border-color:var(--bad);font-
size:12.5px;">Cancelar toda a série recorrente</button>`: ''}
        ` : `
        <div style="font-size:12.5px;color:var(--muted);padding:10px 0;">
            ${a.status=== 'concluido' ? 'Atendimento já finalizado' +
(a.paymentMethod?` – recebido em ${methodLabelPay(a.paymentMethod)}.` : '')} : 'Este

```

```

    agendamento foi cancelado.')}
  </div>
  ${a.status==='concluido' && isOwnerLevel() ? `
    <button class="btn btn-ghost" id="ac-reabrir" style="justify-
content:center;padding:12px;border-color:var(--warn);color:var(--warn);">
      ${ICON.clock} Reabrir comanda
    </button>
    <div style="font-size:11px;color:var(--muted-2);text-
align:center;line-height:1.5;margin-top:-3px;">
      Desfaz o fechamento: tira o valor do caixa e da comissão, e o
atendimento volta para "agendado".
    </div>
    ` : a.status==='concluido' ? `
    <div style="font-size:11.5px;color:var(--muted-2);line-height:1.5;">
      Só o dono pode reabrir uma comanda já finalizada.
    </div>
    ` : ''}
  ${a.status==='cancelado' && podeMexer ? `
    <button class="btn btn-ghost" id="ac-reativar" style="justify-
content:center;padding:12px;">Reativar agendamento</button>
    ` : ''}
  `}
  ${messageChannelRow(

(state.clients.find(cl=>cl.name.trim().toLowerCase()===a.client.trim().toLowerCase(
{})).phone || membro?.phone,
  `Olá ${a.client}! Passando para confirmar seu horário em ${new
Date(a.date+'T12:00').toLocaleDateString('pt-BR')} às ${a.time} na
${state.shopProfile.name}.`
  )}
  <button class="btn btn-ghost" id="ac-excluir" style="justify-
content:center;font-size:12.5px;color:var(--muted);margin-top:4px;">Excluir da
agenda</button>
  </div>
  `}
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  const recarrega = ()=>{ closeModal(); if(view) renderAgenda(view); };

  root.querySelector('#ac-concluir')?.addEventListener('click', ()=>{
closeModal(); concludeAppt(a.id); });
  root.querySelector('#ac-editar-servico')?.addEventListener('click', ()=>{
closeModal(); editarServicoModal(a.id, view); });
  root.querySelector('#ac-reagendar')?.addEventListener('click', ()=>{
closeModal(); reagendarModal(a.id, view); });
  root.querySelector('#ac-confirmar')?.addEventListener('click', ()=>{

```

```

        update(s=>{ const x=s.appointments.find(y=>y.id===a.id); x.confirmed=true;
x.status='confirmado'; });
        showToast('Agendamento confirmado.');
```

recarrega();

```

    });
    root.querySelector('#ac-faltou')?.addEventListener('click', ()=>{
        update(s=>{ s.appointments.find(y=>y.id===a.id).status='faltou'; });
        showToast('Marcado como falta.');
```

recarrega();

```

    });
    root.querySelector('#ac-cancelar')?.addEventListener('click', ()=>{
        if(!confirm('Cancelar este agendamento?')) return;
        update(s=>{ s.appointments.find(y=>y.id===a.id).status='cancelado'; });
        showToast('Agendamento cancelado.');
```

recarrega();

```

    });
    root.querySelector('#ac-cancelar-serie')?.addEventListener('click', ()=>{
        const futuros = state.appointments.filter(x=>x.seriesId===a.seriesId &&
x.status!=='cancelado' && x.status!=='concluido' && x.date>=a.date);
        if(!confirm(`Cancelar ${futuros.length} agendamento(s) desta série
recorrente (a partir de hoje)?`)) return;
        update(s=>{
            s.appointments.forEach(x=>{
                if(x.seriesId===a.seriesId && x.status!=='cancelado' &&
x.status!=='concluido' && x.date>=a.date){
                    x.status='cancelado';
                }
            });
        });
        showToast(`${futuros.length} agendamento(s) da série cancelados.`);
recarrega();
    });
    root.querySelector('#ac-reabrir')?.addEventListener('click', ()=>{
closeModal(); reabrirComanda(a.id, view); });
    root.querySelector('#ac-reativar')?.addEventListener('click', ()=>{
        update(s=>{ s.appointments.find(y=>y.id===a.id).status='agendado'; });
        showToast('Agendamento reativado.');
```

recarrega();

```

    });
    root.querySelector('#ac-excluir')?.addEventListener('click', ()=>{
        if(!confirm('Excluir este agendamento da agenda? Não dá para desfazer.'))
return;
        update(s=>{ s.appointments = s.appointments.filter(y=>y.id!==a.id); });
        showToast('Agendamento excluído.');
```

recarrega();

```

    });
    });
}

```

/* Reagendar: muda dia, horário e profissional */

/* Troca o(s) serviço(s) de um agendamento já criado – sem precisar
cancelar e refazer. Revalida conflito de horário, já que a duração

```

    total pode mudar (ex: trocar um corte por corte+barba). */
function editarServicoModal(apptId, view){
    const a = state.appointments.find(x=>x.id===apptId);
    if(!a) return;
    const marcadosAtuais = new Set(appointmentServiceIds(a));
    openModal(`
        <div style="padding:22px;">
            <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:6px;">
                <div class="font-display" style="font-size:19px;font-weight:600;">Editar
serviço</div>
                <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
            </div>
            <div style="font-size:12.5px;color:var(--muted);margin-bottom:16px;">
                ${esc(a.client)} - ${new Date(a.date+'T12:00').toLocaleDateString('pt-
BR')} às ${a.time}
            </div>
            <label>Serviço(s) <span style="color:var(--muted-2);font-weight:400;">- pode
marcar mais de um</span></label>
            <div id="es-services-box" style="display:flex;flex-
direction:column;gap:6px;max-height:220px;overflow-y:auto;padding:10px
12px;border:1px solid var(--border);border-radius:10px;margin-top:6px;">
                ${state.services.map(s=>`
                    <label style="display:flex;align-items:center;gap:9px;font-
size:13px;cursor:pointer;padding:4px 0;">
                        <input type="checkbox" class="es-service-chk" value="${s.id}"
${marcadosAtuais.has(s.id)?'checked':''} style="width:auto;">
                        <span style="flex:1;">${esc(s.name)}</span>
                        <span class="font-mono" style="color:var(--muted-2);font-
size:12px;">${BRL(s.price)} · ${s.duration}min</span>
                    </label>
                `).join('')}
            </div>
            <div id="es-total" style="font-size:12px;color:var(--muted-2);margin-
top:6px;"></div>
            <div id="es-aviso" style="font-size:12px;margin-top:8px;"></div>
            <button class="btn btn-gold" id="es-save" style="width:100%;justify-
content:center;margin-top:16px;">Salvar</button>
            <div style="font-size:11px;color:var(--muted-2);margin-top:10px;line-
height:1.6;">
                Se o atendimento já foi finalizado, isso não muda o valor lançado no caixa
- só corrige o registro do serviço.
            </div>
        </div>
    `, root=>{
        root.querySelector('#m-close').addEventListener('click', closeModal);
    }
}

```

```

const servicosMarcados = ()=>[...root.querySelectorAll('.es-service-
chk:checked')].map(el=>serviceById(el.value)).filter(Boolean);

const checar = ()=>{
  const svcs = servicosMarcados();
  const durTotal = svcs.reduce((t,s)=>t+(Number(s.duration)||30),0);
  const precoTotal = svcs.reduce((t,s)=>t+Number(s.price),0);
  root.querySelector('#es-total').textContent = svcs.length
    ? `${svcs.length} serviço(s) - ${durTotal} min · ${BRL(precoTotal)}`
    : 'Nenhum serviço selecionado.';
  const aviso = root.querySelector('#es-aviso');
  if(svcs.length===0){ aviso.textContent=''; return; }
  // só revalida conflito se o agendamento ainda não foi concluído/cancelado
  if(a.status==='concluído' || a.status==='cancelado'){ aviso.textContent='';
return; }

  const choque = state.appointments.some(x=>x.id!==a.id &&
x.professionalId===a.professionalId && x.date===a.date &&
  x.status!=='cancelado' && x.status!=='faltou' &&
  timesOverlap(x.time, appointmentDuration(x), a.time, durTotal));
  if(choque){ aviso.style.color='var(--bad)'; aviso.textContent='⚠ Com essa
duração, o horário passa a bater com outro agendamento.'; }
  else { aviso.textContent=''; }
};

root.querySelectorAll('.es-service-
chk').forEach(el=>el.addEventListener('change', checar));
checar();

root.querySelector('#es-save').addEventListener('click', ()=>{
  const svcs = servicosMarcados();
  if(svcs.length===0){ showToast('Marque ao menos um serviço.');
```

 return; }
 const serviceIds = svcs.map(s=>s.id);
 update(s=>{
 const ap = s.appointments.find(x=>x.id===apptId);
 if(!ap) return;
 ap.serviceIds = serviceIds;
 ap.serviceId = serviceIds[0];
 });
 closeModal();
 showToast('Serviço atualizado.');
 if(view) renderAgenda(view);
});
});
}

function reagendarModal(apptId, view){
 const a = state.appointments.find(x=>x.id===apptId);
 if(!a) return;

```

const profs = servingProfessionals();
openModal(`
  <div style="padding:22px;">
    <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:6px;">
      <div class="font-display" style="font-size:19px;font-
weight:600;">Reagendar</div>
      <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
    </div>
    <div style="font-size:12.5px;color:var(--muted);margin-bottom:18px;">
      ${esc(a.client)} - hoje marcado para ${new
Date(a.date+'T12:00').toLocaleDateString('pt-BR')} às ${a.time}
    </div>
    <div style="display:flex;flex-direction:column;gap:13px;">
      <div><label>Profissional</label>
      <select id="r-prof" style="width:100%;">
        ${profs.map(p=><option value="${p.id}"
${a.professionalId===p.id?'selected':''}>${esc(p.name)}</option>`).join('')}
      </select>
    </div>
    <div style="display:flex;gap:10px;">
      <div style="flex:1;"><label>Nova data</label><input id="r-date"
type="date" value="${a.date}" style="width:100%;"></div>
      <div style="flex:1;"><label>Novo horário</label><input id="r-time"
type="time" value="${a.time}" style="width:100%;"></div>
    </div>
    <div id="r-aviso" style="font-size:12px;"></div>
    <button class="btn btn-gold" id="r-save" style="justify-
content:center;">Salvar novo horário</button>
  </div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  const checar = ()=>{
    const pid = root.querySelector('#r-prof').value;
    const d = root.querySelector('#r-date').value;
    const t = root.querySelector('#r-time').value;
    const aviso = root.querySelector('#r-aviso');
    if(!d || !t){ aviso.textContent=''; return; }
    const dur = appointmentDuration(a);
    const choque = state.appointments.some(x=>x.id!==a.id &&
x.professionalId===pid && x.date===d &&
    x.status!=='cancelado' && x.status!=='faltou' &&
    timesOverlap(x.time, appointmentDuration(x), t, dur));
    const bloqueado = blocksForDay(pid, d).some(b=> t >= b.start && t < b.end);
    const prof = state.professionals.find(p=>p.id===pid);

```

```

        if(choque){ aviso.style.color='var(--bad)'; aviso.textContent='⚠ Já existe
outro agendamento nesse horário.'; }
        else if(bloqueado){ aviso.style.color='var(--bad)'; aviso.textContent='⚠
Esse horário está bloqueado (folga ou almoço).'; }
        else if(prof && !isProfessionalWorking(prof, d)){ aviso.style.color='var(--
warn)'; aviso.textContent='⚠ Esse profissional não trabalha neste dia.'; }
        else { aviso.style.color='var(--ok)'; aviso.textContent='✓ Horário livre.';
}
    };
    ['#r-prof', '#r-date', '#r-
time'].forEach(sel=>root.querySelector(sel).addEventListener('change', checar));
    checar();

    root.querySelector('#r-save').addEventListener('click', ()=>{
        const pid = root.querySelector('#r-prof').value;
        const d = root.querySelector('#r-date').value;
        const t = root.querySelector('#r-time').value;
        if(!d || !t){ showToast('Preencha data e horário.');
```

return; }
 update(s=>{
 const x = s.appointments.find(y=>y.id===a.id);
 x.professionalId = pid; x.date = d; x.time = t;
 x.confirmed = false;
 if(x.status==='confirmado') x.status='agendado';
 });
 closeModal();
 showToast(`Reagendado para \${new Date(d+'T12:00').toLocaleDateString('pt-
BR')} às \${t}.`);
 if(view) renderAgenda(view);
 });
});
}

function blockModal(pre, view){
 const isProfView = userSession.role==='profissional';
 const profs = isProfView
 ? state.professionals.filter(p=>p.id===userSession.professionalId)
 : servingProfessionals();
 if(profs.length===0){ showToast('Cadastre um profissional primeiro.');
return; }
 const hoje = pre?.date || todayISO();
 openModal(`
 <div style="padding:22px;">
 <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
 <div class="font-display" style="font-size:19px;font-weight:600;">Bloquear
horário</div>
 <button class="btn btn-ghost" style="padding:6px;" id="m-close">\${ICON.x}
</button>

```

</div>
<div style="font-size:12.5px;color:var(--muted);margin-bottom:18px;">Folga,
almoço, médico – ninguém consegue agendar nesse período.</div>
<div style="display:flex;flex-direction:column;gap:13px;">
  <div><label>Profissional</label>
  <select id="b-prof" style="width:100%;">
    ${profs.map(p=>`<option value="${p.id}"
    ${pre?.professionalId===p.id?'selected':''}>${esc(p.name)}</option>`).join('')}
  </select>
</div>
<div><label>Motivo</label>
  <input id="b-motivo" list="b-motivos" value="${esc(pre?.motivo)}"
placeholder="Ex: Almoço, Folga, Médico" style="width:100%;">
  <datalist id="b-motivos">
    <option value="Almoço"><option value="Folga"><option value="Médico">
    <option value="Compromisso"><option value="Manutenção">
  </datalist>
</div>
<div style="display:flex;gap:10px;">
  <div style="flex:1;"><label>Início</label><input id="b-start"
type="time" value="${pre?.time||'12:00'}" style="width:100%;"></div>
  <div style="flex:1;"><label>Fim</label><input id="b-end" type="time"
value="13:00" style="width:100%;"></div>
</div>
<label style="display:flex;align-items:flex-start;gap:10px;padding:12px
14px;border:1px solid var(--border);border-radius:10px;cursor:pointer;">
  <input type="checkbox" id="b-repete" style="width:auto;margin-top:2px;">
  <div>
    <div style="font-size:13.5px;font-weight:600;">Repetir toda
semana</div>
    <div style="font-size:11.5px;color:var(--muted-2);margin-top:2px;"
id="b-repete-hint">Ideal para o horário de almoço.</div>
  </div>
</label>
  <div id="b-date-wrap"><label>Data</label><input id="b-date" type="date"
value="${hoje}" style="width:100%;"></div>
  <button class="btn btn-gold" id="b-save" style="justify-
content:center;">Bloquear</button>
</div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  const atualizaHint = ()=>{
    const rep = root.querySelector('#b-repete').checked;
    const d = root.querySelector('#b-date').value || hoje;
    const nomeDia =
['domingo','segunda','terça','quarta','quinta','sexta','sábado'] [new

```

```

Date(d+'T12:00').getDay()];
    root.querySelector('#b-repete-hint').textContent = rep
        ? `Vai bloquear toda ${nomeDia}, todas as semanas.`
        : 'Ideal para o horário de almoço.';
    };
    root.querySelector('#b-repete').addEventListener('change', atualizaHint);
    root.querySelector('#b-date').addEventListener('change', atualizaHint);

    root.querySelector('#b-save').addEventListener('click', ()=>{
        const professionalId = root.querySelector('#b-prof').value;
        const motivo = root.querySelector('#b-motivo').value.trim() || 'Bloqueado';
        const start = root.querySelector('#b-start').value;
        const end = root.querySelector('#b-end').value;
        const repeteSemanal = root.querySelector('#b-repete').checked;
        const date = root.querySelector('#b-date').value;
        if(!start || !end){ showToast('Informe o horário de início e fim.');
```

```

function openApptModal(pre) {
  if(servingProfissionaisOk()===false) return;
  // pre = { professionalId, time, date } quando vem de um clique na grade
  const preProf = pre?.professionalId || '';
  const preTime = pre?.time || '';
  const preDate = pre?.date || todayISO();
  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:16px;">
        <div class="font-display" style="font-size:19px;font-weight:600;">Novo
agendamento</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
      </div>
      <div style="display:flex;flex-direction:column;gap:12px;">
        <div style="position:relative;">
          <label>Cliente</label>
          <input id="f-client" placeholder="Digite o nome para buscar"
autocomplete="off" style="width:100%;">
          <div id="client-sug"
style="display:none;position:absolute;left:0;right:0;top:100%;z-index:30;margin-
top:4px;
            background:var(--panel);border:1px solid var(--gold);border-
radius:11px;max-height:230px;overflow-y:auto;
            box-shadow:0 14px 34px -10px rgba(0,0,0,.45);"></div>
          </div>
          <div id="club-note" style="font-size:12px;"></div>
          <div style="display:flex;gap:10px;">
            <div style="flex:1;"><label>Horário</label><input id="f-time"
type="time" value="${preTime}" style="width:100%;"></div>
            <div style="flex:1;"><label>Data</label><input id="f-date" type="date"
value="${preDate}" style="width:100%;"></div>
          </div>
          <div><label>Profissional</label><select id="f-prof"
style="width:100%;">${servingProfessionals().map(p=>`<option value="${p.id}"
${preProf===p.id?'selected':''}>${esc(p.name)}</option>`).join('')}</select></div>
          <div>
            <label>Serviço(s) <span style="color:var(--muted-2);font-weight:400;">—
pode marcar mais de um</span></label>
            <div id="f-services-box" style="display:flex;flex-
direction:column;gap:6px;max-height:190px;overflow-y:auto;padding:10px
12px;border:1px solid var(--border);border-radius:10px;">
              ${state.services.map(s=>`
                <label style="display:flex;align-items:center;gap:9px;font-
size:13px;cursor:pointer;padding:4px 0;">
                  <input type="checkbox" class="f-service-chk" value="${s.id}"

```

```

style="width:auto;">
    <span style="flex:1;">${esc(s.name)}</span>
    <span class="font-mono" style="color:var(--muted-2);font-size:12px;">${BRL(s.price)} · ${s.duration}min</span>
</label>
    `).join('')
</div>
    <div id="f-services-total" style="font-size:12px;color:var(--muted-2);margin-top:6px;"></div>
</div>
    <div id="deposit-note" style="font-size:12px;color:var(--warn);"></div>
    <div id="availability-note" style="font-size:12px;color:var(--muted-2);">
</div>

    <label style="display:flex;align-items:center;gap:9px;font-size:13.5px;color:var(--ink);cursor:pointer;margin-top:2px;">
        <input type="checkbox" id="f-recorrente" style="width:auto;padding:0;">
Repetir automaticamente (cliente fixo)
    </label>
    <div id="recorrendia-fields" style="display:none;flex-direction:column;gap:10px;padding:12px 14px;background:var(--panel-2);border-radius:10px;">
        <div><label>A cada</label>
            <select id="f-rec-freq" style="width:100%;">
                <option value="7">7 dias (toda semana)</option>
                <option value="14">14 dias (quinzenal)</option>
                <option value="21">21 dias</option>
                <option value="30">30 dias (mensal)</option>
            </select>
        </div>
        <div><label>Repetir por quantas vezes</label>
            <input id="f-rec-vezes" type="number" min="2" max="26" value="8"
style="width:100%;">
            <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">Cria os próximos horários já na agenda, sempre no mesmo dia da semana, horário e profissional. Dá para cancelar um ou todos depois.</div>
        </div>
    </div>

    <button class="btn btn-gold" id="f-submit" style="justify-content:center;margin-top:6px;">Agendar</button>
</div>
</div>
    ` , root=>{
        function servicosMarcados(){
            return [...root.querySelectorAll('.f-service-chk:checked')].map(el=>serviceById(el.value)).filter(Boolean);

```

```

    }
    function updateNotes(){
        const svcs = servicosMarcados();
        const durTotal = svcs.reduce((t,s)=>t+(Number(s.duration)||30),0);
        const precoTotal = svcs.reduce((t,s)=>t+Number(s.price),0);
        const totalEl = root.querySelector('#f-services-total');
        totalEl.textContent = svcs.length
            ? `${svcs.length} serviço(s) selecionado(s) - ${durTotal} min ·
    ${BRL(precoTotal)}`
            : 'Nenhum serviço selecionado ainda.';
        const prof = state.professionals.find(p=>p.id===root.querySelector('#f-
prof').value);
        const date = root.querySelector('#f-date').value;
        const dep = svcs.length===1 ? depositAmountFor(svcs[0]) : 0;
        root.querySelector('#deposit-note').textContent = dep>0 ? `Este serviço
exige sinal de ${BRL(dep)}.` : '';
        const el = root.querySelector('#availability-note');
        const hora = root.querySelector('#f-time').value;
        const avisos = [];
        if(prof && date && !isProfessionalWorking(prof, date)){
            avisos.push(`${prof.name} não trabalha neste dia (folga ou fora da
escala).`);
        }
        // fora do horário de funcionamento do profissional
        if(prof && hora){
            const ini = prof.workStart || '09:00';
            const fim = prof.workEnd || '19:00';
            if(hora < ini || hora >= fim){
                avisos.push(`${hora} está fora do expediente de ${prof.name} (${ini} às
${fim}). O agendamento será criado como exceção e aparecerá normalmente na
agenda.`);
            }
        }
        // já tem alguém nesse horário? (usa a duração somada de todos os serviços
marcados)
        if(prof && date && hora && svcs.length){
            const choque = state.appointments.some(x=>x.professionalId===prof.id &&
x.date===date &&
                x.status!=='cancelado' && x.status!=='faltou' &&
                timesOverlap(x.time, appointmentDuration(x), hora, durTotal));
            if(choque) avisos.push('Já existe outro agendamento nesse horário.');
```

```

    ${esc(t)}</div>`).join('')
        : '';
    }
    root.querySelectorAll('.f-service-
chk').forEach(el=>el.addEventListener('change', updateNotes));
    root.querySelector('#f-prof').addEventListener('change', updateNotes);
    root.querySelector('#f-date').addEventListener('change', updateNotes);
    root.querySelector('#f-time').addEventListener('change', updateNotes);
    const inputCliente = root.querySelector('#f-client');
    const caixaSug = root.querySelector('#client-sug');
    const note = root.querySelector('#club-note');

    const atualizaNota = ()=>{
        const m = clubMemberForClient(inputCliente.value);
        if(m && m.status==='atrasado'){
            note.style.color = 'var(--bad)';
            note.textContent = `⚠ ${m.name} está com o clube de assinatura em atraso -
o serviço será cobrado em valor cheio, sem desconto/inclusão do plano.`;
        } else if(m && m.status==='pago'){
            note.style.color = 'var(--ok)';
            note.textContent = `✓ ${m.name} é assinante do clube e está em dia.`;
        } else {
            note.textContent = '';
        }
    };

    // Busca clientes já cadastrados enquanto o dono digita
    const norm = s =>
String(s||'').toLowerCase().normalize('NFD').replace(/[\u0300-
\u036f]/g,'').trim();
    const ultimaVisitaDe = (nome)=>{
        const dts = state.appointments
            .filter(a=>a.status==='concluido' && norm(a.client)===norm(nome))
            .map(a=>a.date).sort();
        return dts.length ? dts[dts.length-1] : null;
    };

    const fecharSugestoes = ()=>{ caixaSug.style.display='none'; };
    const escolher = (nome)=>{
        inputCliente.value = nome;
        fecharSugestoes();
        atualizaNota();
    };

    const buscar = ()=>{
        const termo = norm(inputCliente.value);
        if(termo.length < 2){ fecharSugestoes(); atualizaNota(); return; }
    };

```

```

// Junta a lista de Clientes com a de assinantes do clube - muitos
// assinantes vieram da importação do Asaas e não estão em Clientes.
const mapa = {};
(state.clients||[]).forEach(cl=>{
  if(!cl.name) return;
  mapa[norm(cl.name)] = { name: cl.name, phone: cl.phone || '' };
});
(state.subscriptionMembers||[]).forEach(m=>{
  if(!m.name || m.status==='cancelado') return;
  const k = norm(m.name);
  if(mapa[k]){ if(!mapa[k].phone) mapa[k].phone = m.phone || ''; }
  else mapa[k] = { name: m.name, phone: m.phone || '', soAssinante: true };
});
const todos = Object.values(mapa);

const achados = todos
  .filter(cl=>norm(cl.name).includes(termo))
  .sort((a,b)=>{
    // quem começa com o que foi digitado vem primeiro
    const ai = norm(a.name).startsWith(termo) ? 0 : 1;
    const bi = norm(b.name).startsWith(termo) ? 0 : 1;
    return ai!==bi ? ai-bi : a.name.localeCompare(b.name,'pt-BR');
  })
  .slice(0, 8);

if(achados.length===0){
  caixaSug.innerHTML = `<div style="padding:12px 14px;font-
size:12.5px;color:var(--muted-2);">
    Nenhum cliente com esse nome. Ao salvar, será criado um cadastro novo.
  </div>`;
  caixaSug.style.display='block';
  atualizaNota();
  return;
}

caixaSug.innerHTML = achados.map(cl=>{
  const membro = clubMemberForClient(cl.name);
  const ult = ultimaVisitaDe(cl.name);
  const detalhes = [
    cl.phone || 'sem telefone',
    ult ? 'última visita ' + new Date(ult+'T12:00').toLocaleDateString('pt-
BR') : 'nunca atendido',
    cl.soAssinante ? 'assinante do clube' : null,
  ].filter(Boolean).join(' · ');
  return `<div class="sug-item" data-nome="${esc(cl.name)}"
style="padding:10px 14px;cursor:pointer;border-bottom:1px solid var(--border);">
    <div style="font-size:13.5px;font-weight:500;display:flex;align-

```

```

items:center;gap:6px;">
    ${esc(cl.name)} ${clubStarBadge(cl.name)}
    ${membro && membro.status=== 'atrasado' ? `<span class="pill pill-bad"
style="font-size:10px;">em atraso</span>` : ''}
</div>
<div style="font-size:11.5px;color:var(--muted-2);margin-
top:2px;">${esc(detalhes)}</div>
</div>`;
)).join('');
caixaSug.style.display='block';
caixaSug.querySelectorAll('.sug-item').forEach(el=>{
    el.addEventListener('mousedown', e=>{ e.preventDefault();
escolher(el.dataset.nome); });
    el.addEventListener('mouseenter', ()=>{ el.style.background='var(--panel-
2)'; });
    el.addEventListener('mouseleave', ()=>{ el.style.background='transparent';
});
});
atualizaNota();
});

inputCliente.addEventListener('input', buscar);
inputCliente.addEventListener('focus', buscar);
inputCliente.addEventListener('blur', ()=>setTimeout(fecharSugestoes, 120));
inputCliente.addEventListener('keydown', e=>{ if(e.key==='Escape')
fecharSugestoes(); });
updateNotes();
root.querySelector('#f-recorrente').addEventListener('change', e=>{
    root.querySelector('#recorrencia-fields').style.display = e.target.checked ?
'flex' : 'none';
});
root.querySelector('#m-close').addEventListener('click', closeModal);
root.querySelector('#f-submit').addEventListener('click', ()=>{
    const client = root.querySelector('#f-client').value.trim();
    const time = root.querySelector('#f-time').value;
    const date = root.querySelector('#f-date').value;
    if(!client || !time || !date){ showToast('Preencha cliente, data e
horário.');
```

```

const vezes = recorrente ? Math.max(2, Math.min(26,
parseInt(root.querySelector('#f-rec-vezes').value)||8)) : 1;

// monta as datas da série (a primeira é a que o dono escolheu)
const seriesId = recorrente ? uid() : null;
const datas = [];
for(let i=0; i<vezes; i++){
  const d = new Date(date+'T12:00');
  d.setDate(d.getDate() + i*freqDias);
  datas.push(d.toISOString().slice(0,10));
}

// avisa antes se algum horário da série já está ocupado ou bloqueado
const conflitos = [];
datas.forEach(d=>{
  const choque =
state.appointments.some(x=>x.professionalId===professionalId && x.date===d &&
  x.status!=='cancelado' && x.status!=='faltou' &&
  timesOverlap(x.time, appointmentDuration(x), time, durTotal));
  const bloq = blocksForDay(professionalId, d).some(b=> time >= b.start &&
time < b.end);
  if(choque || bloq) conflitos.push(d);
});
if(conflitos.length>0){
  const lista = conflitos.slice(0,5).map(d=>new
Date(d+'T12:00').toLocaleDateString('pt-BR')).join(', ');
  if(!confirm(`${conflitos.length} data(s) já têm choque de horário ou estão
bloqueadas: ${lista}${conflitos.length>5?'...':''}`)\n\nEssas datas serão puladas.
Continuar com as demais?`)) return;
}

let criados = 0;
update(s=>{
  datas.forEach(d=>{
    if(conflitos.includes(d)) return;
    s.appointments.push({
      id:uid(), date:d, time, client, professionalId,
      serviceId: serviceIds[0], serviceIds,
      status:'agendado', confirmed:false,
      depositStatus: (dep>0 && d===date) ?'pendente':'nao_aplicavel',
depositAmount: d===date?dep:0,
      createdAt:new Date().toISOString(), rating:null,
      seriesId, seriesFreqDias: seriesId?freqDias:undefined,
      origem: 'painel', criadoPor: userSession.role==='profissional' ?
profName(userSession.professionalId) : (state.ownerName || 'equipe'),
    });
    criados++;
  });
});

```

```

    });
  });
  closeModal();
  showToast(recorrente
    ? `${criados} agendamento(s) criado(s) na série (a cada ${freqDias}
dias).`
    : (dep>0 ? `Agendamento criado. Sinal de ${BRL(dep)} pendente.` :
'Agendamento criado.'));
  });
});
}

/* ===== CAIXA ===== */
function renderCaixa(view){
  const session = openSession();
  const totals = sessionTotals(session);
  const history =
state.cashSessions.filter(c=>c.status==='fechado').sort((a,b)=>b.date.localeCompare

  view.innerHTML = `
    ${pageHeader('Caixa', 'Fluxo de caixa do dia', session ? `<button class="btn
btn-ghost" id="btn-close-cash" style="border-color:var(--bad);color:var(--
bad);">Fechar caixa</button>` : `<button class="btn btn-gold" id="btn-open-
cash">${ICON.plus} Abrir caixa</button>`)}

    ${!session ? emptyState('O caixa de hoje ainda não foi aberto.', 'Abra o caixa
informando o valor inicial em dinheiro.') : `
      <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(180px,1fr));gap:14px;margin-bottom:18px;">
        ${statCard('Abertura', BRL(session.openingAmount), 'às '+new
Date(session.openedAt).toLocaleTimeString('pt-BR',{hour:'2-digit',minute:'2-
digit'}), 'ok'))}
        ${statCard('Faturamento do dia', BRL(totals.in), 'todas as formas de
pagamento', 'ok'))}
        ${statCard('Saídas', BRL(totals.out), 'no dia', 'bad')}
        ${statCard('Dinheiro na gaveta', BRL(totals.balance), 'abertura + dinheiro
- saídas', 'ok')}
      </div>

      <div class="card" style="padding:16px 18px;margin-bottom:22px;">
        <div style="font-size:12.5px;color:var(--muted-2);margin-
bottom:12px;">Entradas por forma de pagamento</div>
        <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(130px,1fr));gap:14px;">
          ${PAYMENT_METHODS.map(m=>`
            <div>
              <div style="font-size:11.5px;color:var(--muted-

```

```

2);display:flex;align-items:center;gap:5px;">
    ${m.label}${m.inDrawer?`<span style="font-size:9.5px;color:var(--
gold-bright);">na gaveta</span>`:''}
    </div>
    <div class="font-mono" style="font-
size:17px;color:${totals.byMethod[m.id]>0?(m.inDrawer?'var(--gold-bright)':'var(--
ink)'):'var(--muted-2)'};">${BRL(totals.byMethod[m.id]||0)}</div>
    </div>
    `).join('')}
</div>
</div>

<div style="display:flex;gap:10px;margin-bottom:18px;">
    <button class="btn btn-ghost" style="color:var(--ok);border-color:var(--
ok);" id="btn-add-in">${ICON.arrowUp} Lançar entrada</button>
    <button class="btn btn-ghost" style="color:var(--bad);border-color:var(--
bad);" id="btn-add-out">${ICON.arrowDown} Lançar saída</button>
    <button class="btn btn-ghost" id="btn-sangria">${ICON.cash}
Sangria</button>
</div>

<div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Movimentações de hoje</div>
<div style="display:flex;flex-direction:column;gap:7px;margin-bottom:26px;">
    ${session.entries.length===0 ? emptyState('Nenhuma movimentação ainda.') :
session.entries.slice().reverse().map(e=>`
    <div style="display:flex;align-items:center;gap:12px;padding:10px
14px;background:var(--panel-2);border:1px solid var(--border);border-radius:9px;">
        <div style="color:${e.type==='entrada'?'var(--ok)':'var(--
bad)'};">${e.type==='entrada'?ICON.arrowUp:ICON.arrowDown}</div>
        <div style="flex:1;">
            <div style="font-size:13.5px;">${esc(e.desc)}</div>
            <div style="font-size:11.5px;color:var(--muted-2);">
                ${e.time} · ${methodLabelPay(e.method)} ${e.auto?'· automático':'·
manual'}
                ${e.estorno?' · <span style="color:var(--
warn);">estorno</span>':''}
                ${e.sangria?' · <span style="color:var(--
muted);">sangria</span>':''}
            </div>
        </div>
        <div class="font-mono" style="color:${e.type==='entrada'?'var(--
ok)':'var(--bad)'};font-size:14px;">${e.type==='entrada'?'+':'-'} ${BRL(e.amount)}
</div>
    </div>
    `).join('')}
</div>

```

```

    `}

    <div class="divider-blade" style="margin:8px 0 22px;"></div>
    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Histórico de caixas fechados</div>
    ${history.length===0 ? emptyState('Nenhum caixa fechado ainda.') : `
    <table>
      <thead><tr><th>Data</th><th>Abertura</th><th>Entradas</th><th>Saídas</th>
<th>Fechamento (esperado)</th><th>Contado</th><th>Diferença</th></tr></thead>
      <tbody>
        ${history.map(h=>{
          const t = sessionTotals(h);
          const diff = (h.closingCounted ?? t.balance) - t.balance;
          return `<tr>
            <td>${new Date(h.date+'T00:00').toLocaleDateString('pt-BR')}</td>
            <td class="font-mono">${BRL(h.openingAmount)}</td>
            <td class="font-mono" style="color:var(--ok);">${BRL(t.in)}</td>
            <td class="font-mono" style="color:var(--bad);">${BRL(t.out)}</td>
            <td class="font-mono">${BRL(t.balance)}</td>
            <td class="font-mono">${h.closingCounted!=null?
BRL(h.closingCounted):'-'}</td>
            <td class="font-mono" style="color:${diff===0?'var(--
muted)':'diff>0?'var(--ok)':'var(--bad)'};">
              ${diff===0?'-':(diff>0?'+' : '' )+BRL(diff)}
              ${h.obsFechamento?`<div style="font-size:10.5px;color:var(--muted-
2);font-family:inherit;font-weight:400;margin-top:2px;max-width:160px;white-
space:normal;">${esc(h.obsFechamento)}</div>`:''}
            </td>
          </tr>`;
        }).join('')}
      </tbody>
    </table>
    `}
  `;

  view.querySelector('#btn-open-cash')?.addEventListener('click', openCashModal);
  view.querySelector('#btn-close-cash')?.addEventListener('click',
closeCashModal);
  view.querySelector('#btn-add-in')?.addEventListener('click',
()=>addEntryModal('entrada'));
  view.querySelector('#btn-add-out')?.addEventListener('click',
()=>addEntryModal('saida'));
  view.querySelector('#btn-sangria')?.addEventListener('click',
()=>sangriaModal(view));
}

// Quanto ficou na gaveta no último fechamento
function saldoUltimoFechamento(){

```

```

const fechados = (state.cashSessions||[])
    .filter(c=>c.status==='fechado')
    .sort((a,b)=>(b.closedAt||b.date||'').localeCompare(a.closedAt||a.date||''));
if(!fechados.length) return null;
const ult = fechados[0];
// o que foi contado de verdade vale mais que o calculado
const valor = (ult.closingCounted!=null) ? Number(ult.closingCounted) :
sessionTotals(ult).balance;
return { valor, data: ult.date, contado: ult.closingCounted!=null };
}

function openCashModal(){
    const ant = saldoUltimoFechamento();
    const sugerido = ant ? ant.valor : 0;
    openModal(`
        <div style="padding:22px;">
            <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:16px;">
                <div class="font-display" style="font-size:19px;font-weight:600;">Abrir
caixa</div>
                <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
            </div>
            ${ant ? `
                <div class="card" style="padding:13px 15px;background:var(--panel-
2);margin-bottom:14px;">
                    <div style="font-size:12px;color:var(--muted-2);margin-
bottom:4px;">Sobrou no último fechamento (${new
Date(ant.data+'T12:00').toLocaleDateString('pt-BR')})</div>
                    <div class="font-mono" style="font-size:20px;color:var(--gold-
bright);">${BRL(ant.valor)}</div>
                </div>
                ` : `
                <div style="font-size:12.5px;color:var(--muted-2);margin-bottom:14px;line-
height:1.6;">
                    Primeiro caixa do sistema. Informe quanto tem de troco na gaveta agora.
                </div>
            `}
            <label>Valor na gaveta agora</label>
            <input id="f-open" type="number" step="0.01" min="0"
value="${sugerido.toFixed(2)}" style="width:100%;margin-bottom:8px;">
            <div id="open-dif" style="font-size:12.5px;margin-bottom:14px;min-
height:18px;"></div>
            <button class="btn btn-gold" id="f-submit" style="width:100%;justify-
content:center;">Abrir caixa</button>
            ${ant?`<div style="font-size:11px;color:var(--muted-2);margin-top:10px;line-
height:1.6;">Já veio preenchido com o que sobrou ontem. Confira o dinheiro e

```

```

corrija se estiver diferente - a diferença fica registrada.</div>`:''}
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  const inp = root.querySelector('#f-open');
  const dif = root.querySelector('#open-dif');
  const conferir = ()=>{
    if(!ant){ dif.textContent=''; return; }
    const v = parseFloat(inp.value);
    if(isNaN(v)){ dif.textContent=''; return; }
    const d = v - ant.valor;
    if(Math.abs(d) < 0.01){ dif.innerHTML = `<span style="color:var(--ok);">✓
Bate com o fechamento anterior.</span>`; }
    else if(d < 0){ dif.innerHTML = `<span style="color:var(--bad);">⚠ Faltam
${BRL(Math.abs(d))} em relação ao fechamento anterior.</span>`; }
    else { dif.innerHTML = `<span style="color:var(--warn);">⚠ Sobrando
${BRL(d)} em relação ao fechamento anterior.</span>`; }
  };
  inp.addEventListener('input', conferir);
  conferir();

  root.querySelector('#f-submit').addEventListener('click', ()=>{
    const val = parseFloat(inp.value);
    if(isNaN(val) || val<0){ showToast('Informe um valor válido.');
```

```

        <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:16px;">
            <div class="font-display" style="font-size:19px;font-weight:600;">Fechar caixa</div>
            <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}</button>
        </div>
        <div style="font-size:13.5px;color:var(--muted);margin-bottom:14px;line-height:1.7;">
            Abertura: <span class="font-mono" style="color:var(--ink);">${BRL(session.openingAmount)}</span><br>
            Entradas: <span class="font-mono" style="color:var(--ok);">${BRL(totals.in)}</span><br>
            Saídas: <span class="font-mono" style="color:var(--bad);">${BRL(totals.out)}</span><br>
            Saldo esperado: <span class="font-mono" style="color:var(--ink);">${BRL(totals.balance)}</span>
        </div>
        <label>Conte o dinheiro e informe quanto tem na gaveta</label>
        <input id="f-count" type="number" step="0.01" min="0" placeholder="0,00" style="width:100%;margin-bottom:8px;">
        <div id="close-dif" style="font-size:13px;margin-bottom:14px;min-height:20px;"></div>
        <div><label>Observação (opcional)</label><input id="f-obs" placeholder="Ex: faltou troco de uma venda" style="width:100%;margin-bottom:16px;"></div>
        <button class="btn btn-gold" id="f-submit" style="width:100%;justify-content:center;">Confirmar fechamento</button>
    </div>
    `, root=>{
        root.querySelector('#m-close').addEventListener('click', closeModal);
        const inp = root.querySelector('#f-count');
        const dif = root.querySelector('#close-dif');
        const conferir = ()=>{
            const v = parseFloat(inp.value);
            if(isNaN(v)){ dif.textContent=''; return null; }
            const d = v - totals.balance;
            if(Math.abs(d) < 0.01){
                dif.innerHTML = `<span style="color:var(--ok);">✓ Bateu certinho com o esperado.</span>`;
            } else if(d < 0){
                dif.innerHTML = `<div style="padding:10px 12px;border-radius:8px;background:var(--bad-bg);color:var(--bad);line-height:1.5;">
                    <b>Faltando ${BRL(Math.abs(d))}</b><br><span style="font-size:11.5px;">Confira se algum troco saiu sem registro ou se faltou lançar uma saída.</span>
                </div>`;
            } else {

```

```

        dif.innerHTML = `<div style="padding:10px 12px;border-
radius:8px;background:var(--panel-2);color:var(--warn);line-height:1.5;">
        <b>Sobrando ${BRL(d)}</b><br><span style="font-size:11.5px;">Pode ser
uma venda em dinheiro que não foi lançada.</span>
        </div>`;
    }
    return d;
};
inp.addEventListener('input', conferir);

root.querySelector('#f-submit').addEventListener('click', ()=>{
    const val = parseFloat(inp.value);
    if(isNaN(val) || val<0){ showToast('Informe o valor contado.');
```

 return; }
 const d = val - totals.balance;
 if(Math.abs(d) >= 0.01){
 const txt = d<0 ? `Está faltando \${BRL(Math.abs(d))} na gaveta.` : `Está
sobrando \${BRL(d)} na gaveta.`;
 if(!confirm(`\${txt}\n\nFechar o caixa assim mesmo? A diferença fica
registrada no histórico.``)) return;
 }
 update(s=>{
 const sess = s.cashSessions.find(c=>c.status==='aberto');
 sess.status='fechado'; sess.closedAt=new Date().toISOString();
 sess.closingCounted=val;
 sess.diferencaFechamento = d;
 sess.obsFechamento = root.querySelector('#f-obs').value.trim();
 });
 closeModal();
 showToast(Math.abs(d)>=0.01
 ? `Caixa fechado com \${d<0?'falta':'sobra'} de \${BRL(Math.abs(d))}.`
 : 'Caixa fechado - tudo certo!');
 });
});
}
/* Sangria: tira dinheiro da gaveta durante o dia (banco, cofre, fornecedor) */
function sangriaModal(view){
 if(!isOwnerLevel() && userSession.role!=='recepcionista'){ showToast('Você não
tem permissão para fazer sangria.');
 return; }
 const session = openSession();
 if(!session){ showToast('Abra o caixa primeiro.');
 return; }
 const totals = sessionTotals(session);
 openModal(`
 <div style="padding:22px;">
 <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
 <div class="font-display" style="font-size:19px;font-
weight:600;">Sangria</div>

```

        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
    </button>
    </div>
    <div style="font-size:12.5px;color:var(--muted);margin-bottom:16px;line-
height:1.6;">
        Retirada de dinheiro da gaveta. Use para levar ao banco, guardar no cofre
        ou pagar algo em espécie.
    </div>
    <div class="card" style="padding:12px 15px;background:var(--panel-2);margin-
bottom:16px;">
        <div style="font-size:11.5px;color:var(--muted-2);">Dinheiro na gaveta
        agora</div>
        <div class="font-mono" style="font-size:20px;color:var(--gold-
bright);">${BRL(totals.balance)}</div>
    </div>
    <div style="display:flex;flex-direction:column;gap:13px;">
        <div><label>Valor a retirar</label><input id="sg-valor" type="number"
step="0.01" min="0" placeholder="0,00" style="width:100%;"></div>
        <div><label>Para onde vai</label>
            <input id="sg-destino" list="sg-destinos" placeholder="Ex: Depósito no
banco" style="width:100%;">
            <datalist id="sg-destinos">
                <option value="Depósito no banco"><option value="Cofre">
                <option value="Pagamento de fornecedor"><option value="Retirada do
dono">
            </datalist>
        </div>
        <div id="sg-aviso" style="font-size:12.5px;min-height:18px;"></div>
        <button class="btn btn-gold" id="sg-save" style="justify-
content:center;">Registrar sangria</button>
        <div style="font-size:11px;color:var(--muted-2);text-align:center;line-
height:1.5;">
            Sai do saldo da gaveta, mas não conta como despesa do negócio – o
            dinheiro só mudou de lugar.
        </div>
    </div>
    </div>
    `, root=>{
        root.querySelector('#m-close').addEventListener('click', closeModal);
        const inp = root.querySelector('#sg-valor');
        const aviso = root.querySelector('#sg-aviso');
        const conferir = ()=>{
            const v = parseFloat(inp.value);
            if(isNaN(v) || v<=0){ aviso.textContent=''; return; }
            if(v > totals.balance){
                aviso.innerHTML = `<span style="color:var(--bad);">⚠ Só há
                ${BRL(totals.balance)} na gaveta.</span>`;
            }
        }
    }

```

```

    } else {
      aviso.innerHTML = `<span style="color:var(--muted);">Restam
    ${BRL(totals.balance - v)} na gaveta.</span>`;
    }
  };
  inp.addEventListener('input', conferir);

  root.querySelector('#sg-save').addEventListener('click', ()=>{
    const valor = parseFloat(inp.value);
    const destino = root.querySelector('#sg-destino').value.trim() || 'Sangria';
    if(isNaN(valor) || valor<=0){ showToast('Informe um valor válido.');
```

```

</option>`).join('')}
    </select>
    <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">Só o que
for em dinheiro entra no saldo físico da gaveta.</div></div>
    <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">Lançar</button>
  </div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#f-submit').addEventListener('click', ()=>{
    const desc = root.querySelector('#f-desc').value.trim();
    const amount = parseFloat(root.querySelector('#f-amount').value);
    const method = root.querySelector('#f-method').value;
    if(!desc || isNaN(amount) || amount<=0){ showToast('Preencha descrição e
valor.');
```

valor.');

```

    return; }
    update(s=>{
      s.cashSessions.find(c=>c.status==='aberto').entries.push({id:uid(), type,
method, desc, amount, time:new Date().toString().slice(0,5), auto:false});
    });
    closeModal();
  });
});
}

/* ===== COMISSÕES ===== */
function periodLabel(period){
  const [y,m] = period.split('-');
  const meses =
['Janeiro','Fevereiro','Março','Abril','Maio','Junho','Julho','Agosto','Setembro','
  return `${meses[parseInt(m)-1]}/${y}`;
}
function monthAlreadyClosed(period){
  return state.commissionPayments.some(c=>c.source==='pote' && c.period===period);
}
/* Congela o valor cobrado nos atendimentos antigos.
  Depois disso, mudar plano/status não altera mais comissões do passado. */
function congelarValoresAntigos(view){
  if(!isOwnerLevel()){ showToast('Apenas o dono ou administrador pode fazer
isso.');
```

isso.');

```

  return; }
  const pendentes = state.appointments.filter(a=>a.status==='concluido' &&
a.valorCobrado===null);
  if(pendentes.length===0){ showToast('Todos os atendimentos já estão com o valor
registrado.');
```

registrado.');

```

  return; }

// calcula antes de gravar, para mostrar o resumo
const prev = pendentes.map(a=>{

```

```

    const svcs = appointmentServices(a);
    const multi = svcs.length ? precoParaCobrarMultiplo(a) : null;
    return { a, svcs, valor: multi ? multi.total : 0, coberto: multi ?
multi.todosCobertos : false };
  });
  const precoTabelaDe = a =>
appointmentServices(a).reduce((t,s)=>t+Number(s.price),0);
  const cobertos = prev.filter(x=>x.coberto).length;
  const comDesconto = prev.filter(x=>!x.coberto && x.svcs.length && x.valor <
precoTabelaDe(x.a) && x.valor > 0).length;
  const cheios = prev.length - cobertos - comDesconto;

  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
        <div class="font-display" style="font-size:19px;font-
weight:600;">Registrar valores dos atendimentos</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
      </div>
      <div style="font-size:12.5px;color:var(--muted);line-height:1.7;margin-
bottom:16px;">
        Atendimentos concluídos antes desta atualização não têm o valor cobrado
registrado –
        a comissão deles é recalculada toda hora. Isso significa que, se você
mudar um plano depois,
        a comissão de dias passados mudaria junto.<br><br>
        Este botão <b>grava o valor de agora</b> em cada um, congelando o passado.
      </div>

      <div class="card" style="padding:15px 17px;background:var(--panel-2);margin-
bottom:16px;">
        <div style="font-size:12px;color:var(--muted-2);margin-
bottom:10px;">${pendentes.length} atendimento(s) serão registrados:</div>
        <div style="display:flex;flex-direction:column;gap:7px;font-size:13px;">
          <div style="display:flex;justify-content:space-between;"><span
style="color:var(--muted);">Cobertos pelo clube (R$ 0,00)</span><b class="font-
mono" style="color:var(--ok);">${cobertos}</b></div>
          ${comDesconto?`<div style="display:flex;justify-content:space-between;">
<span style="color:var(--muted);">Com desconto do clube</span><b class="font-mono"
style="color:var(--warn);">${comDesconto}</b></div>`:''}
          <div style="display:flex;justify-content:space-between;"><span
style="color:var(--muted);">Valor cheio</span><b class="font-mono">${cheios}</b>
</div>
        </div>
      </div>

```

```
<div style="font-size:11.5px;color:var(--muted-2);line-height:1.6;margin-bottom:16px;">
```

Não altera nada que já esteja registrado, não mexe no caixa e não apaga atendimento nenhum.

Só preenche o que estava faltando.

```
</div>
```

```
<button class="btn btn-gold" id="cv-ok" style="width:100%;justify-content:center;padding:13px;">Registrar os ${pendentes.length} valores</button>
```

```
<button class="btn btn-ghost" id="cv-cancelar" style="width:100%;justify-content:center;margin-top:9px;font-size:12.5px;">Agora não</button>
```

```
</div>
```

```
` , root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#cv-cancelar').addEventListener('click', closeModal);
  root.querySelector('#cv-ok').addEventListener('click', ()=>{
    let n = 0;
    update(s=>{
      s.appointments.forEach(ap=>{
        if(ap.status!=='concluido' || ap.valorCobrado!==null) return;
        const svcs = appointmentServices(ap);
        const multi = svcs.length ? precoParaCobrarMultiplo(ap) : null;
        ap.valorCobrado = multi ? multi.total : 0;
        ap.cobertoPeloClube = multi ? multi.todosCobertos : false;
        if(multi){
          ap.valorPorServico = multi.itens.map(i=>({ serviceId:i.service.id,
valor:i.calc.basePrice, coberto:i.calc.planoAtivo }));
        }
        n++;
      });
    });
    closeModal();
    showToast(`${n} atendimento(s) registrados. O passado está congelado.`);
    if(view) renderComissoes(view);
  });
});
}
```

```
function renderComissoes(view){
  const fichasBreakdown = clubFichasBreakdown();
  const period = currentPeriod();
  const closed = monthAlreadyClosed(period);
  const potRows = servingProfessionals().map(p=>({prof:p, owed:
commissionOwed(p.id).potOwed})).filter(r=>r.owed>0);
  const totalPotOwed = potRows.reduce((a,r)=>a+r.owed,0);
  const visibleProfs = userSession.role==='profissional' ?
```

```

state.professionals.filter(p=>p.id===userSession.professionalId) :
servingProfessionals();
  const myProf = userSession.role==='profissional' ?
state.professionals.find(p=>p.id===userSession.professionalId) : null;
  const myWithdrawals = myProf ?
state.withdrawalRequests.filter(w=>w.professionalId===myProf.id).slice().reverse()
: [];
  const pendingWithdrawals =
state.withdrawalRequests.filter(w=>w.status==='pendente').slice().reverse();

view.innerHTML = `
  ${pageHeader('Comissões', 'Serviços avulsos pagos a qualquer momento · pote do
clube pago no fechamento mensal')}

  ${(()=>{
    if(!isOwnerLevel()) return '';
    const pend = state.appointments.filter(a=>a.status==='concluido' &&
a.valorCobrado===null).length;
    if(pend===0) return '';
    return `<div class="card" style="padding:15px 18px;margin-
bottom:16px;border-color:var(--warn);">
      <div style="display:flex;justify-content:space-between;align-
items:center;gap:12px;flex-wrap:wrap;">
        <div style="font-size:12.5px;line-height:1.6;">
          <b>${pend} atendimento(s) sem o valor registrado</b><br>
          <span style="color:var(--muted-2);">São de antes desta atualização. A
comissão deles é recalculada toda vez –
            se você mudar um plano depois, mexeria no passado. Registre o valor
para congelar.</span>
        </div>
        <button class="btn btn-gold" id="btn-congelar" style="flex-
shrink:0;">Registrar valores</button>
      </div>
    </div>`;
  })()}
  ${isOwnerLevel() ? `
    <div class="card" style="padding:14px 18px;margin-bottom:18px;">
      <div style="font-size:12.5px;color:var(--muted);">Comissão por serviço –
configure o % específico de cada profissional na aba Profissionais.</div>
    </div>` : ''}

  ${myProf ? `
    ${(()=>{
      const cm = commissionOwed(myProf.id);
      const ganho = monthlyEarnedForProf(myProf.id);
      return `<div class="card" style="padding:18px 20px;margin-
bottom:20px;border-color:${cm.owed<0?'var(--bad)':'var(--gold)'}";>

```

```

    <div style="display:flex;justify-content:space-between;align-items:flex-
start;flex-wrap:wrap;gap:14px;">
        <div style="min-width:200px;">
            <div style="font-size:11.5px;color:var(--muted-2);">${cm.owed<0?'Saldo
devedor':'A receber agora'}</div>
            <div class="font-mono" style="font-size:26px;color:${cm.owed<0?'var(--
bad)':'var(--gold-bright)'};">${BRL(cm.owed)}</div>
            ${cm.owed<0?`<div style="font-size:11.5px;color:var(--bad);margin-
top:4px;">Vale acima da comissão ganha – vai sendo abatido conforme novos
atendimentos entram.</div>`:''}
            <div style="display:flex;flex-direction:column;gap:4px;margin-
top:12px;padding-top:11px;border-top:1px solid var(--border);font-size:12px;">
                <div style="display:flex;justify-content:space-between;gap:14px;">
                    <span style="color:var(--muted);">Comissão em
${periodLabel(period)}</span>
                    <span class="font-mono">${BRL(ganho)}</span>
                </div>
                ${cm.paidService>0?`<div style="display:flex;justify-content:space-
between;gap:14px;">
                    <span style="color:var(--muted);">Já recebido</span>
                    <span class="font-mono" style="color:var(--bad);">-
${BRL(cm.paidService)}</span></div>`:''}
                ${cm.paidVale>0?`<div style="display:flex;justify-content:space-
between;gap:14px;">
                    <span style="color:var(--muted);">Vale adiantado</span>
                    <span class="font-mono" style="color:var(--bad);">-
${BRL(cm.paidVale)}</span></div>`:''}
                ${cm.paidSaque>0?`<div style="display:flex;justify-content:space-
between;gap:14px;">
                    <span style="color:var(--muted);">Saques pagos</span>
                    <span class="font-mono" style="color:var(--bad);">-
${BRL(cm.paidSaque)}</span></div>`:''}
                ${cm.paidPote>0?`<div style="display:flex;justify-content:space-
between;gap:14px;">
                    <span style="color:var(--muted);">Pote já pago</span>
                    <span class="font-mono" style="color:var(--bad);">-
${BRL(cm.paidPote)}</span></div>`:''}
            </div>
        </div>
        <button class="btn btn-gold" id="btn-request-withdrawal" style="flex-
shrink:0;" ${cm.owed<=0?'disabled title="Sem saldo disponível para
saque"':''}>${ICON.cash} Solicitar saque</button>
    </div>
</div>`;
    })() }
    ${myWithdrawals.length>0 ? `
    <div class="card" style="padding:16px 18px;margin-bottom:20px;">

```

```


Meus pedidos de saque</div>


${myWithdrawals.map(w=>`
    <div style="display:flex;justify-content:space-between;align-items:center;font-size:13px;padding:6px 0;border-bottom:1px solid var(--border);">
      <span>${BRL(w.amount)} <span style="color:var(--muted-2);font-size:11.5px;">- ${new Date(w.createdAt).toLocaleDateString('pt-BR')}</span></span>
      <span class="pill ${w.status==='pago'? 'pill-ok':w.status==='recusado'? 'pill-bad': 'pill-warn'}">${w.status==='pago'? 'Pago':w.status==='recusado'? 'Recusado': 'Pendente'}</span>
    </div>
  `).join('')}
</div>
` : ''}
` : ''}

${isOwnerLevel() && pendingWithdrawals.length>0 ? `


```

```

bottom:10px;">${esc(p.name)} <span style="font-weight:400;font-
size:12px;color:var(--muted-2);">- comissão base de ${p.commission}%</span></div>
    <div style="display:flex;flex-wrap:wrap;gap:10px;">
        <div style="flex:1;min-width:240px;background:var(--panel-
2);border:1px solid var(--border);border-radius:9px;padding:12px
14px;display:flex;justify-content:space-between;align-items:center;gap:10px;">
            <div>
                <div style="font-size:11.5px;color:var(--muted-2);">Serviços
avulsos</div>
                <div class="font-mono" style="font-
size:15px;color:${c.serviceOwed<0?'var(--bad)':(c.serviceOwed>0?'var(--
warn)':'var(--muted)')}";">${BRL(c.serviceOwed)} <span style="font-
size:11px;color:var(--muted-2);">${c.serviceOwed<0?'em débito':'a pagar'}</span>
</div>
                ${c.serviceOwed<0 ? `<div style="font-size:11px;color:var(--
bad);margin-top:3px;">Vale maior que a comissão - será abatido conforme ele
atender mais clientes.</div>` : ''}
                ${c.paidVale>0 || c.paidSaque>0} ? `<div style="font-
size:11px;color:var(--muted-2);margin-top:3px;">
                    ${BRL(c.serviceEarned)} de comissão${c.paidService>0?` -
${BRL(c.paidService)} já pago`:''}${c.paidVale>0?` - ${BRL(c.paidVale)} em
vale`:''}${c.paidSaque>0?` - ${BRL(c.paidSaque)} em saque`:''}
                    </div>` : ''}
                </div>
                ${isOwnerLevel() ? `<button class="btn btn-ghost" style="padding:8px
12px;" data-vale="${p.id}">${ICON.cash} Vale</button>` : ''}
                ${isOwnerLevel() ? `<button class="btn btn-gold" style="padding:8px
12px;" ${c.serviceOwed<=0?'disabled':''} data-pay-
service="${p.id}">Pagar</button>` : ''}
            </div>
            <div style="flex:1;min-width:240px;background:var(--panel-
2);border:1px solid var(--border);border-radius:9px;padding:12px 14px;">
                <div style="font-size:11.5px;color:var(--muted-2);">Pote do clube ·
${periodLabel(period)}</div>
                <div class="font-mono" style="font-
size:15px;color:${c.potOwed>0?'var(--gold-bright)':'var(--muted)'}";">
                    ${BRL(c.potOwed)}
                    ${(()=>{ const f =
(fichasBreakdown.rows.find(r=>r.prof.id===p.id)||{}).fichas || 0;
                    return f>0 ? `<span style="font-size:12px;color:var(--gold-
bright);"> · ${f} ficha${f>1?'s':''}</span>` : ''; })()}
                    <span style="font-size:11px;color:var(--muted-2);">${closed?'já
fechado':'incluído no fechamento mensal'}</span>
                </div>
            </div>
        </div>
    </div>
</div>`;

```

```

    }).join('')}
</div>

<div class="divider-blade" style="margin:26px 0 18px;"></div>
<div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:4px;flex-wrap:wrap;gap:8px;">
    <div class="font-display" style="font-size:16px;font-weight:600;">Pote do
clube – detalhamento de ${periodLabel(period)}</div>
</div>
<div style="font-size:12.5px;color:var(--muted-2);margin-
bottom:14px;">Atendimento de assinante coberto pelo plano <b>não gera comissão de
serviço</b> – o pagamento vem por aqui, pelas fichas. As fichas contam assim que o
profissional atende um assinante, mesmo antes da mensalidade entrar. O pote é
formado pelas mensalidades recebidas no mês: uma % fica para a barbearia e o resto
é dividido entre os profissionais, proporcional às fichas de cada um.</div>

${isOwnerLevel() ? (()=>{
    const { total, grupos } = diagnosticoFichasAgrupado();
    if(total===0) return '';
    return `<div class="card" style="padding:13px 16px;margin-
bottom:14px;border-color:var(--bad);">
        <div style="font-size:12.5px;color:var(--bad);font-weight:600;margin-
bottom:10px;">
            ${total} atendimento(s) de assinante não geraram ficha este mês
        </div>
        <div style="display:flex;flex-direction:column;gap:10px;">
            ${grupos.map((g,gi)=>`
                <div style="display:flex;justify-content:space-between;align-
items:flex-start;gap:10px;flex-wrap:wrap;">
                    <div style="font-size:12px;color:var(--muted);line-
height:1.6;flex:1;min-width:220px;">
                        • ${esc(g.motivo)} – ${g.apps.length} atendimento(s):
${g.apps.slice(0,4).map(a=>esc(a.client)+'
('+esc(profName(a.professionalId))+')').join(', ')}${g.apps.length>4?` e mais
${g.apps.length-4}`:''}
                    </div>
                    <div style="display:flex;gap:6px;flex-wrap:wrap;flex-shrink:0;">
                        ${g.tipo==='servico_sem_fichas'?`<button class="btn btn-ghost"
data-fix-fichas-servico="${esc(g.refId)}" style="padding:6px 12px;font-
size:12px;">Ajustar fichas</button>`:''}
                        ${g.tipo==='servico_sem_fichas' || g.tipo==='servico_excluido'?
`<button class="btn btn-gold" data-trocar-servico-
grupo="${esc(g.tipo)}|${esc(g.refId)}|'|'" style="padding:6px 12px;font-
size:12px;">Trocar serviço</button>`:''}
                        ${g.tipo==='prof_nao_atende'?`<button class="btn btn-gold" data-
fix-fichas-prof="${esc(g.refId)}" style="padding:6px 12px;font-
size:12px;">Corrigir profissional</button>`:''}

```

```

        </div>
    </div>
    `).join('')
</div>
<div style="font-size:11px;color:var(--muted-2);margin-top:10px;line-
height:1.6;">
    "Trocar serviço" corrige os atendimentos que foram marcados com o
    serviço errado (ex: alguém esqueceu de escolher e ficou no padrão). Ao corrigir,
    as fichas são contadas automaticamente – não precisa refazer nada.
</div>
</div>`;
})) () : ''}

    ${fichasBreakdown.potTotal===0 && fichasBreakdown.totalFichas>0 ? `
    <div class="card" style="padding:13px 16px;margin-bottom:14px;border-
    color:var(--warn);">
        <div style="font-size:12.5px;color:var(--muted);line-height:1.6;">
            <b style="color:var(--warn);">${fichasBreakdown.totalFichas} ficha(s) já
            acumuladas</b>, mas nenhuma mensalidade entrou este mês ainda –
            por isso o valor de cada um está zerado. Assim que os assinantes
            pagarem, o pote é dividido por essas fichas.
        </div>
    </div>
    ` : ''}

    <div class="card" style="padding:18px 20px;margin-bottom:16px;">
        <div style="display:grid;grid-template-columns:repeat(auto-
        fit,minmax(150px,1fr));gap:14px;">
            <div><div style="font-size:11px;color:var(--muted-2);">Pote total do
            mês</div><div class="font-mono" style="font-size:18px;color:var(--gold-
            bright);">${BRL(fichasBreakdown.potTotal)}</div></div>
            <div><div style="font-size:11px;color:var(--muted-2);">Fatia da barbearia
            (${100-fichasBreakdown.staffPercent}%)</div><div class="font-mono" style="font-
            size:18px;">${BRL(fichasBreakdown.potBarbearia)}</div></div>
            <div><div style="font-size:11px;color:var(--muted-2);">Fatia dos
            profissionais (${fichasBreakdown.staffPercent}%)</div><div class="font-mono"
            style="font-size:18px;color:var(--ok);">${BRL(fichasBreakdown.potStaff)}</div>
            </div>
            <div><div style="font-size:11px;color:var(--muted-2);">Total de fichas no
            mês</div><div class="font-mono" style="font-
            size:18px;">${fichasBreakdown.totalFichas}</div></div>
        </div>
        ${isOwnerLevel() ? `
            <div style="display:flex;gap:10px;align-items:flex-end;margin-
            top:16px;padding-top:16px;border-top:1px solid var(--border);">
                <div><label>% do pote para os profissionais</label><input id="f-club-
                staff-pct" type="number" min="0" max="100" value="${fichasBreakdown.staffPercent}"

```

```

style="width:100px;"></div>
    <button class="btn btn-ghost" id="btn-save-club-pct">Salvar</button>
</div>
` : ''}
</div>

${(()=>{
  // Só o dono/admin vê o pote inteiro. Profissional vê só a própria
  // linha – o valor que os colegas fizeram não é dele pra ver.
  const rowsVisiveis = isOwnerLevel()
    ? fichasBreakdown.rows
    : fichasBreakdown.rows.filter(r=>r.prof.id === myProf?.id);
  if(rowsVisiveis.length===0){
    return isOwnerLevel()
      ? emptyState('Nenhuma ficha gerada neste mês ainda.', 'Assim que um
profissional concluir um atendimento de um assinante pago, as fichas do serviço
aparecem aqui.')
      : emptyState('Você ainda não tem fichas neste mês.', 'Assim que atender
um assinante pago, sua parte do pote aparece aqui.');
```

```
    <div style="font-size:12.5px;color:var(--muted);margin-top:3px;">${closed ? 'O pote deste mês já foi fechado e pago.' : `Paga de uma vez a comissão do pote de todos os profissionais referente a ${periodLabel(period)}.`}</div>
```

```
  </div>
```

```
  <div style="display:flex;align-items:center;gap:16px;">
```

```
    <div style="text-align:right;"><div style="font-size:11px;color:var(--muted-2);">Total do fechamento</div><div class="font-mono" style="font-size:18px;color:var(--gold-bright);">${BRL(totalPotOwed)}</div></div>
```

```
    <button class="btn btn-gold" id="btn-close-month" ${closed || totalPotOwed<=0 ? 'disabled':''}>${ICON.check} Fechar mês do clube</button>
```

```
  </div>
```

```
</div>
```

```
</div>` : ''}
```

```
    ${state.commissionPayments.filter(x=>x.source==='vale').length>0 ? `
```

```
      <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:10px;">Vales dados</div>
```

```
      <table style="margin-bottom:26px;">
```

```
        <thead><tr><th>Data</th><th>Profissional</th><th>Origem</th><th>Observação</th><th>Valor</th></tr></thead>
```

```
        <tbody>
```

```
    ${state.commissionPayments.filter(x=>x.source==='vale').slice().reverse()
```

```
      .filter(x=>isOwnerLevel() ||
```

```
    x.professionalId===userSession.professionalId)
```

```
      .map(x=>`<tr>
```

```
        <td>${new Date(x.date).toLocaleDateString('pt-BR')}</td>
```

```
        <td>${esc(profName(x.professionalId))}</td>
```

```
        <td><span class="pill pill-muted">${x.origem==='banco'? 'Conta bancária': 'Caixa'}</span></td>
```

```
        <td style="color:var(--muted);">${esc(x.note) || '-'}</td>
```

```
        <td class="font-mono" style="color:var(--bad);">- ${BRL(x.amount)}
```

```
</td>
```

```
      </tr>`).join('')}
```

```
    </tbody>
```

```
  </table>
```

```
` : ''}
```

```
  <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:10px;">Histórico de pagamentos de serviços</div>
```

```
    ${state.commissionPayments.filter(c=>c.source==='servico').length===0 ? emptyState('Nenhum pagamento de comissão de serviço registrado ainda.') : `
```

```
      <table style="margin-bottom:26px;">
```

```
        <thead><tr><th>Data</th><th>Profissional</th><th>Valor pago</th></tr></thead>
```

```
        <tbody>
```

```

    ${state.commissionPayments.filter(c=>c.source==='servico').slice().reverse().map(c=>`
        <td>${new Date(c.date).toLocaleDateString('pt-BR')}</td>
        <td>${profName(c.professionalId)}</td>
        <td class="font-mono">${BRL(c.amount)}</td>
    </tr>`).join('')}
    </tbody>
</table>
`}

    <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:10px;">Histórico de fechamentos do pote</div>
    ${state.commissionPayments.filter(c=>c.source==='pote').length===0 ?
    emptyState('Nenhum fechamento mensal do clube realizado ainda.') : `
        <table>
            <thead><tr><th>Mês</th><th>Data do fechamento</th><th>Profissional</th>
            <th>Valor pago</th></tr></thead>
            <tbody>

    ${state.commissionPayments.filter(c=>c.source==='pote').slice().reverse().map(c=>`
        <td>${periodLabel(c.period)}</td>
        <td>${new Date(c.date).toLocaleDateString('pt-BR')}</td>
        <td>${profName(c.professionalId)}</td>
        <td class="font-mono">${BRL(c.amount)}</td>
    </tr>`).join('')}
    </tbody>
</table>
`}

    `;
    view.querySelector('#btn-congelar')?.addEventListener('click',
    ()=>congelarValoresAntigos(view));
    view.querySelectorAll('[data-pay-
    service]').forEach(b=>b.addEventListener('click',
    ()=>payServiceCommission(b.dataset.payService));
    view.querySelectorAll('[data-vale]').forEach(b=>b.addEventListener('click',
    ()=>valeModal(b.dataset.vale, view));
    view.querySelector('#btn-close-month')?.addEventListener('click',
    closeClubMonthModal);
    view.querySelector('#btn-request-withdrawal')?.addEventListener('click',
    ()=>withdrawalRequestModal(myProf));
    view.querySelector('#btn-save-club-pct')?.addEventListener('click', ()=>{
        const val = parseFloat(view.querySelector('#f-club-staff-pct').value);
        if(isNaN(val) || val<0 || val>100){ showToast('Informe um valor entre 0 e
        100.');
```

```

    view.querySelectorAll('[data-approve-withdrawal]').forEach(b=>b.addEventListener('click', ()=>{
        const w =
state.withdrawalRequests.find(x=>x.id===b.dataset.approveWithdrawal);
        if(!w) return;
        update(s=>{
            s.commissionPayments.push({id:uid(), professionalId:w.professionalId,
amount:w.amount, date:new Date().toISOString(), source:'saque'});
            const target = s.withdrawalRequests.find(x=>x.id===w.id);
            if(target){ target.status='pago'; target.resolvedAt=new
Date().toISOString(); }
        });
        showToast('Saque marcado como pago.');
```

renderComissoes(view);

```

    }));
    view.querySelectorAll('[data-decline-withdrawal]').forEach(b=>b.addEventListener('click', ()=>{
        update(s=>{
            const target =
s.withdrawalRequests.find(x=>x.id===b.dataset.declineWithdrawal);
            if(target){ target.status='recusado'; target.resolvedAt=new
Date().toISOString(); }
        });
        showToast('Pedido de saque recusado.');
```

renderComissoes(view);

```

    }));
    view.querySelectorAll('[data-fix-fichas-servico]').forEach(b=>b.addEventListener('click', ()=>{
        const svc = state.services.find(s=>s.id===b.dataset.fixFichasServico);
        if(!svc){ showToast('Serviço não encontrado.');
```

return; }

```

        svcModal(svc);
    }));
    view.querySelectorAll('[data-fix-fichas-prof]').forEach(b=>b.addEventListener('click', ()=>{
        const prof = state.professionals.find(p=>p.id===b.dataset.fixFichasProf);
        if(!prof){ showToast('Profissional não encontrado.');
```

return; }

```

        profModal(prof);
    }));
    view.querySelectorAll('[data-trocar-servico-grupo]').forEach(b=>b.addEventListener('click', ()=>{
        const [tipo, refId] = b.dataset.trocarServicoGrupo.split('|');
```

const { grupos } = diagnosticoFichasAgrupado();

```

        const g = grupos.find(x=>x.tipo===tipo && String(x.refId||'')===refId);
        if(!g || g.apps.length===0){ showToast('Nada para corrigir - já foi
resolvido.');
```

renderComissoes(view); return; }

```

        trocarServicoGrupoModal(g, view);
    }));

```

```

}
/* Corrige em lote atendimentos que ficaram com o serviço errado (ex: o
   profissional esqueceu de escolher e o sistema pegou o primeiro da
   lista). Troca todos de uma vez para o serviço certo. */
function trocarServicoGrupoModal(g, view){
  const idsAfetados = g.apps.map(a=>a.id);
  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
        <div class="font-display" style="font-size:19px;font-weight:600;">Trocar
serviço</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
      </div>
      <div style="font-size:12.5px;color:var(--muted);margin-bottom:14px;line-
height:1.6;">
        ${idsAfetados.length} atendimento(s) provavelmente foram registrados com o
serviço errado. Escolha o serviço correto e ele é aplicado a todos de uma vez.
      </div>
      <div style="display:flex;flex-direction:column;gap:6px;max-
height:30vh;overflow-y:auto;margin-bottom:16px;padding:10px 12px;background:var(--
panel-2);border-radius:9px;">
        ${g.apps.map(a=>`<div style="font-size:12px;color:var(--muted);">${new
Date(a.date+'T12:00').toLocaleDateString('pt-BR')} · ${esc(a.client)} ·
${esc(profName(a.professionalId))}</div>`).join('')}
      </div>
      <label>Serviço correto</label>
      <select id="tg-service" style="width:100%;margin-bottom:16px;">
        <option value="">- selecione -</option>
        ${state.services.map(s=>`<option value="${s.id}">${esc(s.name)} -
${BRL(s.price)}</option>`).join('')}
      </select>
      <button class="btn btn-gold" id="tg-save" style="width:100%;justify-
content:center;">Corrigir ${idsAfetados.length} atendimento(s)</button>
      <div style="font-size:11px;color:var(--muted-2);margin-top:10px;line-
height:1.6;">
        Isso não muda o valor já lançado no caixa nem a comissão já paga – só
corrige o registro do serviço, para as fichas contarem certinho daqui pra frente.
      </div>
    </div>
  `, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#tg-save').addEventListener('click', ()=>{
      const novoServId = root.querySelector('#tg-service').value;
      if(!novoServId){ showToast('Escolha o serviço correto.');
```

```

        s.appointments.forEach(ap=>{ if(idsAfetados.includes(ap.id)) ap.serviceId
= novoServId; });
    });
    closeModal();
    showToast(`${idsAfetados.length} atendimento(s) corrigido(s).`);
    if(view) renderComissoes(view);
    });
    });
}
function withdrawalRequestModal(prof){
    const owed = commissionOwed(prof.id).owed;
    if(owed<=0){ showToast(owed<0 ? 'Sem saldo - há vale a descontar ainda.' : 'Sem
saldo disponível para saque no momento.');
```

 return;
 }
 openModal(`
 <div style="padding:22px;">
 <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:16px;">
 <div class="font-display" style="font-size:19px;font-
weight:600;">Solicitar saque</div>
 <button class="btn btn-ghost" style="padding:6px;" id="m-close">\${ICON.x}
 </button>
 </div>
 <div style="font-size:12.5px;color:var(--muted);margin-bottom:14px;">Você
tem \${BRL(owed)} disponível a receber. O dono vai revisar e marcar como pago.
 </div>
 <div style="display:flex;flex-direction:column;gap:12px;">
 <div><label>Valor do saque (R\$)</label><input id="f-amount" type="number"
step="0.01" min="0" max="\${owed}" value="\${owed>0?owed.toFixed(2):''}"
style="width:100%;"></div>
 <div><label>Observação (opcional)</label><input id="f-note"
placeholder="Ex: preciso até sexta" style="width:100%;"></div>
 <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">Enviar solicitação</button>
 </div>
 </div>
 `, root=>{
 root.querySelector('#m-close').addEventListener('click', closeModal);
 root.querySelector('#f-submit').addEventListener('click', ()=>{
 const amount = parseFloat(root.querySelector('#f-amount').value);
 const note = root.querySelector('#f-note').value.trim();
 if(isNaN(amount) || amount<=0){ showToast('Informe um valor válido.');
 return;
 }
 if(amount > owed){ showToast('O valor não pode passar do que você tem a
receber.');
 return;
 }
 update(s=>{
 s.withdrawalRequests.push({id:uid(), professionalId:prof.id, amount, note,
status:'pendente', createdAt:new Date().toISOString()});
 });
}

```

    });
    showToast('Solicitação de saque enviada.');
```

```
    closeModal();
```

```
    render();
```

```
  });
```

```
});
```

```
}
```

```
function valeModal(profId, view){
```

```
  if(!isOwnerLevel()){ showToast('Apenas o dono ou administrador pode dar vale.');
```

```
  return; }
```

```
  const prof = state.professionals.find(p=>p.id===profId);
```

```
  if(!prof) return;
```

```
  const c = commissionOwed(profId);
```

```
  const caixaAberto = openSession();
```

```
  openModal(`
```

```
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
        <div class="font-display" style="font-size:19px;font-weight:600;">Dar
vale</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
      </div>
      <div style="font-size:13px;color:var(--muted);margin-
bottom:4px;">${esc(prof.name)}</div>
      <div style="font-size:12.5px;color:var(--muted-2);margin-bottom:18px;">
        ${c.owed<0
          ? `Já deve <b style="color:var(--bad);">${BRL(Math.abs(c.owed))}</b> de
vare anterior – outro vare aumenta essa dívida.`
          : `Tem <b style="color:var(--gold-bright);">${BRL(c.owed)}</b> a receber
no momento.`}
      </div>
      <div style="font-size:11px;color:var(--muted-2);margin:-12px 0 6px;line-
height:1.5;">Pode dar mais vare que ele tem a receber – o saldo fica negativo e é
abatido sozinho conforme ele atende novos clientes.</div>
      <div style="display:flex;flex-direction:column;gap:14px;">
        <div><label>Valor do vare (R$)</label><input id="v-amount" type="number"
step="0.01" min="0" placeholder="0,00" style="width:100%;"></div>
        <div>
          <label>De onde sai o dinheiro?</label>
          <div style="display:flex;flex-direction:column;gap:8px;margin-top:6px;">
            <label style="display:flex;align-items:flex-
start;gap:10px;padding:12px 14px;border:1px solid var(--border);border-
radius:10px;cursor:pointer;">
              <input type="radio" name="v-origem" value="caixa" checked
style="width:auto;margin-top:2px;">
            <div>
```

```

        <div style="font-size:13.5px;font-weight:600;">Caixa da
barbearia</div>

        <div style="font-size:11.5px;color:var(--muted-2);margin-
top:2px;">Lança uma saída no caixa automaticamente${caixaAberto?'': ' - <span
style="color:var(--bad);">é preciso abrir o caixa antes</span>'}</div>
        </div>
    </label>
    <label style="display:flex;align-items:flex-
start;gap:10px;padding:12px 14px;border:1px solid var(--border);border-
radius:10px;cursor:pointer;">
        <input type="radio" name="v-origem" value="banco"
style="width:auto;margin-top:2px;">
        <div>
            <div style="font-size:13.5px;font-weight:600;">Conta bancária /
PIX</div>
            <div style="font-size:11.5px;color:var(--muted-2);margin-
top:2px;">Não mexe no caixa – fica só registrado no acerto do profissional</div>
        </div>
    </label>
</div>
</div>
<div><label>Observação (opcional)</label><input id="v-note"
placeholder="Ex: adiantamento de sexta" style="width:100%;"></div>
<button class="btn btn-gold" id="v-save" style="justify-
content:center;">Registrar vale</button>
<div style="font-size:11px;color:var(--muted-2);text-align:center;">O
valor é descontado do que ele tem a receber.</div>
</div>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#v-save').addEventListener('click', ()=>{
        const amount = parseFloat(root.querySelector('#v-amount').value);
        const origem = root.querySelector('input[name="v-origem"]:checked').value;
        const note = root.querySelector('#v-note').value.trim();
        if(isNaN(amount) || amount<=0){ showToast('Informe um valor válido.')}
return; }
    if(origem==='caixa' && !openSession()){ showToast('Abra o caixa antes de dar
vale em dinheiro.')} return; }
    update(s=>{
        s.commissionPayments.push({
            id:uid(), professionalId:profId, amount, date:new Date().toISOString(),
            source:'vale', origem, note,
        });
        if(origem==='caixa'){
            const sess = s.cashSessions.find(x=>x.status==='aberto');
            sess.entries.push({

```

```

id:uid(), type:'saida', method:'dinheiro',
desc:`Vale - ${prof.name}${note? ' ('+note+') ':''}`,
amount, time:new Date().toString().slice(0,5), auto:true,
});
}
});
closeModal();
showToast(`Vale de ${BRL(amount)} registrado (${origem==='caixa'? 'saiu do
caixa': 'conta bancária'})`);
if(view) renderComissoes(view);
});
});
}
function payServiceCommission(profId){
  if(!isOwnerLevel()){ showToast('Apenas o dono ou administrador pode pagar
comissão. '); return; }
  const c = commissionOwed(profId);
  if(c.serviceOwed<=0){ showToast('Não há valor pendente para pagar - o
profissional está com vale além da comissão. '); return; }
  openModal(`


Pagar
comissão de serviços



>${ICON.x}



>${profName(profId)} · valor pendente <span class="font-mono"
style="color:var(--warn);">${BRL(c.serviceOwed)}</span></div>


Valor a pagar agora



Confirmar pagamento


`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#f-submit').addEventListener('click', ()=>{
    const amount = parseFloat(root.querySelector('#f-amount').value);
    if(isNaN(amount) || amount<=0){ showToast('Informe um valor válido. ');
return; }
    update(s=>{ s.commissionPayments.push({id:uid(), professionalId:profId,
amount, date:new Date().toISOString(), source:'servico'})}); });
  closeModal();
}


```

```

        showToast('Comissão de serviços paga.');
```

```

    });
  });
}
function closeClubMonthModal() {
  const period = currentPeriod();
  if(monthAlreadyClosed(period)){ showToast('O pote deste mês já foi fechado.');
```

```

  return; }

  const rows = servingProfessionals().map(p=>({prof:p, owed:
  commissionOwed(p.id).potOwed})).filter(r=>r.owed>0);
  if(rows.length===0){ showToast('Não há pote a fechar neste mês.');
```

```

  return; }
  const total = rows.reduce((a,r)=>a+r.owed,0);
  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:16px;">
        <div class="font-display" style="font-size:19px;font-weight:600;">Fechar
pote de ${periodLabel(period)}</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
      </div>
      <div style="font-size:13px;color:var(--muted);margin-bottom:12px;">Confirmar
paga a comissão do pote do clube a todos os profissionais abaixo, de uma vez.
</div>
      <div style="display:flex;flex-direction:column;gap:6px;margin-bottom:16px;">
        ${rows.map(r=>`<div style="display:flex;justify-content:space-
between;font-size:13.5px;padding:8px 12px;background:var(--panel-2);border-
radius:8px;">
          <span>${esc(r.prof.name)}</span><span class="font-mono"
style="color:var(--gold-bright);">${BRL(r.owed)}</span>
        </div>`)}
        <div style="display:flex;justify-content:space-between;font-
size:14px;font-weight:600;padding:8px 12px;">
          <span>Total do fechamento</span><span class="font-mono">${BRL(total)}
</span>
        </div>
      </div>
      <button class="btn btn-gold" id="f-submit" style="width:100%;justify-
content:center;">Confirmar fechamento e pagar</button>
    </div>
  `, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#f-submit').addEventListener('click', ()=>{
      update(s=>{
        rows.forEach(r=>{
          s.commissionPayments.push({id:uid(), professionalId:r.prof.id,
amount:r.owed, date:new Date().toISOString(), source:'pote', period});

```

```

    });
  });
  closeModal();
  showToast(`Fechamento de ${periodLabel(period)} concluído.`);
});
});
}

/* ===== PROFISSIONAIS ===== */
const WEEKDAY_LABELS = ['Dom', 'Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb'];
const WEEKDAY_FULL =
['domingo', 'segunda', 'terça', 'quarta', 'quinta', 'sexta', 'sábado'];
// Monta a frase de validade do plano a partir dos dias marcados.
// Ex: [1,2,3,4] -> "Válido de segunda a quinta" | todos -> "Válido a semana toda"
function planDaysLabel(days){
  if(!Array.isArray(days) || days.length===0) return '';
  const set = [...new Set(days)].sort((a,b)=>a-b);
  if(set.length===7) return 'Válido a semana toda';
  // ordem da semana começando na segunda (1..6, depois 0=domingo)
  const ordem = [1,2,3,4,5,6,0];
  const idx = set.map(d=>ordem.indexOf(d)).sort((a,b)=>a-b);
  const sequencial = idx.every((v,i)=> i===0 || v === idx[i-1]+1);
  if(sequencial && set.length>=2){
    const ini = WEEKDAY_FULL[ordem[idx[0]]];
    const fim = WEEKDAY_FULL[ordem[idx[idx.length-1]]];
    return `Válido de ${ini} a ${fim}`;
  }
  if(set.length===1) return `Válido apenas às ${WEEKDAY_FULL[set[0]]}s`;
  const nomes = idx.map(i=>WEEKDAY_FULL[ordem[i]]+'s');
  return `Válido às ${nomes.slice(0,-1).join(', ')} e ${nomes[nomes.length-1]}`;
}
// O plano vale nesta data? (sem dias marcados = vale todos os dias)
function planValidOnDate(plan, dateISO){
  if(!plan || !Array.isArray(plan.validDays) || plan.validDays.length===0) return
true;
  return plan.validDays.includes(new Date(dateISO+'T12:00').getDay());
}
function renderProfissionais(view){
  const limits = planLimits(currentShop);
  const atLimit = state.professionals.length >= limits.maxProfessionals;
  view.innerHTML = `
    ${pageHeader('Profissionais', state.professionals.length+' cadastrado(s)'+
(limits.maxProfessionals!==Infinity?` de ${limits.maxProfessionals} (plano
${limits.label})`:''), `<button class="btn btn-gold" id="btn-new-prof"
${atLimit?'disabled title="Limite do plano Basic atingido":"'}`>${ICON.plus} Novo
profissional</button>`)}
    ${atLimit ? `<div class="pill pill-warn" style="display:block;margin-`

```

```

bottom:14px;padding:10px 14px;">Você atingiu o limite de
${limits.maxProfessionals} profissionais do plano Basic. <a href="#" id="lnk-
upgrade-prof" style="color:var(--gold-bright);text-decoration:underline;">Assine o
plano Pro</a> para cadastrar profissionais ilimitados.</div>` : ''}
    ${isOwnerLevel() && state.onlineBooking?.slug ? `
        <div class="card" style="padding:16px 18px;margin-bottom:18px;">
            <div style="display:flex;justify-content:space-between;align-
items:center;flex-wrap:wrap;gap:12px;">
                <div style="min-width:0;">
                    <div style="font-size:12px;color:var(--muted-2);margin-
bottom:5px;">Link de acesso da equipe – cada profissional entra pelo próprio
celular usando o PIN dele</div>
                    <div class="font-mono" style="font-size:12.5px;color:var(--gold-
bright);word-break:break-
all;">${window.location.origin}${window.location.pathname}?
equipe=${esc(state.onlineBooking.slug)}</div>
                </div>
                <button class="btn btn-ghost" id="btn-copy-team-link" style="flex-
shrink:0;">${ICON.copy} Copiar link</button>
            </div>
            <div style="font-size:11px;color:var(--muted-2);margin-top:10px;">Use PINs
de 6 a 8 dígitos e evite sequências óbvias – quem tiver o link e o PIN entra na
visão daquele profissional.</div>
        </div>
    ` : ''}
    ${state.professionals.length===0 ? emptyState('Nenhum profissional
cadastrado.') : `
        <div style="display:grid;grid-template-columns:repeat(auto-
fill,minmax(260px,1fr));gap:14px;">
            ${state.professionals.slice().sort(byName).map(p=>{
                const rt = ratingStats(p.id);
                return `<div class="card" style="padding:16px 18px;">
                    <div style="display:flex;justify-content:space-between;align-
items:flex-start;">
                        <div style="display:flex;gap:12px;align-items:center;">
                            <div style="width:46px;height:46px;border-
radius:50%;background:var(--panel-2);border:1px solid var(--
border);display:flex;align-items:center;justify-
content:center;overflow:hidden;flex-shrink:0;color:var(--muted-2);">
                                ${p.photoDataUrl ? `` : ICON.team}
                            </div>
                            <div>
                                <div style="font-weight:600;font-size:15px;">${esc(p.name)}
                            </div>
                                <div style="font-size:12.5px;color:var(--muted);margin-
top:3px;">${esc(p.phone)||'Sem telefone'}</div>
                        </div>
                    </div>
                `
            })}
        </div>
    `

```

```

        <span class="pill pill-muted" style="margin-top:6px;display:inline-block;">${roleLabel(p.accessRole)}</span>
      </div>
    </div>
    <span class="pill pill-warn font-mono">${p.commission}%</span>
  </div>
  <div style="font-size:11.5px;color:var(--muted-2);margin-top:10px;">${p.workDays.map(d=>WEEKDAY_LABELS[d]).join(' ')} · ${p.workStart}-${p.workEnd}</div>
  <div style="display:flex;justify-content:space-between;align-items:center;margin-top:6px;">
    <div style="font-size:11.5px;color:var(--muted-2);">${ICON.shield}
    PIN ${p.pin}</div>
    ${rt.count>0 ? `<div style="font-size:12px;color:var(--gold-bright);display:flex;align-items:center;gap:3px;">${ICON.star}
    ${rt.avg.toFixed(1)} (${rt.count})</div>` : ''}
  </div>
  <div style="display:flex;gap:6px;margin-top:14px;">
    <button class="btn btn-ghost" style="flex:1;justify-content:center;padding:7px;" data-edit-prof="${p.id}">${ICON.edit} Editar</button>
    <button class="btn btn-ghost" style="padding:7px 9px;color:var(--bad);border-color:var(--bad);" data-del-prof="${p.id}">${ICON.trash}</button>
  </div>
</div>`;
  }).join('')}
</div>
`}
`;
view.querySelector('#btn-new-prof').addEventListener('click', ()=>{
  if(state.professionals.length >= planLimits(currentShop).maxProfessionals){
    renderPaywallUpgrade('profissionais'); return; }
  profModal();
});
view.querySelector('#lnk-upgrade-prof')?.addEventListener('click', (e)=>{
  e.preventDefault(); renderPaywallUpgrade('profissionais'); });
view.querySelector('#btn-copy-team-link')?.addEventListener('click', ()=>{
  const link = `${window.location.origin}${window.location.pathname}?
  equipe=${state.onlineBooking.slug}`;
  navigator.clipboard?.writeText(link);
  showToast('Link da equipe copiado.');
```

```

    }));
}
function profModal(existing) {
    const isEdit = !!existing;
    const workDays = existing?.workDays || [1,2,3,4,5,6];
    const timeOff = existing?.timeOff || [];
    const overrides = existing?.serviceOverrides || {};
    openModal(`
        <div style="padding:22px;">
            <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:16px;">
                <div class="font-display" style="font-size:19px;font-weight:600;">${isEdit?'Editar':'Novo'} profissional</div>
                <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}</button>
            </div>
            <div style="display:flex;flex-direction:column;gap:12px;">
                <div>
                    <label>Foto do profissional</label>
                    <div style="display:flex;align-items:center;gap:12px;">
                        <div id="prof-photo-preview" style="width:64px;height:64px;border-radius:50%;background:var(--panel-2);border:1px solid var(--border);display:flex;align-items:center;justify-content:center;overflow:hidden;flex-shrink:0;color:var(--muted-2);">
                            ${existing?.photoDataUrl ? `` : ICON.team}
                        </div>
                        <input type="file" id="f-photo-file" accept="image/*" style="display:none;">
                        <button class="btn btn-ghost" id="btn-photo-pick" type="button">${ICON.upload} Escolher foto</button>
                        <button class="btn btn-ghost" id="btn-photo-remove" type="button" style="color:var(--bad);border-color:var(--bad);display:${existing?.photoDataUrl?'inline-flex':'none'};">${ICON.trash}</button>
                    </div>
                    <div style="font-size:10.5px;color:var(--muted-2);margin-top:5px;">Aparece na agenda e na página pública de agendamento. Máx. 300KB.</div>
                </div>
                <div><label>Nome</label><input id="f-name" value="${esc(existing?.name)}" placeholder="Nome completo" style="width:100%;"></div>
                <div><label>Telefone</label><input id="f-phone" value="${esc(existing?.phone)}" placeholder="(00) 00000-0000" style="width:100%;"></div>
                <div><label>E-mail <span style="color:var(--muted-2);font-weight:400;">— opcional, para receber aviso quando um cliente agendar com ele</span></label>
                <input id="f-email" type="email" value="${esc(existing?.email)}"
    
```

```

placeholder="opcional" style="width:100%;"></div>
    <div>
        <label>Nível de acesso ao sistema</label>
        <select id="f-access-role" style="width:100%;">
            <option value="profissional"
                ${(!existing||existing.accessRole==='profissional'||!existing.accessRole)?'selectec
                - vê a própria agenda, comissões e pode solicitar saques</option>
                <option value="recepcionista"
                ${existing?.accessRole==='recepcionista'? 'selected':''}>Recepcionista - agenda,
                clientes, caixa, estoque e mensagens</option>
                <option value="admin"
                ${existing?.accessRole==='admin'? 'selected':''}>Dono / Admin - acesso completo ao
                sistema</option>
            </select>
        </div>
        <label style="display:flex;align-items:center;gap:9px;font-
        size:13.5px;cursor:pointer;padding:10px 12px;border:1px solid var(--
        border);border-radius:9px;">
            <input type="checkbox" id="f-serves" ${existing ?
            (existing.servesClients!==false?'checked':'') : 'checked'} style="width:auto;">
            <span>Atende clientes (aparece na agenda e no agendamento online)</span>
        </label>
        <div style="font-size:11px;color:var(--muted-2);margin-
        top:-6px;">Desmarque para quem trabalha na barbearia mas não corta - recepção,
        gerente, etc. Quem está desmarcado não aparece como opção de barbeiro, nem nas
        comissões e no pote do clube.</div>
        <div style="display:flex;gap:10px;">
            <div style="flex:1;"><label>Comissão base (%)</label><input id="f-comm"
            type="number" min="0" max="100" value="${existing?.commission ?? 40}"
            style="width:100%;"></div>
            <div style="flex:1;"><label>PIN de login (4 a 8 dígitos)</label><input
            id="f-pin" maxlength="8" inputmode="numeric" value="${esc(existing?.pin) ||
            String(100000+Math.floor(Math.random()*900000))}" style="width:100%;"><div
            style="font-size:10.5px;color:var(--muted-2);margin-top:4px;">Usado no link de
            acesso da equipe. Quanto mais dígitos, mais seguro.</div></div>
        </div>

        <label>Dias de trabalho</label>
        <div style="display:flex;gap:5px;flex-wrap:wrap;">
            ${WEEKDAY_LABELS.map((lbl,idx)=>`<label style="display:flex;flex-
            direction:column;align-items:center;gap:3px;font-size:11px;color:var(--
            muted);cursor:pointer;">
                <input type="checkbox" data-workday="${idx}"
                ${workDays.includes(idx)? 'checked':''} style="width:auto;">${lbl}
            </label>`)}.join('')
        </div>
        <div style="display:flex;gap:10px;">

```

```

        <div style="flex:1;"><label>Início do expediente</label><input id="f-
start" type="time" value="\${existing?.workStart||'09:00'}" style="width:100%;">
</div>

        <div style="flex:1;"><label>Fim do expediente</label><input id="f-end"
type="time" value="\${existing?.workEnd||'19:00'}" style="width:100%;"></div>
</div>

<label>Folgas / bloqueios de data</label>
<div style="display:flex;gap:8px;">
    <input id="f-timeoff-date" type="date" style="flex:1;">
    <button class="btn btn-ghost" id="btn-add-timeoff" style="padding:8px
12px;">\${ICON.plus}</button>
</div>
<div id="timeoff-list" style="display:flex;flex-wrap:wrap;gap:6px;">
    \${timeOff.map(d=>`<span class="pill pill-muted" data-timeoff-
chip="\${d}">\${new Date(d+'T00:00').toLocaleDateString('pt-BR')} \${ICON.x}
</span>`).join('')}
</div>

\${state.services.length>0 ? `
<label>Comissão por serviço (opcional – sobrescreve a base)</label>
<div style="display:flex;flex-direction:column;gap:6px;">
    \${state.services.map(s=>`<div style="display:flex;justify-content:space-
between;align-items:center;gap:8px;">
        <span style="font-size:13px;flex:1;">\${esc(s.name)}</span>
        <input type="number" min="0" max="100"
placeholder="\${existing?.commission ?? 40}" data-override="\${s.id}"
value="\${overrides[s.id]??''}" style="width:80px;">
        </div>`).join('')}
    </div>`} : ''}

    <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">\${isEdit?'Salvar':'Cadastrar'}</button>
</div>
</div>
`, root=>{
    let currentTimeOff = [...timeOff];
    let currentPhoto = existing?.photoDataUrl || '';
    root.querySelector('#btn-photo-pick').addEventListener('click',
()=>root.querySelector('#f-photo-file').click());
    root.querySelector('#f-photo-file').addEventListener('change', e=>{
        const file = e.target.files[0];
        if(!file) return;
        readImageAsDataURL(file, 300*1024, dataUrl=>{
            currentPhoto = dataUrl;
            root.querySelector('#prof-photo-preview').innerHTML = ``;

```

```

        root.querySelector('#btn-photo-remove').style.display = 'inline-flex';
    });
    e.target.value = '';
});
root.querySelector('#f-access-role').addEventListener('change', e=>{
    // Recepcionista normalmente não atende; profissional sim.
    const box = root.querySelector('#f-serves');
    if(e.target.value==='recepcionista') box.checked = false;
    else if(e.target.value==='profissional') box.checked = true;
});
root.querySelector('#btn-photo-remove').addEventListener('click', ()=>{
    currentPhoto = '';
    root.querySelector('#prof-photo-preview').innerHTML = ICON.team;
    root.querySelector('#btn-photo-remove').style.display = 'none';
});
root.querySelector('#m-close').addEventListener('click', closeModal);
root.querySelector('#btn-add-timeoff').addEventListener('click', ()=>{
    const d = root.querySelector('#f-timeoff-date').value;
    if(!d || currentTimeOff.includes(d)) return;
    currentTimeOff.push(d);
    const chip = document.createElement('span');
    chip.className='pill pill-muted'; chip.dataset.timeoffChip=d;
    chip.innerHTML = `${new Date(d+'T00:00').toLocaleDateString('pt-BR')}`;
    `${ICON.x}`;
    chip.style.cursor='pointer';
    chip.addEventListener('click', ()=>{ currentTimeOff =
currentTimeOff.filter(x=>x!==d); chip.remove(); });
    root.querySelector('#timeoff-list').appendChild(chip);
});
root.querySelectorAll('[data-timeoff-chip]').forEach(chip=>{
    chip.style.cursor='pointer';
    chip.addEventListener('click', ()=>{ currentTimeOff =
currentTimeOff.filter(x=>x!==chip.dataset.timeoffChip); chip.remove(); });
});
root.querySelector('#f-submit').addEventListener('click', ()=>{
    const name = root.querySelector('#f-name').value.trim();
    const phone = root.querySelector('#f-phone').value.trim();
    const accessRole = root.querySelector('#f-access-role').value;
    const commission = parseFloat(root.querySelector('#f-comm').value)||0;
    const pin = root.querySelector('#f-pin').value.trim() || '0000';
    const selectedDays = [...root.querySelectorAll('[data-workday]:checked')].map(el=>parseInt(el.dataset.workday));
    const workStart = root.querySelector('#f-start').value;
    const workEnd = root.querySelector('#f-end').value;
    const serviceOverrides = {};
    root.querySelectorAll('[data-override]').forEach(inp=>{
        if(inp.value!=='') serviceOverrides[inp.dataset.override] =

```

```

parseFloat(inp.value);
    });
    if(!name){ showToast('Informe o nome.');
```

```

    return; }
    const servesClients = root.querySelector('#f-serves').checked;
```

```

    const email = root.querySelector('#f-email').value.trim();
```

```

    const data = {name, phone, email, accessRole, commission, pin,
```

```

    servesClients, photoDataUrl: currentPhoto, workDays:selectedDays, workStart,
    workEnd, timeOff:currentTimeOff, serviceOverrides};
```

```

    update(s=>{
```

```

        if(isEdit){ const alvo = s.professionals.find(x=>x.id===existing.id);
```

```

        if(alvo) Object.assign(alvo, data); }
```

```

        else{ s.professionals.push({id:uid(), ...data}); }
```

```

    });
```

```

    closeModal();
```

```

    });
```

```

});
```

```

}
```

```

/* ===== SERVIÇOS ===== */
```

```

function renderServicos(view){
```

```

    view.innerHTML = `
```

```

        ${pageHeader('Serviços', state.services.length+' cadastrado(s)', `
```

```

        <div style="display:flex;gap:8px;flex-wrap:wrap;">
```

```

            <input type="file" id="f-import-svc" accept=".csv,.xlsx,.xls,.xlsm"
            style="display:none;">
```

```

            <button class="btn btn-ghost" id="btn-import-svc">${ICON.arrowUp} Importar
            planilha</button>
```

```

            <button class="btn btn-gold" id="btn-new-svc">${ICON.plus} Novo
            serviço</button>
```

```

        </div>
```

```

    `)}
```

```

    <div style="font-size:12px;color:var(--muted-2);margin:-12px 0
    18px;">Importação aceita Excel (.xlsx) ou CSV. Colunas esperadas, nesta ordem:
    Serviço, Valor, Duração (min), Fichas do clube.</div>
```

```

    ${state.services.length===0 ? emptyState('Nenhum serviço cadastrado.') : `
```

```

    <table>
```

```

        <thead><tr><th>Serviço</th><th>Valor</th><th>Duração</th><th>Sinal</th>
        <th>Fichas</th><th></th></tr></thead>
```

```

        <tbody>
```

```

        ${state.services.map(s=>`<tr>
```

```

            <td style="font-weight:500;">${esc(s.name)}${s.hideOnline?`<span
            class="pill pill-muted" style="font-size:10px;">só no painel</span>`:''}</td>
```

```

            <td class="font-mono">${BRL(s.price)}</td>
```

```

            <td class="font-mono" style="color:var(--muted);">${s.duration} min</td>
```

```

            <td>${s.requireDeposit ? `<span class="pill pill-warn font-
            mono">${ICON.timer} ${BRL(depositAmountFor(s))}</span>` : '<span class="pill pill-
            muted">Não exige</span>'}</td>
```

```

        <td class="font-mono" style="color:${s.clubFichas}>0?'var(--gold-bright)':'var(--muted-2)';">${s.clubFichas||0}</td>
        <td style="text-align:right;">
            <button class="btn btn-ghost" style="padding:6px 8px;" data-edit-svc="${s.id}">${ICON.edit}</button>
            <button class="btn btn-ghost" style="padding:6px 8px;color:var(--bad);border-color:var(--bad);" data-del-svc="${s.id}">${ICON.trash}</button>
        </td>
    </tr>`)).join('')
</tbody>
</table>
`}
`;
view.querySelector('#btn-new-svc').addEventListener('click', ()=>svcModal());
view.querySelector('#btn-import-svc').addEventListener('click',
()=>view.querySelector('#f-import-svc').click());
view.querySelector('#f-import-svc').addEventListener('change', (e)=>{
    const file = e.target.files[0];
    if(!file) return;
    readSpreadsheetFile(file, importServicesFromRows);
    e.target.value = '';
});
view.querySelectorAll('[data-edit-svc]').forEach(b=>b.addEventListener('click',
()=>svcModal(state.services.find(s=>s.id===b.dataset.editSvc))));
view.querySelectorAll('[data-del-svc]').forEach(b=>b.addEventListener('click',
()=>{
    update(s=>{ s.services = s.services.filter(sv=>sv.id!==b.dataset.delSvc); });
}));
}
function svcModal(existing){
    const isEdit = !!existing;
    openModal(`
        <div style="padding:22px;">
            <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:16px;">
                <div class="font-display" style="font-size:19px;font-weight:600;">${isEdit?'Editar':'Novo'} serviço</div>
                <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}</button>
            </div>
            <div style="display:flex;flex-direction:column;gap:12px;">
                <div><label>Nome do serviço</label><input id="f-name" value="${esc(existing?.name)}" placeholder="Ex: Corte Masculino" style="width:100%;"></div>
                <div style="display:flex;gap:10px;">
                    <div style="flex:1;"><label>Valor (R$)</label><input id="f-price" type="number" step="0.01" min="0" value="${esc(existing?.price)}"

```

```

style="width:100%;"></div>
    <div style="flex:1;"><label>Duração (min)</label><input id="f-dur"
type="number" min="1" value="{esc(existing?.duration)}" style="width:100%;">
</div>
    </div>
    <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;color:var(--ink);cursor:pointer;"><input type="checkbox" id="f-req-
deposit" {existing?.requireDeposit?'checked':''} style="width:auto;padding:0;">
Exigir sinal antecipado neste serviço</label>
    <div id="deposit-fields"
style="display:{existing?.requireDeposit?'flex':'none'};gap:10px;">
        <div style="flex:1;"><label>Tipo</label><select id="f-dep-type"
style="width:100%;">
            <option value="percentual"
${existing?.depositType==='percentual'?'selected':''}>% do valor</option>
            <option value="fixo"
${existing?.depositType==='fixo'?'selected':''}>Valor fixo (R$)</option>
        </select></div>
        <div style="flex:1;"><label>Valor</label><input id="f-dep-value"
type="number" min="0" step="0.01" value="{existing?.depositValue??30}"
style="width:100%;"></div>
    </div>
    <div><label>Fichas no Clube de Assinatura</label><input id="f-fichas"
type="number" min="0" step="1" value="{existing?.clubFichas??0}"
style="width:100%;">
        <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">Quantas
fichas o profissional ganha ao fazer este serviço em um cliente assinante do clube
– usado para dividir o pote no fechamento do mês.</div>
    </div>
    <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;color:var(--ink);cursor:pointer;"><input type="checkbox" id="f-online"
${existing?.hideOnline?'':'checked'} style="width:auto;padding:0;"> Disponível no
agendamento online</label>
    <div style="font-size:11px;color:var(--muted-2);margin-
top:-6px;">Desmarque para serviços que só a barbearia marca por dentro (ex:
procedimentos especiais). O cliente não verá esta opção no link público, mas você
continua agendando normalmente.</div>
    <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">${isEdit?'Salvar':'Cadastrar'}</button>
</div>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#f-req-deposit').addEventListener('change', e=>{
        root.querySelector('#deposit-fields').style.display = e.target.checked ?
'flex' : 'none';
    });
}

```

```

    root.querySelector('#f-submit').addEventListener('click', ()=>{
        const name = root.querySelector('#f-name').value.trim();
        const price = parseFloat(root.querySelector('#f-price').value);
        const duration = parseInt(root.querySelector('#f-dur').value);
        if(!name || isNaN(price) || isNaN(duration)){ showToast('Preencha todos os
campos.');
```

return; }

```

        const requireDeposit = root.querySelector('#f-req-deposit').checked;
        const depositType = root.querySelector('#f-dep-type').value;
        const depositValue = parseFloat(root.querySelector('#f-dep-
value').value)||0;
        const clubFichas = parseInt(root.querySelector('#f-fichas').value)||0;
        const hideOnline = !root.querySelector('#f-online').checked;
        update(s=>{
            if(isEdit){ const alvo = s.services.find(x=>x.id===existing.id); if(alvo)
Object.assign(alvo, {name, price, duration, requireDeposit, depositType,
depositValue, clubFichas, hideOnline}); }
            else{ s.services.push({id:uid(), name, price, duration, requireDeposit,
depositType, depositValue, clubFichas, hideOnline}); }
        });
        closeModal();
    });
}

/* ===== CLUBE DE ASSINATURA ===== */
function statusPillMember(status){
    if(status==='pago') return '<span class="pill pill-ok">Pago</span>';
    if(status==='pendente') return '<span class="pill pill-warn">Pendente</span>';
    if(status==='atrasado') return '<span class="pill pill-bad">Atrasado</span>';
    return '<span class="pill pill-muted">Cancelado</span>';
}

function methodLabel(m){
    if(m.paymentMethod==='dinheiro') return '<span class="pill pill-
muted">Dinheiro</span>';
    return '<span class="pill pill-muted">'+ICON.link+' Pix/Cartão (Asaas)</span>';
}

async function loadClientRequests(view){
    const listEl = view.querySelector('#cli-req-list');
    const countEl = view.querySelector('#cli-req-count');
    if(!listEl || !currentShop) return;
    const { data, error } = await supa.from('client_requests')
        .select('*').eq('shop_id',
currentShop.id).eq('status','pendente').order('created_at',{ascending:false});
    if(error){ listEl.innerHTML = '<div style="font-size:12.5px;color:var(--
muted);">Não foi possível carregar.</div>'; return; }
    countEl.textContent = (data||[]).length + ' pendente(s)';
    if(!data || !data.length){ listEl.innerHTML = '<div style="font-
```

```

size:12.5px;color:var(--muted);">Nenhum pedido pendente.</div>'; return; }
  listEl.innerHTML = data.map(r=>{
    const txt = r.type==='renovacao'
      ? `quer renovar o plano ${esc(r.current_plan||'')}`
      : `quer trocar${r.current_plan?` de ${esc(r.current_plan)}`:''} para
<b>${esc(r.requested_plan_name||'')}</b>`;
    const msg = r.type==='renovacao'
      ? `Olá ${r.client_name}! Recebemos seu pedido de renovação. Já te envio o
link de pagamento.`
      : `Olá ${r.client_name}! Recebemos seu pedido de troca para o
${r.requested_plan_name}. Vamos acertar isso pra você.`;
    return `
    <div style="display:flex;justify-content:space-between;align-
items:center;gap:10px;padding:10px 0;border-bottom:1px solid var(--border);flex-
wrap:wrap;">
      <div style="font-size:13px;min-width:200px;">
        <b>${esc(r.client_name)}</b> ${txt}<br>
        <span style="color:var(--muted);font-
size:11.5px;">${esc(r.client_phone||'')} · ${new
Date(r.created_at).toLocaleDateString('pt-BR')}</span>
      </div>
      <div style="display:flex;gap:6px;flex-shrink:0;flex-wrap:wrap;">
        ${messageChannelRow(r.client_phone, msg)}
        <button class="btn btn-gold" data-cli-done="${r.id}" style="padding:6px
10px;font-size:12px;">Marcar resolvido</button>
      </div>
    </div>`;
  }).join('');
  listEl.querySelectorAll('[data-cli-
done]').forEach(b=>b.addEventListener('click', async ()=>{
    await supa.from('client_requests').update({status:'resolvido'}).eq('id',
b.dataset.cliDone);
    showToast('Pedido marcado como resolvido.');
```

loadClientRequests(view);

```

  }));
}

function clientAccessModal(member){
  if(!member) return;
  const senha = 'Alpha' + Math.floor(1000 + Math.random()*9000);
  const link = `${window.location.origin}${window.location.pathname}?
cliente=${state.onlineBooking?.slug||''}`;
  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
        <div class="font-display" style="font-size:19px;font-weight:600;">Acesso
ao portal</div>

```

```

        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
    </div>
    <div style="font-size:13px;color:var(--muted);margin-bottom:18px;">Crie o
login de <b>${esc(member.name)}</b> para ele acessar o portal, agendar e renovar
sozinho.</div>
    <div style="display:flex;flex-direction:column;gap:12px;">
        <div><label>E-mail do cliente</label><input id="ca-email" type="email"
value="${esc(member.email)}" placeholder="cliente@email.com" style="width:100%;">
</div>
        <div><label>Senha provisória</label><input id="ca-pass" value="${senha}"
style="width:100%;">
        <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">O
cliente será obrigado a trocar essa senha no primeiro acesso.</div>
    </div>
    <button class="btn btn-gold" id="ca-save" style="justify-
content:center;">Criar acesso</button>
    <div id="ca-result"></div>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#ca-save').addEventListener('click', async (e)=>{
        const email = root.querySelector('#ca-email').value.trim();
        const pass = root.querySelector('#ca-pass').value.trim();
        if(!email || pass.length<6){ showToast('Informe o e-mail e uma senha com ao
menos 6 caracteres.');
```

```

    root.querySelector('#ca-result').innerHTML = `
      <div class="card" style="padding:14px 16px;background:var(--panel-
2);margin-top:4px;">
        <div style="color:var(--ok);font-size:13px;font-weight:600;margin-
bottom:10px;">✓ Acesso criado!</div>
        <div style="font-size:12.5px;color:var(--muted);line-height:1.7;word-
break:break-all;">
          Link: <span style="color:var(--gold-bright);">${link}</span><br>
          E-mail: <b style="color:var(--ink);">${esc(email)}</b><br>
          Senha provisória: <b style="color:var(--ink);">${esc(pass)}</b>
        </div>
        <div style="display:flex;gap:8px;margin-top:12px;flex-wrap:wrap;">
          ${messageChannelRow(member.phone, msg)}
          <button class="btn btn-ghost" id="ca-copy" style="padding:7px
11px;font-size:12px;">${ICON.copy} Copiar dados</button>
        </div>
      </div>`;

    root.querySelector('#ca-copy').addEventListener('click', ()=>{
navigator.clipboard?.writeText(msg); showToast('Dados copiados.');
```

```

    });
  });
}

async function loadSubscriptionRequests(view){
  const listEl = view.querySelector('#sub-req-list');
  const countEl = view.querySelector('#sub-req-count');
  if(!listEl || !currentShop) return;
  const { data, error } = await supa.from('subscription_requests')
    .select('*').eq('shop_id',
currentShop.id).eq('status','pendente').order('created_at',{ascending:false});
  if(error){ listEl.innerHTML = '<div style="font-size:12.5px;color:var(--
muted);">Não foi possível carregar.</div>'; return; }
  countEl.textContent = (data||[]).length + ' pendente(s)';
  if(!data || !data.length){ listEl.innerHTML = '<div style="font-
size:12.5px;color:var(--muted);">Nenhum pedido pendente.</div>'; return; }
  listEl.innerHTML = data.map(r=>`
    <div style="display:flex;justify-content:space-between;align-
items:center;gap:10px;padding:10px 0;border-bottom:1px solid var(--border);flex-
wrap:wrap;">
      <div style="font-size:13px;min-width:200px;">
        <b>${esc(r.client_name)}</b> quer assinar <b>${esc(r.plan_name)}</b>
        (${BRL(r.plan_price)})<br>
        <span style="color:var(--muted);font-
size:11.5px;">${esc(r.client_phone)}${r.client_email?' ·
'+esc(r.client_email):''}${r.client_cpf?' · CPF '+esc(r.client_cpf):''}</span>
      </div>
      <div style="display:flex;gap:6px;flex-shrink:0;flex-wrap:wrap;">
        ${messageChannelRow(r.client_phone, `Olá ${r.client_name}! Recebemos seu

```

```

pedido de assinatura do ${r.plan_name}. Vou te enviar o link de pagamento.`))
    <button class="btn btn-gold" data-sub-accept="${r.id}" style="padding:6px
10px;font-size:12px;">Cadastrar assinante</button>
    <button class="btn btn-ghost" data-sub-decline="${r.id}"
style="padding:6px 10px;font-size:12px;">Recusar</button>
  </div>
</div>
`).join('');
  listEl.querySelectorAll('[data-sub-
accept]').forEach(b=>b.addEventListener('click', async ()=>{
    const req = data.find(r=>r.id===b.dataset.subAccept);
    await supa.from('subscription_requests').update({status:'aceito'}).eq('id',
req.id);
    memberModal(null, {
      name:req.client_name, phone:req.client_phone, planId:req.plan_id,
      email:req.client_email || '', cpf:req.client_cpf || '',
      paymentMethod:'asaas',
    });
    loadSubscriptionRequests(view);
  }));
  listEl.querySelectorAll('[data-sub-
decline]').forEach(b=>b.addEventListener('click', async ()=>{
    await supa.from('subscription_requests').update({status:'recusado'}).eq('id',
b.dataset.subDecline);
    showToast('Pedido recusado.');
```

loadSubscriptionRequests(view);

```

  }));
}
/* Mostra quais assinantes o sistema não conseguiu casar com um cliente,
   e deixa o dono ligar cada um manualmente. */
function vinculosModal(view){
  const ativos = (state.subscriptionMembers||
[]).filter(m=>m.status!=='cancelado');
  // um assinante está "ok" se algum cliente cadastrado aponta para ele
  const clientes = (state.clients||[]);
  const casados = new Set();
  clientes.forEach(cl=>{
    const m = clubMemberForClient(cl.name);
    if(m) casados.add(m.id);
  });
  const semVinculo = ativos.filter(m=>!casados.has(m.id));

  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:6px;">
        <div class="font-display" style="font-size:19px;font-
```

```

weight:600;">Assinantes sem cliente vinculado</div>
    <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
</div>
    <div style="font-size:12.5px;color:var(--muted);line-height:1.6;margin-
bottom:16px;">
        Estes assinantes o sistema não conseguiu ligar a nenhum cliente da
barbearia -
        por isso a estrela não aparece na agenda e o plano não é aplicado.
        Escolha o cliente certo de cada um.
    </div>

    ${semVinculo.length===0 ? `
        <div style="text-align:center;padding:24px 0;">
            <div style="color:var(--ok);font-size:15px;font-weight:600;margin-
bottom:6px;">✓ Todos vinculados!</div>
            <div style="font-size:12.5px;color:var(--muted);">Os ${ativos.length}
assinantes estão ligados a um cliente.</div>
        </div>
        ` : `
        <div style="font-size:12px;color:var(--muted-2);margin-
bottom:10px;">${semVinculo.length} de ${ativos.length} sem vínculo:</div>
        <div style="display:flex;flex-direction:column;gap:12px;max-
height:52vh;overflow-y:auto;">
            ${semVinculo.map(m=>{
                const plano = state.subscriptionPlans.find(p=>p.id===m.planId);
                return `<div style="padding:12px 14px;border:1px solid var(--
border);border-radius:10px;">
                    <div style="font-size:13.5px;font-weight:600;">${esc(m.name)}</div>
                    <div style="font-size:11.5px;color:var(--muted-2);margin-
bottom:9px;">
                        ${plano?esc(plano.name):'sem plano'}${m.phone?' ·
'+esc(m.phone):''}
                    </div>
                    <select data-vinc="${m.id}" style="width:100%;font-size:13px;">
                        <option value="">— escolher o cliente —</option>
                        ${clientes.slice().sort(byName).map(cl=>`<option
value="${esc(cl.name)}">${esc(cl.name)}${cl.phone?' · '+esc(cl.phone):''}
</option>`).join('')}
                    </select>
                </div>`;
            }).join('')}
        </div>
        <button class="btn btn-gold" id="vinc-salvar" style="width:100%;justify-
content:center;margin-top:16px;padding:13px;">Salvar vínculos</button>
        <div style="font-size:11px;color:var(--muted-2);margin-top:10px;line-
height:1.6;">

```

Se o cliente ainda não existe na lista, cadastre ele em Clientes primeiro.

```
    </div>
  `}
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#vinc-salvar')?.addEventListener('click', ()=>{
    let n = 0;
    update(s=>{
      root.querySelectorAll('[data-vinc]').forEach(sel=>{
        if(!sel.value) return;
        const m = s.subscriptionMembers.find(x=>x.id===sel.dataset.vinc);
        if(m){ m.linkedClientName = sel.value; n++; }
      });
    });
    closeModal();
    showToast(n>0 ? `${n} assinante(s) vinculado(s).` : 'Nenhum vínculo escolhido.');
```

if(view) renderClube(view);

```
  });
});
}

function renderClube(view){
  if(!planLimits(currentShop).club){
    view.innerHTML = `
      ${pageHeader('Clube de Assinatura', 'Recurso exclusivo do plano Pro')}
      <div class="card" style="padding:32px;text-align:center;">
        <div style="width:48px;height:48px;border-radius:12px;background:var(--gold);display:flex;align-items:center;justify-content:center;color:#1a1610;margin:0 auto 16px;">${ICON.lock}</div>
        <div class="font-display" style="font-size:17px;font-weight:600;margin-bottom:8px;">Disponível no plano Pro</div>
        <div style="font-size:13.5px;color:var(--muted);max-width:420px;margin:0 auto 20px;line-height:1.6;">
          O Clube de Assinatura (pote mensal, cobrança recorrente de clientes) faz parte do plano Pro.
          Faça upgrade para ativar este recurso.
        </div>
        <button class="btn btn-gold" id="btn-upgrade-club" style="margin:0 auto;">Fazer upgrade para o Pro</button>
      </div>
    `;
    view.querySelector('#btn-upgrade-club').addEventListener('click',
    ()=>renderPaywallUpgrade('clube'));
    return;
  }
}
```

```

    }
    const mrr =
state.subscriptionMembers.filter(m=>m.status!=='cancelado').reduce((a,m)=>{
    const plan = state.subscriptionPlans.find(p=>p.id===m.planId); return a+(plan?
plan.price:0);
    },0);
    const sum = asaasSummary();
    const recebidoMes = sum.totalRecebidoAsaas + sum.totalRecebidoDinheiro;

    view.innerHTML = `
        ${pageHeader('Clube de Assinatura', 'Planos recorrentes e membros', `<div
style="display:flex;gap:8px;flex-wrap:wrap;"><button class="btn btn-ghost"
id="btn-new-plan">${ICON.plus} Plano</button><button class="btn btn-gold" id="btn-
new-member">${ICON.plus} Membro</button></div>`)}

        <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(180px,1fr));gap:14px;margin-bottom:14px;">
            ${statCard('Receita recorrente (MRR)', BRL(mrr), 'potencial mensal -
assinaturas não canceladas', 'ok')}
            ${statCard('Recebido em '+new Date().toLocaleDateString('pt-BR',
{month:'long'}), BRL(recebidoMes),
`${state.subscriptionMembers.filter(m=>pagouNoMes(m,
todayISO().slice(0,7))).length} de
${state.subscriptionMembers.filter(m=>m.status!=='cancelado').length} assinantes
já pagaram`, 'ok')}
            ${statCard('Planos disponíveis', state.subscriptionPlans.length,
'configurados', 'ok')}
        </div>

        ${(()=>{
            // marcados como pago mas sem data → não entram no recebido do mês
            const mes = todayISO().slice(0,7);
            const semData = (state.subscriptionMembers||[]).filter(m=>m.status==='pago'
&& !m.lastPaymentDate);
            if(semData.length===0) return '';
            const viaAsaas = semData.filter(m=>m.paymentMethod==='asaas').length;
            return `<div class="card" style="padding:15px 18px;margin-
bottom:14px;border-color:var(--warn);">
                <div style="font-size:12.5px;line-height:1.7;margin-bottom:12px;">
                    <b style="color:var(--warn);">${semData.length} assinante(s) com status
"pago" sem data de pagamento</b><br>
                    <span style="color:var(--muted-2);">
                        Provavelmente é o status que veio da importação, referente a meses
anteriores.
                    </span>
                    ${viaAsaas>0?`<b>${viaAsaas} deles pagam pelo Asaas</b> - o botão
abaixo consulta o Asaas e corrige o status de cada um conforme o mês atual.`:''}
                </div>
            `;
        })}
    `;

```

```

    </div>
    <div style="display:flex;gap:8px;flex-wrap:wrap;">
      ${viaAsaas>0?`<button class="btn btn-gold" id="btn-sync-
corrigir">${ICON.link} Consultar Asaas e corrigir</button>`:''}
      <button class="btn btn-ghost" id="btn-datar-pgto" style="font-
size:12.5px;">Marcar como pagos HOJE</button>
    </div>
    <div style="font-size:11px;color:var(--muted-2);margin-top:9px;line-
height:1.6;">
      Use "marcar como pagos hoje" apenas se eles realmente pagaram neste mês
em dinheiro.
      Para quem paga pelo Asaas, prefira consultar – assim o mês certo é
respeitado.
    </div>
  </div>`;
  })() }

  ${(()=>{
    const orfaos = assinantesOrfaos();
    if(orfaos.length===0) return '';
    return `<div class="card" style="padding:15px 18px;margin-
bottom:14px;border-color:var(--warn);">
      <div style="display:flex;justify-content:space-between;align-
items:center;gap:12px;flex-wrap:wrap;">
        <div style="font-size:12.5px;line-height:1.6;">
          <b style="color:var(--warn);">${orfaos.length} assinante(s) com plano
removido</b><br>
          <span style="color:var(--muted-2);">0 plano deles foi excluído, então
ficaram sem cobrança e sem benefício.
          Escolha um plano novo ou cancele a assinatura.</span>
        </div>
        <button class="btn btn-gold" id="btn-orfaos" style="flex-
shrink:0;">Resolver</button>
      </div>
    </div>`;
  })() }

  ${(()=>{
    // assinantes que nenhum cliente consegue "achar" – estrela não aparece
    const ativos = (state.subscriptionMembers||
[]).filter(m=>m.status!=='cancelado');
    const casados = new Set();
    (state.clients||[]).forEach(cl=>{ const m = clubMemberForClient(cl.name);
if(m) casados.add(m.id); });
    const semVinculo = ativos.filter(m=>!casados.has(m.id)).length;
    if(semVinculo===0) return '';
    return `<div class="card" style="padding:15px 18px;margin-

```

```

bottom:14px;border-color:var(--bad);">
    <div style="display:flex;justify-content:space-between;align-
items:center;gap:12px;flex-wrap:wrap;">
        <div style="font-size:12.5px;line-height:1.6;">
            <b style="color:var(--bad);">${semVinculo} assinante(s) sem cliente
vinculado</b><br>
            <span style="color:var(--muted-2);">O nome no clube não bate com
nenhum cliente cadastrado, então a estrela
            não aparece na agenda e o plano não é aplicado no fechamento.</span>
        </div>
        <button class="btn btn-gold" id="btn-vinculos" style="flex-
shrink:0;">Resolver agora</button>
    </div>
</div>`;
})() }

${(()=>{
    const mesAtual = todayISO().slice(0,7);
    const faltando = state.subscriptionMembers.filter(m=>m.status!=='cancelado'
&& !pagouNoMes(m, mesAtual));
    if(!faltando.length) return '';
    const valorFaltando = faltando.reduce((a,m)=>{
        const p = state.subscriptionPlans.find(x=>x.id===m.planId); return a+(p?
Number(p.price):0);
    },0);
    return `<div class="card" style="padding:15px 18px;margin-
bottom:14px;border-color:var(--warn);">
        <div style="display:flex;justify-content:space-between;align-
items:center;gap:12px;flex-wrap:wrap;">
            <div style="font-size:13px;line-height:1.6;">
                <b>${faltando.length} assinante(s) ainda não pagaram este mês</b> -
${BRL(valorFaltando)} a receber.<br>
                <span style="color:var(--muted-2);font-size:11.5px;">O valor recebido
zera todo dia 1º e vai subindo conforme os pagamentos entram.</span>
            </div>
            <button class="btn btn-ghost" id="btn-ver-faltando" style="flex-
shrink:0;">Ver quem falta</button>
        </div>
    </div>`;
})() }

${state.onlineBooking?.slug ? `
    <div class="card" style="padding:16px 18px;margin-bottom:20px;">
        <div style="display:flex;justify-content:space-between;align-
items:center;flex-wrap:wrap;gap:12px;">
            <div style="min-width:0;">
                <div style="font-size:12px;color:var(--muted-2);margin-

```

```

bottom:5px; ">Página pública de planos – mande esse link para seus clientes
assinarem</div>
    <div class="font-mono" style="font-size:12.5px;color:var(--gold-
bright);word-break:break-
all;">${window.location.origin}${window.location.pathname}?
planos=${esc(state.onlineBooking.slug)}</div>
    </div>
    <div style="display:flex;gap:8px;flex-shrink:0;">
        <a href="?planos=${esc(state.onlineBooking.slug)}" target="_blank"
class="btn btn-ghost" style="text-decoration:none;">${ICON.globe} Ver página</a>
        <button class="btn btn-ghost" id="btn-copy-plans-link">${ICON.copy}
Copiar</button>
        <button class="btn btn-ghost" id="btn-copy-portal-link">${ICON.lock}
Link do portal</button>
    </div>
</div>
</div>
` : ''}

    <div class="card" style="padding:18px 20px;margin-bottom:20px;">
        <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:4px;">
            <div class="font-display" style="font-size:16px;font-weight:600;">Pedidos
dos clientes (renovação / troca)</div>
            <span class="pill pill-warn" id="cli-req-count">carregando...</span>
        </div>
        <div style="font-size:12px;color:var(--muted-2);margin-bottom:12px;">Vindos
do portal do cliente.</div>
        <div id="cli-req-list">Carregando...</div>
    </div>

    <div class="card" style="padding:18px 20px;margin-bottom:20px;">
        <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:4px;">
            <div class="font-display" style="font-size:16px;font-weight:600;">Pedidos
de assinatura</div>
            <span class="pill pill-warn" id="sub-req-count">carregando...</span>
        </div>
        <div style="font-size:12px;color:var(--muted-2);margin-
bottom:12px;">Clientes que pediram assinatura pela página pública, aguardando você
criar a cobrança.</div>
        <div id="sub-req-list">Carregando...</div>
    </div>

    <div class="card" style="padding:18px 20px;margin-bottom:26px;">
        <div style="display:flex;justify-content:space-between;align-
items:center;flex-wrap:wrap;gap:10px;margin-bottom:14px;">

```

```

        <div class="font-display" style="font-size:16px;font-weight:600;">Recebimento via Asaas (Pix/Cartão)</div>
        <button class="btn btn-ghost" id="btn-sync-asaas">${ICON.link} Sincronizar com Asaas</button>
    </div>
    <div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:12px;">
        ${statCard('Pagos', sum.asaasPago.length, BRL(sum.totalRecebidoAsaas)+' recebidos', 'ok')}
        ${statCard('Pendentes', sum.asaasPendente.length, 'aguardando cobrança', sum.asaasPendente.length>0?'warn':'ok')}
        ${statCard('Em atraso', sum.asaasAtrasado.length, 'cobrança vencida', sum.asaasAtrasado.length>0?'bad':'ok')}
    </div>
    <div style="font-size:11.5px;color:var(--muted-2);margin-top:12px;">Considera apenas o valor das cobranças de assinatura pagas no Asaas – não o saldo disponível na conta.</div>
</div>

<div class="card" style="padding:18px 20px;margin-bottom:26px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:14px;">Recebimento em dinheiro</div>
    <div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:12px;">
        ${statCard('Pagos em dinheiro', sum.cashPago.length, BRL(sum.totalRecebidoDinheiro)+' recebidos', 'ok')}
        ${statCard('Aguardando pagamento', sum.cashPendente.length, 'marcar manualmente ao receber', sum.cashPendente.length>0?'warn':'ok')}
    </div>
</div>

<div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:10px;">Planos</div>
<div style="display:grid;grid-template-columns:repeat(auto-fill,minmax(220px,1fr));gap:14px;margin-bottom:26px;">
    ${state.subscriptionPlans.map(p=>`
        <div class="card" style="padding:16px 18px;">
            <div style="display:flex;justify-content:space-between;align-items:flex-start;gap:8px;">
                <div style="font-weight:600;font-size:15px;">${esc(p.name)}</div>
                ${p.category ? `<span class="pill" style="flex-shrink:0;">${esc(p.category)}</span>` : ''}
            </div>
            <div class="font-mono" style="font-size:19px;color:var(--gold-bright);margin:6px 0;">${BRL(p.price)}<span style="font-size:12px;color:var(--muted-2);">/mês</span></div>
            ${p.includedServiceIds && p.includedServiceIds.length ? `<div

```

```

style="font-size:11.5px;color:var(--muted);margin-bottom:4px;">Inclui:
${esc(p.includedServiceIds.map(id=>serviceById(id)?.name).filter(Boolean).join(',
'))}</div>` : ''}

    ${p.discountPercent>0 ? `<div style="font-size:11.5px;color:var(--
muted);margin-bottom:4px;">${p.discountPercent}% de desconto em serviços
extras</div>` : ''}

    ${p.benefits ? `<div style="font-size:12.5px;color:var(--
muted);">${esc(p.benefits)}</div>` : ''}

    <div style="display:flex;gap:6px;margin-top:12px;">
      <button class="btn btn-ghost" style="flex:1;justify-
content:center;padding:6px;" data-edit-plan="${p.id}">${ICON.edit}</button>
      <button class="btn btn-ghost" style="padding:6px 9px;color:var(--
bad);border-color:var(--bad);" data-del-plan="${p.id}">${ICON.trash}</button>
    </div>
  </div>
  `).join('')
  ${state.subscriptionPlans.length===0?emptyState('Nenhum plano criado
ainda.'):''}
</div>

  <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Membros</div>

  ${state.subscriptionMembers.length===0 ? emptyState('Nenhum membro cadastrado
ainda.') : `
    <table>
      <thead><tr><th>Membro</th><th>Plano</th><th>Método</th><th>Vencimento</th>
<th>Status</th><th></th></tr></thead>
      <tbody>
        ${state.subscriptionMembers.map(m=>{
          const plan = state.subscriptionPlans.find(p=>p.id===m.planId);
          return `<tr>
            <td style="font-weight:500;">${esc(m.name)}<div style="font-
size:11.5px;color:var(--muted-2);font-weight:400;">${esc(m.phone)}</div></td>
            <td>${esc(plan?plan.name:'-')}</td>
            <td>${methodLabel(m)}</td>
            <td class="font-mono">${m.dueDate?new
Date(m.dueDate+'T00:00').toLocaleDateString('pt-BR'):'-'}</td>
            <td>${statusPillMember(m.status)}</td>
            <td style="text-align:right;white-space:nowrap;">
              ${m.invoiceUrl ? `<a href="${m.invoiceUrl}" target="_blank"
class="btn btn-ghost" style="padding:6px 10px;text-decoration:none;">${ICON.link}
Ver cobrança</a>` : ''}
              ${m.paymentMethod==='dinheiro' && m.status!=='pago' &&
m.status!=='cancelado' ? `<button class="btn btn-ghost" style="padding:6px
10px;color:var(--ok);border-color:var(--ok);" data-mark-cash="${m.id}">Marcar pago
(dinheiro)</button>` : ''}
            <button class="btn btn-ghost" style="padding:6px 10px;font-

```

```

size:12px;" data-access-member="{m.id}" title="Criar acesso ao portal do
cliente">${ICON.lock} Acesso</button>
        <button class="btn btn-ghost" style="padding:6px 8px;" data-edit-
member="{m.id}">${ICON.edit}</button>
        <button class="btn btn-ghost" style="padding:6px 8px;color:var(--
bad);border-color:var(--bad);" data-del-member="{m.id}">${ICON.trash}</button>
    </td>
</tr>`;
    }).join('')
</tbody>
</table>
`}

<div style="margin-top:14px;font-size:12px;color:var(--muted-
2);display:flex;align-items:center;gap:6px;">${ICON.link} Assinantes com pagamento
via Pix/Cartão são sincronizados com o Asaas em <span data-goto="config"
style="color:var(--gold-bright);cursor:pointer;text-
decoration:underline;">Integração Asaas</span>. Pagamentos em dinheiro são
marcados manualmente aqui.</div>
`;
view.querySelector('#btn-new-plan').addEventListener('click', ()=>planModal());
view.querySelector('#btn-new-member').addEventListener('click',
()=>memberModal());
view.querySelector('#btn-ver-faltando')?.addEventListener('click', ()=>{
    const mesAtual = todayISO().slice(0,7);
    const faltando = state.subscriptionMembers
        .filter(m=>m.status!=='cancelado' && !pagouNoMes(m, mesAtual))
        .sort((a,b)=>{
            // quem deve há mais tempo aparece primeiro
            const da = a.lastPaymentDate || '0000-00';
            const db = b.lastPaymentDate || '0000-00';
            return String(da).localeCompare(String(db));
        });
    openModal(`
        <div style="padding:22px;">
            <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:6px;">
                <div class="font-display" style="font-size:19px;font-weight:600;">Ainda
não pagaram este mês</div>
                <button class="btn btn-ghost" style="padding:6px;" id="m-
close">${ICON.x}</button>
            </div>
            <div style="font-size:12.5px;color:var(--muted);margin-
bottom:16px;">${faltando.length} assinante(s). Quem paga pelo Asaas atualiza
sozinho na sincronização.</div>
            <div style="display:flex;flex-direction:column;gap:8px;max-
height:52vh;overflow-y:auto;">
                ${faltando.map(m=>{

```

```

const p = state.subscriptionPlans.find(x=>x.id===m.planId);
const msg = `Olá ${m.name}! Passando para lembrar da mensalidade do
${p?p.name:'clube'}. Qualquer dúvida é só chamar!`;
// há quantos meses não paga?
const mesesSemPagar = (()=>{
  if(!m.lastPaymentDate) return null;
  const [ay,am] = String(m.lastPaymentDate).slice(0,7).split('-')
    .map(Number);
  const [hy,hm] = todayISO().slice(0,7).split('-').map(Number);
  return (hy-ay)*12 + (hm-am);
})();
return `<div style="display:flex;justify-content:space-between;align-items:center;gap:10px;padding:9px 0;border-bottom:1px solid var(--border);flex-wrap:wrap;">
  <div style="font-size:13px;min-width:160px;">
    <b>${esc(m.name)}</b> <span class="pill pill-muted" style="font-size:10px;">${esc(m.paymentMethod==='asaas'? 'Asaas': 'manual')}</span>
    ${mesesSemPagar!=null && mesesSemPagar>1 ? `<span class="pill pill-bad" style="font-size:10px;">${mesesSemPagar} meses sem pagar</span>` : ''}
    ${mesesSemPagar===null ? `<span class="pill pill-warn" style="font-size:10px;">nunca pagou</span>` : ''}
    ${m.status==='pago' && !pagouNoMes(m, todayISO().slice(0,7)) ? `<span class="pill pill-warn" style="font-size:10px;">marcado pago, sem data</span>` : ''}<br>
    <span style="color:var(--muted-2);font-size:11.5px;">${p? esc(p.name)+' · '+BRL(p.price):'plano removido'}${m.lastPaymentDate? ' · último pgto '+new Date(String(m.lastPaymentDate).slice(0,10)+'T12:00').toLocaleDateString('pt-BR'):' · sem data de pagamento'}</span>
  </div>
  <div style="display:flex;gap:6px;flex-shrink:0;flex-wrap:wrap;">
    ${messageChannelRow(m.phone, msg)}
    ${m.paymentMethod!=='asaas'?`<button class="btn btn-gold" data-marcar-pago="${m.id}" style="padding:6px 10px;font-size:12px;">Recebi</button>`:''}
  </div>
</div>`;
}).join('')}
</div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelectorAll('[data-marcar-pago]').forEach(b=>b.addEventListener('click', ()=>{
    update(s=>{
      const m = s.subscriptionMembers.find(x=>x.id===b.dataset.marcarPago);
      const p = s.subscriptionPlans.find(x=>x.id===m.planId);

```

```

        m.status = 'pago';
        m.lastPaymentDate = todayISO();
        m.lastPaymentAmount = p ? Number(p.price) : 0;
    });
    showToast('Pagamento registrado.');
```

closeModal();

renderClube(view);

}});

});

view.querySelector('#btn-orfaos')?.addEventListener('click', ()=>{

const orfaos = assinantesOrfaos();

openModal(`

<div style="padding:22px;">

<div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:6px;">

<div class="font-display" style="font-size:19px;font-weight:600;">Assinantes sem plano</div>

<button class="btn btn-ghost" style="padding:6px;" id="m-close">\${ICON.x}</button>

</div>

<div style="font-size:12.5px;color:var(--muted);line-height:1.6;margin-bottom:16px;">

O plano destes assinantes foi excluído. Escolha um plano novo para cada um,

ou deixe em branco para cancelar a assinatura.

</div>

<div style="display:flex;flex-direction:column;gap:11px;max-height:52vh;overflow-y:auto;">

`\${orfaos.map(m=>`

<div style="padding:12px 14px;border:1px solid var(--border);border-radius:10px;">

<div style="font-size:13.5px;font-weight:600;">\${esc(m.name)}</div>

<div style="font-size:11.5px;color:var(--muted-2);margin-bottom:9px;">

status atual: \${esc(m.status)} \${m.phone?' · '+esc(m.phone):''}

</div>

<select data-orfao="\${m.id}" style="width:100%;font-size:13px;">

<option value="">- cancelar esta assinatura -</option>

\${state.subscriptionPlans.map(p=>`<option`

value="\${p.id}">\${esc(p.name)} - \${BRL(p.price)}</option>`).join('')}

</select>

</div>`).join('')}

</div>

<button class="btn btn-gold" id="orf-salvar" style="width:100%;justify-content:center;margin-top:16px;padding:13px;">Salvar</button>

</div>

```

`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#orf-salvar').addEventListener('click', ()=>{
    let movidos=0, cancelados=0;
    update(s=>{
      root.querySelectorAll('[data-orfao]').forEach(sel=>{
        const m = s.subscriptionMembers.find(x=>x.id===sel.dataset.orfao);
        if(!m) return;
        if(sel.value){ m.planId = sel.value; movidos++; }
        else { m.status = 'cancelado'; cancelados++; }
      });
    });
    closeModal();
    showToast(`${movidos} movido(s) para outro plano, ${cancelados}
cancelado(s).`);
    renderClube(view);
  });
});
view.querySelector('#btn-sync-corrigir')?.addEventListener('click', syncAsaas);
view.querySelector('#btn-datar-pgto')?.addEventListener('click', ()=>{
  const semData = (state.subscriptionMembers||[]).filter(m=>m.status==='pago' &&
!m.lastPaymentDate);
  if(!confirm(`Registrar ${semData.length} pagamento(s) com a data de hoje?
\n\nUse isso só se eles realmente pagaram neste mês.`)) return;
  update(s=>{
    s.subscriptionMembers.forEach(m=>{
      if(m.status==='pago' && !m.lastPaymentDate){
        const p = s.subscriptionPlans.find(x=>x.id===m.planId);
        m.lastPaymentDate = todayISO();
        m.lastPaymentAmount = p ? Number(p.price) : 0;
      }
    });
  });
  showToast('Pagamentos registrados. O valor do mês foi atualizado.');
```

renderClube(view);

```

});
view.querySelector('#btn-vinculos')?.addEventListener('click',
()=>vinculosModal(view));
view.querySelector('#btn-sync-asaas').addEventListener('click', syncAsaas);
view.querySelector('#btn-copy-plans-link')?.addEventListener('click', ()=>{

navigator.clipboard?.writeText(`${window.location.origin}${window.location.pathname}
planos=${state.onlineBooking.slug}`);
  showToast('Link da página de planos copiado.');
```

});

```

view.querySelector('#btn-copy-portal-link')?.addEventListener('click', ()=>{

```

```

navigator.clipboard?.writeText(`${window.location.origin}${window.location.pathname}
cliente=${state.onlineBooking.slug}`);
    showToast('Link do portal do cliente copiado.');
```

```
    });
```

```
    loadSubscriptionRequests(view);
```

```
    loadClientRequests(view);
```

```
    view.querySelectorAll('[data-edit-plan]').forEach(b=>b.addEventListener('click',
    ()=>planModal(state.subscriptionPlans.find(p=>p.id===b.dataset.editPlan))));
```

```
    view.querySelectorAll('[data-del-plan]').forEach(b=>b.addEventListener('click',
    ()=>{
```

```
        const planoId = b.dataset.delPlan;
```

```
        const plano = state.subscriptionPlans.find(p=>p.id===planoId);
```

```
        const usando = state.subscriptionMembers.filter(m=>m.planId===planoId &&
        m.status!=='cancelado');
```

```
        if(usando.length > 0){
```

```
            const nomes = usando.slice(0,5).map(m=>m.name).join(', ') +
```

```
(usando.length>5?` e mais ${usando.length-5}:`:'');
```

```
            if(!confirm(
```

```
                `${usando.length} assinante(s) estão neste plano:\n${nomes}\n\n` +
```

```
                `Ao excluir o plano, essas assinaturas serão CANCELADAS – elas somem das
                cobranças e dos avisos de atraso.\n\n` +
```

```
                `Continuar?`
```

```
            )) return;
```

```
        }
```

```
        update(s=>{
```

```
            s.subscriptionPlans = s.subscriptionPlans.filter(p=>p.id!==planoId);
```

```
            // não deixa assinante órfão: quem estava nesse plano é cancelado
```

```
            s.subscriptionMembers.forEach(m=>{
```

```
                if(m.planId===planoId){
```

```
                    m.status = 'cancelado';
```

```
                    m.canceladoPorPlanoRemovido = true;
```

```
                }
```

```
            });
```

```
        });
```

```
        showToast(usando.length>0
```

```
            ? `Plano excluído e ${usando.length} assinatura(s) cancelada(s).`
```

```
            : 'Plano excluído.');
```

```
        ));
```

```
        view.querySelectorAll('[data-access-
        member]').forEach(b=>b.addEventListener('click',
        ()=>clientAccessModal(state.subscriptionMembers.find(m=>m.id===b.dataset.accessMemk
```

```
        view.querySelectorAll('[data-edit-
```

```
        member]').forEach(b=>b.addEventListener('click',
```

```
        ()=>memberModal(state.subscriptionMembers.find(m=>m.id===b.dataset.editMember))));
```

```
        view.querySelectorAll('[data-del-
```

```
        member]').forEach(b=>b.addEventListener('click', ()=>update(s=>{
```

```

s.subscriptionMembers =
s.subscriptionMembers.filter(m=>m.id!==b.dataset.delMember); }));
  view.querySelectorAll('[data-mark-cash]').forEach(b=>b.addEventListener('click',
  ()=>{
    update(s=>{
      const m = s.subscriptionMembers.find(mm=>mm.id===b.dataset.markCash);
      const plan = s.subscriptionPlans.find(p=>p.id===m.planId);
      m.status = 'pago'; m.lastPaymentDate = todayISO(); m.lastPaymentAmount =
plan?plan.price:0;
    });
    showToast('Pagamento em dinheiro registrado.');
```

```

  }));
  view.querySelector('[data-goto="config"]')?.addEventListener('click', ()=>{
route='config'; render(); });
}
let asaasImportData = null;
function confirmAsaasImport(view){
  if(!asaasImportData) return;
  const { members, plans } = asaasImportData;
  let planos=0, novos=0, atualizados=0;
  update(s=>{
    // cria os planos que ainda não existem, guardando o mapeamento
    const keyToPlanId = {};
    plans.forEach(p=>{
      const existente =
s.subscriptionPlans.find(x=>x.name.trim().toLowerCase()===p.name.trim().toLowerCase
&& Number(x.price)===Number(p.price));
      if(existente){ keyToPlanId[p.key] = existente.id; return; }
      const id = uid();
      s.subscriptionPlans.push({id, name:p.name, category:'Importado do Asaas',
price:p.price, discountPercent:0, includedServiceIds:[], benefits:''});
      keyToPlanId[p.key] = id;
      planos++;
    });
    // cria/atualiza os assinantes
    members.forEach(m=>{
      const existente =
s.subscriptionMembers.find(x=>x.asaasSubscriptionId===m.asaasSubscriptionId);
      const dados = {
        name:m.name, phone:m.phone, planId:keyToPlanId[m.planKey],
        paymentMethod:'asaas', dueDate:m.dueDate, status:m.status,
        asaasSubscriptionId:m.asaasSubscriptionId,
asaasCustomerId:m.asaasCustomerId,
      };
      if(existente){ Object.assign(existente, dados); atualizados++; }
      else { s.subscriptionMembers.push({id:uid(), asaasChargeId:'',
invoiceUrl:'', lastPaymentDate:null, lastPaymentAmount:0, ...dados}); novos++; }
```

```

    // cria também na lista de Clientes, para aparecer na busca do agendamento
    const nomeNorm = m.name.trim().toLowerCase();
    const jaCliente = s.clients.find(x=>x.name.trim().toLowerCase()===nomeNorm);
    if(!jaCliente){
        s.clients.push({id:uid(), name:m.name.trim(), phone:m.phone||'',
birthday:'', points:0, notes:''});
    } else if(!jaCliente.phone && m.phone){
        jaCliente.phone = m.phone;
    }
    });
    });
    asaasImportData = null;
    showToast(`Importado: ${novos} assinante(s) novo(s), ${atualizados}
atualizado(s), ${planos} plano(s) criado(s).`);
    renderConfig(view);
}
async function syncAsaas(){
    if(!state.asaas.apiKey){
        showToast('Configure sua API Key do Asaas em Integração Asaas antes de
sincronizar.');
```

return;

```

    }
    showToast('Sincronizando com o Asaas...');
    const { data: result, error } = await supa.functions.invoke('asaas-sync', {
body: { action:'sync_status' } });
    if(error){ showToast('Falha ao chamar o servidor: '+(error.message||'erro
desconhecido'))}; return; }
    if(result?.error){ showToast(result.error); return; }
    if(result?.note){ showToast(result.note); return; }
    let novosPagamentos = 0;
    const mesAtual = todayISO().slice(0,7);
    update(s=>{
        (result.results||[]).forEach(r=>{
            const m = s.subscriptionMembers.find(x=>x.id===r.memberId);
            if(!m) return;
            const antes = m.lastPaymentDate;
            // Se você já registrou o pagamento deste mês na mão (dinheiro, por
            // exemplo), a sincronização não desfaz isso.
            const jaRegistradoNoMes = antes && String(antes).slice(0,7) === mesAtual;
            if(jaRegistradoNoMes && !r.pagouEsteMes){
                return; // mantém o que você marcou
            }
            m.status = r.status;
            if(r.lastPaymentDate) m.lastPaymentDate = r.lastPaymentDate;
            if(r.lastPaymentAmount) m.lastPaymentAmount = r.lastPaymentAmount;
            if(r.invoiceUrl) m.invoiceUrl = r.invoiceUrl;

```

```

    // Atualiza o vencimento com o que o Asaas tem de mais recente - sem
    // isso, uma data antiga ficava guardada e o sistema achava "venceu"
    // de novo mesmo já corrigido o status, tirando o benefício do plano.
    if(r.dueDate) m.dueDate = r.dueDate;
    // Status "pago" sem data era resquício da importação. Se o Asaas diz
    // que não pagou este mês, limpamos para não parecer em dia.
    if(!r.pagouEsteMes && !r.lastPaymentDate) m.lastPaymentDate = null;
    if(r.pagouEsteMes && r.lastPaymentDate !== antes) novosPagamentos++;
  });
});
const total = (result.results||[]).length;
showToast(novosPagamentos > 0
  ? `Sincronizado - ${novosPagamentos} pagamento(s) novo(s) deste mês
encontrado(s).`
  : `Sincronizado - ${total} assinante(s) verificados, nenhum pagamento novo
neste mês.`);
}
function planModal(existing){
  const isEdit = !!existing;
  const selectedIds = new Set(existing?.includedServiceIds || []);
  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:16px;">
        <div class="font-display" style="font-size:19px;font-
weight:600;">${isEdit?'Editar':'Novo'} plano</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
      </div>
      <div style="display:flex;flex-direction:column;gap:12px;">
        <div><label>Nome do plano</label><input id="f-name"
value="${esc(existing?.name)}" placeholder="Ex: Plano Ouro" style="width:100%;">
</div>
        <div><label>Categoria</label><input id="f-category" list="f-category-list"
value="${esc(existing?.category)}" placeholder="Ex: Corte, Combo, Premium"
style="width:100%;">
          <datalist id="f-category-list">${[...new
Set(state.subscriptionPlans.map(p=>p.category).filter(Boolean))].map(c=>`<option
value="${esc(c)}">`).join('')}</datalist>
        </div>
        <div><label>Valor mensal (R$)</label><input id="f-price" type="number"
step="0.01" min="0" value="${esc(existing?.price)}" style="width:100%;"></div>
        <div>
          <label>Serviços incluídos no plano</label>
          <div style="display:flex;flex-direction:column;gap:6px;max-
height:160px;overflow-y:auto;border:1px solid var(--border);border-
radius:9px;padding:10px;">

```

```

        ${state.services.length===0 ? '<div style="font-size:12px;color:var(--
muted);">Cadastro serviços primeiro.</div>' : state.services.map(s=>`
        <label style="display:flex;align-items:center;gap:8px;font-
size:13px;cursor:pointer;">
            <input type="checkbox" class="f-service-check" value="${s.id}"
${selectedIds.has(s.id)?'checked':''} style="width:auto;">
            ${esc(s.name)} – ${BRL(s.price)}
        </label>
        `).join('')}
    </div>
</div>
<div>
    <label>Dias em que o plano é válido</label>
    <div style="display:flex;gap:5px;flex-wrap:wrap;margin-bottom:6px;">
        ${WEEKDAY_LABELS.map((lbl,idx)=>`<label style="display:flex;flex-
direction:column;align-items:center;gap:3px;font-size:11px;color:var(--
muted);cursor:pointer;">
            <span>${lbl}</span>
            <input type="checkbox" class="f-plan-day" data-day="${idx}"
${(existing?.validDays||[0,1,2,3,4,5,6]).includes(idx)?'checked':''}
style="width:auto;">
        </label>`).join('')}
    </div>
    <div id="plan-days-preview" style="font-size:11.5px;color:var(--gold-
bright);"></div>
    <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">Fora
desses dias, o assinante paga o valor cheio. Essa frase aparece automaticamente na
página pública.</div>
</div>
    <div><label>Desconto do assinante em serviços e produtos extras (%)
</label><input id="f-discount" type="number" step="1" min="0" max="100"
value="${existing?.discountPercent??0}" style="width:100%;">
        <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">Vale
para qualquer serviço fora dos incluídos no plano e também para a venda de
produtos.</div>
    </div>
    <div>
        <label>Benefícios para a página pública (um por linha)</label>
        <textarea id="f-benefits-list" rows="5" placeholder="Corte
ilimitado&#10;Barba ilimitada&#10;15% de desconto em produtos&#10;Válido de Seg a
Qui" style="width:100%;resize:vertical;">${esc((existing?.benefitsList||
[]).join('\n'))}</textarea>
        <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">Cada
linha vira um item com ✓ na página de planos que seus clientes veem.</div>
    </div>
    <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;cursor:pointer;"><input type="checkbox" id="f-featured"

```

```

    ${existing?.featured?'checked':''} style="width:auto;"> Destacar como "Melhor
opção" na página pública</label>
    <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;cursor:pointer;"><input type="checkbox" id="f-hidden"
    ${existing?.hidden?'checked':''} style="width:auto;"> Não mostrar este plano na
página pública</label>
    <div><label>Observação interna (opcional)</label><input id="f-benefits"
value="${esc(existing?.benefits)}" placeholder="Só você vê isso"
style="width:100%;"></div>
    <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">${isEdit?'Salvar':'Criar plano'}</button>
    </div>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    const atualizaPreviewDias = ()=>{
        const dias = [...root.querySelectorAll('.f-plan-
day:checked')].map(el=>parseInt(el.dataset.day));
        root.querySelector('#plan-days-preview').textContent = dias.length ?
planDaysLabel(dias) : 'Nenhum dia marcado – o plano não valeria nunca.';
    };
    root.querySelectorAll('.f-plan-day').forEach(el=>el.addEventListener('change',
atualizaPreviewDias));
    atualizaPreviewDias();
    root.querySelector('#f-submit').addEventListener('click', ()=>{
        const name = root.querySelector('#f-name').value.trim();
        const category = root.querySelector('#f-category').value.trim();
        const price = parseFloat(root.querySelector('#f-price').value);
        const discountPercent = parseFloat(root.querySelector('#f-
discount').value)||0;
        const includedServiceIds = [...root.querySelectorAll('.f-service-
check:checked')].map(c=>c.value);
        const benefits = root.querySelector('#f-benefits').value.trim();
        const benefitsList = root.querySelector('#f-benefits-
list').value.split('\n').map(l=>l.trim()).filter(Boolean);
        const featured = root.querySelector('#f-featured').checked;
        const hidden = root.querySelector('#f-hidden').checked;
        const validDays = [...root.querySelectorAll('.f-plan-
day:checked')].map(el=>parseInt(el.dataset.day));
        if(!name || isNaN(price)){ showToast('Preencha nome e valor.');
```

```

    else{ s.subscriptionPlans.push({id:uid(), ...dados}); }
  });
  closeModal();
});
});
}

function memberModal(existing, prefill){
  if(state.subscriptionPlans.length===0){ showToast('Crie um plano antes de
adicionar membros.');
```

```

2);border:1px solid var(--gold);font-size:11.5px;color:var(--muted);line-
height:1.6;">
    ${ICON.link} <b style="color:var(--gold-bright);">Cobrança ativa no
Asaas.</b>

    Se você trocar o plano, o valor lá é atualizado automaticamente –
inclusive nas cobranças ainda não pagas.

    </div>
    ` : ''}
    ${isEdit ? `<div>
        <label>Cliente vinculado na agenda</label>
        <select id="f-linked" style="width:100%;">
            <option value="">Automático (pelo nome)</option>
            ${ (state.clients||[]).slice().sort(byName).map(cl=>`<option
value="${esc(cl.name)}"
${existing?.linkedClientName===cl.name?'selected':''}>${esc(cl.name)}${cl.phone?'
'+esc(cl.phone):''}</option>`).join('')}
        </select>
        <div style="font-size:11px;color:var(--muted-2);margin-top:4px;">Use
quando o nome no clube for diferente do nome na agenda (apelido, nome incompleto,
etc).</div>
    </div>` : ''}
    <div><label>Plano</label><select id="f-plan"
style="width:100%;">${state.subscriptionPlans.map(p=>`<option value="${p.id}"
${existing?.planId===p.id?'selected':''}>${esc(p.name)} –
${BRL(p.price)}/mês</option>`).join('')}</select></div>
    <div><label>Forma de pagamento</label><select id="f-method"
style="width:100%;">
        <option value="asaas"
${!existing||existing.paymentMethod==='asaas'? 'selected':''}>Pix / Cartão (via
Asaas)</option>
        <option value="dinheiro"
${existing?.paymentMethod==='dinheiro'? 'selected':''}>Dinheiro</option>
    </select></div>
    <div style="display:flex;gap:10px;">
        <div style="flex:1;"><label>Vencimento</label><input id="f-due"
type="date" value="${existing?.dueDate||todayISO()}" style="width:100%;"></div>
        <div style="flex:1;"><label>Status</label><select id="f-status"
style="width:100%;">
            <option value="pago"
${existing?.status==='pago'? 'selected':''}>Pago</option>
            <option value="pendente"
${!existing||existing.status==='pendente'? 'selected':''}>Pendente</option>
            <option value="atrasado"
${existing?.status==='atrasado'? 'selected':''}>Atrasado</option>
            <option value="cancelado"
${existing?.status==='cancelado'? 'selected':''}>Cancelado</option>
        </select></div>

```

```

    </div>
    <div style="font-size:11.5px;color:var(--muted-2);">Membros via Pix/Cartão
têm o status atualizado pela sincronização com o Asaas. Membros em dinheiro são
marcados como pagos manualmente na lista.</div>
    <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">${isEdit?'Salvar':'Adicionar membro'}</button>
  </div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);

  // Busca clientes já cadastrados na barbearia para virar assinante
  if(!isEdit){
    const inpNome = root.querySelector('#f-name');
    const caixa = root.querySelector('#mem-sug');
    const nrm = s =>
String(s||'').toLowerCase().normalize('NFD').replace(/[\u0300-
\u036f]/g, '').trim();
    const fechar = ()=>{ caixa.style.display='none'; };
    const buscar = ()=>{
      const termo = nrm(inpNome.value);
      if(termo.length < 2){ fechar(); return; }
      // não sugere quem já é assinante
      const jaSao = new Set((state.subscriptionMembers||[])
        .filter(m=>m.status!=='cancelado').map(m=>nrm(m.name)));
      const achados = (state.clients||[])
        .filter(cl=>nrm(cl.name).includes(termo) && !jaSao.has(nrm(cl.name)))
        .sort((a,b)=>{
          const ai = nrm(a.name).startsWith(termo)?0:1, bi =
nrm(b.name).startsWith(termo)?0:1;
          return ai!==bi ? ai-bi : a.name.localeCompare(b.name,'pt-BR');
        }).slice(0,8);
      if(!achados.length){ fechar(); return; }
      caixa.innerHTML = achados.map(cl=>`
        <div class="mem-item" data-nome="${esc(cl.name)}" data-
tel="${esc(cl.phone)}" style="padding:10px 14px;cursor:pointer;border-bottom:1px
solid var(--border);">
          <div style="font-size:13.5px;font-weight:500;">${esc(cl.name)}</div>
          <div style="font-size:11.5px;color:var(--muted-2);margin-
top:2px;">${esc(cl.phone)}||'sem telefone'</div>
        </div>`).join('');
      caixa.style.display='block';
      caixa.querySelectorAll('.mem-item').forEach(el=>{
        el.addEventListener('mousedown', e=>{
          e.preventDefault();
          inpNome.value = el.dataset.nome;
          const tel = root.querySelector('#f-phone');

```

```

        if(!tel.value && el.dataset.tel) tel.value = el.dataset.tel;
        fechar();
    });
    el.addEventListener('mouseenter', ()=>{ el.style.background='var(--
panel-2)'; });
    el.addEventListener('mouseleave', ()=>{
el.style.background='transparent'; });
    });
};
inpNome.addEventListener('input', buscar);
inpNome.addEventListener('focus', buscar);
inpNome.addEventListener('blur', ()=>setTimeout(fechar, 120));
}

root.querySelector('#f-submit').addEventListener('click', async ()=>{
    const name = root.querySelector('#f-name').value.trim();
    if(!name){ showToast('Informe o nome.');
```

return; }

```

    const phone = root.querySelector('#f-phone').value.trim();
    const planId = root.querySelector('#f-plan').value;
    const paymentMethod = root.querySelector('#f-method').value;
    const email = root.querySelector('#f-email')?.value.trim() || '';
    const linkedClientName = root.querySelector('#f-linked')?.value ||
(existing?.linkedClientName || '');
    const cpf = root.querySelector('#f-cpf').value.trim();
    const novoStatus = root.querySelector('#f-status').value;
    const data = {
        name, phone, email, cpf, planId, paymentMethod, linkedClientName,
        dueDate: root.querySelector('#f-due').value,
        status: novoStatus,
    };
    // Marcar como "pago" na mão precisa registrar a DATA do pagamento -
    // é ela que faz o valor entrar no "recebido no mês" e no pote.
    const mesAtual = todayISO().slice(0,7);
    if(novoStatus === 'pago'){
        const jaContabilizado = existing?.lastPaymentDate &&
            String(existing.lastPaymentDate).slice(0,7) === mesAtual;
        if(!jaContabilizado){
            const planoEscolhido = state.subscriptionPlans.find(p=>p.id===planId);
            data.lastPaymentDate = todayISO();
            data.lastPaymentAmount = planoEscolhido ? Number(planoEscolhido.price) :
0;
        }
    } else if(existing?.lastPaymentDate &&
String(existing.lastPaymentDate).slice(0,7) === mesAtual){
        // tirou de "pago" ainda no mesmo mês: desfaz o recebimento
        data.lastPaymentDate = null;
        data.lastPaymentAmount = 0;
    }
}

```

```

    }
    // Trocou de plano num assinante que já paga pelo Asaas?
    // Atualiza o valor lá também, senão continuaria cobrando o preço antigo.
    // Sempre reenvia o valor do plano atual para o Asaas ao salvar um
    // assinante Pix/Cartão. Se já estiver certo, nada muda; se estiver
    // defasado (troca feita antes desta correção), acerta agora.
    if(isEdit && paymentMethod==='asaas' && existing?.asaasSubscriptionId){
      const novoPlano = state.subscriptionPlans.find(p=>p.id===planId);
      if(novoPlano){
        const btnEdit = root.querySelector('#f-submit');
        btnEdit.disabled = true; btnEdit.textContent = 'Atualizando no
Asaas...';
        const { data: res, error } = await supa.functions.invoke('asaas-sync', {
          body: { action:'update_subscription', subscriptionId:
existing.asaasSubscriptionId,
                    planPrice: novoPlano.price, planName: novoPlano.name }
        });
        btnEdit.disabled = false; btnEdit.textContent = 'Salvar';
        if(error || res?.error){
          const msg = res?.error || error?.message || 'erro desconhecido';
          if(!confirm(
            `Não foi possível atualizar o valor no Asaas:\n${msg}\n\n` +
            `O cliente pode continuar sendo cobrado o valor antigo ` +
            `em vez de ${BRL(novoPlano.price)}.\n\nSalvar mesmo assim?`
          )) return;
        } else {
          showToast(`Valor no Asaas confirmado: ${BRL(novoPlano.price)}
(${novoPlano.name}).`);
        }
      }
    }
  }

  if(!isEdit && paymentMethod==='asaas' && !cpf.replace(/\D/g, '')){
    showToast('Para cobrar pelo Asaas é preciso informar o CPF do cliente.');
```

root.querySelector('#f-cpf').focus();

```

    return;
  }

  if(!isEdit && paymentMethod==='asaas'){
    if(!state.asaas.apiKey){ showToast('Configure sua API Key do Asaas em
Integração Asaas antes de cadastrar via Pix/Cartão.');
```

return; }

```

    const submitBtn = root.querySelector('#f-submit');
    submitBtn.disabled = true; submitBtn.textContent = 'Criando cobrança no
Asaas...';

    const plan = state.subscriptionPlans.find(p=>p.id===planId);
    const newMemberId = uid();
    const { data: result, error } = await supa.functions.invoke('asaas-sync',
{

```

```

        body: { action:'create_subscription', memberId: newMemberId,
memberName:name, memberPhone:phone,
                memberEmail:email, memberCpf:cpf, planPrice:plan.price,
planName:plan.name }
        });
        submitBtn.disabled = false; submitBtn.textContent =
isEdit?'Salvar':'Adicionar membro';
        if(error){ showToast('Falha ao chamar o servidor: '+(error.message||'erro
desconhecido'))}; return; }
        if(result?.error){ showToast(result.error); return; }
        update(s=>{ s.subscriptionMembers.push({id:newMemberId, asaasChargeId:'',
asaasSubscriptionId:result.subscriptionId, asaasCustomerId:result.customerId,
invoiceUrl:result.invoiceUrl, lastPaymentDate:null, lastPaymentAmount:0,
...data}); });
        showToast('Assinante cadastrado e cobrança recorrente criada no Asaas.');
```

closeModal();

return;

}

update(s=>{

if(isEdit){ const alvo =

s.subscriptionMembers.find(x=>x.id===existing.id); if(alvo) Object.assign(alvo,

data); }

else{ s.subscriptionMembers.push({id:uid(), asaasChargeId:'',

asaasSubscriptionId:'', asaasCustomerId:'', invoiceUrl:'', lastPaymentDate:null,

lastPaymentAmount:0, ...data}); }

// garante que o assinante também exista na lista de Clientes

const nn = name.trim().toLowerCase();

const jaCl = s.clients.find(x=>x.name.trim().toLowerCase()===nn);

if(!jaCl) s.clients.push({id:uid(), name:name.trim(), phone:phone||'',

birthday:'', points:0, notes:''});

else if(!jaCl.phone && phone) jaCl.phone = phone;

});

closeModal();

});

});

}

/* ===== CONFIG ASAAS ===== */

/* ===== CONFIGURAÇÕES DA CONTA ===== */

/* ===== MEUS LINKS (todos os endereços públicos num lugar só) =====

*/

/* Caixinha de feedback: o dono da barbearia conta o que travou ou o que

poderia melhorar, e isso chega por e-mail para o dono do sistema. */

function feedbackModal(){

const telaAtual = (NAV.find(n=>n.id===route)||{}).label || route;

openModal(`

<div style="padding:22px;">

```

    <div style="display:flex;justify-content:space-between;align-items:flex-
start;gap:10px;margin-bottom:8px;">
      <div>
        <div class="font-display" style="font-size:19px;font-weight:600;">O que
podemos melhorar?</div>
        <div style="font-size:12.5px;color:var(--muted);margin-top:4px;line-
height:1.6;">
          Achou um erro? Sentiu falta de alguma coisa? Escreve aqui – eu leio
tudo e vou ajustando o sistema.
        </div>
      </div>
      <button class="btn btn-ghost" style="padding:6px;flex-shrink:0;" id="m-
close">${ICON.x}</button>
    </div>

    <div id="fb-form" style="display:flex;flex-direction:column;gap:14px;margin-
top:16px;">
      <div>
        <label>Do que se trata?</label>
        <div style="display:grid;grid-template-columns:1fr 1fr;gap:8px;margin-
top:6px;">
          ${[
            {id:'bug', txt:'🐛 Algo com erro'},
            {id:'sugestao', txt:'💡 Uma ideia'},
            {id:'duvida', txt:'❓ Uma dúvida'},
            {id:'elogio', txt:'⭐ Um elogio'},
          ].map((o,i)=>`
            <label style="display:flex;align-items:center;gap:8px;padding:11px
12px;border:1px solid var(--border);border-radius:10px;cursor:pointer;font-
size:13px;">
              <input type="radio" name="fb-cat" value="${o.id}"
${i===0?'checked':''} style="width:auto;">
              <span>${o.txt}</span>
            </label>`)}
        </div>
      </div>
      <div>
        <label>Conta pra gente</label>
        <textarea id="fb-msg" rows="5" placeholder="Ex: na tela de comissões o
valor do assinante aparece cheio, mas ele não pagou nada..."
style="width:100%;resize:vertical;"></textarea>
      </div>
      <div>
        <label>Seu contato (para eu te responder)</label>
        <input id="fb-contato" value="${esc(currentShop?.email ||
state.shopProfile?.phone || '')}" placeholder="e-mail ou WhatsApp"
style="width:100%;">

```

```

    </div>
    <button class="btn btn-gold" id="fb-enviar" style="justify-
content:center;padding:13px;">Enviar</button>
    <div style="font-size:11px;color:var(--muted-2);text-align:center;line-
height:1.6;">
        Vai junto em qual tela você estava (${esc(telaAtual)}), para eu achar o
problema mais rápido.
    </div>
</div>

<div id="fb-done" style="display:none;text-align:center;padding:26px 0;">
    <div style="font-size:40px;margin-bottom:10px;">✔</div>
    <div style="font-size:16px;font-weight:600;margin-bottom:6px;">Recebido,
obrigado!</div>
    <div style="font-size:12.5px;color:var(--muted);line-height:1.6;">
        Seu retorno chega direto para quem cuida do sistema.<br>Se precisar,
entro em contato.
    </div>
    <button class="btn btn-ghost" id="fb-fechar" style="margin-
top:18px;">Fechar</button>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#fb-fechar')?.addEventListener('click', closeModal);
    root.querySelector('#fb-enviar').addEventListener('click', async (e)=>{
        const btn = e.currentTarget;
        const mensagem = root.querySelector('#fb-msg').value.trim();
        const contato = root.querySelector('#fb-contato').value.trim();
        const categoria = root.querySelector('input[name="fb-cat"]:checked').value;
        if(mensagem.length < 5){ showToast('Escreva um pouco mais para eu
entender.');
```

```

    btn.disabled = false; btn.textContent = 'Enviar';
    if(error){ showToast('Não foi possível enviar agora. Tente de novo em instantes.');
```

```

    return; }
    root.querySelector('#fb-form').style.display = 'none';
    root.querySelector('#fb-done').style.display = 'block';
  });
});
}

function renderMeusLinks(view){
  const slug = state.onlineBooking?.slug || '';
  const base = `${window.location.origin}${window.location.pathname}`;
  if(!slug){
    view.innerHTML = `${pageHeader('Meus Links','Endereços para divulgar sua barbearia')}`
    ${emptyState('Configure o identificador do link primeiro.','Vá em Agendamento Online e defina o endereço da sua barbearia.')};
    return;
  }
  const links = [
    { id:'agendar', titulo:'Agendamento rápido', icone:ICON.agenda,
      desc:'O cliente escolhe serviço, profissional e horário sem precisar de login. Ideal para divulgar no Instagram e no WhatsApp.',
      url:`${base}?agendar=${slug}` },
    { id:'cliente', titulo:'Área do cliente', icone:ICON.lock,
      desc:'Com login e senha. O cliente agenda, vê o plano dele, pede renovação e troca de plano. Ideal para os assinantes do clube.',
      url:`${base}?cliente=${slug}` },
    { id:'planos', titulo:'Página de planos', icone:ICON.club,
      desc:'Mostra seus planos do clube com preços e benefícios, para atrair novos assinantes.',
      url:`${base}?planos=${slug}` },
    { id:'equipe', titulo:'Acesso da equipe', icone:ICON.team,
      desc:'Somente para seus profissionais. Cada um entra pelo celular com o PIN dele. Não divulgue para clientes.',
      url:`${base}?equipe=${slug}`, restrito:true },
  ];

  view.innerHTML = `
    ${pageHeader('Meus Links','Todos os endereços da sua barbearia num lugar só')}
    <div style="display:flex;flex-direction:column;gap:16px;max-width:820px;">
      ${links.map(l=>`
        <div class="card" style="padding:20px 22px;${l.restrito?'border-color:var(--warn);':''}">
          <div style="display:flex;gap:18px;align-items:flex-start;flex-wrap:wrap;">

```

```

    <div style="flex:1;min-width:230px;">
      <div style="display:flex;align-items:center;gap:8px;margin-
bottom:6px;">
        <span style="color:var(--gold-bright);">${l.icone}</span>
        <span class="font-display" style="font-size:16px;font-
weight:600;">${l.titulo}</span>
        ${l.restrito?`<span class="pill pill-warn" style="font-
size:10.5px;">Uso interno</span>`:''}
      </div>
      <div style="font-size:12.5px;color:var(--muted);line-
height:1.6;margin-bottom:12px;">${l.desc}</div>
      <div class="font-mono" style="font-size:12px;color:var(--gold-
bright);word-break:break-all;background:var(--panel-2);padding:9px 11px;border-
radius:8px;margin-bottom:12px;">${esc(l.url)}</div>
      <div style="display:flex;gap:8px;flex-wrap:wrap;">
        <button class="btn btn-gold" data-copy="${esc(l.url)}"
style="padding:8px 13px;font-size:12.5px;">${ICON.copy} Copiar</button>
        <a href="${esc(l.url)}" target="_blank" class="btn btn-ghost"
style="padding:8px 13px;font-size:12.5px;text-decoration:none;">${ICON.globe}
Abrir</a>
        ${!l.restrito?`<button class="btn btn-ghost" data-
share="${esc(l.url)}" data-titulo="${esc(l.titulo)}" style="padding:8px 13px;font-
size:12.5px;">${ICON.send} Compartilhar</button>`:''}
      </div>
    </div>
    <div style="text-align:center;flex-shrink:0;">
      <div id="qr-${l.id}" style="background:#fff;padding:9px;border-
radius:10px;display:inline-block;"></div>
      <div style="font-size:10.5px;color:var(--muted-2);margin-
top:7px;">Aponte a câmera</div>
    </div>
  </div>
  `).join('')
</div>
<div style="font-size:12px;color:var(--muted-2);margin-top:18px;max-
width:820px;line-height:1.7;">
  Dica: imprima o QR Code do <b>Agendamento rápido</b> e deixe no balcão – o
cliente aponta a câmera e agenda sozinho, sem baixar nada.
</div>
`;

// desenha os QR Codes
links.forEach(l=>{
  const box = view.querySelector(`#qr-${l.id}`);
  if(box && typeof QRCode!=='undefined'){
    new QRCode(box, { text:l.url, width:118, height:118, correctLevel:

```

```

QRCode.CorrectLevel.M });
    } else if(box){
        box.innerHTML = `

QR indisponível<br>sem internet</div>`;
    }
});

view.querySelectorAll('[data-copy]').forEach(b=>b.addEventListener('click', ()=>{
    navigator.clipboard?.writeText(b.dataset.copy);
    showToast('Link copiado!');
}));
view.querySelectorAll('[data-share]').forEach(b=>b.addEventListener('click', ()=>{
    const url = b.dataset.share, titulo = b.dataset.titulo;
    if(navigator.share) navigator.share({ title:`${state.shopProfile.name} - ${titulo}`, url });
    else { navigator.clipboard?.writeText(url); showToast('Link copiado!'); }
}));
}

function renderMinhaAssinatura(view){
    const shop = currentShop;
    const limits = planLimits(shop);
    const statusInfo = {
        trialing: {label:'Em teste grátis', tone:'warn'},
        active: {label:'Ativa', tone:'ok'},
        past_due: {label:'Pagamento atrasado', tone:'bad'},
        canceled: {label:'Cancelada', tone:'bad'},
    }[shop.subscription_status] || {label: shop.subscription_status, tone:'muted'};

    let periodInfo = '';
    if(shop.subscription_status==='trialing' && shop.trial_ends_at){
        const daysLeft = Math.max(0, Math.ceil((new Date(shop.trial_ends_at) - new Date())/86400000));
        periodInfo = `Seu teste grátis termina em ${new Date(shop.trial_ends_at).toLocaleDateString('pt-BR')} (${daysLeft} dia(s) restante(s)).`;
    } else if(shop.subscription_status==='active' && shop.current_period_end){
        const daysLeft = Math.max(0, Math.ceil((new Date(shop.current_period_end) - new Date())/86400000));
        periodInfo = `Próxima renovação em ${new Date(shop.current_period_end).toLocaleDateString('pt-BR')} (${daysLeft} dia(s)). Se o pagamento falhar nessa data, a assinatura passa para "Pagamento atrasado".`;
    } else if(shop.subscription_status==='past_due'){
        periodInfo = 'Houve um problema com o último pagamento. Regularize pelo botão


```

```

abaixo para não perder o acesso.';
    } else if(shop.subscription_status==='canceled'){
        periodInfo = 'Sua assinatura foi cancelada. Assine novamente para recuperar o
acesso.';
    }

    view.innerHTML = `
        ${pageHeader('Configurações da Conta', 'Seu perfil de acesso e os dados da sua
assinatura do sistema')}

        <div class="card" style="padding:22px;margin-bottom:20px;max-width:560px;">
            <div class="font-display" style="font-size:15px;font-weight:600;margin-
bottom:14px;">Perfil da conta</div>
            <div style="display:flex;flex-direction:column;gap:14px;">
                <div><label>Seu nome (dono/gerente)</label><input id="f-owner-name"
value="${esc(state.ownerName)}" placeholder="Como podemos te chamar"
style="width:100%;"></div>
                <div><label>Email cadastrado</label><input value="${esc(shop.email)}"
disabled style="width:100%;opacity:.7;"></div>
                <button class="btn btn-ghost" id="btn-save-owner-name" style="width:fit-
content;">Salvar nome</button>
            </div>
            <div class="divider-blade" style="margin:18px 0;"></div>
            <div style="font-size:13px;font-weight:600;margin-bottom:4px;">🔔
Notificação mesmo com o site fechado</div>
            <div style="font-size:11.5px;color:var(--muted-2);margin-bottom:12px;line-
height:1.6;">
                Ative isso pra chegar na barra de notificações do celular ou do computador
– igual WhatsApp ou Facebook.
                Precisa ativar <b>em cada aparelho</b> que você quiser receber (celular,
PC, etc).
            </div>
            <button class="btn btn-gold" id="btn-ativar-push" style="font-
size:13px;">${ICON.bell} Ativar notificação neste dispositivo</button>
            <div id="push-status" style="font-size:11.5px;color:var(--muted-2);margin-
top:8px;"></div>

            <div class="divider-blade" style="margin:18px 0;"></div>
            <div style="font-size:13px;font-weight:600;margin-bottom:4px;">Alertas de
novo agendamento</div>
            <div style="font-size:11.5px;color:var(--muted-2);margin-bottom:12px;line-
height:1.6;">
                Som e vibração – funcionam só enquanto o sistema estiver aberto no
navegador.
            </div>
            <div style="display:flex;flex-direction:column;gap:10px;max-width:420px;">
                <label style="display:flex;align-items:center;gap:10px;font-

```

```

size:13.5px;cursor:pointer;">
    <input type="checkbox" id="f-alert-sound"
    ${state.alertSound!==false?'checked':''} style="width:auto;">
    <span>Tocar som</span>
</label>
<label style="display:flex;align-items:center;gap:10px;font-
size:13.5px;cursor:pointer;">
    <input type="checkbox" id="f-alert-vibrate"
    ${state.alertVibrate!==false?'checked':''} style="width:auto;">
    <span>Vibrar <span style="font-size:11px;color:var(--muted-2);">(Android
    – o iPhone não permite)</span></span>
</label>
<label style="display:flex;align-items:center;gap:10px;font-
size:13.5px;cursor:pointer;">
    <input type="checkbox" id="f-alert-push"
    ${state.alertPush!==false?'checked':''} style="width:auto;">
    <span>Mostrar notificação do sistema</span>
</label>
<div style="display:flex;gap:8px;flex-wrap:wrap;margin-top:4px;">
    <button class="btn btn-ghost" id="btn-test-alert" style="font-
size:12.5px;">${ICON.bell} Testar alerta</button>
    <button class="btn btn-ghost" id="btn-perm-alert" style="font-
size:12.5px;">${ICON.check} Permitir notificações</button>
</div>
<div id="alert-perm-status" style="font-size:11.5px;color:var(--muted-
2);"></div>
</div>

<div class="divider-blade" style="margin:18px 0;"></div>
<div style="font-size:13px;font-weight:600;margin-bottom:10px;">Trocar
senha</div>
<div style="display:flex;flex-direction:column;gap:12px;max-width:340px;">
    <div><label>Nova senha</label><input id="f-new-pass" type="password"
placeholder="Mínimo 6 caracteres" style="width:100%;"></div>
    <div><label>Confirmar nova senha</label><input id="f-new-pass2"
type="password" style="width:100%;"></div>
    <button class="btn btn-ghost" id="btn-change-pass" style="width:fit-
content;">Atualizar senha</button>
</div>
</div>

<div class="card" style="padding:24px;margin-bottom:20px;max-
width:560px;border-color:var(--gold);">
    <div class="font-display" style="font-size:15px;font-weight:600;margin-
bottom:14px;">Minha Assinatura</div>
    <div style="display:flex;justify-content:space-between;align-items:flex-
start;flex-wrap:wrap;gap:14px;">

```

```

    <div>
      <div style="font-size:11.5px;color:var(--muted-2);text-
transform:uppercase;letter-spacing:.06em;margin-bottom:6px;">Plano atual</div>
      <div class="font-display" style="font-size:24px;font-
weight:600;color:var(--gold-bright);">${limits.label}</div>
    </div>
    <span class="pill pill-${statusInfo.tone}" style="font-
size:13px;padding:6px 14px;">${statusInfo.label}</span>
  </div>
  ${periodInfo ? `<div style="font-size:13px;color:var(--muted);margin-
top:14px;padding-top:14px;border-top:1px solid var(--border);">${periodInfo}</div>` : ''}
  <div style="display:flex;gap:10px;flex-wrap:wrap;margin-top:16px;">
    <button class="btn btn-gold" id="btn-change-plan">Trocar de plano</button>
    <button class="btn btn-ghost" id="btn-open-portal">${ICON.link} Ver fatura
/ pagar / trocar cartão</button>
  </div>
</div>

<div class="card" style="padding:22px;max-width:560px;">
  <div class="font-display" style="font-size:15px;font-weight:600;margin-
bottom:14px;">O que está incluso no seu plano</div>
  <div style="display:flex;flex-direction:column;gap:10px;">
    <div style="display:flex;justify-content:space-between;font-size:13.5px;">
      <span style="color:var(--muted);">Profissionais cadastrados</span>
      <span class="font-
mono">${state.professionals.length}${limits.maxProfessionals===Infinity?'
(ilimitado)':'} de '+limits.maxProfessionals}</span>
    </div>
    <div style="display:flex;justify-content:space-between;font-size:13.5px;">
      <span style="color:var(--muted);">Clube de Assinatura</span>
      <span class="pill ${limits.club?'pill-ok':'pill-
muted'}">${limits.club?'Ativado':'Não incluso no Basic'}</span>
    </div>
  </div>
</div>
</div>
`;
view.querySelector('#btn-change-plan').addEventListener('click',
()=>renderPaywallUpgrade('minha-assinatura'));
['sound','vibrate','push'].forEach(k=>{
  const el = view.querySelector('#f-alert-'+k);
  el?.addEventListener('change', ()=>{
    const campo = 'alert' + k.charAt(0).toUpperCase() + k.slice(1);
    update(s=>{ s[campo] = el.checked; });
    if(k==='push' && el.checked) pedirPermissaoAlertas();
  });
});
});

```

```

view.querySelector('#btn-ativar-push')?.addEventListener('click', async (e)=>{
  const btn = e.currentTarget;
  btn.disabled = true; btn.textContent = 'Ativando...';
  await ativarNotificacaoPush();
  btn.disabled = false; btn.textContent = `${ICON.bell} Ativar notificação neste
dispositivo`;
  atualizaStatusPush();
});
const atualizaStatusPush = async ()=>{
  const el = view.querySelector('#push-status');
  if(!el) return;
  const st = await statusNotificacaoPush();
  if(st==='on') el.innerHTML = '<span style="color:var(--ok);">✓ Ativado neste
dispositivo.</span>';
  else if(st==='unsupported') el.textContent = 'Este navegador não suporta
(funciona no Chrome, Edge e similares).';
  else el.textContent = 'Ainda não ativado neste dispositivo.';
};
atualizaStatusPush();
view.querySelector('#btn-test-alert')?.addEventListener('click', ()=>{
  alertarNovoAgendamento({
    client_name:'Cliente Teste', service_name:'Corte', professional_name:'Você',
    date: todayISO(), time:'14:00'
  });
  showToast('Alerta de teste disparado.');
```

```

});
view.querySelector('#btn-perm-alert')?.addEventListener('click', async ()=>{
  await pedirPermissaoAlertas();
  atualizaStatusPermissao();
});
const atualizaStatusPermissao = ()=>{
  const el = view.querySelector('#alert-perm-status');
  if(!el) return;
  if(!('Notification' in window)){ el.textContent = 'Este navegador não suporta
notificações do sistema.'; return; }
  const p = Notification.permission;
  el.innerHTML = p==='granted'
    ? '<span style="color:var(--ok);">✓ Notificações permitidas neste
dispositivo.</span>'
    : p==='denied'
    ? '<span style="color:var(--bad);">Notificações bloqueadas – libere nas
configurações do navegador para este site.</span>'
    : 'Clique em "Permitir notificações" para ativar.';
};
atualizaStatusPermissao();
view.querySelector('#btn-save-owner-name').addEventListener('click', ()=>{
  const name = view.querySelector('#f-owner-name').value.trim();

```

```

    update(s=>{ s.ownerName = name; });
    showToast('Nome salvo.');
```

});

```

view.querySelector('#btn-change-pass').addEventListener('click', async ()=>{
    const p1 = view.querySelector('#f-new-pass').value;
    const p2 = view.querySelector('#f-new-pass2').value;
    if(p1.length<6){ showToast('A senha precisa ter pelo menos 6 caracteres.');
```

return; }

```

    if(p1!==p2){ showToast('As senhas não coincidem.');
```

return; }

```

    const { error } = await supa.auth.updateUser({ password: p1 });
    if(error){ showToast('Não foi possível trocar a senha: '+error.message);
return; }
```

```

    view.querySelector('#f-new-pass').value = '';
    view.querySelector('#f-new-pass2').value = '';
    showToast('Senha atualizada com sucesso.');
```

});

```

view.querySelector('#btn-open-portal').addEventListener('click', async (e)=>{
    const btn = e.currentTarget;
    const original = btn.textContent;
    btn.disabled = true; btn.textContent = 'Abrindo...';
    const { data: result, error } = await supa.functions.invoke('stripe-portal', {
body: { returnUrl: window.location.href } }));
    btn.disabled = false; btn.textContent = original;
    if(error || result?.error){ showToast(result?.error || 'Não foi possível abrir
o portal de faturas.');
```

return; }

```

    window.open(result.url, '_blank');
```

});

}

```

function renderConfig(view){
    const a = state.asaas;
    const sum = asaasSummary();
    view.innerHTML = `
        ${pageHeader('Integração Asaas', 'Cobranças recorrentes do clube de
assinatura')}
```

```

        <div class="card" style="padding:18px 20px;margin-bottom:20px;border-
color:var(--gold);">
```

```

            <div style="display:flex;gap:10px;align-items:flex-start;">
```

```

                <div style="color:var(--gold-bright);margin-top:2px;">${ICON.lock}</div>
```

```

                <div style="font-size:13.5px;color:var(--muted);line-height:1.7;">
```

Sua chave de API do Asaas fica guardada com segurança e é usada apenas pelo servidor

(nunca é exposta no navegador). Assinantes cadastrados com "Pix / Cartão" geram uma cobrança

recorrente de verdade no Asaas – o cliente recebe o link de pagamento, e o status é atualizado

```

        automaticamente aqui assim que ele paga.
    </div>
</div>
</div>

<div class="card" style="padding:20px;margin-bottom:20px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:14px;">Status das cobranças de assinatura (Pix/Cartão)</div>
    <div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:12px;">
        ${statCard('Pagos', sum.asaasPago.length, BRL(sum.totalRecebidoAsaas)+' já recebidos', 'ok')}
        ${statCard('Pendentes', sum.asaasPendente.length, 'ainda não venceram', sum.asaasPendente.length>0?'warn':'ok')}
        ${statCard('Em atraso', sum.asaasAtrasado.length, 'cobrança vencida sem pagamento', sum.asaasAtrasado.length>0?'bad':'ok')}
    </div>
    <div style="font-size:11.5px;color:var(--muted-2);margin-top:12px;">O valor somado é sempre o das cobranças de assinatura efetivamente pagas no Asaas – nunca o saldo disponível na conta.</div>
</div>

<div class="card" style="padding:20px;margin-bottom:20px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:14px;">Credenciais</div>
    <div style="display:flex;flex-direction:column;gap:12px;max-width:440px;">
        <div><label>Ambiente</label>
        <select id="f-env" style="width:100%;">
            <option value="sandbox" ${a.env==='sandbox'? 'selected':''}>Sandbox (testes)</option>
            <option value="production" ${a.env==='production'? 'selected':''}>Produção</option>
        </select>
    </div>
    <div><label>API Key do Asaas</label><input id="f-key" type="password" value="${esc(a.apiKey)}" placeholder="$aact_..." style="width:100%;"></div>
    <div><label>Wallet ID (opcional)</label><input id="f-wallet" value="${esc(a.walletId)}" placeholder="deixe em branco se não souber" style="width:100%;"></div>
    <div style="display:flex;gap:8px;flex-wrap:wrap;">
        <button class="btn btn-gold" id="f-save" style="justify-content:center;">Salvar configuração</button>
        <button class="btn btn-ghost" id="f-test" style="justify-content:center;">${ICON.check} Testar conexão</button>
    </div>
    <div id="asaas-test-result" style="font-size:12.5px;line-height:1.6;">
</div>

```

```

    </div>
  </div>

  <div class="card" style="padding:20px;">
    <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:10px;flex-wrap:wrap;gap:10px;">
      <div class="font-display" style="font-size:16px;font-weight:600;">Sincronizar cobranças</div>
      <button class="btn btn-ghost" id="f-sync">${ICON.link} Sincronizar com Asaas</button>
    </div>
    <div style="font-size:13px;color:var(--muted-2);">Busca no Asaas o status mais recente de cada assinante (pago, pendente ou atrasado) e atualiza aqui. Isso também acontece automaticamente quando o Asaas avisa o sistema de um pagamento novo – este botão é só para forçar uma checagem manual.</div>
  </div>

  <div class="card" style="padding:20px;margin-top:20px;">
    <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:10px;flex-wrap:wrap;gap:10px;">
      <div class="font-display" style="font-size:16px;font-weight:600;">Importar assinaturas que já existem no Asaas</div>
      <button class="btn btn-ghost" id="f-import-asaas">${ICON.arrowUp} Buscar assinaturas</button>
    </div>
    <div style="font-size:13px;color:var(--muted-2);">Se você já usava o Asaas em outro sistema, isso traz os planos e assinantes de lá para cá, mantendo a cobrança recorrente que já está rodando. Nada é criado ou cancelado no Asaas – só copiamos as informações para o BarberPro.</div>
    <div id="asaas-import-result" style="margin-top:14px;"></div>
  </div>

  <div class="card" style="padding:20px;margin-top:20px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:10px;">Atualização automática (webhook)</div>
    <div style="font-size:13px;color:var(--muted-2);margin-bottom:10px;">Para que o status mude sozinho assim que um assinante pagar (sem precisar clicar em "Sincronizar"), cole este endereço no painel do Asaas em <b>Integrações → Webhooks</b>:</div>
    <div class="font-mono" style="font-size:12.5px;background:var(--bg);border:1px solid var(--border);border-radius:8px;padding:10px 12px;word-break:break-all;color:var(--gold-bright);">${SUPABASE_URL}/functions/v1/asaas-webhook</div>
  </div>
`;

view.querySelector('#f-save').addEventListener('click', ()=>{
  update(s=>{

```

```

        s.asaas.env = view.querySelector('#f-env').value;
        s.asaas.apiKey = view.querySelector('#f-key').value.trim();
        s.asaas.walletId = view.querySelector('#f-wallet').value.trim();
    });
    showToast('Configuração salva.');
```

```

    });
    view.querySelector('#f-test').addEventListener('click', async (e)=>{
        const btn = e.currentTarget;
        const out = view.querySelector('#asaas-test-result');
        if(!state.asaas.apiKey){
            out.innerHTML = `<span style="color:var(--bad);">Salve a API Key primeiro
(botão "Salvar configuração").</span>`;
            return;
        }
        btn.disabled = true; out.innerHTML = '<span style="color:var(--muted);">Testando conexão com o Asaas...</span>';
        const { data: res, error } = await supa.functions.invoke('asaas-sync', { body:
{ action:'test_connection' } });
        btn.disabled = false;
        if(error){
            out.innerHTML = `<span style="color:var(--bad);">Não foi possível falar com
o servidor.<br><span style="color:var(--muted);">Detalhe:
${esc(error.message||'erro desconhecido')}</span><br>Publique a função com
<b>supabase functions deploy asaas-sync</b> na versão mais recente (a anterior não
respondia ao navegador).</span>`;
            return;
        }
        if(res?.error){
            out.innerHTML = `<span style="color:var(--bad);">${esc(res.error)}</span>`;
            return;
        }
        out.innerHTML = `<span style="color:var(--ok);">✓ Conectado com sucesso ao
Asaas (${res.env==='production'?'Produção':'Sandbox'}) - conta:
<b>${esc(res.accountName)}</b></span>`;
    });
    view.querySelector('#f-import-asaas').addEventListener('click', async (e)=>{
        const btn = e.currentTarget;
        const out = view.querySelector('#asaas-import-result');
        if(!state.asaas.apiKey){
            out.innerHTML = `<span style="color:var(--bad);font-size:12.5px;">Salve e
teste a API Key primeiro.</span>`;
            return;
        }
        btn.disabled = true; out.innerHTML = '<span style="color:var(--muted);font-size:12.5px;">Buscando no Asaas... isso pode levar alguns segundos.</span>';
        const { data: res, error } = await supa.functions.invoke('asaas-sync', { body:
{ action:'import_existing' } });

```

```

    btn.disabled = false;
    if(error){ out.innerHTML = `">Falha ao chamar o servidor: ${esc(error.message||'')}</span>`;
return; }

    if(res?.error){ out.innerHTML = `">${esc(res.error)}</span>`; return; }

    if(res?.note){ out.innerHTML = `">${esc(res.note)}</span>`; return; }

    asaasImportData = res;
    const jaExiste =
res.members.filter(m=>state.subscriptionMembers.some(x=>x.asaasSubscriptionId===m.a
    const novos = res.members.length - jaExiste;
    const porStatus = {};
    res.members.forEach(m=>{ porStatus[m.status] = (porStatus[m.status]||0)+1; });

    out.innerHTML = `
        <div class="card" style="padding:16px 18px;background:var(--panel-2);">
            <div style="font-size:13.5px;font-weight:600;margin-bottom:10px;">Encontramos no seu Asaas:</div>
            <div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(120px,1fr));gap:12px;margin-bottom:12px;">
                <div><div style="font-size:11px;color:var(--muted-2);">Assinantes</div>
<div class="font-mono" style="font-size:18px;color:var(--gold-bright);">${res.members.length}</div></div>
                <div><div style="font-size:11px;color:var(--muted-2);">Planos
distintos</div><div class="font-mono" style="font-size:18px;">${res.plans.length}
</div></div>
                <div><div style="font-size:11px;color:var(--muted-2);">Novos para
importar</div><div class="font-mono" style="font-size:18px;color:var(--
ok);">${novos}</div></div>
                ${jaExiste?`<div><div style="font-size:11px;color:var(--muted-2);">Já
cadastrados aqui</div><div class="font-mono" style="font-size:18px;color:var(--
muted);">${jaExiste}</div></div>`:''}
            </div>
            <div style="font-size:12px;color:var(--muted);margin-bottom:10px;">
                Status: ${Object.entries(porStatus).map(([k,v])=>`${v} ${k}`).join('
')}
            </div>
            <div style="font-size:12.5px;margin-bottom:6px;font-weight:600;">Planos
que serão criados:</div>
            <div style="display:flex;flex-direction:column;gap:4px;margin-bottom:14px;">
                ${res.plans.map(p=>`<div style="font-size:12.5px;color:var(--muted);">•
${esc(p.name)} - ${BRL(p.price)}/mês <span style="color:var(--muted-2);">
(${p.count} assinante(s))</span></div>`).join('')}
            </div>

```

```

        <button class="btn btn-gold" id="f-confirm-import">Importar ${novos}
assinante(s) e ${res.plans.length} plano(s)</button>
        <div style="font-size:11px;color:var(--muted-2);margin-
top:8px;">Assinantes que já existem aqui serão atualizados, não duplicados.</div>
    </div>
    `;
    view.querySelector('#f-confirm-import').addEventListener('click',
    ()=>confirmAsaasImport(view));
    });
    view.querySelector('#f-sync').addEventListener('click', syncAsaas);
    view.querySelector('[data-goto="clube"]')?.addEventListener('click', ()=>{
route='clube'; render(); });
}

/* ===== CLIENTES (CRM) ===== */
function clientsToCSV(){
    const header = ['Nome','Telefone','Aniversario(DD/MM)','Pontos','Observacoes'];
    const rows = state.clients.map(c=>[
        c.name||'', c.phone||'', c.birthday?c.birthday.split('-
').reverse().join('/'):'', c.points||0, c.notes||''
    ]);
    const escape = v => {
        const s = String(v).replace(/"/g, '"');
        return /[;,"\\n]/.test(s) ? `"$${s}"` : s;
    };
    return [header, ...rows].map(r=>r.map(escape).join(';')).join('\n');
}

function downloadCSV(){
    const csv = '\uFEFF' + clientsToCSV();
    const blob = new Blob([csv], {type:'text/csv;charset=utf-8;'});
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url; a.download = `clientes-primer-barber-${todayISO()}.csv`;
    document.body.appendChild(a); a.click(); a.remove();
    URL.revokeObjectURL(url);
    showToast('Exportação gerada.');
```

```

        else if(ch===delim){ out.push(cur); cur=''; }
        else cur+=ch;
    }
}
out.push(cur);
return out.map(v=>v.trim());
}
/* Lê um arquivo (.xlsx, .xls ou .csv) e devolve as linhas como matriz. */
function readSpreadsheetFile(file, onRows){
    const isExcel = /\.(xlsx|xls|xlsxm)$/i.test(file.name);
    const reader = new FileReader();
    reader.onerror = () => showToast('Não foi possível ler o arquivo.');
```

Verifique sua conexão e recarregue a página.');

```

    reader.onload = () => {
        try{
            if(isExcel){
                if(typeof XLSX==='undefined'){ showToast('Leitor de Excel não carregou.
                const wb = XLSX.read(new Uint8Array(reader.result), {type:'array'});
                const sheet = wb.Sheets[wb.SheetNames[0]];
                const rows = XLSX.utils.sheet_to_json(sheet, {header:1, blankrows:false,
                defval:''});
                onRows(rows.map(r=>r.map(cell=>String(cell??'').trim())));
            } else {
                const text = reader.result;
                const lines = String(text).split(/\r?\n/).filter(l=>l.trim().length>0);
                if(lines.length===0){ showToast('Arquivo vazio.');
```

Arquivo vazio.');

```

                const delim = lines[0].includes(';') ? ';' : ',';
                onRows(lines.map(l=>parseCSVLine(l, delim)));
            }
        }catch(err){
            console.error(err);
            showToast('Não foi possível interpretar o arquivo. Verifique o formato.');
```

Não foi possível interpretar o arquivo. Verifique o formato.');

```

        }
    };
    if(isExcel) reader.readAsArrayBuffer(file);
    else reader.readAsText(file, 'UTF-8');
}

/* Acha em qual coluna está cada informação, lendo o cabeçalho da planilha.
    Assim funciona com qualquer formato de exportação, em qualquer ordem. */
function mapColumns(headerRow, mapping){
    const norm = s => String(s||'').toLowerCase()
        .normalize('NFD').replace(/[\u0300-\u036f]/g, '').trim();
    const header = headerRow.map(norm);
    const found = {};
    Object.entries(mapping).forEach(([key, aliases])=>{
        let idx = -1;
```

```
// Testa os apelidos na ordem de prioridade: o primeiro da lista vence.
// Assim "Telefone" ganha de "Zap" quando as duas colunas existem.
for(const a of aliases){
  const exact = header.findIndex(h => h === a);
  if(exact >= 0){ idx = exact; break; }
  const partial = header.findIndex(h => h && h.includes(a));
  if(partial >= 0){ idx = partial; break; }
}
found[key] = idx;
});
return found;
}

function cellValue(row, idx){
  if(idx==null || idx < 0) return '';
  let v = row[idx];
  if(v==null) return '';
  v = String(v).trim();
  // Excel às vezes guarda telefone/número como 5522981296800.0
  if(/^\d+\.\d$/.test(v)) v = v.slice(0,-2);
  return v;
}

function importClientsFromRows(rows){
  if(!rows || rows.length===0){ showToast('Arquivo vazio.');
```

```

let added=0, updated=0, skipped=0;
update(s=>{
  dataRows.forEach(r=>{
    const name = cellValue(r, cols.name);
    if(!name){ skipped++; return; }
    const phone = cellValue(r, cols.phone);
    const birthdayRaw = cellValue(r, cols.birthday);
    const pointsRaw = cellValue(r, cols.points);
    const notes = cellValue(r, cols.notes);

    let birthday = '';
    const m = birthdayRaw.match(/^(\\d{1,2})[\\/\\-](\\d{1,2})/);
    if(m) birthday = m[2].padStart(2,'0')+'-'+m[1].padStart(2,'0');
    const points = parseInt(pointsRaw)||0;

    const existing =
s.clients.find(c=>c.name.trim().toLowerCase()===name.toLowerCase());
    if(existing){
      Object.assign(existing, {
        phone: phone||existing.phone,
        birthday: birthday||existing.birthday,
        points: pointsRaw ? points : existing.points,
        notes: notes||existing.notes,
      });
      updated++;
    }else{
      s.clients.push({id:uid(), name, phone, birthday, points, notes});
      added++;
    }
  });
});

showToast(`Importação concluída: ${added} novo(s), ${updated}
atualizado(s)${skipped?``, ${skipped} linha(s) sem nome ignorada(s):'`.`);
}

function importServicesFromRows(rows){
  if(!rows || rows.length===0){ showToast('Arquivo vazio.');
```

```

    fichas:    ['ficha','fichas','clube'],
  });

  if(cols.name < 0){
    showToast('Não encontrei uma coluna com o nome do serviço. Verifique o
cabeçalho.');
```

return;

```
  }

  const dataRows = rows.slice(headerIdx+1);
  let added=0, updated=0;
  update(s=>{
    dataRows.forEach(r=>{
      const name = cellValue(r, cols.name);
      if(!name) return;
      const priceRaw = cellValue(r, cols.price);
      const durationRaw = cellValue(r, cols.duration);
      const fichasRaw = cellValue(r, cols.fichas);
      const price = parseFloat(priceRaw.replace(/^[^\\d,.-]/g,'').replace(',','.'))
|| 0;
      const duration = parseInt(durationRaw) || 30;
      const clubFichas = parseInt(fichasRaw) || 0;

      const existing =
s.services.find(x=>x.name.trim().toLowerCase()===name.toLowerCase());
      if(existing){
        Object.assign(existing, {
          price: priceRaw ? price : existing.price,
          duration: durationRaw ? duration : existing.duration,
          clubFichas: fichasRaw ? clubFichas : existing.clubFichas,
        });
        updated++;
      }else{
        s.services.push({id:uid(), name, price, duration, requireDeposit:false,
depositType:'percentual', depositValue:30, clubFichas});
        added++;
      }
    });
  });

  showToast(`Importação concluída: ${added} novo(s), ${updated} atualizado(s).`);
}

function renderClientes(view){
  const duplicados = encontrarClientesDuplicados();
  view.innerHTML = `
    ${pageHeader('Clientes', state.clients.length+' cadastrado(s)', `
      <div style="display:flex;gap:8px;flex-wrap:wrap;">
        <input type="file" id="f-import-file" accept=".csv,.xlsx,.xls,.xlsm"

```

```

style="display:none;">
    <button class="btn btn-ghost" id="btn-import-client">${ICON.arrowUp}
Importar planilha</button>
    <button class="btn btn-ghost" id="btn-export-client">${ICON.arrowDown}
Exportar CSV</button>
    <button class="btn btn-gold" id="btn-new-client">${ICON.plus} Novo
cliente</button>
</div>
`))
${duplicados.length>0 ? `
    <div class="card" style="padding:14px 16px;margin-bottom:16px;border-
color:var(--warn);">
        <div style="display:flex;justify-content:space-between;align-
items:center;gap:12px;flex-wrap:wrap;">
            <div style="font-size:12.5px;line-height:1.6;">
                <b style="color:var(--warn);">${duplicados.length} grupo(s) de
clientes possivelmente duplicados</b><br>
                <span style="color:var(--muted-2);">Mesmo telefone, ou nome idêntico
cadastrado mais de uma vez.</span>
            </div>
            <button class="btn btn-gold" id="btn-ver-duplicados" style="flex-
shrink:0;">Revisar duplicados</button>
        </div>
    </div>
    ` : ``}

    <div style="font-size:12px;color:var(--muted-2);margin:-12px 0 18px;">Aceita
Excel (.xlsx) ou CSV. Colunas esperadas, nesta ordem: Nome, Telefone,
Aniversario(DD/MM), Pontos, Observacoes.</div>
    ${state.clients.length===0 ? emptyState('Nenhum cliente cadastrado ainda.') :
    `
        <table>
            <thead><tr><th>Cliente</th><th>Contato</th><th>Indicações</th>
<th>Visitas</th><th>Total gasto</th><th>Última visita</th><th></th></tr></thead>
            <tbody>
                ${state.clients.map(c=>{
                    const st = clientStats(c.name);
                    const referrals = state.clients.filter(x=>x.referredBy &&
x.referredBy.trim().toLowerCase()===c.name.trim().toLowerCase()).length;
                    return `<tr>
                        <td style="font-weight:500;">${esc(c.name)}
                        ${clubStarBadge(c.name)}${c.referredBy?`<div style="font-size:11px;color:var(--
muted-2);font-weight:400;">${ICON.gift} indicado por ${esc(c.referredBy)}
</div>`:''}${c.notes?`<div style="font-size:11.5px;color:var(--muted-2);font-
weight:400;max-width:220px;">${esc(c.notes)}</div>`:''}</td>
                        <td class="font-mono" style="color:var(--muted);">${esc(c.phone)}||'-' }
                    </td>
                        <td class="font-mono" style="color:${referrals>0?'var(--gold-

```

```

bright) ':' + var(--muted-2) '});">${referrals>0?referrals+' indicado(s) ':'-'}</td>
    <td class="font-mono">${st.visits}</td>
    <td class="font-mono">${BRL(st.total)}</td>
    <td class="font-mono" style="color:var(--muted);">${st.lastVisit?new
Date(st.lastVisit+'T00:00').toLocaleDateString('pt-BR'):'-'}</td>
    <td style="text-align:right;white-space:nowrap;">
        <button class="btn btn-ghost" style="padding:6px 8px;" data-edit-
client="${c.id}">${ICON.edit}</button>
        <button class="btn btn-ghost" style="padding:6px 8px;color:var(--
bad);border-color:var(--bad);" data-del-client="${c.id}">${ICON.trash}</button>
    </td>
</tr>`;
    }).join('')
</tbody>
</table>
`
`;
view.querySelector('#btn-new-client').addEventListener('click',
()=>clientModal());
view.querySelector('#btn-export-client').addEventListener('click', downloadCSV);
view.querySelector('#btn-import-client').addEventListener('click',
()=>view.querySelector('#f-import-file').click());
view.querySelector('#f-import-file').addEventListener('change', (e)=>{
    const file = e.target.files[0];
    if(!file) return;
    readSpreadsheetFile(file, importClientsFromRows);
    e.target.value = '';
});
view.querySelectorAll('[data-edit-
client]').forEach(b=>b.addEventListener('click',
()=>clientModal(state.clients.find(c=>c.id===b.dataset.editClient))));
view.querySelectorAll('[data-del-
client]').forEach(b=>b.addEventListener('click', ()=>update(s=>{ s.clients =
s.clients.filter(c=>c.id!==b.dataset.delClient); })));
view.querySelector('#btn-ver-duplicados')?.addEventListener('click',
()=>duplicadosModal(view));
}

function duplicadosModal(view){
    const grupos = encontrarClientesDuplicados();
    openModal(`
    <div style="padding:22px;">
        <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:6px;">
            <div class="font-display" style="font-size:19px;font-weight:600;">Clientes
duplicados</div>
            <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}

```

```

</button>
</div>
<div style="font-size:12.5px;color:var(--muted);margin-bottom:18px;line-
height:1.6;">
    Escolha qual cadastro manter em cada grupo. O histórico de agendamentos,
    pontos e assinatura dos outros é transferido para ele antes de removê-los – nada
    se perde.
</div>
<div id="dup-lista" style="display:flex;flex-direction:column;gap:16px;max-
height:56vh;overflow-y:auto;"></div>
</div>
`, root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    const desenhar = ()=>{
        const grupos = encontrarClientesDuplicados();
        const lista = root.querySelector('#dup-lista');
        if(grupos.length===0){
            lista.innerHTML = `<div style="text-align:center;padding:20px
0;color:var(--ok);font-size:14px;">✓ Nenhum duplicado encontrado.</div>`;
            return;
        }
        lista.innerHTML = grupos.map((g,gi)=>`
            <div style="border:1px solid ${g.confianca==='alta'?'var(--warn)':'var(--
border)'};border-radius:10px;padding:14px;">
                <div style="font-size:11.5px;color:${g.confianca==='alta'?'var(--
warn)':'var(--muted-2)'};margin-bottom:10px;">
                    ${g.tipo==='telefone' ? `${ICON.phone}||'`} Mesmo telefone` : 'Nome
idêntico'} · ${g.confianca==='alta'?'alta confiança':'confira antes de mesclar'}
                </div>
                <div style="display:flex;flex-direction:column;gap:8px;margin-
bottom:12px;">
                    ${g.clientes.map(c=>{
                        const st = clientStats(c.name);
                        return `<label style="display:flex;align-items:flex-
start;gap:10px;padding:9px 11px;border:1px solid var(--border);border-
radius:8px;cursor:pointer;">
                            <input type="radio" name="dup-manter-${gi}" value="${c.id}"
${c===g.clientes.reduce((a,b)=>
(clientStats(b.name).visits>clientStats(a.name).visits?b:a))?'checked':''}
style="width:auto;margin-top:2px;">
                                <div style="flex:1;min-width:0;">
                                    <div style="font-size:13px;font-weight:500;">${esc(c.name)}
                                </div>
                                    <div style="font-size:11.5px;color:var(--muted-
2);">${esc(c.phone)}||'sem telefone'} · ${st.visits} visita(s) · ${BRL(st.total)}
gasto</div>
                                </div>
                            </div>
                        `;
                    })}
                </div>
            `;
    }
}

```

```

        </label>`;
    })).join('')}
</div>
<button class="btn btn-gold dup-mesclar" data-grupo="${gi}"
style="padding:8px 14px;font-size:12.5px;">Mesclar este grupo</button>
</div>
`).join('');
lista.querySelectorAll('.dup-
mesclar').forEach(btn=>btn.addEventListener('click', ()=>{
    const gi = parseInt(btn.dataset.grupo);
    const grupo = grupos[gi].clientes;
    const escolhido = root.querySelector(`input[name="dup-manter-
${gi}"]:checked`);
    if(!escolhido){ showToast('Escolha qual cadastro manter.');
```

```

    <div><label>${ICON.gift} Indicado por (programa de indicação)</label>
<select id="f-referred" style="width:100%;">
    <option value="">Ninguém / não sei</option>
    ${state.clients.filter(c=>c.id!==existing?.id).map(c=>`<option
value="${esc(c.name)}"
${existing?.referredBy===c.name?'selected':''}>${esc(c.name)}</option>`).join('')}
    </select></div>
    <div><label>Observações</label><input id="f-notes"
value="${esc(existing?.notes)}" placeholder="Preferências do cliente"
style="width:100%;"></div>
    <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">${isEdit?'Salvar':'Cadastrar'}</button>
</div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#f-submit').addEventListener('click', ()=>{
    const name = root.querySelector('#f-name').value.trim();
    if(!name){ showToast('Informe o nome.');
```

```

bottom:4px;">Estoque baixo em ${low.length} produto(s)</div>
    <div style="font-size:12.5px;color:var(--
muted);">${low.map(p=>p.name).join(', ')} – considere repor.</div>
    </div>` : ''}
    ${state.products.length===0 ? emptyState('Nenhum produto cadastrado.') : `
    <table>
        <thead><tr><th>Produto</th><th>Estoque</th><th>Mínimo</th><th>Custo</th>
<th>Venda</th><th></th></tr></thead>
        <tbody>
            ${state.products.map(p=>`<tr>
                <td style="font-weight:500;">${esc(p.name)}</td>
                <td class="font-mono">${p.stock<=p.minStock?`<span class="pill pill-
bad">${p.stock}</span>`:p.stock}</td>
                <td class="font-mono" style="color:var(--muted);">${p.minStock}</td>
                <td class="font-mono" style="color:var(--muted);">${BRL(p.cost)}</td>
                <td class="font-mono">${p.price>0?BRL(p.price):'-'}</td>
                <td style="text-align:right;white-space:nowrap;">
                    ${p.price>0 && p.stock>0 ? `<button class="btn btn-gold"
style="padding:6px 10px;font-size:12px;" data-sell-product="${p.id}">${ICON.cash}
Vender</button>` : ''}
                    <button class="btn btn-ghost" style="padding:6px 8px;" data-edit-
product="${p.id}">${ICON.edit}</button>
                    <button class="btn btn-ghost" style="padding:6px 8px;color:var(--
bad);border-color:var(--bad);" data-del-product="${p.id}">${ICON.trash}</button>
                </td>
            </tr>`).join('')}
        </tbody>
    </table>
    `}
    `;
    view.querySelector('#btn-new-product').addEventListener('click',
    ()=>productModal());
    view.querySelectorAll('[data-sell-
product]').forEach(b=>b.addEventListener('click',
    ()=>sellProductModal(state.products.find(p=>p.id===b.dataset.sellProduct))));
    view.querySelectorAll('[data-edit-
product]').forEach(b=>b.addEventListener('click',
    ()=>productModal(state.products.find(p=>p.id===b.dataset.editProduct))));
    view.querySelectorAll('[data-del-
product]').forEach(b=>b.addEventListener('click', ()=>update(s=>{ s.products =
s.products.filter(p=>p.id!==b.dataset.delProduct); })));
}
function sellProductModal(prod){
    if(!prod) return;
    if(!openSession()){ showToast('Abra o caixa antes de registrar vendas.')}
    route='caixa'; render(); return; }
    const clientes =

```

```

state.clients.slice().sort((a,b)=>a.name.localeCompare(b.name));
openModal(`
  <div style="padding:22px;">
    <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:8px;">
      <div class="font-display" style="font-size:19px;font-weight:600;">Vender
produto</div>
      <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
    </div>
    <div style="font-size:13px;color:var(--muted);margin-
bottom:16px;">${esc(prod.name)} · ${prod.stock} em estoque</div>
    <div style="display:flex;flex-direction:column;gap:12px;">
      <div><label>Quantidade</label><input id="sp-qty" type="number" min="1"
max="${prod.stock}" value="1" style="width:100%;"></div>
      <div><label>Cliente (opcional – para aplicar desconto do clube)</label>
      <select id="sp-client" style="width:100%;">
        <option value="">Sem cliente identificado</option>
        ${clientes.map(cl=><option value="${esc(cl.name)}">${esc(cl.name)}
</option>`).join('')}
      </select>
    </div>
    <div id="sp-total" class="card" style="padding:14px 16px;background:var(--
panel-2);"></div>
    <label>Forma de pagamento</label>
    <div style="display:grid;grid-template-columns:1fr 1fr;gap:8px;">
      ${PAYMENT_METHODS.map(m=><button class="btn btn-ghost sp-pay" data-
method="${m.id}" style="justify-content:center;padding:13px 10px;font-
size:13px;">${m.label}</button>`).join('')}
    </div>
  </div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  const calc = ()=>{
    const qty = Math.max(1, Math.min(prod.stock,
parseInt(root.querySelector('#sp-qty').value)||1));
    const nome = root.querySelector('#sp-client').value;
    const bruto = prod.price * qty;
    let desconto = 0, nota = '';
    const m = nome ? clubMemberForClient(nome) : null;
    if(m && m.status==='pago'){
      const plan = state.subscriptionPlans.find(p=>p.id===m.planId);
      if(plan && !planValidOnDate(plan, todayISO())){
        nota = `${planDaysLabel(plan.validDays)} – hoje o desconto não vale.`;
      } else if(plan && plan.discountPercent>0){
        desconto = bruto * (plan.discountPercent/100);
      }
    }
  }
}

```

```

        nota = `Desconto de assinante: ${plan.discountPercent}%
        (${esc(plan.name)})`;
    }
    } else if(m && m.status==='atrasado'){
        nota = 'Assinante em atraso - sem desconto.';
    }
    const total = bruto - desconto;
    root.querySelector('#sp-total').innerHTML = `
        <div style="display:flex;justify-content:space-between;font-
        size:13px;color:var(--muted);"><span>${qty}× ${esc(prod.name)}</span><span
        class="font-mono">${BRL(bruto)}</span></div>
        ${desconto>0?`<div style="display:flex;justify-content:space-between;font-
        size:13px;color:var(--ok);margin-top:5px;"><span>Desconto</span><span class="font-
        mono">- ${BRL(desconto)}</span></div>`:''}
        <div style="display:flex;justify-content:space-between;align-
        items:center;margin-top:10px;padding-top:10px;border-top:1px solid var(--
        border);">
            <span style="font-size:13px;font-weight:600;">Total</span>
            <span class="font-mono" style="font-size:21px;color:var(--gold-
        bright);">${BRL(total)}</span>
        </div>
        ${nota?`<div style="font-size:11.5px;color:var(--muted-2);margin-
        top:8px;">${nota}</div>`:''}
    `;
    return {qty, total, nome, desconto};
};
root.querySelector('#sp-qty').addEventListener('input', calc);
root.querySelector('#sp-client').addEventListener('change', calc);
calc();
root.querySelectorAll('.sp-pay').forEach(b=>b.addEventListener('click', ()=>{
    const {qty, total, nome, desconto} = calc();
    const method = b.dataset.method;
    update(s=>{
        const p = s.products.find(x=>x.id===prod.id);
        p.stock = Math.max(0, p.stock - qty);
        const sess = s.cashSessions.find(x=>x.status==='aberto');
        sess.entries.push({
            id:uid(), type:'entrada', method,
            desc:`${qty}× ${prod.name}${nome?' - '+nome:''}${desconto>0?' (com
desconto do clube)':''}`,
            amount: total, time:new Date().toTimeString().slice(0,5), auto:false,
        });
    });
    closeModal();
    showToast(`Venda registrada - ${BRL(total)} em ${methodLabelPay(method)}.`);
}));
});

```

```

}
function productModal(existing) {
  const isEdit = !!existing;
  openModal(`
    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:16px;">
        <div class="font-display" style="font-size:19px;font-
weight:600;">${isEdit?'Editar':'Novo'} produto</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
      </div>
      <div style="display:flex;flex-direction:column;gap:12px;">
        <div><label>Nome do produto</label><input id="f-name"
value="${esc(existing?.name)}" style="width:100%;"></div>
        <div style="display:flex;gap:10px;">
          <div style="flex:1;"><label>Estoque atual</label><input id="f-stock"
type="number" min="0" value="${existing?.stock??0}" style="width:100%;"></div>
          <div style="flex:1;"><label>Estoque mínimo</label><input id="f-min"
type="number" min="0" value="${existing?.minStock??0}" style="width:100%;"></div>
        </div>
        <div style="display:flex;gap:10px;">
          <div style="flex:1;"><label>Custo (R$)</label><input id="f-cost"
type="number" step="0.01" min="0" value="${existing?.cost??0}"
style="width:100%;"></div>
          <div style="flex:1;"><label>Preço de venda (R$)</label><input id="f-
price" type="number" step="0.01" min="0" value="${existing?.price??0}"
style="width:100%;"></div>
        </div>
        <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">${isEdit?'Salvar':'Cadastrar'}</button>
      </div>
    </div>
  ` , root=>{
    root.querySelector('#m-close').addEventListener('click', closeModal);
    root.querySelector('#f-submit').addEventListener('click', ()=>{
      const name = root.querySelector('#f-name').value.trim();
      if(!name){ showToast('Informe o nome.');
```

```

        else{ s.products.push({id:uid(), ...data}); }
    });
    closeModal();
  });
});
}

/* ===== RELATÓRIOS ===== */
/* ===== CENTRAL DE INTELIGÊNCIA - tela ===== */
let ciMes = null;

function ciCard(titulo, valor, sub, variacaoPct, tone){
  const cor = tone==='bad' ? 'var(--bad)' : tone==='warn' ? 'var(--warn)' : 'var(--gold-bright)';
  const seta = variacaoPct===null ? '' : (variacaoPct>=0
    ? `<span style="color:var(--ok);font-size:12.5px;white-space:nowrap;">↑
    ${variacaoPct.toFixed(1)}%</span>`
    : `<span style="color:var(--bad);font-size:12.5px;white-space:nowrap;">↓
    ${Math.abs(variacaoPct).toFixed(1)}%</span>`);
  return `
    <div class="card" style="padding:18px 20px;">
      <div style="font-size:12px;color:var(--muted-2);margin-bottom:8px;">${titulo}</div>
      <div style="display:flex;justify-content:space-between;align-items:baseline;gap:10px;">
        <div class="font-mono" style="font-size:22px;font-weight:600;color:${cor}">${valor}</div>
        ${seta}
      </div>
      ${sub?`<div style="font-size:11.5px;color:var(--muted);margin-top:5px;">${sub}</div>`:''}
    </div>`;
}

function barrasHtml(dados, cor){
  const max = Math.max(1, ...dados.map(d=>d.valor));
  return `<div style="display:flex;align-items:flex-end;gap:6px;height:130px;">
    ${dados.map(d=>`
      <div style="flex:1;display:flex;flex-direction:column;align-items:center;gap:5px;min-width:0;">
        <div style="font-size:9.5px;color:var(--muted-2);">${d.valor>0?
d.rotuloValor||d.valor:''}</div>
        <div title="${d.label}: ${d.rotuloValor||d.valor}"
style="width:100%;background:${cor};border-radius:4px 4px 0 0;height:${Math.max(2,
(d.valor/max)*95)}px;"></div>
        <div style="font-size:10px;color:var(--muted-2);white-space:nowrap;">${d.label}</div>
      </div>
    `)}
  </div>`;
}

```

```

        </div>`).join('')}
</div>`;
}

function renderCentral(view){
    if(!ciMes) ciMes = currentYM();
    const ym = ciMes, ant = shiftYM(ym,-1);
    const fat = faturamentoMes(ym), fatAnt = faturamentoMes(ant);
    const at = atendimentosMes(ym), atAnt = atendimentosMes(ant);
    const ocup = ocupacaoMes(ym);
    const st = statusClientes();
    const pico = diasDePico(ym);
    const rank = rankingClientes(ym);

    const ticket = at.length ? fat.servicos/at.length : 0;
    const ticketAnt = atAnt.length ? fatAnt.servicos/atAnt.length : 0;
    const totalClientes = st.ativos + st.semi + st.inativos;

    // faturamento por categoria
    const cat = [
        {nome:'Serviços', valor: fat.servicos, cor:'var(--gold)'},
        {nome:'Assinaturas', valor: fat.assinaturas, cor:'var(--ok)'},
    ];
    const somaCat = cat.reduce((t,x)=>t+x.valor,0) || 1;

    // evolução dos últimos 6 meses
    const meses = [];
    for(let i=5;i>=0;i--){
        const m = shiftYM(ym,-i);
        const f = faturamentoMes(m);
        meses.push({ label: ymLabel(m).split(' ')[0], valor: Math.round(f.servicos),
        rotuloValor: f.servicos>0?BRL(f.servicos).replace('R$ ',''):'' });
    }

    view.innerHTML = `
        ${pageHeader('Central de Inteligência', 'Os números que mostram onde está o
        dinheiro do seu negócio', `
            <div style="display:flex;gap:6px;align-items:center;">
                <button class="btn btn-ghost" id="ci-prev" style="padding:8px
11px;"></button>
                <span class="pill" style="padding:8px 14px;font-size:13px;">${ymLabel(ym)}
</span>
                <button class="btn btn-ghost" id="ci-next" style="padding:8px 11px;"
${ym>currentYM()?'disabled':''}></button>
            </div>
        `)}
    
```

```

<div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(210px,1fr));gap:14px;margin-bottom:20px;">
  ${ciCard('Faturamento em serviços', BRL(fat.servicos), `${at.length}
atendimento(s)`, variacao(fat.servicos, fatAnt.servicos))}
  ${ciCard('Resultado do caixa', BRL(fat.liquido), `Entradas
${BRL(fat.entradas)} · Saídas ${BRL(fat.saidas)}`, variacao(fat.liquido,
fatAnt.liquido), fat.liquido<0?'bad':'ok'')}
  ${ciCard('Ticket médio', BRL(ticket), 'por atendimento', variacao(ticket,
ticketAnt))}
  ${ciCard('Receita recorrente', BRL(fat.assinaturas), 'assinantes em dia
hoje', null)}
</div>

```

```

<div style="display:grid;grid-template-columns:1.1fr 1fr;gap:16px;align-
items:start;margin-bottom:20px;" class="dash-grid">

```

```

  <div class="card" style="padding:20px;border-color:${ocup.taxa<30?'var(--
bad)':'var(--border)'};">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:4px;">Taxa de ocupação da agenda</div>
    <div style="font-size:11.5px;color:var(--muted-2);margin-
bottom:16px;">Quanto da jornada disponível virou atendimento.</div>
    <div style="display:flex;align-items:center;gap:22px;flex-wrap:wrap;">
      <div style="position:relative;width:132px;height:70px;flex-shrink:0;">
        <svg viewBox="0 0 132 70" style="width:100%;height:100%;">
          <path d="M 12 66 A 54 54 0 0 1 120 66" fill="none" stroke="var(--
panel-2)" stroke-width="13" stroke-linecap="round"/>
          <path d="M 12 66 A 54 54 0 0 1 120 66" fill="none"
            stroke="${ocup.taxa<30?'var(--bad)':ocup.taxa<60?'var(--
warn)':'var(--ok)'}"
            stroke-width="13" stroke-linecap="round"
            stroke-dasharray="${(Math.min(100,ocup.taxa)/100)*170} 170"/>
        </svg>
        <div style="position:absolute;bottom:0;width:100%;text-align:center;">
          <span class="font-mono" style="font-size:21px;font-
weight:600;">${ocup.taxa.toFixed(1)}%</span>
        </div>
      </div>
      <div style="flex:1;min-width:190px;display:flex;flex-
direction:column;gap:9px;">
        <div style="display:flex;justify-content:space-between;font-
size:13px;">
          <span style="color:var(--muted);">Faturado</span>
          <span class="font-mono" style="color:var(--
ok);">${BRL(ocup.faturado)}</span>
        </div>
        <div style="display:flex;justify-content:space-between;font-

```

```

size:13px;">
    <span style="color:var(--muted);">Potencial não usado</span>
    <span class="font-mono" style="color:var(--
warn);">${BRL(ocup.perdido)}</span>
    </div>
    <div style="display:flex;justify-content:space-between;font-
size:13px;">
        <span style="color:var(--muted);">Em folga</span>
        <span class="font-mono" style="color:var(--muted-
2);">${BRL(ocup.folgaValor)}</span>
    </div>
</div>
</div>
    ${ocup.taxa<60 && ocup.perdido>0 ? `<div style="margin-
top:14px;padding:11px 13px;border-radius:9px;background:var(--panel-2);font-
size:12.5px;color:var(--muted);line-height:1.6;">
        Se você subisse a ocupação para <b style="color:var(--ink);">60%</b>,
seriam cerca de
        <b style="color:var(--gold-bright);">${BRL(Math.max(0,
(ocup.perdido+ocup.faturado)*0.6 - ocup.faturado))}</b> a mais no mês.
        Vale olhar os dias mais vazios e os clientes inativos.
    </div>`:''}
    <div style="font-size:10.5px;color:var(--muted-2);margin-top:10px;line-
height:1.5;">
        O "potencial não usado" considera a agenda 100% cheia, o que nenhuma
barbearia atinge na prática –
        use como referência de espaço para crescer, não como prejuízo real.
        Calculado pelo preço e duração médios dos seus serviços.
    </div>
</div>

    <div class="card" style="padding:20px;">
        <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:4px;">Status dos clientes</div>
        <div style="font-size:11.5px;color:var(--muted-2);margin-
bottom:16px;">Baseado na última vez que cada um foi atendido.</div>
        ${totalClientes===0 ? emptyState('Ainda sem histórico de atendimentos.') :
        ,

        <div style="display:flex;height:13px;border-
radius:99px;overflow:hidden;margin-bottom:16px;">
            <div style="width:${(st.ativos/totalClientes)*100}%;background:var(--
ok);"></div>
            <div style="width:${(st.semi/totalClientes)*100}%;background:var(--
warn);"></div>
            <div style="width:${(st.inativos/totalClientes)*100}%;background:var(--
bad);"></div>
        </div>

```

```

        <div style="display:flex;flex-direction:column;gap:9px;">
            <div style="display:flex;justify-content:space-between;font-size:13px;"><span style="color:var(--muted);">● Ativos <span style="font-size:11px;color:var(--muted-2);">(até 30 dias)</span></span><b class="font-mono" style="color:var(--ok);">${st.ativos}</b></div>
            <div style="display:flex;justify-content:space-between;font-size:13px;"><span style="color:var(--muted);">● Semiativos <span style="font-size:11px;color:var(--muted-2);">(31 a 90 dias)</span></span><b class="font-mono" style="color:var(--warn);">${st.semi}</b></div>
            <div style="display:flex;justify-content:space-between;font-size:13px;"><span style="color:var(--muted);">● Inativos <span style="font-size:11px;color:var(--muted-2);">(mais de 90 dias)</span></span><b class="font-mono" style="color:var(--bad);">${st.inativos}</b></div>
            ${st.semHistorico?`<div style="display:flex;justify-content:space-between;font-size:13px;"><span style="color:var(--muted-2);">Nunca atendidos</span><b class="font-mono" style="color:var(--muted-2);">${st.semHistorico}</b></div>`:''}
        </div>
        ${st.inativos>0?`<button class="btn btn-gold" id="ci-recuperar" style="width:100%;justify-content:center;margin-top:16px;">Ver os ${st.inativos} inativos e chamar de volta</button>`:''}
    `}
</div>
</div>

<div style="display:grid;grid-template-columns:1.1fr 1fr;gap:16px;align-items:start;margin-bottom:20px;" class="dash-grid">
    <div class="card" style="padding:20px;">
        <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:16px;">Faturamento nos últimos 6 meses</div>
        ${barrasHtml(meses, 'var(--gold)')}
    </div>
    <div class="card" style="padding:20px;">
        <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:16px;">Dias mais movimentados</div>
        ${barrasHtml(
            ['Dom','Seg','Ter','Qua','Qui','Sex','Sáb'].map((d,i)=>({label:d, valor:pico[i]})),
            'var(--gold-bright)'
        )}
    </div>
</div>

<div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;align-items:start;" class="dash-grid">
    <div class="card" style="padding:20px;">
        <div class="font-display" style="font-size:16px;font-weight:600;margin-

```

```

bottom:16px;">De onde vem o dinheiro</div>
    ${cat.map(x=>`
        <div style="margin-bottom:14px;">
            <div style="display:flex;justify-content:space-between;font-
size:13px;margin-bottom:6px;">
                <span style="color:var(--muted);">${x.nome}</span>
                <span class="font-mono">${BRL(x.valor)} <span style="color:var(--
muted-2);font-size:11.5px;">${((x.valor/somaCat)*100).toFixed(1)}%</span></span>
            </div>
            <div style="height:8px;background:var(--panel-2);border-
radius:99px;overflow:hidden;">
                <div
style="height:100%;width:${(x.valor/somaCat)*100}%;background:${x.cor};"></div>
            </div>
        </div>`)}.join('')}
    </div>

    <div class="card" style="padding:20px;">
        <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:16px;">Melhores clientes do mês</div>
        ${rank.length===0 ? emptyState('Nenhum atendimento neste mês.') : `
            <div style="display:flex;flex-direction:column;gap:11px;">
                ${rank.map((r,i)=>`
                    <div style="display:flex;align-items:center;gap:11px;">
                        <span style="width:20px;font-size:12px;color:var(--muted-2);text-
align:right;">${i+1}</span>
                        <span style="flex:1;font-size:13px;min-
width:0;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;">${esc(r.nome)}
                        ${clubStarBadge(r.nome)}</span>
                        <span class="font-mono" style="font-size:13px;color:var(--gold-
bright);">${BRL(r.total)}</span>
                    </div>`)}.join('')}
            </div>`)
        </div>
    </div>
`;

    view.querySelector('#ci-prev')?.addEventListener('click', ()=>{ ciMes =
shiftYM(ym,-1); renderCentral(view); });
    view.querySelector('#ci-next')?.addEventListener('click', ()=>{ ciMes =
shiftYM(ym,1); renderCentral(view); });
    view.querySelector('#ci-recuperar')?.addEventListener('click',
()=>inativosModal(st.listaInativos));
}

function inativosModal(lista){
    openModal(`

```

```

    <div style="padding:22px;">
      <div style="display:flex;justify-content:space-between;align-items:center;margin-bottom:6px;">
        <div class="font-display" style="font-size:19px;font-weight:600;">Clientes para chamar de volta</div>
        <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}</button>
      </div>
      <div style="font-size:12.5px;color:var(--muted);margin-bottom:16px;">${lista.length} cliente(s) sem vir há mais de 90 dias. Começa pelos que sumiram há menos tempo – são os mais fáceis de trazer de volta.</div>
      <div style="display:flex;flex-direction:column;gap:9px;max-height:52vh;overflow-y:auto;">
        ${lista.slice(0,60).map(cl=>{
          const msg = `Olá ${cl.name}! Faz um tempinho que você não aparece aqui na ${state.shopProfile.name}. Que tal marcar um horário? Vamos adorar te atender de novo!`;
          return `<div style="display:flex;justify-content:space-between;align-items:center;gap:10px;padding:9px 0;border-bottom:1px solid var(--border);flex-wrap:wrap;">
            <div style="font-size:13px;min-width:150px;">
              <b>${esc(cl.name)}</b><br>
              <span style="color:var(--muted-2);font-size:11.5px;">Última visita há ${cl.diasSemVir} dias</span>
            </div>
            ${messageChannelRow(cl.phone, msg)}
          </div>`;
        })}.join('')}
      </div>
      ${lista.length>60?`<div style="font-size:11.5px;color:var(--muted-2);margin-top:12px;">Mostrando os 60 primeiros.</div>`:''}
    </div>
  `, root=>root.querySelector('#m-close').addEventListener('click', closeModal));
}

/* ===== CENTRAL DE INTELIGÊNCIA – cálculos ===== */
function ymOf(dateStr){ return (dateStr||'').slice(0,7); } // "2026-08"
function currentYM(){ return todayISO().slice(0,7); }
function shiftYM(ym, delta){
  const [y,m] = ym.split('-').map(Number);
  const d = new Date(y, m-1+delta, 1);
  return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2,'0')}`;
}
function ymLabel(ym){
  const [y,m] = ym.split('-').map(Number);
  return new Date(y, m-1, 1).toLocaleDateString('pt-BR',{month:'short',year:'numeric'}).replace('.','').toUpperCase();
}

```

```

}

/* ===== RELATÓRIOS - desempenho por semana/mês ===== */
let relatSemana = null; // segunda-feira da semana em exibição (YYYY-MM-DD)
let relatMes = null;
function addDaysISO(dateStr, dias){
  const d = new Date(dateStr+'T12:00');
  d.setDate(d.getDate()+dias);
  return d.toISOString().slice(0,10);
}
function mondayOfWeek(dateStr){
  const d = new Date(dateStr+'T12:00');
  const dow = (d.getDay()+6)%7; // 0=segunda
  return addDaysISO(dateStr, -dow);
}
function weekRangeLabel(inicioISO){
  const fim = addDaysISO(inicioISO, 6);
  const f = d => new Date(d+'T12:00').toLocaleDateString('pt-BR',{day:'2-digit',month:'2-digit'});
  return `${f(inicioISO)} a ${f(fim)}`;
}

function variacao(atual, anterior){
  if(!anterior) return atual>0 ? 100 : 0;
  return ((atual - anterior) / Math.abs(anterior)) * 100;
}

// Fatura de um mês: serviços concluídos + entradas manuais do caixa
function faturamentoMes(ym){
  const servicos = state.appointments
    .filter(a=>a.status==='concluido' && ymOf(a.date)===ym)
    .reduce((t,a)=>t+valorRealAtendimento(a),0);
  let entradas = 0, saidas = 0;
  (state.cashSessions||[]).forEach(s=>{
    if(ymOf(s.date||s.openedAt)!==ym) return;
    (s.entries||[]).forEach(e=>{
      if(e.type==='entrada') entradas += Number(e.amount)||0;
      else saidas += Number(e.amount)||0;
    });
  });
  const assinaturas = (state.subscriptionMembers||[])
    .filter(m=>m.status==='pago')
    .reduce((t,m)=>{ const p=state.subscriptionPlans.find(x=>x.id===m.planId);
  return t+(p?Number(p.price):0); },0);
  return { servicos, entradas, saidas, assinaturas, liquido: entradas - saidas };
}

```

```

function atendimentosMes(ym){
  return state.appointments.filter(a=>a.status==='concluido' &&
ymOf(a.date)===ym);
}

// Quanto a agenda poderia ter faturado x quanto faturou de fato
function ocupacaoMes(ym){
  const [y,m] = ym.split('-').map(Number);
  const diasNoMes = new Date(y, m, 0).getDate();
  const hoje = todayISO();
  const precoMedio = state.services.length
    ? state.services.reduce((t,s)=>t+Number(s.price),0)/state.services.length : 0;
  const duracaoMedia = state.services.length
    ?
state.services.reduce((t,s)=>t+Number(s.duration||30),0)/state.services.length :
30;

  let minutosPossiveis = 0, minutosFolga = 0;
  // No mês corrente, só conta até hoje - dias que ainda não chegaram
  // não podem ser considerados "perdidos".
  const ultimoDia = (ym === currentYM()) ? Number(hoje.slice(8,10)) : diasNoMes;
  servingProfessionals().forEach(p=>{
    for(let d=1; d<=ultimoDia; d++){
      const dia = `${ym}-${String(d).padStart(2,'0')}`;
      const [sh,sm] = (p.workStart||'09:00').split(':').map(Number);
      const [eh,em] = (p.workEnd||'19:00').split(':').map(Number);
      const jornada = (eh*60+em) - (sh*60+sm);
      if(jornada<=0) continue;
      if(isProfessionalWorking(p, dia)){
        minutosPossiveis += jornada;
        // desconta bloqueios (almoço, folga)
        blocksForDay(p.id, dia).forEach(b=>{
          const [bsh,bsm] = b.start.split(':').map(Number);
          const [beh,bem] = b.end.split(':').map(Number);
          minutosPossiveis -= Math.max(0, (beh*60+bem)-(bsh*60+bsm));
        });
      } else {
        minutosFolga += jornada;
      }
    }
  });

  const concluidos = atendimentosMes(ym);
  const minutosUsados = concluidos.reduce((t,a)=>t+appointmentDuration(a),0);
  const faturado = concluidos.reduce((t,a)=>t+valorRealAtendimento(a),0);
  const taxa = minutosPossiveis>0 ? (minutosUsados/minutosPossiveis)*100 : 0;
  const minutosVagos = Math.max(0, minutosPossiveis - minutosUsados);

```

```

// valor estimado que ficou na mesa por cadeira vazia
const perdido = duracaoMedia>0 ? (minutosVagos/duracaoMedia)*precoMedio : 0;
const folgaValor = duracaoMedia>0 ? (minutosFolga/duracaoMedia)*precoMedio : 0;

return { taxa, faturado, perdido, folgaValor, minutosPossiveis, minutosUsados,
ym, futuro: ym>=currentYM() };
}

// Ativo = veio nos últimos 30 dias | Semiativo = 31 a 90 | Inativo = mais de 90
function statusClientes(){
  const hoje = new Date(todayISO()+ 'T12:00');
  const ultima = {};
  state.appointments.filter(a=>a.status==='concluido').forEach(a=>{
    const nome = (a.client||'').trim().toLowerCase();
    if(!nome) return;
    if(!ultima[nome] || a.date > ultima[nome]) ultima[nome] = a.date;
  });
  let ativos=0, semi=0, inativos=0, semHistorico=0;
  const listaInativos = [];
  (state.clients||[]).forEach(cl=>{
    const nome = (cl.name||'').trim().toLowerCase();
    const ult = ultima[nome];
    if(!ult){ semHistorico++; return; }
    const dias = Math.floor((hoje - new Date(ult+'T12:00')) / 86400000);
    if(dias <= 30) ativos++;
    else if(dias <= 90) semi++;
    else { inativos++; listaInativos.push({...cl, ultimaVisita: ult, diasSemVir:
dias}); }
  });
  listaInativos.sort((a,b)=>a.diasSemVir-b.diasSemVir);
  return { ativos, semi, inativos, semHistorico, listaInativos };
}

function diasDePico(ym){
  const contagem = [0,0,0,0,0,0,0,0];
  atendimentosMes(ym).forEach(a=>{ contagem[new Date(a.date+'T12:00').getDay()]++});
  return contagem;
}

function rankingClientes(ym){
  const porCliente = {};
  atendimentosMes(ym).forEach(a=>{
    const nome = (a.client||'').trim();
    if(!nome) return;
    porCliente[nome] = (porCliente[nome]||0) + valorRealAtendimento(a);
  });
}

```

```

    const lista = Object.entries(porCliente).map(([nome, total])=>
({nome, total})).sort((a, b)=>b.total-a.total);
    const soma = lista.reduce((t, x)=>t+x.total, 0) || 1;
    return lista.slice(0, 8).map(x=>({...x, pct: (x.total/soma)*100}));
}

```

```

function renderRelatorios(view) {
    const done = state.appointments.filter(a=>a.status==='concluido');
    const ticketMedio = done.length ?
done.reduce((a, ap)=>a+valorRealAtendimento(ap), 0)/done.length : 0;
    const totalAppts = state.appointments.length;
    const ocupacao = totalAppts ? Math.round((done.length/totalAppts)*100) : 0;
    const ns = noShowStats();

    const byProf = state.professionals.map(p=>({
        name: p.name,
        value:
done.filter(a=>a.professionalId===p.id).reduce((a, ap)=>a+valorRealAtendimento(ap), 0)
    })).sort((a, b)=>b.value-a.value);
    const maxProf = Math.max(1, ...byProf.map(b=>b.value));

    const bySvc = state.services.map(s=>({
        name: s.name,
        count: done.filter(a=>a.serviceId===s.id).length
    })).sort((a, b)=>b.count-a.count);
    const maxSvc = Math.max(1, ...bySvc.map(b=>b.count));

    // ---- Desempenho por profissional: semana e mês navegáveis ----
    if(!relatSemana) relatSemana = mondayOfWeek(todayISO());
    if(!relatMes) relatMes = todayISO().slice(0, 7);

    const inicioSemana = relatSemana;
    const fimSemana = addDaysISO(inicioSemana, 6);
    const doneSemana = state.appointments.filter(a=>a.status==='concluido' &&
a.date>=inicioSemana && a.date<=fimSemana);
    const porProfSemana = servingProfessionals().map(p=>{
        const apps = doneSemana.filter(a=>a.professionalId===p.id);
        const bruto = apps.reduce((t, a)=>t+valorRealAtendimento(a), 0);
        const comissao = apps.reduce((t, a)=>{
            const v = valorRealAtendimento(a);
            if(v<=0) return t; // coberto pelo clube: não gera comissão de serviço
            return t + v*(effectiveCommissionPercent(p, a.serviceId)/100);
        }, 0);
        return { prof: p, valor: bruto, comissao, lucro: bruto-comissao, qtd:
apps.length };
    })).sort((a, b)=>b.valor-a.valor);
    const maxSemana = Math.max(1, ...porProfSemana.map(r=>r.valor));

```

```

const totSemana = porProfSemana.reduce((t,r)=>({
  bruto:t.bruto+r.valor, comissao:t.comissao+r.comissao, lucro:t.lucro+r.lucro
})), {bruto:0, comissao:0, lucro:0});

const doneMes = state.appointments.filter(a=>a.status==='concluido' &&
a.date.startsWith(relatMes));
const porProfMes = servingProfessionals().map(p=>{
  const apps = doneMes.filter(a=>a.professionalId===p.id);
  const bruto = apps.reduce((t,a)=>t+valorRealAtendimento(a),0);
  const comissao = apps.reduce((t,a)=>{
    const v = valorRealAtendimento(a);
    if(v<=0) return t;
    return t + v*(effectiveCommissionPercent(p, a.serviceId)/100);
  },0);
  return { prof:p, valor:bruto, comissao, lucro: bruto-comissao, qtd:
apps.length };
}).sort((a,b)=>b.valor-a.valor);
const maxMes = Math.max(1, ...porProfMes.map(r=>r.valor));
const totMes = porProfMes.reduce((t,r)=>({
  bruto:t.bruto+r.valor, comissao:t.comissao+r.comissao, lucro:t.lucro+r.lucro
})), {bruto:0, comissao:0, lucro:0});

view.innerHTML = `
  ${pageHeader('Relatórios', 'Desempenho com base nos atendimentos concluídos')}
  <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(180px,1fr));gap:14px;margin-bottom:24px;">
    ${statCard('Ticket médio', BRL(ticketMedio), 'por atendimento concluído',
'ok')}
    ${statCard('Taxa de ocupação', ocupacao+'%', 'agendamentos concluídos vs.
total', 'ok')}
    ${statCard('Atendimentos concluídos', done.length, 'no histórico atual',
'ok')}
    ${statCard('Taxa de no-show', ns.rate.toFixed(0)+'%', ns.total+' falta(s)
registrada(s)', ns.rate>10?'bad':'ok')}
  </div>

  <div class="card" style="padding:20px;margin-bottom:20px;">
    <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:4px;flex-wrap:wrap;gap:10px;">
      <div class="font-display" style="font-size:16px;font-
weight:600;">Desempenho por profissional – semana</div>
      <div style="display:flex;gap:6px;align-items:center;">
        <button class="btn btn-ghost" id="rel-sem-prev" style="padding:6px
10px;"><</button>
        <span class="pill" style="padding:6px 12px;font-
size:12.5px;">${esc(weekRangeLabel(inicioSemana))}</span>
        <button class="btn btn-ghost" id="rel-sem-next" style="padding:6px

```

```

10px;" ${fimSemana}>todayISO()?'disabled':''>></button>
    </div>
</div>
<div style="font-size:11.5px;color:var(--muted-2);margin-
bottom:16px;">Visível apenas para dono/admin. Não inclui o pote do clube (esse é
sempre mensal – veja em Comissões).</div>
    ${porProfSemana.length===0 ? emptyState('Nenhum profissional cadastrado.') :
    ,

    <div style="display:flex;flex-direction:column;gap:14px;">
        ${porProfSemana.map(r=>`
            <div style="padding:12px 14px;border:1px solid var(--border);border-
radius:10px;">
                <div style="display:flex;justify-content:space-between;font-
size:13.5px;font-weight:600;margin-bottom:8px;">
                    <span>${esc(r.prof.name)} <span style="color:var(--muted-2);font-
weight:400;font-size:11.5px;">${r.qtd} atend.</span></span>
                    <span class="font-mono" style="color:var(--gold-
bright);">${BRL(r.valor)}</span>
                </div>
                <div style="height:7px;background:var(--panel-2);border-
radius:99px;overflow:hidden;margin-bottom:9px;display:flex;">
                    <div style="height:100%;width:${r.valor>0?
(r.comissao/r.valor*100).toFixed(0):0}%;background:var(--gold);"></div>
                    <div style="height:100%;flex:1;background:var(--muted-
2);opacity:.4;"></div>
                </div>
                <div style="display:flex;justify-content:space-
between;gap:10px;font-size:12px;flex-wrap:wrap;">
                    <span style="color:var(--muted);">🟡 Comissão do
profissional</span>
                    <span class="font-mono" style="color:var(--gold-
bright);">${BRL(r.comissao)}</span>
                </div>
                <div style="display:flex;justify-content:space-
between;gap:10px;font-size:12px;flex-wrap:wrap;">
                    <span style="color:var(--muted);">🟤 Lucro da barbearia</span>
                    <span class="font-mono">${BRL(r.lucro)}</span>
                </div>
            </div>
        `).join('')}
    </div>
    <div style="display:flex;justify-content:space-between;gap:10px;font-
size:12.5px;margin-top:16px;padding-top:14px;border-top:1px solid var(--
border);flex-wrap:wrap;">
        <span style="color:var(--muted);">Total bruto: <b class="font-mono"
style="color:var(--ink);">${BRL(totSemana.bruto)}</b></span>
        <span style="color:var(--muted);">Comissões: <b class="font-mono"

```

```

style="color:var(--gold-bright);">${BRL(totSemana.comissao)}</b></span>
    <span style="color:var(--muted);">Lucro barbearia: <b class="font-mono"
style="color:var(--ink);">${BRL(totSemana.lucro)}</b></span>
    </div>
  `}
</div>

```

```

<div class="card" style="padding:20px;margin-bottom:24px;">
  <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:4px;flex-wrap:wrap;gap:10px;">
    <div class="font-display" style="font-size:16px;font-
weight:600;">Desempenho por profissional – mês</div>
    <div style="display:flex;gap:6px;align-items:center;">
      <button class="btn btn-ghost" id="rel-mes-prev" style="padding:6px
10px;"></button>
      <span class="pill" style="padding:6px 12px;font-
size:12.5px;">${ymLabel(relatMes)}</span>
      <button class="btn btn-ghost" id="rel-mes-next" style="padding:6px
10px;" ${relatMes}>currentYM()?'disabled':''>></button>
    </div>
  </div>
  <div style="font-size:11.5px;color:var(--muted-2);margin-
bottom:16px;">Visível apenas para dono/admin. Não inclui o pote do clube (esse é
sempre mensal – veja em Comissões).</div>
  ${porProfMes.length===0 ? emptyState('Nenhum profissional cadastrado.') : `
    <div style="display:flex;flex-direction:column;gap:14px;">
      ${porProfMes.map(r=>`
        <div style="padding:12px 14px;border:1px solid var(--border);border-
radius:10px;">
          <div style="display:flex;justify-content:space-between;font-
size:13.5px;font-weight:600;margin-bottom:8px;">
            <span>${esc(r.prof.name)} <span style="color:var(--muted-2);font-
weight:400;font-size:11.5px;">${r.qtd} atend.</span></span>
            <span class="font-mono" style="color:var(--gold-
bright);">${BRL(r.valor)}</span>
          </div>
          <div style="height:7px;background:var(--panel-2);border-
radius:99px;overflow:hidden;margin-bottom:9px;display:flex;">
            <div style="height:100%;width:${r.valor>0?
(r.comissao/r.valor*100).toFixed(0):0}%;background:var(--gold);"></div>
            <div style="height:100%;flex:1;background:var(--muted-
2);opacity:.4;"></div>
          </div>
          <div style="display:flex;justify-content:space-
between;gap:10px;font-size:12px;flex-wrap:wrap;">
            <span style="color:var(--muted);">🟡 Comissão do
profissional</span>

```

```

        <span class="font-mono" style="color:var(--gold-bright);">${BRL(r.comissao)}</span>
      </div>
      <div style="display:flex;justify-content:space-between;gap:10px;font-size:12px;flex-wrap:wrap;">
        <span style="color:var(--muted);">■ Lucro da barbearia</span>
        <span class="font-mono">${BRL(r.lucro)}</span>
      </div>
    </div>
    `).join('')
  </div>

  <div style="display:flex;justify-content:space-between;gap:10px;font-size:12.5px;margin-top:16px;padding-top:14px;border-top:1px solid var(--border);flex-wrap:wrap;">
    <span style="color:var(--muted);">Total bruto: <b class="font-mono" style="color:var(--ink);">${BRL(totMes.bruto)}</b></span>
    <span style="color:var(--muted);">Comissões: <b class="font-mono" style="color:var(--gold-bright);">${BRL(totMes.comissao)}</b></span>
    <span style="color:var(--muted);">Lucro barbearia: <b class="font-mono" style="color:var(--ink);">${BRL(totMes.lucro)}</b></span>
  </div>
  `}
</div>

<div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;" class="dash-grid">
  <div class="card" style="padding:20px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:16px;">Faturamento por profissional</div>
    ${byProf.length===0?emptyState('Sem dados ainda.'):byProf.map(b=>`
      <div style="margin-bottom:12px;">
        <div style="display:flex;justify-content:space-between;font-size:13px;margin-bottom:4px;">
          <span>${esc(b.name)}</span><span class="font-mono" style="color:var(--gold-bright);">${BRL(b.value)}</span>
        </div>
        <div style="height:7px;background:var(--panel-2);border-radius:99px;overflow:hidden;">
          <div style="height:100%;width:${(b.value/maxProf*100).toFixed(0)}%;background:var(--gold);border-radius:99px;"></div>
        </div>
      </div>
    `).join('')}
  </div>
  <div class="card" style="padding:20px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-

```

```

bottom:16px;">Serviços mais procurados</div>
    ${bySvc.length===0?emptyState('Sem dados ainda.'):bySvc.map(b=>`
      <div style="margin-bottom:12px;">
        <div style="display:flex;justify-content:space-between;font-
size:13px;margin-bottom:4px;">
          <span>${esc(b.name)}</span><span class="font-mono"
style="color:var(--muted);">${b.count}x</span>
        </div>
        <div style="height:7px;background:var(--panel-2);border-
radius:99px;overflow:hidden;">
          <div
style="height:100%;width:${(b.count/maxSvc*100).toFixed(0)}%;background:var(--
ok);border-radius:99px;"></div>
        </div>
      </div>
    `).join('')}
  </div>
</div>

<div class="divider-blade" style="margin:22px 0 18px;"></div>
<div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;" class="dash-
grid">
  <div class="card" style="padding:20px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:14px;">No-show por profissional</div>
    ${ns.byProf.filter(b=>b.total>0).length===0 ? emptyState('Sem dados
suficientes ainda.') : ns.byProf.filter(b=>b.total>0).map(b=>`
      <div style="display:flex;justify-content:space-between;align-
items:center;font-size:13px;padding:6px 0;border-top:1px solid var(--border);">
        <span>${esc(b.prof.name)}</span>
        <span class="font-mono" style="color:${b.rate>10?'var(--bad)':'var(--
muted)'};">${b.faltas}/${b.total} · ${b.rate.toFixed(0)}%</span>
      </div>
    `).join('')}
  </div>
  <div class="card" style="padding:20px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:14px;">Avaliação média por profissional</div>
    ${state.professionals.slice().sort(byName).map(p=>{
      const rt = ratingStats(p.id);
      return `<div style="display:flex;justify-content:space-between;align-
items:center;font-size:13px;padding:6px 0;border-top:1px solid var(--border);">
        <span>${esc(p.name)}</span>
        ${rt.count>0 ? `<span style="color:var(--gold-
bright);display:flex;align-items:center;gap:4px;">${ICON.star} <span class="font-
mono">${rt.avg.toFixed(1)}</span> <span style="color:var(--muted-2);font-
size:11px;">(${rt.count})</span></span>` : `<span style="color:var(--muted-

```

```

2);font-size:12px;">sem avaliações</span>`}
    </div>`;
    }).join('')}
</div>
</div>
<div style="margin-top:14px;font-size:12px;color:var(--muted-2);">Os
relatórios são calculados em tempo real a partir da agenda e do caixa – sem
necessidade de exportação manual.</div>
`;
view.querySelector('#rel-sem-prev')?.addEventListener('click', ()=>{ relatSemana
= addDaysISO(relatSemana,-7); renderRelatorios(view); });
view.querySelector('#rel-sem-next')?.addEventListener('click', ()=>{ relatSemana
= addDaysISO(relatSemana,7); renderRelatorios(view); });
view.querySelector('#rel-mes-prev')?.addEventListener('click', ()=>{ relatMes =
shiftYM(relatMes,-1); renderRelatorios(view); });
view.querySelector('#rel-mes-next')?.addEventListener('click', ()=>{ relatMes =
shiftYM(relatMes,1); renderRelatorios(view); });
}

/* ===== FIDELIDADE ===== */
function renderFidelidade(view){
    const L = state.loyalty;
    const now = new Date();
    const thisMonth = String(now.getMonth()+1).padStart(2,'0');
    const birthdays = state.clients.filter(c=>c.birthday &&
c.birthday.startsWith(thisMonth+'-'))
        .sort((a,b)=>a.birthday.localeCompare(b.birthday));

    view.innerHTML = `
        ${pageHeader('Fidelidade', 'Pontos, bonificação e aniversariantes')}
        <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(180px,1fr));gap:14px;margin-bottom:24px;">
            ${statCard('Regra de pontos', `${L.pointsPerReal} pt / R$1`, 'gasto em
serviços', 'ok')}
            ${statCard('Resgate', `${L.redeemThreshold} pts`, `= ${BRL(L.redeemValue)}
de desconto`, 'ok')}
            ${statCard('Aniversariantes do mês', birthdays.length, 'clientes para
lembrar', birthdays.length>0?'warn':'ok')}
        </div>

        <div class="card" style="padding:20px;margin-bottom:22px;">
            <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:14px;">Configurar programa de pontos</div>
            <div style="display:flex;gap:12px;flex-wrap:wrap;align-items:flex-end;">
                <div><label>Pontos por R$1 gasto</label><input id="f-ppr" type="number"
min="0" step="0.1" value="${L.pointsPerReal}" style="width:140px;"></div>
                <div><label>Pontos para resgate</label><input id="f-thr" type="number"

```

```

min="1" value="${L.redeemThreshold}" style="width:140px;"></div>
    <div><label>Valor do desconto (R$)</label><input id="f-val" type="number"
min="0" step="0.01" value="${L.redeemValue}" style="width:140px;"></div>
    <button class="btn btn-gold" id="f-save-loyalty">Salvar regra</button>
</div>
</div>

<div class="card" style="padding:20px;margin-bottom:22px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:14px;">${ICON.gift} Programa de indicação</div>
    <div style="font-size:12.5px;color:var(--muted);margin-bottom:12px;">Quando
um cliente indicado (campo "Indicado por" no cadastro) tem seu primeiro
atendimento concluído, quem indicou ganha pontos de bônus automaticamente.</div>
    <div style="display:flex;gap:12px;flex-wrap:wrap;align-items:flex-end;">
        <div><label>Pontos de bônus por indicação</label><input id="f-referral"
type="number" min="0" value="${L.referralBonusPoints}" style="width:180px;"></div>
        <button class="btn btn-gold" id="f-save-referral">Salvar</button>
    </div>
</div>

<div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Aniversariantes deste mês</div>
    ${birthdays.length===0 ? emptyState('Nenhum aniversariante este mês.') : `
    <div style="display:flex;flex-direction:column;gap:8px;margin-bottom:24px;">
        ${birthdays.map(c=>`<div style="display:flex;align-
items:center;gap:12px;padding:10px 14px;background:var(--panel-2);border:1px solid
var(--border);border-radius:9px;">
            <div style="color:var(--gold-bright);">${ICON.cake}</div>
            <div style="flex:1;"><div style="font-size:13.5px;font-
weight:500;">${esc(c.name)}</div><div style="font-size:12px;color:var(--
muted);">${esc(c.phone)}</div></div>
            <span class="pill pill-warn font-mono">${monthDayLabel(c.birthday)}
</span>
        </div>`}).join('')}
    </div>
`}

    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Pontuação por cliente</div>
    ${state.clients.length===0 ? emptyState('Nenhum cliente cadastrado.') : `
    <table>
        <thead><tr><th>Cliente</th><th>Pontos</th><th>Progresso</th><th></th></tr>
    </thead>
        <tbody>
            ${state.clients.map(c=>{
                const pct = Math.min(100, (c.points/L.redeemThreshold)*100);
                return `<tr>

```

```

        <td style="font-weight:500;">${esc(c.name)}</td>
        <td class="font-mono" style="color:var(--gold-bright);">${c.points}
</td>

        <td><div style="height:7px;width:120px;background:var(--panel-
2);border-radius:99px;overflow:hidden;"><div
style="height:100%;width:${pct}%;background:var(--gold);"></div></div></td>
        <td style="text-align:right;"><button class="btn btn-ghost"
style="padding:6px 10px;" data-add-points="${c.id}">+10 pts</button></td>
    </tr>`;
    }).join('')
</tbody>
</table>
`
`;
view.querySelector('#f-save-loyalty').addEventListener('click', ()=>{
  update(s=>{
    s.loyalty.pointsPerReal = parseFloat(view.querySelector('#f-ppr').value)||0;
    s.loyalty.redeemThreshold = parseInt(view.querySelector('#f-thr').value)||1;
    s.loyalty.redeemValue = parseFloat(view.querySelector('#f-val').value)||0;
  });
  showToast('Regra de fidelidade atualizada.');
```

```

  });
  view.querySelectorAll('[data-add-
points]').forEach(b=>b.addEventListener('click', ()=>{
    update(s=>{ s.clients.find(c=>c.id===b.dataset.addPoints).points += 10; });
  }));
}

/* ===== AGENDAMENTO ONLINE ===== */
function renderOnlineBooking(view){
  const ob = state.onlineBooking;
  const link = ob.slug ? publicBookingUrl(ob.slug) : '(configure um identificador
de link abaixo)';
  view.innerHTML = `
    ${pageHeader('Agendamento Online', 'Link público para seus clientes marcarem
horário sozinhos')}

    <div class="card" style="padding:22px;margin-bottom:20px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;flex-wrap:wrap;gap:14px;">
        <div style="min-width:0;">
          <div style="font-size:12px;color:var(--muted);margin-bottom:6px;">Seu
link personalizado</div>
          <div class="font-mono" style="font-size:13px;color:var(--gold-
bright);word-break:break-all;">${link}</div>
        </div>
        <div style="display:flex;gap:8px;flex-shrink:0;">

```

```

        <button class="btn btn-ghost" id="btn-copy-link">${ICON.copy} Copiar
link</button>
        <span class="pill ${ob.active?'pill-ok':'pill-
bad'}">${ob.active?'Ativo':'Desativado'}</span>
    </div>
</div>
</div>

<div class="card" style="padding:22px;margin-bottom:20px;">
    <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:4px;">
        <div class="font-display" style="font-size:16px;font-
weight:600;">Solicitações de agendamento</div>
        <span class="pill pill-warn" id="pb-pending-count">carregando...</span>
    </div>
    <div style="font-size:12px;color:var(--muted);margin-bottom:14px;">Pedidos
que clientes fizeram pelo link público, aguardando sua confirmação.</div>
    <div id="pb-requests-list">Carregando...</div>
</div>

<div class="card" style="padding:20px;margin-bottom:20px;max-width:460px;">
    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:14px;">Comportamento do link</div>
    <div style="display:flex;flex-direction:column;gap:14px;">
        <div><label>Identificador do link (slug)</label><input id="f-slug"
value="${ob.slug}" style="width:100%;"></div>
        <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;color:var(--ink);cursor:pointer;"><input type="checkbox" id="f-active"
${ob.active?'checked':''} style="width:auto;padding:0;"> Página de agendamento
ativa</label>

        <label style="display:flex;align-items:flex-start;gap:10px;padding:13px
15px;border:1px solid ${ob.autoSubscribe?'var(--gold)':'var(--border)'};border-
radius:10px;cursor:pointer;">
            <input type="checkbox" id="f-auto-subscribe"
${ob.autoSubscribe?'checked':''} style="width:auto;padding:0;margin-top:2px;">
            <div>
                <div style="font-size:13.5px;font-weight:600;">Criar assinatura
automaticamente</div>
                <div style="font-size:11.5px;color:var(--muted-2);margin-top:3px;line-
height:1.6;">
                    ${ob.autoSubscribe
                        ? 'O cliente assina pela página de planos e a cobrança já é gerada
no Asaas, com o link de pagamento na tela.'
                        : 'Hoje o pedido espera você confirmar. Ative para o cliente já
sair com a cobrança gerada.'}
                    <br><span style="color:var(--warn);">Precisa da API Key do Asaas

```

```

configurada.</span>
    </div>
</div>
</label>

    <label style="display:flex;align-items:flex-start;gap:10px;padding:13px
15px;border:1px solid ${ob.autoAccept?'var(--gold)':'var(--border)'};border-
radius:10px;cursor:pointer;">
        <input type="checkbox" id="f-auto-accept" ${ob.autoAccept?'checked':''}
style="width:auto;padding:0;margin-top:2px;">
        <div>
            <div style="font-size:13.5px;font-weight:600;">Aceitar agendamentos
automaticamente</div>
            <div style="font-size:11.5px;color:var(--muted-2);margin-top:3px;line-
height:1.6;">
                ${ob.autoAccept
                    ? 'O horário do cliente entra direto na agenda, sem você precisar
confirmar. Você continua recebendo o aviso.'
                    : 'Hoje cada agendamento espera você aceitar. Ative para o cliente
já sair com o horário garantido.'}
            </div>
        </div>
    </label>

    <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;color:var(--ink);cursor:pointer;"><input type="checkbox" id="f-cancel"
${ob.allowCancel?'checked':''} style="width:auto;padding:0;"> Cliente pode
cancelar online</label>

    <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;color:var(--ink);cursor:pointer;"><input type="checkbox" id="f-
resched" ${ob.allowReschedule?'checked':''} style="width:auto;padding:0;"> Cliente
pode remarcar online</label>

    <button class="btn btn-gold" id="f-save-ob" style="justify-
content:center;">Salvar configurações</button>
</div>
</div>

    <div style="font-size:12px;color:var(--muted-2);margin-
bottom:20px;display:flex;align-items:center;gap:6px;">${ICON.image} Logo, banner,
bio e redes sociais da página ficam em <span data-goto="configuracoes"
style="color:var(--gold-bright);cursor:pointer;text-
decoration:underline;">Configurações da Página</span></div>
`;

view.querySelector('[data-goto="configuracoes"]')?.addEventListener('click',
()=>{ route='configuracoes'; render(); });
view.querySelector('#btn-copy-link').addEventListener('click', ()=>{
    if(ob.slug){ navigator.clipboard?.writeText(link); showToast('Link copiado:
'+link); }

```

```

    else showToast('Configure um identificador de link antes de copiar.');
```

```

  });
  view.querySelector('#f-save-ob').addEventListener('click', ()=>{
    update(s=>{
      s.onlineBooking.slug = slugify(view.querySelector('#f-slug').value.trim())
    || s.onlineBooking.slug;
      s.onlineBooking.active = view.querySelector('#f-active').checked;
      s.onlineBooking.autoAccept = view.querySelector('#f-auto-accept').checked;
      s.onlineBooking.autoSubscribe = view.querySelector('#f-auto-
subscribe').checked;
      s.onlineBooking.allowCancel = view.querySelector('#f-cancel').checked;
      s.onlineBooking.allowReschedule = view.querySelector('#f-resched').checked;
    });
    showToast('Configurações salvas.');
```

```

    renderOnlineBooking(view);
  });
  loadBookingRequests(view);
}

```

```

async function acceptBookingRequest(req){
  // Avisa se o pedido cair num dia de folga ou fora do expediente do
  // profissional, antes de jogar direto na agenda sem querer.
  const prof = state.professionals.find(p=>p.id===req.professional_id);
  if(prof){
    const dow = new Date(req.date+'T12:00').getDay();
    const dias = Array.isArray(prof.workDays) ? prof.workDays : [1,2,3,4,5,6];
    const foraDoDia = !dias.includes(dow);
    const foraDoHorario = req.time < (prof.workStart||'09:00') || req.time >=
    (prof.workEnd||'18:00');
    if(foraDoDia || foraDoHorario){
      const motivo = foraDoDia
        ? `${prof.name} não trabalha neste dia da semana`
        : `${req.time} está fora do expediente de ${prof.name}
        (${prof.workStart||'09:00'} às ${prof.workEnd||'18:00'})`;
      if(!confirm(`⚠️ ${motivo}.\n\nAceitar mesmo assim como uma exceção?`))
        return;
    }
  }
  update(s=>{
    s.appointments.push({id:uid(), date:req.date, time:req.time,
    professionalId:req.professional_id, serviceId:req.service_id,
    client:req.client_name, status:'agendado', confirmed:false,
    depositStatus:'nao_aplicavel', depositAmount:0, createdAt:new
    Date().toISOString(), rating:null, origem:'online',
    clientPhone:req.client_phone||'', clientEmail:req.client_email||''});
  });
  await supa.from('booking_requests').update({status:'aceito'}).eq('id', req.id);
}

```

```

showToast('Agendamento aceito e adicionado à agenda.');
```

```

await refreshNotifItems();

// Só agora que virou de verdade um agendamento – dispara os e-mails
const shopNameEmail = state.shopProfile?.name || '';
if(req.client_email){
  enviarEmailAgendamento({
    tipo:'confirmacao_cliente', to: req.client_email, shopName: shopNameEmail,
    clientName: req.client_name, serviceName: req.service_name, profName:
req.professional_name,
    date: req.date, time: req.time,
  });
}
if(prof?.email){
  enviarEmailAgendamento({
    tipo:'aviso_profissional', to: prof.email, shopName: shopNameEmail,
    clientName: req.client_name, serviceName: req.service_name, profName:
prof.name,
    date: req.date, time: req.time,
  });
}
}

async function declineBookingRequest(id){
  await supa.from('booking_requests').update({status:'recusado'}).eq('id', id);
  showToast('Pedido recusado.');
```

```

  await refreshNotifItems();
}

async function loadBookingRequests(view){
  const listEl = view.querySelector('#pb-requests-list');
  const countEl = view.querySelector('#pb-pending-count');
  if(!listEl || !currentShop) return;
  const { data, error } = await
supa.from('booking_requests').select('*').eq('shop_id',
currentShop.id).eq('status', 'pendente').order('created_at',{ascending:false});
  if(error){ listEl.innerHTML = '<div style="font-size:12.5px;color:var(--
muted);">Não foi possível carregar.</div>'; return; }
  countEl.textContent = (data||[]).length + ' pendente(s)';
  if(!data || !data.length){ listEl.innerHTML = '<div style="font-
size:12.5px;color:var(--muted);">Nenhum pedido pendente.</div>'; return; }
  listEl.innerHTML = data.map(r=>`
    <div style="display:flex;justify-content:space-between;align-
items:center;gap:10px;padding:10px 0;border-bottom:1px solid var(--border);">
      <div style="font-size:13px;">
        <b>${esc(r.client_name)}</b> – ${esc(r.service_name)} com
${esc(r.professional_name)}<br>
        <span style="color:var(--muted);font-size:11.5px;">${esc(r.date)} às

```

```

    ${esc(r.time)} · ${esc(r.client_phone)}</span>
    </div>
    <div style="display:flex;gap:6px;flex-shrink:0;">
        <button class="btn btn-gold" data-accept="${r.id}" style="padding:6px
10px;font-size:12px;">Aceitar</button>
        <button class="btn btn-ghost" data-decline="${r.id}" style="padding:6px
10px;font-size:12px;">Recusar</button>
    </div>
</div>
` ).join('');
    listEl.querySelectorAll('[data-accept]').forEach(b=>b.addEventListener('click',
async ()=>{
        const req = data.find(r=>r.id===b.dataset.accept);
        await acceptBookingRequest(req);
        loadBookingRequests(view);
    }));
    listEl.querySelectorAll('[data-decline]').forEach(b=>b.addEventListener('click',
async ()=>{
        await declineBookingRequest(b.dataset.decline);
        loadBookingRequests(view);
    }));
}

/* ===== SININHO DE NOTIFICAÇÕES (novos agendamentos em tempo real)
===== */
async function refreshNotifItems(){
    if(!supa || !currentShop) return;
    const { data } = await supa.from('booking_requests').select('*').eq('shop_id',
currentShop.id).eq('status','pendente').order('created_at',{ascending:false});
    notifState.items = data || [];
    renderNotifUI();
}

/* ===== ALERTAS: som, vibração e notificação do sistema ===== */
// O som é gerado na hora, sem precisar de arquivo de áudio.
function tocarAlerta(){
    if(state?.alertSound === false) return;
    try{
        const Ctx = window.AudioContext || window.webkitAudioContext;
        if(!Ctx) return;
        const ctx = new Ctx();
        // dois toques curtos, tipo "din-don"
        [[880, 0], [1180, 0.18]].forEach(([freq, atraso])=>{
            const osc = ctx.createOscillator();
            const gain = ctx.createGain();
            osc.type = 'sine';
            osc.frequency.value = freq;
            osc.connect(gain); gain.connect(ctx.destination);

```

```

        const t = ctx.currentTime + atraso;
        gain.gain.setValueAtTime(0, t);
        gain.gain.linearRampToValueAtTime(0.28, t + 0.02);
        gain.gain.exponentialRampToValueAtTime(0.001, t + 0.32);
        osc.start(t); osc.stop(t + 0.34);
    });
    setTimeout(()=>ctx.close().catch(()=>{}), 1200);
} catch(e) { /* som é extra, nunca deve quebrar nada */ }
}
function vibrarCelular(){
    if(state?.alertVibrate === false) return;
    // Só funciona em Android. iPhone não permite vibração pelo navegador.
    if(navigator.vibrate) { try{ navigator.vibrate([220, 90, 220]); }catch(e){} }
}
function notificarSistema(titulo, corpo){
    if(state?.alertPush === false) return;
    if(!('Notification' in window) || Notification.permission !== 'granted') return;
    try{
        const n = new Notification(titulo, {
            body: corpo,
            icon: APP_LOGO_ICON,
            badge: APP_LOGO_ICON,
            tag: 'novo-agendamento',
            renotify: true,
        });
        n.onclick = ()=>{ window.focus(); n.close(); };
    }catch(e){}
}
function alertarNovoAgendamento(r){
    tocarAlerta();
    vibrarCelular();
    notificarSistema(
        'Novo agendamento!',
        `${r.client_name} - ${r.service_name} com ${r.professional_name}, ${new
Date(r.date+'T'+r.time).toLocaleDateString('pt-BR')} às ${r.time}`
    );
}
async function pedirPermissaoAlertas(){
    if(!('Notification' in window)){
        showToast('Este navegador não suporta notificações do sistema.');
```

```

    }
    const p = await Notification.requestPermission();
    if(p === 'granted'){ tocarAlerta(); showToast('Alertas ativados!'); return true;
  }
  showToast('Permissão não concedida.');
```

return false;

```

}

/* ===== NOTIFICAÇÃO PUSH DE VERDADE (chega com o site fechado) =====
  Diferente do alerta sonoro (que só funciona com a aba aberta), isto
  usa o Service Worker do navegador – a mesma tecnologia por trás das
  notificações do Facebook, WhatsApp Web, etc. */
const VAPID_PUBLIC_KEY =
'BCbZm9QgNYBtE7Lg_urxVLUoXvlCRN8A5Bo5cbFVqjNunQyldFqtooNy9H3PDcLbuNmwwT0cRUoVuX7_D1

function urlBase64ToUint8Array(base64String) {
  const padding = '='.repeat((4 - base64String.length % 4) % 4);
  const base64 = (base64String + padding).replace(/-/g, '+').replace(/_/g, '/');
  const raw = atob(base64);
  return Uint8Array.from([...raw].map(c=>c.charCodeAt(0)));
}

async function ativarNotificacaoPush(){
  if(!('serviceWorker' in navigator) || !('PushManager' in window)){
    showToast('Este navegador não suporta notificações em segundo plano. Tente
pelo Chrome ou Edge.');
```

return false;

```

  }
  const ok = await pedirPermissaoAlertas();
  if(!ok) return false;
  try{
    const reg = await navigator.serviceWorker.register('/sw.js');
    await navigator.serviceWorker.ready;
    let sub = await reg.pushManager.getSubscription();
    if(!sub){
      sub = await reg.pushManager.subscribe({
        userVisibleOnly: true,
        applicationServerKey: urlBase64ToUint8Array(VAPID_PUBLIC_KEY),
      });
    }
    const json = sub.toJSON();
    const { error } = await supabase.from('push_subscriptions').upsert({
      shop_id: currentShop.id,
      endpoint: sub.endpoint,
      p256dh: json.keys.p256dh,
      auth: json.keys.auth,
    }, { onConflict: 'endpoint' });
  }
}

```

```

        if(error){ showToast('Não foi possível ativar: '+error.message); return false;
    }

    showToast('Notificações ativadas neste dispositivo! Você vai receber avisos
mesmo com o site fechado.');
```

```

        return true;
    }catch(e){
        console.error(e);
        showToast('Não foi possível ativar as notificações neste navegador.');
```

```

        return false;
    }
}

```

```

async function statusNotificacaoPush(){
    if(!('serviceWorker' in navigator) || !('PushManager' in window)) return
'unsupported';
    try{
        const reg = await navigator.serviceWorker.getRegistration('/sw.js');
        if(!reg) return 'off';
        const sub = await reg.pushManager.getSubscription();
        return sub ? 'on' : 'off';
    }catch(e){ return 'off'; }
}

```

```

async function enviarPushParaLoja(shopId, title, message, url){
    try{
        await supa.functions.invoke('push-notify', { body: { shopId, title, message,
url } });
    }catch(e){ /* notificação é extra, nunca deve travar o fluxo principal */ }
}

```

```

// Dispara e-mail de confirmação (cliente) ou aviso (profissional) na
// hora do agendamento. "Fire and forget": se falhar, não trava o
// agendamento – o cliente já está marcado de qualquer forma.

```

```

async function enviarEmailAgendamento(payload){
    try{
        await supa.functions.invoke('appointment-email', { body: payload });
    }catch(e){ /* e-mail é extra, nunca deve travar o fluxo principal */ }
}

```

```

function subscribeBookingNotifications(){
    if(!supa || !currentShop || notifState.channel) return;
    notifState.channel = supa.channel('booking-notify-'+currentShop.id)
        .on('postgres_changes', {event:'INSERT', schema:'public',
table:'booking_requests', filter:`shop_id=eq.${currentShop.id}`}, payload=>{
        notifState.items.unshift(payload.new);
        showToast(`Novo agendamento: ${payload.new.client_name} –
${payload.new.date} às ${payload.new.time}`);
    });
}

```

```

        alertarNovoAgendamento(payload.new);
        renderNotifUI();
    })
    // pedidos de assinatura vindos da página pública de planos
    .on('postgres_changes', {event:'INSERT', schema:'public',
table:'subscription_requests', filter:`shop_id=eq.${currentShop.id}`}, payload=>{
        const r = payload.new;
        showToast(`Novo pedido de assinatura: ${r.client_name} - ${r.plan_name}`);
        tocarAlerta(); vibrarCelular();
        notificarSistema('Novo pedido de assinatura!', `${r.client_name} quer
assinar o ${r.plan_name}`);
    })
    // Solicitações de saque ficam dentro do JSON da barbearia, então
    // escutamos a própria linha da loja para avisar o dono na hora.
    .on('postgres_changes', {event:'UPDATE', schema:'public', table:'shops',
filter:`id=eq.${currentShop.id}`}, payload=>{
        // Alguém salvou de outro dispositivo (profissional no celular, por ex.).
        // Se a tela local ficar desatualizada, o próximo salvamento daqui
        // apagaria o trabalho do outro – então recarregamos os dados.
        const remoto = payload.new?.data;
        if(remoto && !salvandoAgora){
            const qtdRemoto = (remoto.appointments||[]).length;
            const qtdLocal = (state?.appointments||[]).length;
            const concluidosRemoto = (remoto.appointments||
[]).filter(a=>a.status==='concluido').length;
            const concluidosLocal = (state?.appointments||
[]).filter(a=>a.status==='concluido').length;
            if(qtdRemoto !== qtdLocal || concluidosRemoto !== concluidosLocal){
                // mantém a tela em dia com o que outro aparelho fez.
                // O salvamento já é protegido por merge – isto é só para a exibição.
                state = remoto;
                migrateState();
                render();
                return;
            }
        }
        if(!isOwnerLevel()) return;
        const novos = (payload.new?.data?.withdrawalRequests ||
[]).filter(w=>w.status==='pendente');
        const atuais = (state?.withdrawalRequests ||
[]).filter(w=>w.status==='pendente');
        const idsAtuais = new Set(atuais.map(w=>w.id));
        const chegaram = novos.filter(w=>!idsAtuais.has(w.id));
        if(chegaram.length===0) return;
        // traz os dados novos para a tela sem precisar recarregar
        state.withdrawalRequests = payload.new.data.withdrawalRequests;
        chegaram.forEach(w=>{

```

```

        const nome = profName(w.professionalId);
        showToast(`${nome} solicitou saque de ${BRL(w.amount)}`);
        notificarSistema('Pedido de saque', `${nome} solicitou
        ${BRL(w.amount)} ${w.note? ' - '+w.note:''}`);
    });
    tocarAlerta(); vibrarCelular();
    renderNotifUI();
    if(route==='comissoes') render();
  })
  .subscribe();
}

function renderNotifUI(){
  const badge = document.getElementById('notif-badge');
  const bell = document.getElementById('btn-notif-bell');
  if(bell){
    const saquesPend = isOwnerLevel() ? (state?.withdrawalRequests||
  []).filter(w=>w.status==='pendente').length : 0;
    const count = notifState.items.length + saquesPend;
    let b = document.getElementById('notif-badge');
    if(count>0){
      if(!b){ b = document.createElement('span'); b.id='notif-badge';
b.style.cssText='position:absolute;top:-4px;right:-4px;background:var(--
bad);color:#fff;font-size:10px;font-weight:700;border-radius:999px;min-
width:16px;height:16px;display:flex;align-items:center;justify-
content:center;padding:0 4px;'; bell.appendChild(b); }
      b.textContent = count>9?'9+':String(count);
    } else if(b){ b.remove(); }
  }
  const dropdown = document.getElementById('notif-dropdown');
  if(dropdown && notifState.open) dropdown.innerHTML = notifDropdownContent();
}

function notifDropdownContent(){
  const saques = isOwnerLevel() ? (state?.withdrawalRequests||
  []).filter(w=>w.status==='pendente') : [];
  const blocoSaques = saques.map(w=>`
    <div style="padding:10px 12px;border-bottom:1px solid var(--
border);background:var(--panel-2);">
      <div style="font-size:13px;">${ICON.cash}
<b>${esc(profName(w.professionalId))}</b> pediu saque</div>
      <div style="font-size:12px;color:var(--muted);margin:2px 0 8px;">
        <span class="font-mono" style="color:var(--gold-bright);">${BRL(w.amount)}
</span>${w.note? ' · '+esc(w.note):''}
        · ${new Date(w.createdAt).toLocaleDateString('pt-BR')}
      </div>
      <button class="btn btn-ghost" data-ir-comissoes="1" style="padding:5px
10px;font-size:11.5px;">Ver em Comissões</button>
    </div>
  `);
}

```

```

    `).join('');

    if(!notifState.items.length && !saques.length){
        return `

Nada de novo por aqui.</div>`;
    }

    return blocoSaques + notifState.items.map(r=>`
        <div style="padding:10px 12px;border-bottom:1px solid var(--border);">
            <div style="font-size:13px;"><b>${esc(r.client_name)}</b> agendou
            ${r.status==='aceito'?`<span class="pill pill-ok" style="font-size:10px;">já na agenda</span>`:''}</div>
            <div style="font-size:12px;color:var(--muted);margin:2px 0 8px;">${esc(r.service_name)} com ${esc(r.professional_name)} · ${new
            Date(r.date+'T'+r.time).toLocaleDateString('pt-BR')} às ${esc(r.time)}</div>
            ${r.status==='aceito' ? '' : `<div style="display:flex;gap:6px;">
                <button class="btn btn-gold" data-notif-accept="${r.id}"
                style="padding:5px 10px;font-size:11.5px;">Aceitar</button>
                <button class="btn btn-ghost" data-notif-decline="${r.id}"
                style="padding:5px 10px;font-size:11.5px;">Recusar</button>
            </div>`}
            </div>
        `).join('');
    }

/* ===== MENSAGENS AUTOMÁTICAS (view) ===== */
function messageChannelRow(phone, text, onSend){
    if(!phone){
        return `<button class="btn btn-ghost" disabled title="Cliente sem telefone cadastrado">${ICON.chat} Sem telefone</button>`;
    }

    return `<a class="btn btn-gold" href="${waLink(phone, text)}" target="_blank" rel="noopener" data-wa-send="1">${ICON.send} Enviar via WhatsApp</a>`;
}

function renderMensagens(view){
    const T = state.messageTemplates;
    const MS = state.messageSettings;
    const confirmations = pendingConfirmations();
    const reminders = pendingReminders();
    const absent = absentClients();

    view.innerHTML = `
        ${pageHeader('Mensagens Automáticas', 'Confirmação, lembrete, retorno de clientes ausentes e oferta de serviços/produtos')}

        <div class="card" style="padding:16px 18px;margin-bottom:22px;">
            <div style="font-size:13px;color:var(--muted);line-height:1.7;">Este painel


```

identifica automaticamente quem precisa receber cada mensagem com base na agenda e no histórico de clientes. Toque em <strong style="color:var(--ink);">Enviar via WhatsApp para abrir a conversa já com o texto pronto – o envio 100% automático (disparado sozinho, sem toque, mesmo com o sistema fechado) exige um servidor com a API oficial do WhatsApp Business rodando em segundo plano.</div>

```
<div style="display:grid;grid-template-columns:repeat(auto-fit,minmax(180px,1fr));gap:14px;margin-bottom:24px;">
  ${statCard('Aguardando confirmação', confirmations.length, 'agendamentos futuros', confirmations.length>0?'warn':'ok')}
  ${statCard('Lembretes de hoje', reminders.length, `próximas ${MS.reminderHours}h`, reminders.length>0?'warn':'ok')}
  ${statCard('Clientes ausentes', absent.length, `sem voltar há ${MS.winbackDays}+ dias`, absent.length>0?'bad':'ok')}
</div>
```

```
<div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:10px;">1 · Confirmação de horário</div>
```

```
${confirmations.length===0 ? emptyState('Nenhum agendamento aguardando confirmação.') : `
```

```
<div style="display:flex;flex-direction:column;gap:8px;margin-bottom:24px;">
  ${confirmations.map(a=>{
    const nomeServ = appointmentServiceNames(a); const phone =
    phoneForClientName(a.client);
    const text = fillTemplate(T.confirmation, {cliente:a.client,
    servico:nomeServ, data:new Date(a.date+'T00:00').toLocaleDateString('pt-BR'),
    hora:a.time, profissional:profName(a.professionalId)});
    return `<div class="card" style="padding:12px 16px;display:flex;align-items:center;gap:14px;flex-wrap:wrap;">
      <div style="flex:1;min-width:180px;">
        <div style="font-weight:600;font-size:13.5px;">${esc(a.client)}
</div>
        <div style="font-size:12px;color:var(--muted);">${new
Date(a.date+'T00:00').toLocaleDateString('pt-BR')} às ${a.time} · ${esc(nomeServ)}
</div>
      </div>
      ${messageChannelRow(phone, text)}
      <button class="btn btn-ghost" style="padding:9px 10px;" data-mark-confirmed="${a.id}" title="Marcar como confirmado">${ICON.check}</button>
    </div>`;
  }).join('')}
</div>
`}
```

```
<div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:10px;">2 · Lembrete antes do horário</div>
```

```

    <div style="font-size:12px;color:var(--muted-2);margin-
bottom:12px;display:flex;align-items:center;gap:8px;flex-wrap:wrap;">
        Enviar lembrete <input id="f-reminder-hours" type="number" min="1" max="48"
value="${MS.reminderHours}" style="width:60px;padding:5px 8px;display:inline-
block;"> hora(s) antes do horário marcado.
        <button class="btn btn-ghost" id="btn-save-reminder-hours"
style="padding:5px 10px;">Salvar</button>
    </div>

    ${reminders.length===0 ? emptyState('Nenhum lembrete pendente no momento.',
`Aparecem aqui os horários de hoje que começam em até ${MS.reminderHours}h.`) : `
    <div style="display:flex;flex-direction:column;gap:8px;margin-bottom:24px;">
        ${reminders.map(a=>{
            const nomeServ = appointmentServiceNames(a); const phone =
phoneForClientName(a.client);
            const text = fillTemplate(T.reminder, {cliente:a.client,
servico:nomeServ, hora:a.time, profissional:profName(a.professionalId)});
            return `<div class="card" style="padding:12px 16px;display:flex;align-
items:center;gap:14px;flex-wrap:wrap;">
                <div style="flex:1;min-width:180px;">
                    <div style="font-weight:600;font-size:13.5px;">${esc(a.client)}
</div>
                    <div style="font-size:12px;color:var(--muted);">Hoje às ${a.time} ·
${esc(nomeServ)}</div>
                </div>
                ${messageChannelRow(phone, text)}
                <button class="btn btn-ghost" style="padding:9px 10px;" data-mark-
reminded="${a.id}" title="Marcar como enviado">${ICON.check}</button>
            </div>`;
        })).join('')}
    </div>
`}

    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">3 · Cliente ausente – chamar de volta</div>
    <div style="font-size:12px;color:var(--muted-2);margin-
bottom:12px;display:flex;align-items:center;gap:8px;flex-wrap:wrap;">
        Considerar ausente após <input id="f-winback-days" type="number" min="1"
max="180" value="${MS.winbackDays}" style="width:60px;padding:5px
8px;display:inline-block;"> dias sem visita.
        <button class="btn btn-ghost" id="btn-save-winback-days" style="padding:5px
10px;">Salvar</button>
    </div>

    ${absent.length===0 ? emptyState('Nenhum cliente ausente pelo critério
configurado.') : `
    <div style="display:flex;flex-direction:column;gap:8px;margin-bottom:24px;">
        ${absent.map(x=>{
            const text = fillTemplate(T.winback, {cliente:x.client.name,

```

```

dias:x.daysSince));
    return `

```

<div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">4 · Impulsionar serviços e produtos</div>
<div class="card" style="padding:18px 20px;margin-bottom:24px;">
 <div style="display:flex;flex-direction:column;gap:12px;max-width:520px;">
 <div><label>Cliente</label><select id="f-upsell-client"
style="width:100%;">
 <option value="">Selecione um cliente</option>
 ${state.clients.map(c=>`<option value="${c.id}">${esc(c.name)}
</option>`).join('')}
 </select></div>
 <div><label>Sugerir</label>
 <div style="display:flex;gap:8px;">
 <button class="btn btn-ghost" style="flex:1;justify-content:center;"
data-upsell-type="servico">Serviço ainda não experimentado</button>
 <button class="btn btn-ghost" style="flex:1;justify-content:center;"
data-upsell-type="produto">Produto da loja</button>
 </div>
 </div>
 <div><label>Mensagem</label><textarea id="f-upsell-text" rows="3"
style="width:100%;resize:vertical;" placeholder="Selecione um cliente e um tipo de
sugestão acima"></textarea></div>
 <div id="upsell-send-slot"></div>
</div>
</div>

```



```

<div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Editar modelos de mensagem</div>
<div style="display:flex;flex-direction:column;gap:14px;margin-bottom:10px;">
 ${[
 ['confirmation','Confirmação de horário','{cliente} {servico} {data}
{hora} {profissional}'],
 ['reminder','Lembrete antes do horário','{cliente} {servico} {hora}

```


```

```

{profissional}']],
    ['winback','Cliente ausente','{cliente} {dias}'],
    ['upsellServices','Sugestão de serviço','{cliente} {sugestao}'],
    ['upsellProducts','Sugestão de produto','{cliente} {sugestao}'],
  ].map(([key,label,tokens])=>`
    <div class="card" style="padding:14px 16px;">
      <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:6px;flex-wrap:wrap;gap:6px;">
        <div style="font-weight:600;font-size:13.5px;">${label}</div>
        <div style="font-size:11px;color:var(--muted-2);" class="font-
mono">${tokens}</div>
      </div>
      <textarea data-template="${key}" rows="2"
style="width:100%;resize:vertical;">${T[key]}</textarea>
    </div>
  `).join('')
  <button class="btn btn-gold" id="btn-save-templates" style="width:fit-
content;">Salvar modelos</button>
</div>

<div class="divider-blade" style="margin:26px 0 18px;"></div>
<div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Histórico de envios</div>
${state.messageLog.length===0 ? emptyState('Nenhuma mensagem enviada ainda.')}
: `
  <table>
    <thead><tr><th>Data</th><th>Cliente</th><th>Tipo</th></tr></thead>
    <tbody>
      ${state.messageLog.slice().reverse().slice(0,20).map(l=>`<tr>
        <td class="font-mono" style="color:var(--muted);">${new
Date(l.date).toLocaleString('pt-BR')}</td>
        <td>${l.clientName}</td>
        <td>${esc(l.note)}</td>
      </tr>`).join('')}
    </tbody>
  </table>
`
  `;

view.querySelector('#btn-save-reminder-hours').addEventListener('click', ()=>{
  const v = parseInt(view.querySelector('#f-reminder-hours').value)||3;
  update(s=>{ s.messageSettings.reminderHours = v; });
  showToast('Configuração de lembrete salva.');
```

```

    showToast('Configuração de ausência salva.');
```

```

  });
  view.querySelector('#btn-save-templates').addEventListener('click', ()=>{
    update(s=>{
      view.querySelectorAll('[data-template]').forEach(ta=>{
s.messageTemplates[ta.dataset.template] = ta.value; });
    });
    showToast('Modelos de mensagem salvos.');
```

```

  });
  view.querySelectorAll('[data-mark-confirmed]').forEach(b=>b.addEventListener('click', ()=>{
    update(s=>{ s.appointments.find(a=>a.id===b.dataset.markConfirmed).confirmed =
true; });
    showToast('Marcado como confirmado.');
```

```

  }));
  view.querySelectorAll('[data-mark-reminded]').forEach(b=>b.addEventListener('click', ()=>{
    update(s=>{ s.appointments.find(a=>a.id===b.dataset.markReminded).reminderSent
= true; });
    showToast('Lembrete marcado como enviado.');
```

```

  }));
  view.querySelectorAll('[data-wa-send]').forEach(a=>a.addEventListener('click',
()=>{
    const card = a.closest('.card');
    const name = card?.querySelector('div[style*="font-weight:600"]')?.textContent
|| '';
    logMessage('auto', name, '', 'Mensagem aberta no WhatsApp');
```

```

  }));

// Upsell builder
let upsellType = null;
function renderUpsellText(){
  const clientId = view.querySelector('#f-upsell-client').value;
  const client = state.clients.find(c=>c.id===clientId);
  const slot = view.querySelector('#upsell-send-slot');
  if(!client || !upsellType){ slot.innerHTML=''; return; }
  let text = '';
  if(upsellType==='servico'){
    const opts = suggestServicesFor(client.name);
    const names = opts.length ? opts.slice(0,2).map(s=>s.name).join(' e ') :
'nossos serviços premium';
    text = fillTemplate(T.upsellServices, {cliente:client.name,
sugestao:names});
  }else{
    const opts = state.products.filter(p=>p.price>0);
    const names = opts.length ? opts.slice(0,2).map(p=>p.name).join(' e ') :
'nossos produtos de cuidado';

```

```

        text = fillTemplate(T.upsellProducts, {cliente:client.name,
sugestao:names});
    }
    view.querySelector('#f-upsell-text').value = text;
    slot.innerHTML = messageChannelRow(client.phone, text);
    slot.querySelector('[data-wa-send]')?.addEventListener('click',
()=>logMessage('upsell', client.name, client.phone, 'Sugestão de '+
(upsellType==='servico'? 'serviço': 'produto')));
    }
    view.querySelectorAll('[data-upsell-
type]').forEach(b=>b.addEventListener('click', ()=>{
upsellType=b.dataset.upsellType; renderUpsellText(); }));
    view.querySelector('#f-upsell-client').addEventListener('change',
renderUpsellText);
    view.querySelector('#f-upsell-text').addEventListener('input', ()=>{
        const clientId = view.querySelector('#f-upsell-client').value;
        const client = state.clients.find(c=>c.id===clientId);
        const slot = view.querySelector('#upsell-send-slot');
        if(client){ slot.innerHTML = messageChannelRow(client.phone,
view.querySelector('#f-upsell-text').value);
            slot.querySelector('[data-wa-send]')?.addEventListener('click',
()=>logMessage('upsell', client.name, client.phone, 'Sugestão personalizada'));
        }
    });
}

/* ===== #2 LISTA DE ESPERA (view) ===== */
function renderWaitlist(view){
    view.innerHTML = `
        ${pageHeader('Lista de Espera', 'Clientes aguardando um horário', `<button
class="btn btn-gold" id="btn-new-wait">${ICON.plus} Adicionar à espera</button>`)}
        <div class="card" style="padding:14px 18px;margin-bottom:20px;">
            <div style="font-size:13px;color:var(--muted);">Quando um agendamento é
cancelado ou vira falta, o sistema procura automaticamente alguém na lista de
espera compatível com o horário e avisa você para confirmar por WhatsApp.</div>
        </div>
        ${state.waitlist.length===0 ? emptyState('Ninguém na lista de espera no
momento.') : `
            <div style="display:flex;flex-direction:column;gap:8px;">
                ${state.waitlist.map(w=>{
                    const s = serviceById(w.serviceId);
                    const text = fillTemplate(state.messageTemplates.waitlistOpen,
{cliente:w.clientName, data: w.preferredDate?new
Date(w.preferredDate+'T00:00').toLocaleDateString('pt-BR'):'em breve', hora:'a
combinar', profissional: w.professionalId?profName(w.professionalId):'qualquer
profissional'}});
                    return `<div class="card" style="padding:12px 16px;display:flex;align-

```

```

items:center;gap:14px;flex-wrap:wrap;">
    <div style="flex:1;min-width:180px;">
        <div style="font-weight:600;font-size:13.5px;">${w.clientName}</div>
        <div style="font-size:12px;color:var(--muted);">${s?s.name:'Qualquer
serviço'} · ${w.professionalId?profName(w.professionalId):'Qualquer profissional'}
        ${w.preferredDate?' · ' +new Date(w.preferredDate+'T00:00').toLocaleDateString('pt-
BR'):''}</div>
        ${w.note?`<div style="font-size:11.5px;color:var(--muted-
2);">${esc(w.note)}</div>`:''}
    </div>
    ${messageChannelRow(w.phone, text)}
    <button class="btn btn-ghost" style="padding:8px 9px;color:var(--
bad);border-color:var(--bad);" data-del-wait="${w.id}">${ICON.trash}</button>
</div>`;
    }).join('')}
</div>
`}
`;
view.querySelector('#btn-new-wait').addEventListener('click', waitlistModal);
view.querySelectorAll('[data-del-wait]').forEach(b=>b.addEventListener('click',
())=>{
    update(s=>{ s.waitlist = s.waitlist.filter(w=>w.id!==b.dataset.delWait); });
    });
view.querySelectorAll('[data-wa-send]').forEach(a=>a.addEventListener('click',
())=>logMessage('waitlist','', '', 'Aviso de vaga na lista de espera')));
}
function waitlistModal(){
    openModal(`
        <div style="padding:22px;">
            <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:16px;">
                <div class="font-display" style="font-size:19px;font-
weight:600;">Adicionar à lista de espera</div>
                <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
            </button>
        </div>
        <div style="display:flex;flex-direction:column;gap:12px;">
            <div><label>Cliente</label><input id="f-name" placeholder="Nome do
cliente" style="width:100%;"></div>
            <div><label>Telefone</label><input id="f-phone" placeholder="(00) 00000-
0000" style="width:100%;"></div>
            <div><label>Serviço desejado (opcional)</label><select id="f-service"
style="width:100%;"><option value="">Qualquer
serviço</option>${state.services.map(s=>`<option value="${s.id}">${esc(s.name)}
</option>`).join('')}</select></div>
            <div><label>Profissional preferido (opcional)</label><select id="f-prof"
style="width:100%;"><option value="">Qualquer

```

```

profissional</option>${servingProfessionals().map(p=>`<option
value="${p.id}">${esc(p.name)}</option>`).join('')}</select></div>
    <div><label>Data preferida (opcional)</label><input id="f-date"
type="date" style="width:100%;"></div>
    <div><label>Observação</label><input id="f-note" placeholder="Ex: prefere
fim de tarde" style="width:100%;"></div>
    <button class="btn btn-gold" id="f-submit" style="justify-
content:center;margin-top:4px;">Adicionar</button>
  </div>
</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#f-submit').addEventListener('click', ()=>{
    const clientName = root.querySelector('#f-name').value.trim();
    if(!clientName){ showToast('Informe o nome do cliente.');

```

```
saldo do caixa.', imagem:null },
  { titulo:'Faturamento do dia', texto:'Mostra o valor já recebido em serviços
concluídos hoje. Se algum atendimento foi de um assinante do clube e saiu de
graça, ele conta como atendimento mas não entra no valor – o painel avisa quantos
foram assim.', imagem:null },
  { titulo:'Saldo do caixa (dono e recepcionista)', texto:'Mostra quanto tem no
caixa aberto agora, com o total de entradas e saídas do dia. Se aparecer "Nenhum
caixa aberto", é preciso abrir o caixa na tela de Caixa antes de começar a receber
pagamentos.', imagem:null },
  { titulo:'Comissões pendentes', texto:'O dono vê o total a pagar pra todos os
profissionais somado. O profissional vê só quanto ele mesmo tem a receber.',
imagem:null },
  { titulo:'Clube de assinatura e atrasados (dono e recepcionista)',
texto:'Mostra quantos assinantes estão ativos, a receita recorrente mensal (MRR) e
quantos estão com a mensalidade atrasada – um alerta rápido pra saber se precisa
cobrar alguém.', imagem:null },
  { titulo:'Agenda de hoje', texto:'Lista os próximos atendimentos do dia. O
profissional vê só os dele; o dono e a recepcionista veem a agenda inteira da
barbearia.', imagem:null },
] },
{ id:'agenda', titulo:'Agenda', papeis:['dono','recepcionista','profissional'],
imagemGeral:'ajuda/ajuda-agenda.png', passos:[
  { titulo:'Escolhendo como ver a agenda', texto:'No topo da tela existem três
formas de ver os horários: Grade (colunas por profissional, é a mais usada no dia
a dia), Lista (todos os horários em sequência) e Semana (visão geral da semana
inteira, ótima pra ver quem está mais ocupado). Clique em qualquer uma pra
trocar.', imagem:null },
  { titulo:'Navegando entre os dias', texto:'Use as setas "<" e ">" no topo para
ir pro dia anterior ou seguinte, ou clique em "Hoje" pra voltar direto pro dia
atual. Também dá pra clicar no calendário e pular direto pra uma data
específica.', imagem:null },
  { titulo:'Criando um agendamento novo', texto:'Clique no botão "+ Novo
agendamento" no canto superior, ou direto num espaço vazio da grade no horário
desejado – os dados já vêm preenchidos com o profissional e o horário daquela
coluna. Preencha o nome do cliente (o sistema sugere quem já é cadastrado enquanto
você digita), escolha um ou mais serviços e confirme.', imagem:null },
  { titulo:'Marcando mais de um serviço no mesmo horário', texto:'Ao criar o
agendamento, marque quantas caixinhas de serviço quiser (ex: Corte + Barba). O
sistema soma a duração e o valor de todos automaticamente e já avisa se algum
deles bate com outro horário.', imagem:null },
  { titulo:'Abrindo as opções de um agendamento', texto:'Clique em cima de
qualquer agendamento já criado (na grade ou na lista) pra abrir o menu com as
ações: Finalizar e receber, Editar serviço, Reagendar, Confirmar, Marcar falta ou
Cancelar.', imagem:null },
  { titulo:'A estrelinha ao lado do nome do cliente', texto:'Quando aparece uma
estrela dourada, o cliente é assinante do Clube e está em dia – alguns serviços
podem sair grátis ou com desconto. Estrela vermelha significa que a mensalidade
```

```
dele está atrasada, então ele paga o valor cheio.', imagem:null },
    { titulo:'Bloqueando um horário (folga, almoço)', texto:'Clique em "Bloquear
horário" no topo da tela, escolha o profissional, o motivo e o período. Dá pra
marcar pra repetir toda semana no mesmo dia – ótimo pro horário de almoço, por
exemplo. Ninguém mais consegue agendar em cima de um horário bloqueado.',
imagem:null },
    { titulo:'Finalizando um atendimento e recebendo o pagamento', texto:'Depois
de atender o cliente, clique no agendamento e depois em "Finalizar e receber". O
sistema já calcula o valor certo (considerando desconto do clube, se for o caso) e
permite dividir o pagamento em mais de uma forma, ou dar um desconto na hora.',
imagem:null },
  ] },
  { id:'caixa', titulo:'Caixa', papeis:['dono','recepcionista'],
imagemGeral:'ajuda/ajuda-caixa.png', passos:[
    { titulo:'Abrindo o caixa no início do dia', texto:'Ao entrar na tela de Caixa
pela primeira vez no dia, clique em "Abrir caixa". O valor já vem sugerido com o
que sobrou no fechamento anterior – confira o dinheiro na gaveta e corrija se
estiver diferente. O sistema avisa na hora se faltar ou sobrar algo em relação ao
esperado.', imagem:null },
    { titulo:'Como o dinheiro entra sozinho no caixa', texto:'Você não precisa
lançar nada na mão quando finaliza um atendimento – o valor recebido já entra
automaticamente no caixa, separado por forma de pagamento (dinheiro, PIX, débito
ou crédito).', imagem:null },
    { titulo:'Lançando uma entrada ou saída manual', texto:'Use os botões "Lançar
entrada" ou "Lançar saída" pra registrar algo que não veio de um atendimento – por
exemplo, pagar um fornecedor ou receber algo fora do sistema. Preencha o valor, a
forma de pagamento e uma descrição.', imagem:null },
    { titulo:'Vendendo um produto do estoque', texto:'Na tela de Estoque, clique
em "Vender" ao lado do produto desejado. Escolha a quantidade e, se o comprador
for assinante do clube, o desconto dele é aplicado automaticamente. O valor cai
direto no caixa.', imagem:null },
    { titulo:'Fazendo uma sangria', texto:'Sangria é quando você tira dinheiro
físico da gaveta pra guardar no cofre ou levar ao banco, sem que isso seja uma
despesa do negócio. Clique em "Sangria", informe o valor e pra onde ele vai.',
imagem:null },
    { titulo:'Fechando o caixa no final do dia', texto:'Clique em "Fechar caixa",
conte o dinheiro físico da gaveta e digite o valor. O sistema mostra na hora se
bateu certinho com o esperado, ou se está faltando/sobrando algo – assim dá pra
conferir antes de ir embora.', imagem:null },
    { titulo:'Consultando o histórico de caixas', texto:'Mais abaixo na mesma tela
fica a lista de todos os caixas já fechados, com a data, o valor de abertura,
entradas, saídas e a diferença encontrada no fechamento – útil pra revisar
qualquer dia passado.', imagem:null },
  ] },
  { id:'comissoes', titulo:'Comissões', papeis:['dono','profissional'],
imagemGeral:'ajuda/ajuda-comissoes.png', passos:[
    { titulo:'O que aparece nesta tela', texto:'O dono vê a comissão de todos os
```

profissionais. Cada profissional vê apenas a própria comissão – não é possível ver o quanto os colegas estão ganhando.', imagem:null },

{ titulo:'Comissão de serviços avulsos', texto:'O card "A receber agora" mostra a comissão do mês, já descontando o que já foi pago, vales adiantados e saques. É calculada sobre o valor que realmente foi cobrado do cliente – se o serviço estava incluso no clube e saiu de graça, ele não gera comissão de serviço (a recompensa dele vem pelas fichas do pote).', imagem:null },

{ titulo:'Pagando a comissão (dono)', texto:'No card do profissional, clique em "Pagar" pra quitar o valor pendente de serviços avulsos. O botão fica desabilitado se não houver nada a pagar.', imagem:null },

{ titulo:'Dando um vale / adiantamento (dono)', texto:'Clique em "Vale" no card do profissional, informe o valor e escolha se sai do caixa da barbearia ou da conta bancária. O valor é descontado automaticamente do que ele tem a receber – se o vale for maior que a comissão, o saldo fica negativo e vai sendo abatido sozinho conforme ele atende mais clientes.', imagem:null },

{ titulo:'Pote do Clube de Assinatura', texto:'É a parte do dinheiro das mensalidades que é dividida entre os profissionais, proporcional às "fichas" que cada um acumulou no mês (cada serviço tem um valor em fichas, configurado na tela de Serviços). O pote só existe se as mensalidades tiverem realmente entrado naquele mês.', imagem:null },

{ titulo:'Quando uma ficha não conta (dono)', texto:'Se um atendimento de assinante não gerou ficha, a tela mostra um aviso explicando o motivo (serviço sem fichas configuradas, ou o profissional marcado como "não atende clientes") com um botão pra corrigir na hora – sem precisar refazer o atendimento.', imagem:null },

{ titulo:'Solicitando um saque (profissional)', texto:'Clique em "Solicitar saque" no seu card, informe o valor (até o limite que você tem a receber) e envie. O pedido fica esperando o dono aceitar.', imagem:null },

{ titulo:'Aceitando ou recusando um saque (dono)', texto:'Os pedidos pendentes aparecem no sininho de notificação e numa lista na própria tela. Confira o valor e clique em aceitar ou recusar.', imagem:null },

] },

{ id:'clube', titulo:'Clube de Assinatura', papeis:['dono'],
imagemGeral:'ajuda/ajuda-clube.png', passos:[

{ titulo:'Criando um plano', texto:'Clique em "+ Plano", dê um nome, defina o preço mensal, marque quais serviços estão inclusos (saem de graça pro assinante) e em quais dias da semana o plano vale. Também dá pra definir um desconto percentual pra serviços extras não inclusos.', imagem:null },

{ titulo:'Adicionando um assinante', texto:'Clique em "+ Membro", escolha um cliente já cadastrado (ou digite um nome novo), selecione o plano e a forma de pagamento. Pagamento pelo Asaas exige o CPF do cliente, porque é exigência do próprio Asaas pra gerar a cobrança.', imagem:null },

{ titulo:'Pagamento automático (Asaas) x manual (dinheiro)', texto:'Assinantes cadastrados como "Pix/Cartão via Asaas" têm o status atualizado sozinho quando a cobrança é paga. Assinantes em dinheiro precisam ser marcados como "Recebi" manualmente todo mês, na lista de quem ainda não pagou.', imagem:null },

{ titulo:'Cobrando quem está em atraso', texto:'A tela mostra quantos assinantes ainda não pagaram no mês, com um botão pra ver a lista completa –

```
ordenada por quem deve há mais tempo – e enviar cobrança pelo WhatsApp direto dali.', imagem:null },
    { titulo:'Página pública de planos', texto:'Todo plano cadastrado aparece automaticamente na sua página pública (o link fica em "Meus Links"). O cliente pode assinar sozinho por lá, se a opção "Criar assinatura automaticamente" estiver ligada em Agendamento Online.', imagem:null },
    { titulo:'Sincronizando com o Asaas', texto:'Clique em "Sincronizar com Asaas" pra atualizar o status de todos os assinantes que pagam por lá de uma vez – útil se algum pagamento recente ainda não refletiu no sistema.', imagem:null },
    { titulo:'Assinantes sem cliente vinculado', texto:'Às vezes o nome cadastrado no clube não bate exatamente com o nome na lista de Clientes (apelido, nome incompleto), e a estrelinha do assinante não aparece na agenda. A tela avisa quando isso acontece, com um botão pra vincular manualmente.', imagem:null },
    { titulo:'Fechando o pote do mês', texto:'No fim do mês, revise o detalhamento do pote (na tela de Comissões) e feche – isso registra o pagamento de cada profissional referente às fichas que ele acumulou.', imagem:null },
  ] },
  { id:'clientes', titulo:'Clientes', papeis:['dono','recepcionista'],
  imagemGeral:'ajuda/ajuda-clientes.png', passos:[
    { titulo:'Cadastrando um cliente novo', texto:'Clique em "+ Novo cliente" e preencha nome, telefone e, se quiser, aniversário e observações. O telefone ajuda o sistema a reconhecer o mesmo cliente em outras telas, como o clube de assinatura.', imagem:null },
    { titulo:'Editando um cliente', texto:'Clique no ícone de lápis na linha do cliente pra atualizar os dados.', imagem:null },
    { titulo:'Importando vários clientes de uma vez', texto:'Clique em "Importar planilha" e escolha um arquivo Excel ou CSV. O sistema detecta sozinho quais colunas são nome, telefone, etc, não precisa estar numa ordem específica.', imagem:null },
    { titulo:'Encontrando clientes duplicados', texto:'Se o sistema perceber dois cadastros com o mesmo telefone (ou nome idêntico), aparece um aviso com o botão "Revisar duplicados". Escolha qual cadastro manter – o histórico de atendimentos dos outros é transferido pra ele automaticamente antes de remover os duplicados.', imagem:null },
    { titulo:'Exportando a lista', texto:'Clique em "Exportar CSV" pra baixar todos os clientes cadastrados numa planilha.', imagem:null },
  ] },
  { id:'profissionais', titulo:'Profissionais', papeis:['dono'],
  imagemGeral:'ajuda/ajuda-profissionais.png', passos:[
    { titulo:'Cadastrando um profissional', texto:'Clique em "+ Novo profissional", preencha nome e telefone, defina o PIN de acesso dele (usado pra entrar pelo celular) e o nível de acesso: Dono/Admin, Recepcionista ou Profissional.', imagem:null },
    { titulo:'Configurando a comissão', texto:'Defina o percentual padrão de comissão dele. Se quiser um percentual diferente pra um serviço específico (ex: ele ganha mais em coloração), use os ajustes por serviço dentro do próprio cadastro.', imagem:null },
  ]
}
```

```
{ titulo:'Dias e horário de trabalho', texto:'Marque em quais dias da semana
ele atende e o horário de início e fim do expediente. Isso é usado pra calcular os
horários disponíveis na agenda e na página pública.', imagem:null },

{ titulo:'A caixinha "Atende clientes"', texto:'Desmarque essa opção pra quem
trabalha na barbearia mas não corta cabelo (recepcionista, por exemplo). Quem está
desmarcado não aparece como opção de barbeiro na agenda nem na página pública, mas
continua com acesso normal ao sistema.', imagem:null },

{ titulo:'Foto do profissional', texto:'Adicione uma foto pra ele aparecer
identificado na agenda em grade e na página pública de agendamento.', imagem:null
},

{ titulo:'Acesso da equipe pelo celular', texto:'Cada profissional pode entrar
direto pelo celular dele usando o link de equipe (em "Meus Links") e o PIN
cadastrado – sem precisar do login e senha do dono.', imagem:null },

] },

{ id:'servicos', titulo:'Serviços', papeis:['dono','recepcionista'],
imagemGeral:'ajuda/ajuda-servicos.png', passos:[

{ titulo:'Cadastrando um serviço', texto:'Clique em "+ Novo serviço" e
preencha nome, preço e duração em minutos. A duração é usada pra calcular o
tamanho do bloco na agenda e evitar choque de horário.', imagem:null },

{ titulo:'Fichas do Clube de Assinatura', texto:'Defina quantas fichas esse
serviço vale quando feito num cliente assinante – é isso que determina a fatia do
pote mensal que cada profissional recebe.', imagem:null },

{ titulo:'Exigir sinal pra agendar', texto:'Ative essa opção pra serviços que
precisam de um pagamento antecipado (sinal) antes de confirmar o horário, e defina
se é um valor fixo ou percentual do preço.', imagem:null },

{ titulo:'Disponível no agendamento online', texto:'Desmarque essa caixinha
pra serviços que só a barbearia agenda internamente (procedimentos especiais, por
exemplo). O cliente não vê essa opção no link público, mas continua disponível pra
você agendarem pelo painel.', imagem:null },

] },

{ id:'agendamento-online', titulo:'Agendamento Online', papeis:['dono'],
imagemGeral:'ajuda/ajuda-agendamento-online.png', passos:[

{ titulo:'Definindo o link da sua barbearia', texto:'Escolha um identificador
único (o "slug") que vai aparecer no final do link, por exemplo
primerbarber.com.br/?agendar=sua-barbearia.', imagem:null },

{ titulo:'Ativando a página', texto:'Marque "Página de agendamento ativa" pra
liberar o link pro público. Desmarcando, o link para de funcionar sem precisar
apagar nada.', imagem:null },

{ titulo:'Aceitar agendamentos automaticamente', texto:'Ligado, o horário que
o cliente escolhe entra direto na agenda, sem precisar da sua confirmação.
Desligado, cada pedido fica esperando você aceitar manualmente.', imagem:null },

{ titulo:'Criar assinatura automaticamente', texto:'Ligado, o cliente pode
assinar um plano do clube sozinho pela página pública de planos, com a cobrança já
sendo gerada no Asaas na hora. Exige a integração com o Asaas configurada.',
imagem:null },

{ titulo:'Permitir cancelar e remarcar', texto:'Defina se o próprio cliente
pode cancelar ou remarcar o horário dele pelo link, sem precisar ligar pra
```

```
barbearia.', imagem:null },
  ] },
  { id:'central', titulo:'Central de Inteligência', papeis:['dono'],
imagemGeral:'ajuda/ajuda-central.png', passos:[
    { titulo:'Navegando por mês', texto:'Use as setas no topo pra ver os números
de meses anteriores e comparar com o mês atual.', imagem:null },
    { titulo:'Faturamento e variação', texto:'Os cards do topo mostram o
faturamento em serviços, o resultado do caixa e o ticket médio, sempre com a
variação em relação ao mês anterior.', imagem:null },
    { titulo:'Taxa de ocupação', texto:'Mostra quanto da agenda disponível dos
profissionais virou atendimento de verdade, e uma estimativa de quanto daria pra
faturar a mais se a ocupação subisse.', imagem:null },
    { titulo:'Status dos clientes', texto:'Classifica os clientes em Ativos
(vieram nos últimos 30 dias), Semiativos (31 a 90 dias) e Inativos (mais de 90
dias sem aparecer). Clique pra ver a lista dos inativos e mandar uma mensagem de
"sentimos sua falta" direto por ali.', imagem:null },
    { titulo:'Ranking de clientes e dias de pico', texto:'Mostra quem mais gastou
no mês e quais dias da semana têm mais movimento – útil pra planejar promoções ou
reforçar a equipe nos dias mais cheios.', imagem:null },
  ] },
  { id:'relatorios', titulo:'Relatórios', papeis:['dono'],
imagemGeral:'ajuda/ajuda-relatorios.png', passos:[
    { titulo:'Indicadores gerais', texto:'Ticket médio, taxa de ocupação e taxa de
no-show (clientes que faltaram) dão um retrato rápido da saúde do negócio.',
imagem:null },
    { titulo:'Desempenho por profissional – semana', texto:'Mostra, pra cada
profissional, o valor bruto gerado, quanto foi comissão dele e quanto ficou de
lucro pra barbearia, navegando semana a semana.', imagem:null },
    { titulo:'Desempenho por profissional – mês', texto:'A mesma divisão de bruto,
comissão e lucro, mas olhando o mês inteiro – útil pra comparar o desempenho de
cada um ao longo do tempo.', imagem:null },
    { titulo:'Faturamento por serviço', texto:'Mostra quais serviços mais
venderam, ajudando a identificar o que vale a pena divulgar mais.', imagem:null },
  ] },
  { id:'configuracoes', titulo:'Configurações da Página', papeis:['dono'],
imagemGeral:'ajuda/ajuda-configuracoes.png', passos:[
    { titulo:'Nome da barbearia', texto:'É o nome que aparece na página pública de
agendamento, na página de planos e no topo do sistema.', imagem:null },
    { titulo:'Logo', texto:'Envie uma imagem quadrada, de preferência 512x512 px,
com no máximo 600KB. PNG com fundo transparente costuma ficar melhor.',
imagem:null },
    { titulo:'Banner', texto:'É a faixa que aparece no topo da página de
agendamento e da página de planos. Tamanho recomendado: 1200x400 px, até 600KB. A
imagem é ajustada pra preencher a largura, então evite colocar texto muito perto
das bordas.', imagem:null },
    { titulo:'Bio, endereço, telefone e horário', texto:'Essas informações
aparecem pro cliente na página pública, ajudando ele a saber onde fica a barbearia
```

```

e quando ela funciona antes mesmo de agendar.', imagem:null },
    { titulo:'Texto sobre o Clube de Assinatura', texto:'Um espaço livre pra
explicar como funcionam seus planos, regras e vantagens – esse texto aparece na
página pública de planos, ajudando a convencer quem está em dúvida se assina ou
não.', imagem:null },
    { titulo:'Redes sociais', texto:'Adicione os links do Instagram, Facebook,
TikTok, WhatsApp e site – eles aparecem como botões na sua página pública.',
imagem:null },
  ] },
  { id:'fidelidade', titulo:'Fidelidade', papeis:['dono'],
imagemGeral:'ajuda/ajuda-fidelidade.png', passos:[
    { titulo:'Configurando a regra de pontos', texto:'Defina quantos pontos o
cliente ganha a cada R$1 gasto em serviços, quantos pontos são necessários pra
resgatar, e o valor do desconto que esse resgate vale.', imagem:null },
    { titulo:'Programa de indicação', texto:'Quando um cliente indicado
(preenchido no campo "Indicado por" no cadastro dele) tem o primeiro atendimento
concluído, quem indicou ganha pontos de bônus automaticamente. Defina quantos
pontos valem essa indicação.', imagem:null },
    { titulo:'Aniversariantes do mês', texto:'A tela lista automaticamente os
clientes que fazem aniversário no mês atual, com o telefone à mão – bom pra mandar
uma mensagem especial.', imagem:null },
    { titulo:'Pontuação por cliente', texto:'A tabela mostra o total de pontos de
cada cliente e o progresso até o próximo resgate. Dá pra adicionar pontos
manualmente com o botão "+10 pts", útil pra alguma bonificação especial.',
imagem:null },
  ] },
];
const FAQ_AJUDA = [
  // { pergunta, resposta, papeis:[...] }
  { pergunta:'Esqueci minha senha, como faço?', resposta:'Na tela de login, clique
em "Esqueci minha senha" e siga as instruções enviadas por e-mail.', papeis:
['dono','recepcionista','profissional'] },
  { pergunta:'Como meu profissional acessa o sistema pelo celular dele?',
resposta:'Cadastre-o na tela de Profissionais com um PIN de acesso, e envie o link
de equipe (disponível em "Meus Links"). Ele entra direto pelo celular, sem
precisar do seu login e senha.', papeis:['dono'] },
  { pergunta:'Por que um cliente assinante está sendo cobrado valor cheio?',
resposta:'Confira três coisas: se o serviço realmente está marcado como incluso no
plano dele (em Clube de Assinatura, ao editar o plano), se a mensalidade dele está
em dia, e se hoje é um dos dias em que o plano vale.', papeis:['dono'] },
  { pergunta:'A estrelinha do cliente não aparece na agenda, o que fazer?',
resposta:'Isso acontece quando o nome cadastrado no clube não bate exatamente com
o nome na lista de Clientes. Vá em Clube de Assinatura – se houver assinantes sem
vínculo, aparece um aviso com um botão pra resolver isso manualmente.', papeis:
['dono'] },
  { pergunta:'Posso agendar mais de um serviço no mesmo horário?', resposta:'Sim.
Ao criar o agendamento, marque quantas caixinhas de serviço quiser – o sistema

```

```

soma a duração e o valor de todos automaticamente.', papeis:
['dono','recepcionista','profissional'] },
  { pergunta:'Como eu recebo aviso quando um cliente agenda pelo link?',
  resposta:'Ative as notificações em Configurações da Conta. Com o navegador aberto,
  você recebe som e vibração; ativando a notificação push, o aviso chega mesmo com o
  sistema fechado, direto na barra de notificações do celular ou computador.',
  papeis:['dono','recepcionista'] },
  { pergunta:'O profissional pode ver quanto os colegas ganham?', resposta:'Não.
  Cada profissional vê apenas a própria comissão e a própria fatia do pote do clube
  – os valores dos colegas ficam visíveis só pro dono e administrador.', papeis:
  ['dono','profissional'] },
];

let ajudaAba = 'guias'; // 'guias' | 'faq'
let ajudaGuiaAberto = null;

function meuPapelAjuda(){
  if(isOwnerLevel()) return 'dono';
  if(userSession.role==='recepcionista') return 'recepcionista';
  return 'profissional';
}

function renderAjuda(view){
  const papel = meuPapelAjuda();
  const guias = GUIAS_AJUDA.filter(g=>g.papeis.includes(papel));
  const faqs = FAQ_AJUDA.filter(f=>f.papeis.includes(papel));

  view.innerHTML = `
    ${pageHeader('Central de Ajuda', 'Passo a passo de cada tela e respostas para
    as dúvidas mais comuns')}

    <div style="display:inline-flex;background:var(--panel-2);border-
    radius:10px;padding:3px;margin-bottom:20px;">
      <button class="ajuda-tab" data-tab="guias" style="padding:8px 16px;font-
      size:13px;border:none;border-
      radius:8px;cursor:pointer;background:${ajudaAba==='guias'?'var(--
      gold)':'transparent'};color:${ajudaAba==='guias'?'#1a1610':'var(--
      muted)'};">${ICON.book} Guia por tela</button>
      <button class="ajuda-tab" data-tab="faq" style="padding:8px 16px;font-
      size:13px;border:none;border-
      radius:8px;cursor:pointer;background:${ajudaAba==='faq'?'var(--
      gold)':'transparent'};color:${ajudaAba==='faq'?'#1a1610':'var(--
      muted)'};">${ICON.help} Perguntas Frequentes</button>
    </div>

    ${ajudaAba==='guias' ? renderGuiasAjuda(guias) : renderFaqAjuda(faqs)}
  `;

```

```

view.querySelectorAll('.ajuda-tab').forEach(b=>b.addEventListener('click', ()=>{
  ajudaAba = b.dataset.tab; ajudaGuiaAberto = null; renderAjuda(view);
}));
view.querySelectorAll('.ajuda-guia-
header').forEach(el=>el.addEventListener('click', ()=>{
  const id = el.dataset.guia;
  ajudaGuiaAberto = (ajudaGuiaAberto===id) ? null : id;
  renderAjuda(view);
}));
view.querySelectorAll('.ajuda-faq-
header').forEach(el=>el.addEventListener('click', ()=>{
  el.nextElementSibling.style.display =
el.nextElementSibling.style.display==='none' ? 'block' : 'none';
  const seta = el.querySelector('.ajuda-faq-seta');
  if(seta) seta.style.transform = el.nextElementSibling.style.display==='none' ?
'rotate(0deg)' : 'rotate(180deg)';
}));
}

```

```

function renderGuiasAjuda(guias){
  if(guias.length===0){
    return emptyState('Nenhum guia disponível para o seu perfil ainda.');
```

```

  }
  return `
    <div style="display: flex; flex-direction: column; gap: 10px; max-width: 760px;">
      ${guias.map(g=>{
        const aberto = ajudaGuiaAberto===g.id;
        const pronto = g.passos.length>0;
        return `
          <div class="card" style="padding: 0; overflow: hidden;">
            <div class="ajuda-guia-header" data-guia="${g.id}" style="padding: 15px
18px; display: flex; justify-content: space-between; align-
items: center; cursor: pointer;">
              <div style="display: flex; align-items: center; gap: 10px;">
                <span style="font-size: 14.5px; font-weight: 600;">${esc(g.titulo)}
</span>
                ${!pronto ? `<span class="pill pill-muted" style="font-size: 10px;">em
breve</span>` : ''}
              </div>
              <span style="transform: rotate(${aberto ? 180 : 0}deg); transition: transform
.15s; color: var(--muted-2);">${ICON.chevronDown || '▼'}</span>
            </div>
            ${aberto ? `
              <div style="padding: 0 18px 18px; border-top: 1px solid var(--border);">
                ${pronto ? `
                  <div style="margin: 16px 0; max-width: 640px;">

```

```

        
        <div style="display:none;flex-direction:column;align-
items:center;gap:6px;border:1px dashed var(--border);border-
radius:10px;padding:26px 16px;text-align:center;color:var(--muted-2);">
            <div>\${ICON.image}</div>
            <div style="font-size:11px;">Imagem ainda não enviada – os
números abaixo mostram onde cada uma aponta.</div>
        </div>
    </div>
    <div style="display:flex;flex-direction:column;gap:16px;">
        \${g.passos.map((p,i)=>`
            <div style="display:flex;gap:12px;">
                <div style="width:26px;height:26px;border-
radius:50%;background:var(--panel-2);border:1px solid var(--gold);color:var(--
gold-bright);display:flex;align-items:center;justify-content:center;font-
size:12px;font-weight:700;flex-shrink:0;">\${i+1}</div>
                <div style="flex:1;min-width:0;">
                    <div style="font-size:13.5px;font-weight:600;margin-
bottom:3px;">\${esc(p.titulo)}</div>
                    <div style="font-size:12.5px;color:var(--muted);line-
height:1.6;">\${esc(p.texto)}</div>
                </div>
            </div>
            \`).join('')}
        </div>
        ` : `
        <div style="padding:16px 0 4px;font-size:12.5px;color:var(--muted-
2);line-height:1.6;">
            Esse guia ainda está sendo escrito. Se estiver com dúvida agora
sobre <b>\${esc(g.titulo)}</b>, pode chamar no suporte.
        </div>
        `}
    </div>
    ` : ''}
</div>`;
    }).join('')}
</div>
`;
}

```

```

function renderFaqAjuda(faqs) {
    if(faqs.length===0){

```

```

    return emptyState('Nenhuma pergunta cadastrada ainda para o seu perfil.',
'Assim que as dúvidas mais comuns forem adicionadas, elas aparecem aqui.');
```

```

    }
    return `
      <div style="display:flex;flex-direction:column;gap:8px;max-width:760px;">
        ${faqs.map((f,i)=>`
          <div class="card" style="padding:0;overflow:hidden;">
            <div class="ajuda-faq-header" style="padding:14px
18px;display:flex;justify-content:space-between;align-
items:center;cursor:pointer;gap:10px;">
              <span style="font-size:13.5px;font-weight:600;">${esc(f.pergunta)}
</span>
              <span class="ajuda-faq-seta" style="color:var(--muted-2);flex-
shrink:0;transition:transform .15s;">▼</span>
            </div>
            <div style="display:none;padding:0 18px 16px;font-
size:12.5px;color:var(--muted);line-height:1.6;border-top:1px solid var(--
border);padding-top:12px;">
              ${esc(f.resposta)}
            </div>
          </div>
        `).join('')}
      </div>
    `;
  }

```

```

function renderNotaFiscal(view){
  const n = state.nfe;
  const emittable =
state.cashSessions.flatMap(cs=>cs.entries.filter(e=>e.type==='entrada'));
  view.innerHTML = `
    ${pageHeader('Notas Fiscais', 'Emissão de NFe/NFS-e para os atendimentos e
vendas')}
    <div class="card" style="padding:18px 20px;margin-bottom:20px;border-
color:var(--gold);">
      <div style="display:flex;gap:10px;align-items:flex-start;">
        <div style="color:var(--gold-bright);margin-top:2px;">${ICON.lock}</div>
        <div style="font-size:13.5px;color:var(--muted);line-height:1.7;">
          Assim como no Asaas, a emissão de nota fiscal exige um <strong
style="color:var(--ink);">certificado digital e um provedor emissor</strong> (ex:
NFE.io, Focus NFe, eNotas) rodando em um backend – não é seguro nem possível
emitir nota direto do navegador. Esta tela guarda a configuração e mostra
exatamente o que seria emitido; a emissão real acontece assim que esse backend
estiver plugado.
        </div>
      </div>
    </div>
  </div>

```

```

<div class="card" style="padding:20px;margin-bottom:20px;">
  <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:14px;">Configuração</div>
  <div style="display:flex;flex-direction:column;gap:12px;max-width:440px;">
    <div><label>CNPJ do emitente</label><input id="f-cnpj" value="{n.cnpj}"
placeholder="00.000.000/0001-00" style="width:100%;"></div>
    <div><label>Provedor de emissão</label><select id="f-provider"
style="width:100%;">
      <option value="" ${!n.provider?'selected':''}>Selecione...</option>
      <option value="nfeio"
${n.provider==='nfeio'?'selected':''}>NFE.io</option>
      <option value="focus" ${n.provider==='focus'?'selected':''}>Focus
NFe</option>
      <option value="enotas"
${n.provider==='enotas'?'selected':''}>eNotas</option>
    </select></div>
    <div><label>API Key do provedor</label><input id="f-key" type="password"
value="{n.apiKey}" style="width:100%;"></div>
    <label style="display:flex;align-items:center;gap:9px;font-
size:13.5px;cursor:pointer;"><input type="checkbox" id="f-auto"
${n.autoEmit?'checked':''} style="width:auto;"> Emitir nota automaticamente ao
concluir cada atendimento</label>
    <button class="btn btn-gold" id="f-save" style="width:fit-content;">Salvar
configuração</button>
  </div>
</div>

<div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Entradas do caixa pendentes de nota</div>
${emittable.length===0 ? emptyState('Nenhuma entrada de caixa registrada
ainda.') : `
  <div style="display:flex;flex-direction:column;gap:8px;">
    ${emittable.slice(0,15).map(e=>{
      const already = state.nfeLog.find(l=>l.entryId===e.id);
      return `<div class="card" style="padding:12px 16px;display:flex;align-
items:center;gap:14px;flex-wrap:wrap;">
        <div style="flex:1;">
          <div style="font-size:13.5px;font-weight:500;">${esc(e.desc)}</div>
          <div style="font-size:11.5px;color:var(--muted-2);">${e.time}</div>
        </div>
        <div class="font-mono" style="font-size:14px;">${BRL(e.amount)}</div>
        ${already ? `<span class="pill pill-ok">${ICON.receipt} Nota
registrada</span>` : `<button class="btn btn-ghost" data-emit="{e.id}" data-
desc="{esc(e.desc)}" data-amount="{e.amount}">${ICON.receipt} Emitir
nota</button>`}
      </div>`;
    }
  `

```

```

    }).join('')}
</div>
`}

<div class="font-display" style="font-size:16px;font-weight:600;margin:24px 0
10px;">Histórico de notas</div>
    ${state.nfeLog.length===0 ? emptyState('Nenhuma nota emitida ainda.') : `
    <table>
        <thead><tr><th>Data</th><th>Descrição</th><th>Valor</th></tr></thead>
        <tbody>${state.nfeLog.slice().reverse().map(l=>`<tr><td class="font-mono"
style="color:var(--muted);">${new Date(l.date).toLocaleString('pt-BR')}</td>
<td>${esc(l.desc)}</td><td class="font-mono">${BRL(l.amount)}</td>
</tr>`).join('')}</tbody>
    </table>
    `}
    `;
view.querySelector('#f-save').addEventListener('click', ()=>{
    update(s=>{
        s.nfe.cnpj = view.querySelector('#f-cnpj').value.trim();
        s.nfe.provider = view.querySelector('#f-provider').value;
        s.nfe.apiKey = view.querySelector('#f-key').value.trim();
        s.nfe.autoEmit = view.querySelector('#f-auto').checked;
    });
    showToast('Configuração de nota fiscal salva.');
```

```

    reader.readAsDataURL(file);
}
function renderConfiguracoes(view) {
    const sp = state.shopProfile;
    view.innerHTML = `
        ${pageHeader('Configurações da Página', 'Personalize logo, banner, bio e redes sociais da sua página de agendamento')}

        <div class="card" style="padding:20px;margin-bottom:20px;max-width:560px;">
            <div class="font-display" style="font-size:16px;font-weight:600;margin-bottom:14px;">Identidade</div>
            <div style="display:flex;flex-direction:column;gap:14px;">
                <div><label>Nome da barbearia</label><input id="f-shop-name" value="${esc(sp.name)}" style="width:100%;"></div>

                <div>
                    <label>Logo</label>
                    <div style="background:var(--panel-2);border:1px solid var(--border);border-radius:8px;padding:9px 12px;margin-bottom:8px;font-size:11.5px;color:var(--muted);line-height:1.6;">
                         <b style="color:var(--ink);">Recomendado: 512 × 512 px</b>
                        (quadrada) · máx. 600KB · PNG com fundo transparente fica melhor
                    </div>
                    <div style="display:flex;align-items:center;gap:12px;">
                        <div style="width:56px;height:56px;border-radius:10px;background:var(--panel-2);border:1px solid var(--border);display:flex;align-items:center;justify-content:center;overflow:hidden;flex-shrink:0;">
                            ${sp.logoDataUrl ? `` : ICON.image}
                        </div>
                        <input type="file" id="f-logo-file" accept="image/*" style="display:none;">
                        <button class="btn btn-ghost" id="btn-upload-logo">${ICON.upload}
                        Enviar logo</button>
                        ${sp.logoDataUrl ? `<button class="btn btn-ghost" id="btn-remove-logo" style="color:var(--bad);border-color:var(--bad);">${ICON.trash}</button>` : ''}
                    </div>
                </div>

                <div>
                    <label>Banner (topo da página de agendamento e de planos)</label>
                    <div style="background:var(--panel-2);border:1px solid var(--border);border-radius:8px;padding:9px 12px;margin-bottom:8px;font-size:11.5px;color:var(--muted);line-height:1.6;">
                         <b style="color:var(--ink);">Tamanho recomendado: 1200 × 400 px</b>
                        (proporção 3:1) · máximo 600KB<br>
                        A imagem é ajustada para preencher a largura, então o miolo aparece
                    </div>
                </div>
            </div>
        </div>
    `;
}

```

sempre – evite texto muito perto das bordas.

```
</div>

<div style="width:100%;height:100px;border-radius:10px;background:var(--
panel-2);border:1px solid var(--border);display:flex;align-items:center;justify-
content:center;overflow:hidden;margin-bottom:8px;">

    ${sp.bannerDataUrl ? `` : `<span style="color:var(--
muted-2);font-size:12px;">${ICON.image} Nenhum banner enviado</span>`}

</div>

<div style="display:flex;gap:8px;">

    <input type="file" id="f-banner-file" accept="image/*"
style="display:none;">

    <button class="btn btn-ghost" id="btn-upload-banner">${ICON.upload}
Enviar banner</button>

    ${sp.bannerDataUrl ? `<button class="btn btn-ghost" id="btn-remove-
banner" style="color:var(--bad);border-color:var(--bad);">${ICON.trash}</button>`
: ''}

</div>

<div style="font-size:11px;color:var(--muted-2);margin-top:6px;">Imagens
até ~600KB. Prefira arquivos leves (JPG comprimido) para o app carregar rápido.
</div>

</div>

<div><label>Bio / descrição curta</label><textarea id="f-shop-bio"
rows="2" style="width:100%;resize:vertical;">${esc(sp.bio)}</textarea></div>

<div><label>Endereço</label><input id="f-shop-address"
value="${esc(sp.address)}" placeholder="Rua, número, bairro, cidade"
style="width:100%;"></div>

<div><label>Telefone / WhatsApp de contato</label><input id="f-shop-phone"
value="${esc(sp.phone)}" placeholder="(00) 00000-0000" style="width:100%;"></div>

<div><label>Horário de funcionamento</label><input id="f-shop-hours"
value="${esc(sp.hours)}" placeholder="Ex: Seg das 13h às 17h · Ter a Sáb das 9h às
19h" style="width:100%;"></div>

<div><label>Texto sobre o Clube de Assinatura</label><textarea id="f-shop-
clubabout" rows="4" placeholder="Explique aqui como funcionam seus planos, regras
e vantagens. Este texto aparece na página pública de planos."
style="width:100%;resize:vertical;">${esc(sp.clubAbout)}</textarea></div>

</div>

</div>

<div class="card" style="padding:20px;margin-bottom:20px;max-width:560px;">

    <div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:14px;">Redes sociais</div>

    <div style="display:flex;flex-direction:column;gap:12px;">

        <div><label>${ICON.instagram} Instagram</label><input id="f-social-
instagram" value="${esc(sp.social.instagram)}" placeholder="@suabarbearia ou link"
style="width:100%;"></div>
```

```

        <div><label>${ICON.facebook} Facebook</label><input id="f-social-facebook"
value="${esc(sp.social.facebook)}" placeholder="Link da página"
style="width:100%;"></div>
        <div><label>${ICON.tiktok} TikTok</label><input id="f-social-tiktok"
value="${esc(sp.social.tiktok)}" placeholder="@suabarbearia ou link"
style="width:100%;"></div>
        <div><label>${ICON.chat} WhatsApp</label><input id="f-social-whatsapp"
value="${esc(sp.social.whatsapp)}" placeholder="(00) 00000-0000"
style="width:100%;"></div>
        <div><label>${ICON.globe} Site</label><input id="f-social-website"
value="${esc(sp.social.website)}" placeholder="https://" style="width:100%;">
</div>
    </div>
</div>

<button class="btn btn-gold" id="btn-save-shop" style="margin-
bottom:26px;">Salvar configurações</button>

<div class="font-display" style="font-size:16px;font-weight:600;margin-
bottom:10px;">Prévia da página de agendamento</div>
<div class="card" style="padding:0;max-width:380px;overflow:hidden;">
    <div style="width:100%;height:90px;background:${sp.bannerDataUrl?
`url(${sp.bannerDataUrl}) center/cover`:`linear-gradient(135deg, var(--panel-2),
var(--panel))`};"></div>
    <div style="padding:16px;">
        <div style="display:flex;align-items:center;gap:10px;margin-bottom:8px;">
            <div style="width:44px;height:44px;border-radius:9px;background:var(--
panel-2);border:1px solid var(--border);display:flex;align-items:center;justify-
content:center;overflow:hidden;flex-shrink:0;margin-top:-28px;box-shadow:0 2px 8px
rgba(0,0,0,.4);">
                ${sp.logoDataUrl ? `` : ICON.scissors}
            </div>
            <div class="font-display" style="font-size:15px;font-
weight:600;">${esc(sp.name)}</div>
        </div>
        <div style="font-size:12px;color:var(--muted);margin-
bottom:8px;">${sp.bio||'Sem bio cadastrada.'}</div>
        ${sp.address?`<div style="font-size:11.5px;color:var(--muted-2);margin-
bottom:4px;">${esc(sp.address)}</div>`:''}
        <div style="display:flex;gap:8px;margin-top:8px;">
            ${sp.social.instagram?`<span class="pill pill-muted">${ICON.instagram}
</span>`:''}
            ${sp.social.facebook?`<span class="pill pill-muted">${ICON.facebook}
</span>`:''}
            ${sp.social.tiktok?`<span class="pill pill-muted">${ICON.tiktok}
</span>`:''}

```

```

        ${sp.social.whatsapp?`<span class="pill pill-muted">${ICON.chat}
</span>`:''}
        ${sp.social.website?`<span class="pill pill-muted">${ICON.globe}
</span>`:''}
    </div>
</div>
</div>
`;
view.querySelector('#btn-upload-logo').addEventListener('click',
()=>view.querySelector('#f-logo-file').click());
view.querySelector('#f-logo-file').addEventListener('change', e=>{
    const file = e.target.files[0]; if(!file) return;
    readImageAsDataURL(file, 600*1024, dataUrl=>{
        update(s=>{ s.shopProfile.logoDataUrl = dataUrl; });
    });
});
view.querySelector('#btn-remove-logo')?.addEventListener('click', ()=>{
    update(s=>{ s.shopProfile.logoDataUrl = ''; });
});
view.querySelector('#btn-upload-banner').addEventListener('click',
()=>view.querySelector('#f-banner-file').click());
view.querySelector('#f-banner-file').addEventListener('change', e=>{
    const file = e.target.files[0]; if(!file) return;
    readImageAsDataURL(file, 600*1024, dataUrl=>{
        update(s=>{ s.shopProfile.bannerDataUrl = dataUrl; });
    });
});
view.querySelector('#btn-remove-banner')?.addEventListener('click', ()=>{
    update(s=>{ s.shopProfile.bannerDataUrl = ''; });
});
view.querySelector('#btn-save-shop').addEventListener('click', ()=>{
    update(s=>{
        s.shopProfile.name = view.querySelector('#f-shop-name').value.trim() ||
s.shopProfile.name;
        s.shopProfile.bio = view.querySelector('#f-shop-bio').value.trim();
        s.shopProfile.address = view.querySelector('#f-shop-address').value.trim();
        s.shopProfile.phone = view.querySelector('#f-shop-phone').value.trim();
        s.shopProfile.hours = view.querySelector('#f-shop-hours').value.trim();
        s.shopProfile.clubAbout = view.querySelector('#f-shop-
clubabout').value.trim();
        s.shopProfile.social.instagram = view.querySelector('#f-social-
instagram').value.trim();
        s.shopProfile.social.facebook = view.querySelector('#f-social-
facebook').value.trim();
        s.shopProfile.social.tiktok = view.querySelector('#f-social-
tiktok').value.trim();
        s.shopProfile.social.whatsapp = view.querySelector('#f-social-

```

```

whatsapp').value.trim();
    s.shopProfile.social.website = view.querySelector('#f-social-
website').value.trim();
    });
    showToast('Configurações da página salvas.');
```

```

});
}

/* ===== PÁGINA PÚBLICA DE AGENDAMENTO (sem login) ===== */
function publicBookingUrl(slug){
    return `${window.location.origin}${window.location.pathname}?agendar=${slug}`;
}

let pubBooking = { shopId:null, slug:null, info:null, serviceId:null,
professionalId:null, date:null };

async function renderPublicBooking(slug){
    const app = document.getElementById('app');
    app.innerHTML = `
```

```

<div class="card" style="padding:0;overflow:hidden;margin-bottom:16px;">
  <div style="width:100%;height:70px;background:${info.bannerDataUrl?
`url(${info.bannerDataUrl}) center/cover`: 'linear-gradient(135deg, var(--panel-2),
var(--panel))'};"></div>
  <div style="padding:14px 20px 16px;">
    <div style="display:flex;align-items:center;gap:10px;">
      <div style="width:42px;height:42px;border-
radius:9px;background:var(--panel-2);border:1px solid var(--
border);display:flex;align-items:center;justify-
content:center;overflow:hidden;flex-shrink:0;margin-top:-26px;box-shadow:0 2px 8px
rgba(0,0,0,.4);">
        ${info.logoDataUrl ? `` : ''}
      </div>
      <div class="font-display" style="font-size:16px;font-
weight:600;">${esc(info.name) || 'Barbearia'}</div>
    </div>
    ${info.bio ? `<div style="font-size:12.5px;color:var(--muted);margin-
top:10px;line-height:1.5;">${esc(info.bio)}</div>` : ''}
    ${info.address ? `<div style="font-size:11.5px;color:var(--muted-
2);margin-top:6px;">📍 ${esc(info.address)}</div>` : ''}
    ${info.social && Object.values(info.social).some(v=>v)} || info.phone
? `
      <div style="display:flex;gap:8px;margin-top:10px;flex-wrap:wrap;">
        ${info.phone ? `<a
href="https://wa.me/55${info.phone.replace(/\D/g, '')}" target="_blank" class="pill
pill-muted" style="text-decoration:none;">${ICON.chat} WhatsApp</a>` : ''}
        ${info.social?.instagram ? `<a
href="${info.social.instagram.startsWith('http')?
info.social.instagram:'https://instagram.com/'+info.social.instagram.replace('@','')
target="_blank" class="pill pill-muted" style="text-
decoration:none;">${ICON.instagram} Instagram</a>` : ''}
        ${info.social?.facebook ? `<a href="${info.social.facebook}"
target="_blank" class="pill pill-muted" style="text-
decoration:none;">${ICON.facebook} Facebook</a>` : ''}
        ${info.social?.tiktok ? `<a
href="${info.social.tiktok.startsWith('http')?
info.social.tiktok:'https://tiktok.com/@'+info.social.tiktok.replace('@','')}"
target="_blank" class="pill pill-muted" style="text-
decoration:none;">${ICON.tiktok} TikTok</a>` : ''}
        ${info.social?.website ? `<a href="${info.social.website}"
target="_blank" class="pill pill-muted" style="text-
decoration:none;">${ICON.globe} Site</a>` : ''}
      </div>
    ` : ''}
  </div>
</div>

```

```

<div class="card" style="padding:13px 16px;margin-
bottom:14px;display:flex;align-items:center;gap:12px;flex-wrap:wrap;">
  <div style="flex:1;min-width:180px;font-size:12.5px;color:var(--
muted);line-height:1.5;">
    <b style="color:var(--ink);">Já é cliente ou assinante?</b><br>
    Entre na sua conta para ver seu plano e agendar mais rápido.
  </div>
  <a href="?cliente=${esc(pubBooking.slug)}" class="btn btn-ghost"
style="text-decoration:none;flex-shrink:0;">${ICON.lock} Entrar na minha conta</a>
</div>

<div class="card" style="padding:20px;">
  <div style="font-size:13px;font-weight:600;margin-bottom:10px;">1.
Escolha o serviço</div>
  <div id="pb-services" style="display:flex;flex-
direction:column;gap:8px;margin-bottom:16px;">
    ${services.map(s=>`
      <label style="display:flex;align-items:center;gap:10px;padding:10px
12px;border:1px solid var(--border);border-
radius:9px;cursor:pointer;background:${pubBooking.serviceId===s.id?'var(--panel-
2)':'transparent'};">
        <input type="radio" name="pb-service" value="${s.id}"
${pubBooking.serviceId===s.id?'checked':''} style="width:auto;">
        <div style="flex:1;">
          <div style="font-size:13.5px;">${esc(s.name)}</div>
          <div style="font-size:11.5px;color:var(--
muted);">${BRL(s.price)} · ${s.duration}min</div>
        </div>
      </label>
    `).join('') || '<div style="font-size:12.5px;color:var(--
muted);">Nenhum serviço disponível ainda.</div>'}
  </div>

  <div style="font-size:13px;font-weight:600;margin-bottom:10px;">2.
Escolha o profissional</div>
  <div id="pb-profs" style="display:flex;flex-
direction:column;gap:8px;margin-bottom:16px;">
    ${professionals.map(p=>`
      <label style="display:flex;align-items:center;gap:10px;padding:10px
12px;border:1px solid var(--border);border-
radius:9px;cursor:pointer;background:${pubBooking.professionalId===p.id?'var(--
panel-2)':'transparent'};">
        <input type="radio" name="pb-prof" value="${p.id}"
${pubBooking.professionalId===p.id?'checked':''} style="width:auto;">
        <div style="width:38px;height:38px;border-
radius:50%;background:var(--panel-2);border:1px solid var(--

```

```

border);display:flex;align-items:center;justify-
content:center;overflow:hidden;flex-shrink:0;color:var(--muted-2);">
        ${p.photoDataUrl ? `` : ICON.team}
        </div>
        <div style="font-size:13.5px;">${esc(p.name)}</div>
    </label>
    `).join('') || '<div style="font-size:12.5px;color:var(--
muted);">Nenhum profissional disponível ainda.</div>'}
    </div>

    <div style="font-size:13px;font-weight:600;margin-bottom:10px;">3.
Escolha o dia</div>
    <input id="pb-date" type="date" min="${todayISO()}"
value="${pubBooking.date}" style="width:100%;margin-bottom:16px;">

    <div id="pb-slots-wrap"></div>

    <div id="pb-form" style="display:none;flex-
direction:column;gap:10px;margin-top:16px;">
        <div style="font-size:13px;font-weight:600;">4. Seus dados</div>
        <div><label>Seu nome</label><input id="pb-name" style="width:100%;">
</div>
        <div><label>Telefone (WhatsApp)</label><input id="pb-phone"
style="width:100%;" placeholder="(11) 90000-0000"></div>
        <div><label>E-mail <span style="color:var(--muted-2);font-
weight:400;">— para você receber a confirmação do horário</span></label><input
id="pb-email" type="email" style="width:100%;" placeholder="seu@email.com"></div>
        <button class="btn btn-gold" id="pb-confirm" style="justify-
content:center;">Confirmar agendamento</button>
    </div>
    <div id="pb-done" style="display:none;text-align:center;padding:20px
0;">
        <div data-done-title style="font-size:15px;font-weight:600;margin-
bottom:6px;">Pedido enviado! </div>
        <div data-done-sub style="font-size:12.5px;color:var(--muted);">A
barbearia vai confirmar seu horário em breve.</div>
    </div>
</div>
</div>
</div>
`;
    app.querySelectorAll('input[name="pb-
service"]').forEach(r=>r.addEventListener('change', e=>{
pubBooking.serviceId=e.target.value; renderPublicBookingScreen(); }));
    app.querySelectorAll('input[name="pb-
prof"]').forEach(r=>r.addEventListener('change', e=>{

```

```

pubBooking.professionalId=e.target.value; loadPublicSlots(); }));
  app.querySelector('#pb-date').addEventListener('change', e=>{
pubBooking.date=e.target.value; loadPublicSlots(); });
  if(pubBooking.professionalId) loadPublicSlots();
}

async function loadPublicSlots(){
  const wrap = document.getElementById('pb-slots-wrap');
  const prof = (pubBooking.info.professionals||
[]) .find(p=>p.id===pubBooking.professionalId);
  if(!wrap || !prof) return;

  // Confere se o profissional trabalha neste dia da semana ANTES de
  // qualquer coisa. Sem isso, o sistema gerava horários mesmo em dias
  // de folga (ex: domingo, ou antes do profissional começar na barbearia).
  const diaSemana = new Date(pubBooking.date+'T12:00').getDay();
  const dias = Array.isArray(prof.workDays) ? prof.workDays : [1,2,3,4,5,6];
  if(!dias.includes(diaSemana)){
    wrap.innerHTML = `<div style="font-size:12.5px;color:var(--
muted);">${esc(prof.name)} não atende neste dia. Escolha outra data.</div>`;
    return;
  }

  wrap.innerHTML = `<div style="font-size:12px;color:var(--muted);">Carregando
horários...</div>`;
  const { data: booked } = await supa.rpc('get_booked_times', { p_shop_id:
pubBooking.shopId, p_professional_id: prof.id, p_date: pubBooking.date });
  const bookedSet = new Set(booked || []);
  // duração do serviço escolhido: o horário só serve se TODOS os intervalos
  // que ele vai ocupar estiverem livres, e se couber antes do fim do expediente
  const svcEscolhido = (pubBooking.info.services||
[]) .find(s=>s.id===pubBooking.serviceId);
  const durMin = Number(svcEscolhido?.duration) || 30;
  const slots = [];
  let [h,m] = (prof.workStart||'09:00').split(':').map(Number);
  const [endH,endM] = (prof.workEnd||'18:00').split(':').map(Number);
  const fimExpediente = endH*60 + endM;
  while(h < endH || (h===endH && m < endM)){
    const inicio = h*60 + m;
    const t = String(h).padStart(2,'0')+':'+String(m).padStart(2,'0');
    let cabe = (inicio + durMin) <= fimExpediente;
    // confere cada intervalo de 15 min que o serviço ocuparia
    for(let x = inicio; cabe && x < inicio + durMin; x += 15){
      const marca =
String(Math.floor(x/60)).padStart(2,'0')+':'+String(x%60).padStart(2,'0');
      if(bookedSet.has(marca)) cabe = false;
    }
  }
}

```

```

        if(cabe) slots.push(t);
        m += 30; if(m>=60){m=0;h++;}
    }
    wrap.innerHTML = `
        <div style="font-size:13px;font-weight:600;margin-bottom:10px;">Horários
disponíveis</div>
        <div style="display:flex;flex-wrap:wrap;gap:8px;">
            ${slots.map(t=>`<button class="btn btn-ghost pb-slot" data-time="${t}"
style="padding:8px 12px;font-size:12.5px;">${t}</button>`).join('')} || '<div
style="font-size:12.5px;color:var(--muted);">Sem horários livres neste dia.
</div>' }
        </div>
    `;
    wrap.querySelectorAll('.pb-slot').forEach(b=>b.addEventListener('click', ()=>{
        pubBooking.time = b.dataset.time;
        wrap.querySelectorAll('.pb-slot').forEach(x=>x.classList.remove('btn-gold'));
        b.classList.add('btn-gold');
        const form = document.getElementById('pb-form');
        form.style.display = 'flex';
        form.scrollToView({behavior:'smooth', block:'nearest'});
    }));
}

document.addEventListener('click', function pbConfirmDelegate(e){
    if(e.target && e.target.id === 'pb-confirm') submitPublicBooking();
});
async function submitPublicBooking(){
    const name = document.getElementById('pb-name').value.trim();
    const phone = document.getElementById('pb-phone').value.trim();
    const email = document.getElementById('pb-email')?.value.trim() || '';
    if(!name || !phone || !pubBooking.serviceId || !pubBooking.professionalId ||
!pubBooking.time){
        alert('Preencha todos os campos e escolha um horário.');
```

```

const { data: auto, error: erroAuto } = await supa.rpc('create_booking_auto', {
  p_shop_id: pubBooking.shopId,
  p_professional_id: prof.id,
  p_service_id: service.id,
  p_date: pubBooking.date,
  p_time: pubBooking.time,
  p_client_name: name,
  p_client_phone: phone,
  p_client_email: email || null,
});
if(!erroAuto && auto?.error){ alert(auto.error); return; }
if(erroAuto){
  // Sem confirmação da automação: melhor avisar do que deixar o cliente
  // achar que está tudo certo quando pode não estar.
  alert('Não foi possível confirmar seu agendamento agora. Tente novamente ou
fale direto com a barbearia.');
```

return;

```

}
const aceitoNaHora = auto?.auto === true;

// Registra o pedido de qualquer forma – é o que aciona o aviso e fica no
histórico
const { error } = await supa.from('booking_requests').insert({
  shop_id: pubBooking.shopId,
  professional_id: prof.id, professional_name: prof.name,
  service_id: service.id, service_name: service.name,
  client_name: name, client_phone: phone, client_email: email || null,
  date: pubBooking.date, time: pubBooking.time,
  status: aceitoNaHora ? 'aceito' : 'pendente',
});
if(error){
  alert('Não foi possível enviar o pedido. Tente novamente.');
```

return;

```

}

// Avisa a barbearia na hora – chega mesmo com o site fechado
enviarPushParaLoja(
  pubBooking.shopId,
  aceitoNaHora ? 'Novo agendamento confirmado!' : 'Novo pedido de agendamento',
  `${name} – ${service.name} com ${prof.name}, ${new
Date(pubBooking.date+'T'+pubBooking.time).toLocaleDateString('pt-BR')} às
${pubBooking.time}`,
  aceitoNaHora ? '?agenda' : '?route=agendamento-online'
);

// E-mails de confirmação/aviso – só quando o horário já entrou direto na
// agenda (aceite automático). No caminho manual, o e-mail sai quando a
// barbearia aceitar o pedido (função acceptBookingRequest).
```

```

if(aceitoNaHora){
  const shopNameEmail = pubBooking.info?.name || '';
  if(email){
    enviarEmailAgendamento({
      tipo:'confirmacao_cliente', to: email, shopName: shopNameEmail,
      clientName: name, serviceName: auto.serviceName || service.name,
      profName: auto.professionalName || prof.name,
      date: pubBooking.date, time: pubBooking.time,
    });
  }
  if(auto.professionalEmail){
    enviarEmailAgendamento({
      tipo:'aviso_profissional', to: auto.professionalEmail, shopName:
shopNameEmail,
      clientName: name, serviceName: auto.serviceName || service.name,
      profName: auto.professionalName || prof.name,
      date: pubBooking.date, time: pubBooking.time,
    });
  }
}

document.getElementById('pb-form').style.display = 'none';
const done = document.getElementById('pb-done');
if(aceitoNaHora){
  const t = done.querySelector('[data-done-title]');
  const s = done.querySelector('[data-done-sub]');
  if(t) t.textContent = 'Horário confirmado! ';
  if(s) s.textContent = ` ${new
Date(pubBooking.date+'T'+pubBooking.time).toLocaleDateString('pt-BR')} às
${pubBooking.time} com ${prof.name}. Te esperamos!`;
} else {
  // Deixa claro que ISSO NÃO É uma confirmação – evita o cliente achar
  // que já está agendado quando na verdade a barbearia ainda precisa aceitar.
  const t = done.querySelector('[data-done-title]');
  const s = done.querySelector('[data-done-sub]');
  if(t) t.textContent = 'Pedido enviado – aguarde a confirmação';
  if(s) s.innerHTML = `A barbearia precisa <b>confirmar</b> este horário antes
dele valer. Você será avisado. Se não tiver retorno logo, entre em contato direto
para garantir.`;
}
done.style.display = 'block';
}

/* ===== PORTAL DO CLIENTE FINAL (link ?cliente) ===== */
let clientSession = null;

```

```

function clientShell(inner){
  document.getElementById('app').innerHTML = `
    <div class="auth-bg"><div class="auth-card"><div class="auth-inner">${inner}
</div></div></div>`;
}

async function renderClientLogin(slug, errorMsg){
  clientShell(`
    <div class="auth-eyebrow">Área do cliente</div>
    <div class="auth-title font-display" style="font-weight:600;font-size:22px;">Entrar</div>
    <div class="auth-subtitle">Acesse para agendar, ver seu plano e renovar sua assinatura.</div>
    ${errorMsg?`<div class="auth-error">${esc(errorMsg)}</div>`:''}
    <div style="display:flex;flex-direction:column;gap:16px;">
      <div class="auth-field"><label>E-mail</label><input id="cl-email" type="email" placeholder="voce@email.com"></div>
      <div class="auth-field"><label>Senha</label><input id="cl-pass" type="password" placeholder="Sua senha"></div>
      <button class="btn btn-gold auth-submit" id="cl-submit">Entrar</button>
      <button class="btn btn-ghost auth-switch" id="cl-signup">Não tem conta? Criar agora</button>
      <div style="text-align:center;padding-top:6px;border-top:1px solid var(--border);margin-top:4px;">
        <div style="font-size:11.5px;color:var(--muted-2);margin-bottom:8px;padding-top:10px;">Só quer marcar um horário rápido?</div>
        <a href="?agendar=${esc(slug)}" style="font-size:13px;color:var(--gold-bright);text-decoration:underline;">Agendar sem criar conta</a>
      </div>
    </div>
  `);
  document.getElementById('cl-signup').addEventListener('click', ()=>renderClientSignup(slug));
  document.getElementById('cl-submit').addEventListener('click', async (e)=>{
    const email = document.getElementById('cl-email').value.trim();
    const pass = document.getElementById('cl-pass').value;
    if(!email || !pass){ renderClientLogin(slug, 'Preencha e-mail e senha.')}
    return; }
    const btnEntrar = e.currentTarget;
    btnEntrar.disabled = true; btnEntrar.textContent = 'Entrando...';
    const { data: res, error } = await supa.rpc('client_login', { p_slug: slug, p_email: email, p_password: pass });
    btnEntrar.disabled = false; btnEntrar.textContent = 'Entrar';
    if(error || !res || res.error){ renderClientLogin(slug, res?.error || 'Não foi possível entrar.')} return; }
    clientSession = { ...res, slug };
    if(res.mustChangePassword) renderClientChangePassword(slug, true);

```

```

    else renderClientPortal();
  });
}

function renderClientSignup(slug, errorMsg) {
  clientShell(`
    <div class="auth-eyebrow">Área do cliente</div>
    <div class="auth-title font-display" style="font-weight:600;font-size:22px;">Criar minha conta</div>
    <div class="auth-subtitle">Depois é só entrar para agendar e acompanhar seu plano.</div>
    ${errorMsg?`<div class="auth-error">${esc(errorMsg)}</div>`:''}
    <div style="display:flex;flex-direction:column;gap:16px;">
      <div class="auth-field"><label>Seu nome</label><input id="cs-name" placeholder="Nome completo"></div>
      <div class="auth-field"><label>Telefone / WhatsApp</label><input id="cs-phone" placeholder="(00) 00000-0000"></div>
      <div class="auth-field"><label>E-mail</label><input id="cs-email" type="email" placeholder="voce@email.com"></div>
      <div class="auth-field"><label>Crie uma senha</label><input id="cs-pass" type="password" placeholder="Mínimo 6 caracteres"></div>
      <button class="btn btn-gold auth-submit" id="cs-submit">Criar conta</button>
      <button class="btn btn-ghost auth-switch" id="cs-back">Já tenho conta? Entrar</button>
      <div style="text-align:center;font-size:11px;color:var(--muted-2);line-height:1.6;">
        Se você já é assinante, use o mesmo telefone cadastrado na barbearia para seu plano aparecer automaticamente.
      </div>
    </div>
  `);
  document.getElementById('cs-back').addEventListener('click', ()=>renderClientLogin(slug));
  document.getElementById('cs-submit').addEventListener('click', async (e)=>{
    const name = document.getElementById('cs-name').value.trim();
    const phone = document.getElementById('cs-phone').value.trim();
    const email = document.getElementById('cs-email').value.trim();
    const pass = document.getElementById('cs-pass').value;
    if(!name || !email || pass.length<6){ renderClientSignup(slug, 'Preencha nome, e-mail e uma senha de ao menos 6 caracteres.');
```

return; }

```

    const btnCriar = e.currentTarget;
    btnCriar.disabled = true; btnCriar.textContent = 'Criando...';
    const { data: res, error } = await supa.rpc('client_self_register', {
      p_slug: slug, p_name: name, p_phone: phone, p_email: email, p_password: pass
    });
    if(error || res?.error){ renderClientSignup(slug, res?.error || 'Não foi possível criar a conta.');
```

return; }

```

    // entra direto após criar
    const { data: log } = await supa.rpc('client_login', { p_slug: slug, p_email:
email, p_password: pass });
    if(log?.ok){
        clientSession = { ...log, slug };
        showToast(res.linkedToPlan ? 'Conta criada e vinculada ao seu plano!' :
'Conta criada com sucesso!');
        renderClientPortal();
    } else {
        renderClientLogin(slug, 'Conta criada! Agora é só entrar.');
```

```

    }
});
}
function renderClientChangePassword(slug, obrigatorio, errorMsg){
    clientShell(`
        <div class="auth-eyebrow">Área do cliente</div>
        <div class="auth-title font-display" style="font-weight:600;font-
size:22px;">${obrigatorio?'Crie sua senha':'Trocar senha'}</div>
        <div class="auth-subtitle">${obrigatorio?'Por segurança, escolha uma senha só
sua antes de continuar.':'Escolha uma nova senha.'}</div>
        ${errorMsg?`<div class="auth-error">${esc(errorMsg)}</div>`:''}
        <div style="display:flex;flex-direction:column;gap:16px;">
            <div class="auth-field"><label>Senha atual</label><input id="cp-cur"
type="password"></div>
            <div class="auth-field"><label>Nova senha</label><input id="cp-new"
type="password" placeholder="Mínimo 6 caracteres"></div>
            <div class="auth-field"><label>Confirmar nova senha</label><input id="cp-
new2" type="password"></div>
            <button class="btn btn-gold auth-submit" id="cp-save">Salvar senha</button>
            ${!obrigatorio?`<button class="btn btn-ghost auth-switch" id="cp-
back">Voltar</button>`:''}
        </div>
    `);
    document.getElementById('cp-back')?.addEventListener('click',
renderClientPortal);
    document.getElementById('cp-save').addEventListener('click', async ()=>{
        const cur = document.getElementById('cp-cur').value;
        const n1 = document.getElementById('cp-new').value;
        const n2 = document.getElementById('cp-new2').value;
        if(n1.length<6){ renderClientChangePassword(slug, obrigatorio, 'A nova senha
precisa ter ao menos 6 caracteres.');
```

```

        return; }
        if(n1!==n2){ renderClientChangePassword(slug, obrigatorio, 'As senhas não
coincidem.');
```

```

        return; }
        const { data: res, error } = await supa.rpc('client_change_password', {
            p_account_id: clientSession.accountId, p_current: cur, p_new: n1
        });
        if(error || res?.error){ renderClientChangePassword(slug, obrigatorio,
```

```

res?.error || 'Não foi possível trocar a senha.');
```

return; }
 clientSession.mustChangePassword = false;
 showToast('Senha atualizada!');
 renderClientPortal();
 });
}

function renderClientPortal(){
 const s = clientSession;
 const m = s.member, plan = s.plan;
 const statusInfo = m ? ({
 pago: {label:'Em dia', tone:'ok'},
 pendente: {label:'Aguardando pagamento', tone:'warn'},
 atrasado: {label:'Mensalidade atrasada', tone:'bad'},
 cancelado: {label:'Cancelada', tone:'bad'},
 })[m.status] || {label:m.status, tone:'muted'}) : null;
 const bookingLink = s.bookingSlug ?
`\${window.location.origin}\${window.location.pathname}?agendar=\${s.bookingSlug}` :
'';
 const outros = (s.allPlans||[]).filter(p=>!plan || p.id!==plan.id);

 document.getElementById('app').innerHTML = `
 <div style="min-height:100vh;background:var(--bg);padding:26px 18px 60px;">
 <div style="max-width:640px;margin:0 auto;">
 <div style="display:flex;align-items:center;gap:12px;margin-bottom:24px;">
 \${s.logo?``:''}
 <div style="flex:1;">
 <div class="font-display" style="font-size:19px;font-weight:600;">Olá,
\${esc((s.name||'')).split(' ')[0])}!</div>
 <div style="font-size:12px;color:var(--muted-
2);">\${esc(s.shopName||'')}</div>
 </div>
 <button class="btn btn-ghost" id="cl-logout" style="padding:7px
12px;font-size:12.5px;">Sair</button>
 </div>

 \${bookingLink?`<a href="\${bookingLink}" class="btn btn-gold"
style="width:100%;justify-content:center;text-decoration:none;padding:15px;font-
size:15px;margin-bottom:20px;">\${ICON.agenda} Agendar um horário`:''}

 \${m && plan ? `
 <div class="card" style="padding:22px;margin-bottom:18px;border-
color:var(--gold);">
 <div style="display:flex;justify-content:space-between;align-
items:flex-start;gap:12px;flex-wrap:wrap;">
 <div>

```

        <div style="font-size:11px;color:var(--muted-2);text-
transform:uppercase;letter-spacing:.07em;margin-bottom:5px;">Seu plano</div>
        <div class="font-display" style="font-size:22px;font-
weight:600;color:var(--gold-bright);">${esc(plan.name)}</div>
        <div class="font-mono" style="font-size:14px;color:var(--
muted);margin-top:3px;">${BRL(plan.price)}/mês</div>
    </div>
    <span class="pill pill-${statusInfo.tone}" style="padding:6px
13px;">${statusInfo.label}</span>
</div>
    ${m.dueDate?`<div style="font-size:12.5px;color:var(--muted);margin-
top:12px;">Próximo vencimento: ${new
Date(m.dueDate+'T00:00').toLocaleDateString('pt-BR')}</div>`:''}
    ${ (plan.validDays||[]).length && (plan.validDays||[]).length<7 ? `<div
style="margin-top:10px;font-size:12px;color:var(--gold-bright);">📅
${esc(planDaysLabel(plan.validDays))}</div>`:''}
    ${ (plan.benefitsList||[]).length?`
    <div style="border-top:1px solid var(--border);margin-
top:16px;padding-top:14px;">
        <div style="font-size:10.5px;color:var(--muted-2);letter-
spacing:.08em;text-transform:uppercase;margin-bottom:10px;">O que está incluso:
</div>
        <div style="display:flex;flex-direction:column;gap:7px;">
            ${plan.benefitsList.map(b=>`<div
style="display:flex;gap:8px;font-size:13px;color:var(--muted);"><span
style="color:var(--gold-bright);">✓</span><span>${esc(b)}</span></div>`).join('')}
            </div>
        </div>`:''}
    ${m.invoiceUrl?`<a href="${esc(m.invoiceUrl)}" target="_blank"
class="btn btn-ghost" style="width:100%;justify-content:center;margin-
top:16px;text-decoration:none;">${ICON.link} Ver / pagar minha cobrança</a>`:''}
    ${m.status!=='pago'?`<button class="btn btn-gold" id="cl-renew"
style="width:100%;justify-content:center;margin-top:9px;">Pedir
renovação</button>`:''}
    </div>
    `:
    <div class="card" style="padding:22px;margin-bottom:18px;text-
align:center;">
        <div class="font-display" style="font-size:17px;font-
weight:600;margin-bottom:8px;">Você ainda não tem um plano</div>
        <div style="font-size:13px;color:var(--muted);line-
height:1.6;">Escolha um dos planos abaixo e peça sua assinatura.</div>
    </div>
    `}

    ${outros.length?`
    <div class="font-display" style="font-size:16px;font-weight:600;margin-

```

```

bottom:12px;">${plan?'Quer trocar de plano?':'Planos disponíveis'}</div>
    <div style="display:flex;flex-direction:column;gap:12px;">
        ${outros.map(p=>`
            <div class="card" style="padding:18px 20px;">
                <div style="display:flex;justify-content:space-between;align-
items:center;gap:12px;flex-wrap:wrap;">
                    <div style="min-width:0;">
                        <div style="font-weight:600;font-size:15px;">${esc(p.name)}
</div>
                            <div class="font-mono" style="font-size:16px;color:var(--gold-
bright);margin-top:2px;">${BRL(p.price)}<span style="font-size:11.5px;color:var(--
muted-2);">/mês</span></div>
                                ${ (p.validDays||[]).length && (p.validDays||[]).length<7?`<div
style="font-size:11.5px;color:var(--muted-2);margin-top:4px;">📅
${esc(planDaysLabel(p.validDays))}</div>`:''}
                                    </div>
                                        <button class="btn btn-ghost cl-upgrade" data-
plan="${esc(p.id)}" data-name="${esc(p.name)}" style="flex-
shrink:0;">${plan?'Trocar para este':'Quero este'}</button>
                                            </div>
                                                ${ (p.benefitsList||[]).length?`<div style="display:flex;flex-
direction:column;gap:5px;margin-top:12px;padding-top:12px;border-top:1px solid
var(--border);">
                                                    ${p.benefitsList.map(b=>`<div style="display:flex;gap:7px;font-
size:12.5px;color:var(--muted);"><span style="color:var(--gold-bright);">✓</span>
<span>${esc(b)}</span></div>`).join('')}
                                                        </div>`:''}
                                                            </div>
                                                                `).join('')}
                                                                    </div>
                                                                        `:''}

                                <button class="btn btn-ghost" id="cl-changepass"
style="width:100%;justify-content:center;margin-top:24px;font-
size:12.5px;color:var(--muted);">Trocar minha senha</button>
                                    </div>
                                        </div>
                                            `;

                                document.getElementById('cl-logout').addEventListener('click', ()=>{ const
sl=s.slug; clientSession=null; renderClientLogin(sl); });
                                    document.getElementById('cl-changepass').addEventListener('click',
()=>renderClientChangePassword(s.slug, false));
                                    document.getElementById('cl-renew')?.addEventListener('click',
()=>sendClientRequest('renovacao'));
                                    document.querySelectorAll('.cl-upgrade').forEach(b=>b.addEventListener('click',
()=>sendClientRequest('upgrade', b.dataset.plan, b.dataset.name)));
                                }

```

```

async function sendClientRequest(tipo, planId, planName){
  const s = clientSession;
  const { error } = await supa.from('client_requests').insert({
    shop_id: s.shopId, account_id: s.accountId,
    client_name: s.name, client_phone: s.phone,
    type: tipo,
    current_plan: s.plan?.name || null,
    requested_plan_id: planId || null,
    requested_plan_name: planName || null,
  });
  if(error){ showToast('Não foi possível enviar o pedido. Tente de novo.')}
return; }

  showToast(tipo==='renovacao'
    ? 'Pedido de renovação enviado! A barbearia vai te mandar o link de
pagamento.'
    : 'Pedido enviado! A barbearia vai confirmar a troca com você.');"

}

/* ===== PÁGINA PÚBLICA DE PLANOS (link ?planos) ===== */
let planPageData = null;

async function renderPublicPlans(slug){
  const app = document.getElementById('app');
  app.innerHTML = `
```

```

document.getElementById('app').innerHTML = `
  <div style="min-height:100vh;background:var(--bg);">
    <div style="width:100%;height:180px;background:${d.banner?`url(${d.banner})
center/cover`:`linear-gradient(135deg, var(--panel-2), var(--panel))`;}"></div>
    <div style="max-width:1020px;margin:0 auto;padding:0 20px 60px;">
      <div style="display:flex;align-items:flex-end;gap:16px;margin-
top:-42px;margin-bottom:20px;flex-wrap:wrap;">
        <div style="width:88px;height:88px;border-radius:18px;background:var(--
panel);border:1px solid var(--border);display:flex;align-items:center;justify-
content:center;overflow:hidden;flex-shrink:0;box-shadow:0 12px 30px -12px
rgba(0,0,0,.5);">
          ${d.logo ? `` : ''}
        </div>
        <div style="flex:1;min-width:200px;padding-bottom:4px;">
          <h1 class="font-display" style="font-size:28px;font-weight:600;margin-
bottom:6px;">${esc(d.name||'Barbearia')}</h1>
          <div style="font-size:13px;color:var(--muted);line-height:1.7;">
            ${d.address?`<div>📍 ${esc(d.address)}</div>`:''}
            ${d.hours?`<div>🕒 ${esc(d.hours)}</div>`:''}
          </div>
        </div>
        <div style="display:flex;gap:8px;flex-wrap:wrap;padding-bottom:4px;">
          ${bookingLink?`<a href="${bookingLink}" class="btn btn-gold"
style="text-decoration:none;">Agende seu horário</a>`:''}
          <button class="btn btn-ghost" id="pp-share">${ICON.copy}
Compartilhar</button>
        </div>
      </div>

      <h2 class="font-display" style="font-size:22px;font-weight:600;margin-
bottom:18px;text-align:center;">Conheça nossos planos</h2>

      ${plans.length===0 ? `<div style="text-align:center;color:var(--
muted);font-size:13.5px;padding:40px 0;">Nenhum plano disponível no momento.
</div>` : (()=>{
        // Ordena pelos mais assinados. Só entram no "Top" os planos que
        // realmente têm assinantes – plano sem ninguém não vira Top 1.
        const ordenados = [...plans].sort((a,b)=>{
          const d = (b.subscribers||0) - (a.subscribers||0);
          if(d!==0) return d;
          if(a.featured !== b.featured) return a.featured ? -1 : 1;
          return 0;
        });
        const comAssinantes = ordenados.filter(p=>(p.subscribers||0) > 0);
        const top = comAssinantes.slice(0,3);

```

```

const resto = ordenados.filter(p=>!top.includes(p));

const cardHtml = (p, rank)=>{
  const medal = rank===1?'🥇':rank===2?'🥈':rank===3?'🥉': '';
  const destaque = rank<=3 && rank>0;
  return `
    <div class="card" style="padding:26px 22px
22px;position:relative;display:flex;flex-direction:column;${destaque?'border-
color:var(--gold);box-shadow:0 0 0 1px var(--gold);':''}">
      ${destaque ? `<div
style="position:absolute;top:-11px;left:50%;transform:translateX(-50%);background:var(
--gold);color:#12140f;font-size:11px;font-weight:700;padding:4px 14px;border-
radius:999px;white-space:nowrap;">${medal} Top ${rank} mais assinado</div>`
      : (p.featured ? `<div
style="position:absolute;top:-11px;left:50%;transform:translateX(-50%);background:var(
--panel-2);color:var(--gold-bright);border:1px solid var(--gold);font-
size:11px;font-weight:700;padding:4px 14px;border-radius:999px;white-
space:nowrap;">★ Melhor opção</div>` : '')}

      <div class="font-display" style="font-size:19px;font-
weight:600;margin-bottom:4px;">${esc(p.name)}</div>
      ${p.category?`<div style="font-size:11px;color:var(--muted-
2);margin-bottom:12px;">${esc(p.category)}</div>`:''}<div style="margin-
bottom:12px;"></div>' }

      <div style="margin-bottom:6px;">
        <span class="font-mono" style="font-size:32px;font-
weight:600;color:var(--gold-bright);">${BRL(p.price)}</span>
        <span style="font-size:12.5px;color:var(--muted-2);">/mês</span>
      </div>
      ${ (p.subscribers||0)>0?`<div style="font-size:11.5px;color:var(--
muted-2);margin-bottom:16px;">${p.subscribers} assinante${p.subscribers>1?'s':''}
</div>`:''}<div style="margin-bottom:16px;"></div>' }

      ${ (p.validDays||[]).length && (p.validDays||[]).length<7 ? `
        <div style="background:rgba(201,162,75,.1);border:1px solid
rgba(201,162,75,.3);border-radius:8px;padding:8px 11px;margin-bottom:14px;font-
size:12px;color:var(--gold-bright);font-weight:600;">
          📅 ${esc(planDaysLabel(p.validDays))}
        </div>
        ` : (p.validDays||[]).length===7 ? `
        <div style="background:var(--panel-2);border-
radius:8px;padding:8px 11px;margin-bottom:14px;font-size:12px;color:var(--
muted);">
          📅 Válido a semana toda
        </div>
        ` : ''}

```

```

        ${p.benefitsList||[]}.length ? `
            <div style="border-top:1px solid var(--border);padding-
top:14px;margin-bottom:18px;flex:1;">
                <div style="font-size:10.5px;color:var(--muted-2);letter-
spacing:.08em;text-transform:uppercase;margin-bottom:10px;">Incluso:</div>
                <div style="display:flex;flex-direction:column;gap:7px;">
                    ${p.benefitsList.map(b=>`<div
style="display:flex;gap:8px;align-items:flex-start;font-size:13px;color:var(--
muted);line-height:1.5;"><span style="color:var(--gold-bright);flex-
shrink:0;">✓</span><span>${esc(b)}</span></div>`).join('')}
                </div>
            </div>
        ` : `<div style="flex:1;"></div>`

        <button class="btn btn-gold pp-subscribe" data-plan="${esc(p.id)}"
style="justify-content:center;width:100%;">ASSINAR</button>
    </div>`;
};

return `
    ${top.length ? `
        <div style="text-align:center;font-size:12px;color:var(--muted-
2);letter-spacing:.06em;text-transform:uppercase;margin-bottom:20px;">Os mais
escolhidos pelos nossos clientes</div>
        <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(268px,1fr));gap:18px;margin-bottom:${resto.length?'38px':'0'};">
            ${top.map((p,i)=>cardHtml(p, i+1)).join('')}
        </div>
    ` : ''}
    ${resto.length ? `
        ${top.length?'<div style="text-align:center;font-
size:12px;color:var(--muted-2);letter-spacing:.06em;text-
transform:uppercase;margin-bottom:20px;">Outros planos</div>': ''}
        <div style="display:grid;grid-template-columns:repeat(auto-
fit,minmax(268px,1fr));gap:18px;">
            ${resto.map(p=>cardHtml(p, 0)).join('')}
        </div>
    ` : ''}
`;
})();
${(d.clubAbout||d.bio) ? `
    <div class="card" style="padding:22px 24px;margin-top:38px;">
        <div class="font-display" style="font-size:17px;font-
weight:600;margin-bottom:10px;">Sobre</div>
        <div style="font-size:13.5px;color:var(--muted);line-
height:1.75;white-space:pre-line;">${esc(d.clubAbout || d.bio)}</div>

```

```

    ${ (social.instagram||social.facebook||social.tiktok||waNumber||social.website) ? `
        <div style="display:flex;gap:8px;margin-top:16px;flex-wrap:wrap;">
            ${waNumber?`<a href="https://wa.me/55${waNumber}" target="_blank"
class="pill pill-muted" style="text-decoration:none;">${ICON.chat}
WhatsApp</a>`:''}

            ${social.instagram?`<a
href="${social.instagram.startsWith('http')?
esc(social.instagram):'https://instagram.com/'+esc(social.instagram.replace('@',''))"
target="_blank" class="pill pill-muted" style="text-
decoration:none;">${ICON.instagram} Instagram</a>`:''}

            ${social.facebook?`<a href="${esc(social.facebook)}"
target="_blank" class="pill pill-muted" style="text-
decoration:none;">${ICON.facebook} Facebook</a>`:''}

            ${social.tiktok?`<a href="${social.tiktok.startsWith('http')?
esc(social.tiktok):'https://tiktok.com/@'+esc(social.tiktok.replace('@',''))}"
target="_blank" class="pill pill-muted" style="text-
decoration:none;">${ICON.tiktok} TikTok</a>`:''}

            ${social.website?`<a href="${esc(social.website)}" target="_blank"
class="pill pill-muted" style="text-decoration:none;">${ICON.globe} Site</a>`:''}
        </div>
    ` : ''}
</div>
` : ''}

</div>
</div>
`;

document.getElementById('pp-share')?.addEventListener('click', ()=>{
    const url = window.location.href;
    if(navigator.share) navigator.share({title:d.name, url});
    else { navigator.clipboard?.writeText(url); showToast('Link copiado!'); }
});

document.querySelectorAll('.pp-
subscribe').forEach(b=>b.addEventListener('click', ()=>{
    const plan = plans.find(p=>p.id===b.dataset.plan);
    if(plan) publicSubscribeModal(plan);
}));
}

function publicSubscribeModal(plan){
    const d = planPageData;
    openModal(`
        <div style="padding:24px;">
            <div style="display:flex;justify-content:space-between;align-items:flex-
start;margin-bottom:14px;">
                <div>

```

```

        <div class="font-display" style="font-size:19px;font-
weight:600;">Realizar assinatura</div>
        <div style="font-size:13px;color:var(--muted);margin-
top:3px;">${esc(plan.name)}</div>
    </div>
    <button class="btn btn-ghost" style="padding:6px;" id="m-close">${ICON.x}
</button>
</div>
    ${ (plan.benefitsList||[]).length?`
        <div style="font-size:10.5px;color:var(--muted-2);letter-
spacing:.08em;text-transform:uppercase;margin-bottom:8px;">Incluso:</div>
        <div style="display:flex;flex-direction:column;gap:5px;margin-
bottom:14px;">
            ${plan.benefitsList.map(b=>`<div style="display:flex;gap:8px;font-
size:12.5px;color:var(--muted);"><span style="color:var(--gold-bright);">✓</span>
<span>${esc(b)}</span></div>`).join('')}
        </div>
        `:''}
        <div class="font-mono" style="font-size:26px;color:var(--gold-
bright);margin-bottom:20px;">${BRL(plan.price)}<span style="font-
size:12px;color:var(--muted-2);">/mês</span></div>
        <div style="font-size:13px;font-weight:600;margin-bottom:12px;">Seus
dados</div>
        <div style="display:flex;flex-direction:column;gap:12px;">
            <div><label>Nome completo</label><input id="ps-name" style="width:100%;">
</div>
            <div><label>Telefone / WhatsApp</label><input id="ps-phone" placeholder="
(00) 00000-0000" style="width:100%;"></div>
            <div><label>E-mail</label><input id="ps-email" type="email"
style="width:100%;"></div>
            <div><label>CPF</label><input id="ps-cpf" inputmode="numeric"
placeholder="000.000.000-00" style="width:100%;">
            <div style="font-size:11px;color:var(--muted-2);margin-
top:4px;">Necessário para emitir a cobrança da mensalidade.</div>
        </div>
        <button class="btn btn-gold" id="ps-submit" style="justify-
content:center;margin-top:4px;">Enviar pedido de assinatura</button>
        <div style="font-size:11px;color:var(--muted-2);text-align:center;line-
height:1.5;">A barbearia vai receber seu pedido e entrar em contato com o link de
pagamento.</div>
    </div>
    <div id="ps-done" style="display:none;text-align:center;padding:24px 0;">
        <div style="font-size:16px;font-weight:600;margin-bottom:6px;">Pedido
enviado! </div>
        <div style="font-size:12.5px;color:var(--muted);">Em breve a barbearia
entra em contato com você.</div>
    </div>

```

```

</div>
`, root=>{
  root.querySelector('#m-close').addEventListener('click', closeModal);
  root.querySelector('#ps-submit').addEventListener('click', async (e)=>{
    const name = root.querySelector('#ps-name').value.trim();
    const phone = root.querySelector('#ps-phone').value.trim();
    const email = root.querySelector('#ps-email').value.trim();
    const cpf = root.querySelector('#ps-cpf').value.trim();
    if(!name || !phone){ showToast('Preencha ao menos nome e telefone.');
```

```

return; }
    if(cpf.replace(/\D/g,'').length !== 11){ showToast('Informe um CPF válido
(11 dígitos).'); return; }
    const btn = e.currentTarget;
    btn.disabled = true; btn.textContent = 'Criando sua assinatura...';

    // Se a barbearia deixou no automático, a cobrança já é gerada aqui.
    // O servidor devolve auto=false quando a opção está desligada.
    const { data: auto, error: erroAuto } = await supabase.functions.invoke('asaas-
sync', {
      body: { action:'public_subscribe', shopId: d.shop_id, planId: plan.id,
        name, phone, email, cpf }
    });
    if(!erroAuto && auto?.error){
      btn.disabled = false; btn.textContent = 'Enviar pedido de assinatura';
      showToast(auto.error); return;
    }
    const criouSozinho = !erroAuto && auto?.auto === true;

    // registra o pedido de qualquer forma (histórico e aviso para a barbearia)
    const { error } = await supabase.from('subscription_requests').insert({
      shop_id: d.shop_id, plan_id: plan.id, plan_name: plan.name, plan_price:
plan.price,
      client_name: name, client_phone: phone, client_email: email, client_cpf:
cpf,
      status: criouSozinho ? 'aceito' : 'pendente',
    });
    btn.disabled = false; btn.textContent = 'Enviar pedido de assinatura';
    if(error && !criouSozinho){ showToast('Não foi possível enviar. Tente
novamente.');
```

```

    return; }

    enviarPushParaLoja(
      d.shop_id,
      criouSozinho ? 'Nova assinatura do clube!' : 'Novo pedido de assinatura',
      `${name} - ${plan.name} (${BRL(plan.price)}/mês)`,
      '?route=clube'
    );
  });
}

```

```

const done = root.querySelector('#ps-done');
if(criouSozinho){
  done.innerHTML = `
    <div style="font-size:40px;margin-bottom:10px;">🎉</div>
    <div style="font-size:17px;font-weight:600;margin-
bottom:8px;">Assinatura criada!</div>
    <div style="font-size:13px;color:var(--muted);line-height:1.7;margin-
bottom:18px;">
      ${esc(auto.planName)} - <b style="color:var(--gold-
bright);">${BRL(auto.value)}/mês</b><br>
      Sua primeira cobrança já está pronta.
    </div>
    ${auto.invoiceUrl ? `<a href="${esc(auto.invoiceUrl)}" target="_blank"
class="btn btn-gold" style="width:100%;justify-content:center;text-
decoration:none;padding:14px;">Pagar agora</a>` : ''}
    <div style="font-size:11.5px;color:var(--muted-2);margin-top:12px;line-
height:1.6;">
      Você também recebe o link por e-mail. A partir do mês que vem a
cobrança chega sozinha.
    </div>`;
  }
  done.style.display = 'block';
  root.querySelector('#ps-submit').style.display = 'none';
});
});
}

/* ===== ACESSO DA EQUIPE (link ?equipe) ===== */
let teamSession = null; // {shopId, profId, pin} - quando a equipe entra sem o
login do dono
let agendaDate = null; // dia visível na agenda (não é salvo no banco)
let agendaMode = 'grade'; // 'grade' (quadro de horários) ou 'lista'
let agendaEstendida = false; // mostra horas antes/depois do expediente

async function renderTeamLogin(slug, errorMsg){
  const app = document.getElementById('app');
  app.innerHTML = `<div class="auth-bg"><div class="auth-card"><div class="auth-
inner" style="text-align:center;color:var(--muted);font-size:13px;">Carregando...
</div></div></div>`;
  const { data, error } = await supabase.rpc('get_shop_team', { p_slug: slug });
  if(error || !data || !data.shop_id){
    app.innerHTML = `<div class="auth-bg"><div class="auth-card"><div class="auth-
inner" style="text-align:center;color:var(--muted);font-size:13.5px;">Link de
acesso não encontrado.</div></div></div>`;
    return;
  }
  const team = data.team || [];

```

```

app.innerHTML = `
  <div class="auth-bg">
    <div class="auth-card">
      <div class="auth-inner">
        ${data.logo ? `` : ''}
        <div class="auth-eyebrow">Acesso da equipe</div>
        <div class="auth-title font-display" style="font-weight:600;font-
size:22px;">${esc(data.shop_name||'Barbearia')}</div>
        <div class="auth-subtitle">Escolha seu nome e digite seu PIN.</div>
        ${errorMsg?`<div class="auth-error">${esc(errorMsg)}</div>`: ''}
        ${team.length===0 ? `<div style="text-align:center;color:var(--
muted);font-size:13px;">Nenhum profissional cadastrado ainda.</div>` : `
        <div style="display:flex;flex-direction:column;gap:12px;">
          ${team.map(p=>`
            <div style="display:flex;gap:8px;align-items:center;">
              <div style="flex:1;min-width:0;">
                <div style="font-size:13.5px;font-weight:500;">${esc(p.name)}
</div>
                <div style="font-size:11px;color:var(--muted-
2);">${roleLabel(p.accessRole)}</div>
              </div>
              <input type="password" inputmode="numeric" maxlength="8"
placeholder="PIN" data-team-pin="${p.id}" style="width:84px;text-align:center;">
              <button class="btn btn-gold" style="padding:9px 12px;" data-
team-login="${p.id}">Entrar</button>
            </div>
          `).join('')}
        </div>
      `}
    </div>
  </div>
</div>
`;

app.querySelectorAll('[data-team-login]').forEach(b=>b.addEventListener('click',
async ()=>{
  const profId = b.dataset.teamLogin;
  const pin = app.querySelector(`[data-team-pin="${profId}"]`).value.trim();
  if(!pin){ renderTeamLogin(slug, 'Digite seu PIN. '); return; }
  b.disabled = true; b.textContent = '...';
  const { data: res, error: err } = await supa.rpc('team_login', { p_shop_id:
data.shop_id, p_prof_id: profId, p_pin: pin });
  b.disabled = false; b.textContent = 'Entrar';
  if(err || !res || res.error){ renderTeamLogin(slug, res?.error || 'Não foi
possível entrar. '); return; }
  teamSession = { shopId: data.shop_id, profId, pin };

```

```

    currentShop = { id: data.shop_id, data: res.data, plan: 'pro',
subscription_status: 'active', email: '' };
    const prof = (res.data.professionals||[]).find(p=>p.id===profId);
    userSession = { role: prof?.accessRole || 'profissional', professionalId:
profId };
    route = 'dashboard';
    await loadState();
  }));
}

```

```

function authShell(inner){
  document.getElementById('app').innerHTML = `
    <div class="auth-bg">
      <div class="auth-card">
        <div class="auth-inner">
          
            ${inner}
          </div>
        </div>
      </div>
    </div>
  `;
}

```

```

function renderConfigMissing(){
  authShell(`
    <div class="auth-title font-display" style="font-weight:600;">Configuração
pendente</div>
    <div class="auth-subtitle" style="text-align:left;">
      Este app ainda não está ligado a um banco de dados. Preencha <b
style="color:var(--ink);">SUPABASE_URL</b> e <b style="color:var(--
ink);">SUPABASE_ANON_KEY</b>
      no início do código (procure por "CONFIGURAÇÃO – PREENCHA ANTES DE
PUBLICAR") antes de publicar este site.
    </div>
  `);
}

```

```

function renderAuthScreen(mode, errorMsg){
  const isLogin = mode!=='signup';
  authShell(`
    <div class="auth-title font-display" style="font-weight:600;">${isLogin?'Bem-
vindo de volta':'Crie sua barbearia'}</div>
    <div class="auth-subtitle">${isLogin?'Entre para acessar o painel da sua
barbearia.': 'Monte sua agenda, equipe e clube de assinatura em minutos.'}</div>
    ${!isLogin?`<div class="auth-trial-row"><span class="auth-trial-
pill">${ICON.star} ${TRIAL_DAYS} dias grátis · sem cartão</span></div>`: ''}
  `);
}

```

```

    ${errorMsg?`<div class="auth-error">${errorMsg}</div>`:''}
    <div style="display:flex;flex-direction:column;gap:16px;">
      ${!isLogin?`<div class="auth-field"><label>Nome da barbearia</label><input
id="au-shop" placeholder="Ex: Primer Barber"></div>`:''}
      <div class="auth-field"><label>Email</label><input id="au-email"
type="email" placeholder="voce@email.com"></div>
      <div class="auth-field"><label>Senha</label><input id="au-pass"
type="password" placeholder="Mínimo 6 caracteres"></div>
      <button class="btn btn-gold auth-submit" id="au-
submit">${isLogin?'Entrar':'Criar conta grátis'}</button>
      <button class="btn btn-ghost auth-switch" id="au-switch">${isLogin?'Não tem
conta? Criar uma':'Já tem conta? Entrar'}</button>
      ${!isLogin?`<div style="text-align:center;font-size:11.5px;color:var(--
muted-2);line-height:1.6;margin-top:4px;">Ao criar sua conta, você concorda com
os<br><a href="?pagina=termos" target="_blank" style="color:var(--muted);">Termos
de Uso</a> e a <a href="?pagina=privacidade" target="_blank" style="color:var(--
muted);">Política de Privacidade</a>.</div>`:''}
    </div>
  `);
  document.getElementById('au-switch').addEventListener('click',
()=>renderAuthScreen(isLogin?'signup':'login'));
  document.getElementById('au-submit').addEventListener('click', async ()=>{
    const email = document.getElementById('au-email').value.trim();
    const pass = document.getElementById('au-pass').value;
    const shopName = document.getElementById('au-shop')?.value.trim();
    if(!email || !pass){ renderAuthScreen(mode, 'Preencha email e senha.')}
return; }
    if(isLogin){
      const { error } = await supa.auth.signInWithPassword({ email, password: pass
});
      if(error){ renderAuthScreen('login', 'Não foi possível entrar:
'+error.message); return; }
      } else {
        const { error } = await supa.auth.signUp({ email, password: pass });
        if(error){ renderAuthScreen('signup', 'Não foi possível criar a conta:
'+error.message); return; }
        await ensureShop(shopName || 'Minha barbearia');
      }
      boot();
    });
}

async function ensureShop(defaultName){
  const { data: { user } } = await supa.auth.getUser();
  if(!user) return null;
  const { data: existing } = await supa.from('shops').select('*').eq('owner_id',
user.id).maybeSingle();

```

```

if(existing) return existing;
const trialEnds = new Date(Date.now() + TRIAL_DAYS*24*60*60*1000).toISOString();
const seeded = blankShopData();
if(defaultName) seeded.shopProfile.name = defaultName;
const baseSlug = slugify(defaultName || 'barbearia');
const uniqueSlug = baseSlug + '-' + Math.random().toString(36).slice(2,7);
seeded.onlineBooking.slug = uniqueSlug;
const { data: created, error } = await supa.from('shops').insert({
  owner_id: user.id,
  email: user.email,
  data: seeded,
  booking_slug: uniqueSlug,
  subscription_status: 'trialing',
  trial_ends_at: trialEnds,
}).select().single();
if(error){ console.error('Erro ao criar barbearia', error); return null; }
// Avisar o dono do sistema por e-mail. Se falhar, não atrapalha o cadastro.
supa.functions.invoke('notify-signup', {
  body: { event:'signup', shopName: seeded.shopProfile.name, email: user.email }
}).catch(()=>{});
return created;
}

function slugify(text){
  return (text||'').toString().normalize('NFD').replace(/[\u0300-\u036f]/g,'')
    .toLowerCase().trim().replace(/^[a-z0-9]+/g,'-').replace(/(^-|-$/g,'') ||
'barbearia';
}

function shopIsActive(shop){
  if(shop.subscription_status==='active') return true;
  if(shop.subscription_status==='trialing'){
    return new Date(shop.trial_ends_at).getTime() > Date.now();
  }
  return false;
}

function renderPaywallScreen(shop){
  const trialExpired = shop.subscription_status==='trialing' &&
!shopIsActive(shop);
  authShell(`
    <div class="auth-title font-display" style="font-weight:600;font-size:22px;">
      ${trialExpired ? 'Seu teste grátis acabou' : 'Assinatura necessária'}
    </div>
    <div class="auth-subtitle">
      ${trialExpired
        ? `Seus ${TRIAL_DAYS} dias grátis terminaram. Escolha um plano para
continuar usando o painel da sua barbearia.`

```

```

        : 'Escolha um plano para liberar o acesso.')}
    </div>
    ${planPickerHtml(shop)}
    <button class="btn btn-ghost auth-switch" id="pw-refresh" style="margin-
top:14px;">Já assinei, verificar novamente</button>
    <button class="btn btn-ghost auth-switch" id="pw-logout" style="color:var(--
muted);">Sair</button>
    `);
    document.getElementById('pw-refresh').addEventListener('click', boot);
    document.getElementById('pw-logout').addEventListener('click', async ()=>{ await
supa.auth.signOut(); boot(); });
}

function planPickerHtml(shop){
    const mk = (planKey, link) => {
        const limits = PLAN_LIMITS[planKey];
        const url = link.startsWith('http')
        ?
` ${link} ${link.includes('?')?'&':'?'}client_reference_id=${shop.id}&prefilled_email
        : null;
        const isPro = planKey==='pro';
        return `
            <div class="card" style="padding:18px 20px;margin-
bottom:12px;${isPro?'border-color:rgba(201,162,75,.4);background:linear-
gradient(160deg, var(--panel), var(--panel-2));':''}">
                <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:6px;">
                    <div class="font-display" style="font-size:17px;font-
weight:600;">${limits.label}</div>
                    ${shop.plan===planKey && shop.subscription_status==='active' ? `<span
class="pill pill-ok">Plano atual</span>` : isPro ? `<span class="pill"
style="background:rgba(201,162,75,.14);color:var(--gold-bright);border-
color:rgba(201,162,75,.3);">Mais popular</span>` : ''}
                </div>
                <div style="font-size:12.5px;color:var(--muted);margin-bottom:14px;line-
height:1.5;">
                    ${planKey==='basic'
                    ? 'Até 4 profissionais cadastrados. Sem Clube de Assinatura.'
                    : 'Profissionais ilimitados. Clube de Assinatura ativado.'}
                </div>
                ${url
                ? `<a href="${url}" target="_blank" class="btn ${isPro?'btn-gold':'btn-
ghost'} auth-switch" style="text-decoration:none;">Assinar ${limits.label}</a>`
                : `<div class="pill pill-warn" style="display:block;">Link do plano
${limits.label} não configurado</div>`}
            </div>
        `;
    };

```

```

    };
    return mk('basic', STRIPE_PAYMENT_LINK_BASIC) + mk('pro',
STRIPE_PAYMENT_LINK_PRO);
}

function renderPaywallUpgrade(fromRoute) {
  authShell(`
    <div class="auth-title font-display" style="font-weight:600;font-
size:22px;">Fazer upgrade</div>
    <div class="auth-subtitle">Escolha o plano Pro para liberar este recurso.
</div>
    ${planPickerHtml(currentShop)}
    <button class="btn btn-ghost auth-switch" id="pw-back" style="margin-
top:14px;">Voltar</button>
  `);
  document.getElementById('pw-back').addEventListener('click', ()=>{ route =
fromRoute; render(); });
}

function renderLegalPage(type) {
  const isTermos = type==='termos';
  const baseUrl = window.location.origin + window.location.pathname;
  const content = isTermos ? `
    <h1 class="font-display" style="font-size:26px;font-weight:600;margin-
bottom:6px;">Termos de Uso</h1>
    <p style="color:var(--muted-2);font-size:12.5px;margin-bottom:28px;">Última
atualização: 24/08/2026</p>

    <h2 class="font-display">1. Aceitação dos termos</h2>
    <p>Ao criar uma conta ou utilizar o Primer Barber ("Plataforma", "Sistema" ou
"Serviço"), você concorda com estes Termos de Uso. Se você estiver utilizando a
Plataforma em nome de uma empresa, declara ter poderes para vinculá-la a este
contrato.</p>

    <h2 class="font-display">2. O que é o Primer Barber</h2>
    <p>O Primer Barber é um sistema de gestão para barbearias, oferecido no
formato de assinatura mensal (SaaS), que inclui, entre outras funcionalidades:
agenda, cadastro de clientes e profissionais, controle de comissões, controle de
caixa e estoque, clube de assinatura e agendamento online.</p>

    <h2 class="font-display">3. Cadastro e conta</h2>
    <ul>
      <li>Você é responsável por manter a confidencialidade da senha da sua conta
e por todas as atividades realizadas nela.</li>
      <li>As informações fornecidas no cadastro devem ser verdadeiras, completas e
mantidas atualizadas.</li>
      <li>Cada barbearia (conta) é isolada das demais – você só tem acesso aos

```

seus próprios dados.

<h2 class="font-display">4. Planos e pagamento</h2>

O Primer Barber oferece um período de teste gratuito de 14 dias.

Após o período de teste, o uso contínuo da Plataforma exige a contratação de um plano pago (Basic ou Pro), cobrado mensalmente de forma recorrente através do processador de pagamentos Stripe.

Os valores e o que está incluso em cada plano estão descritos na Plataforma e podem ser alterados mediante aviso prévio.

O cancelamento pode ser feito a qualquer momento pelo painel de assinatura. Não há reembolso proporcional de períodos já pagos, salvo quando exigido por lei.

Em caso de falha no pagamento, o acesso à Plataforma pode ser suspenso até a regularização.

<h2 class="font-display">5. Uso aceitável</h2>

<p>Ao usar o Primer Barber, você concorda em não:</p>

utilizar a Plataforma para fins ilegais ou não autorizados;

tentar acessar contas ou dados de outras barbearias sem autorização;

realizar engenharia reversa, copiar ou revender o sistema sem autorização;

inserir conteúdo ofensivo, ilegal ou que viole direitos de terceiros.

<h2 class="font-display">6. Seus dados e os dados dos seus clientes</h2>

Você é o responsável ("controlador", nos termos da LGPD) pelos dados pessoais dos seus clientes finais que inserir na Plataforma (nome, telefone, agendamentos, etc.).

O Primer Barber atua como operador desses dados, armazenando-os com segurança e utilizando-os apenas para viabilizar o funcionamento do sistema que você contratou.

Cabe a você garantir que possui base legal adequada (ex: consentimento ou execução de contrato) para tratar os dados dos seus próprios clientes.

<h2 class="font-display">7. Propriedade intelectual</h2>

<p>O código, design, marca e demais elementos da Plataforma pertencem ao Primer Barber. Você recebe uma licença de uso, não uma transferência de propriedade, ao assinar o serviço.</p>

<h2 class="font-display">8. Disponibilidade e limitação de responsabilidade</h2>

Fazemos o possível para manter a Plataforma disponível, mas não garantimos operação ininterrupta ou livre de falhas.

O Primer Barber não se responsabiliza por perdas financeiras decorrentes de indisponibilidade temporária, falhas de terceiros (como provedores de pagamento) ou uso indevido da Plataforma pelo usuário.

<h2 class="font-display">9. Cancelamento e encerramento</h2>

<p>Você pode cancelar sua assinatura a qualquer momento. Podemos suspender ou encerrar contas que violem estes Termos, mediante aviso, exceto em casos de uso ilegal ou risco à segurança da Plataforma, quando a suspensão pode ser imediata.</p>

<h2 class="font-display">10. Alterações destes termos</h2>

<p>Podemos atualizar estes Termos periodicamente. Mudanças relevantes serão comunicadas com antecedência razoável. O uso continuado da Plataforma após as alterações implica concordância com os novos termos.</p>

<h2 class="font-display">11. Lei aplicável e foro</h2>

<p>Estes Termos são regidos pelas leis brasileiras. Fica eleito o foro da comarca de Nova Friburgo/RJ para dirimir eventuais controvérsias.</p>

<h2 class="font-display">12. Contato</h2>

<p>Dúvidas sobre estes Termos podem ser enviadas para: primerbarbersuporte@gmail.com</p>

` : `

<h1 class="font-display" style="font-size:26px;font-weight:600;margin-bottom:6px;">Política de Privacidade</h1>

<p style="color:var(--muted-2);font-size:12.5px;margin-bottom:28px;">Última atualização: 24/08/2026</p>

<h2 class="font-display">1. Quem somos</h2>

<p>O Primer Barber é um sistema de gestão para barbearias. Esta política explica como coletamos, usamos e protegemos dados pessoais ao operar a Plataforma.</p>

<h2 class="font-display">2. Quais dados coletamos</h2>

<p>Do dono/gerente da barbearia (titular da conta):</p>

Nome, e-mail e senha de acesso

Dados de pagamento (processados diretamente pelo Stripe – não armazenamos números de cartão)

Nome da barbearia, endereço, telefone e redes sociais (se preenchidos)

<p>Dos clientes finais da barbearia (inseridos pelo dono/gerente):</p>

Nome e telefone

Histórico de agendamentos e atendimentos

Dados de assinatura do clube (se aplicável)

>3. Base legal e finalidade do tratamento</h2>

<p>Tratamos os dados acima com base na execução do contrato (para viabilizar o funcionamento do sistema contratado) e, quando aplicável, no legítimo interesse em manter a segurança e o bom funcionamento da Plataforma.</p>

<p>Os dados são usados para: criar e manter sua conta e o acesso ao sistema; processar pagamentos da assinatura; viabilizar as funcionalidades da Plataforma (agenda, clube de assinatura, comissões, etc.); enviar comunicações essenciais sobre sua conta.</p>

>4. Compartilhamento de dados</h2>

<p>Compartilhamos dados apenas com prestadores de serviço necessários para o funcionamento da Plataforma:</p>

Stripe – processamento de pagamentos das assinaturas do sistema

Supabase – armazenamento seguro do banco de dados

Asaas (quando o dono da barbearia ativa essa integração) – processamento de cobranças recorrentes aos clientes finais da própria barbearia

<p>Não vendemos dados pessoais a terceiros.</p>

>5. Onde os dados ficam armazenados</h2>

<p>Os dados são armazenados em servidores do Supabase, com controles de acesso restritos por conta – cada barbearia só acessa os próprios dados.</p>

>6. Por quanto tempo guardamos os dados</h2>

<p>Mantemos os dados enquanto sua conta estiver ativa. Após o cancelamento, os dados podem ser mantidos por um período adicional para cumprimento de obrigações legais ou resolução de disputas, sendo excluídos ou anonimizados depois disso.</p>

>7. Seus direitos (LGPD)</h2>

<p>Você (ou, no caso dos clientes finais, a barbearia responsável por eles) pode solicitar a qualquer momento: confirmação de que tratamos seus dados; acesso aos dados armazenados; correção de dados incompletos ou desatualizados; exclusão dos dados, quando aplicável; portabilidade dos dados a outro fornecedor; informações sobre com quem compartilhamos os dados.</p>

<p>Solicitações podem ser feitas para: primerbarbersuporte@gmail.com</p>

<h2 class="font-display">8. Segurança</h2>

<p>Adotamos medidas técnicas razoáveis para proteger os dados, incluindo controle de acesso por login, criptografia em trânsito (HTTPS) e isolamento de dados entre contas. Nenhum sistema é 100% livre de riscos, e nos comprometemos a agir rapidamente em caso de incidente de segurança.</p>

<h2 class="font-display">9. Cookies e dados de navegação</h2>

<p>A Plataforma pode utilizar armazenamento local do navegador para manter sua sessão de login ativa. Não utilizamos cookies de rastreamento publicitário.</p>

<h2 class="font-display">10. Alterações desta política</h2>

<p>Esta política pode ser atualizada periodicamente. Mudanças relevantes serão comunicadas com antecedência razoável.</p>

<h2 class="font-display">11. Contato</h2>

<p>Dúvidas sobre esta Política de Privacidade ou solicitações relacionadas aos seus dados podem ser enviadas para: primerbarbersuporte@gmail.com</p>

```
`;  
document.getElementById('app').innerHTML = `  
  <div class="auth-bg" style="align-items:flex-start;padding-top:48px;">  
    <div class="auth-card" style="max-width:680px;">  
      <div class="auth-inner" style="padding:38px 44px 44px;">  
        <a href="${baseUrl}" style="display:inline-block;margin-  
bottom:22px;font-size:13px;color:var(--muted);text-decoration:none;">&larr;  
Voltar</a>  
        <div style="font-size:14.5px;line-height:1.75;color:var(--ink);"   
class="legal-content">${content}</div>  
      </div>  
    </div>  
  </div>  
`;  
}
```

```
async function boot(){  
  if(!supa){ renderConfigMissing(); return; }  
  const urlParams = new URLSearchParams(window.location.search);  
  const legalPage = urlParams.get('pagina');  
  if(legalPage==='termos' || legalPage==='privacidade'){  
renderLegalPage(legalPage); return; }  
  const bookSlug = urlParams.get('agendar');  
  if(bookSlug){ await renderPublicBooking(bookSlug); return; }  
  const teamSlug = urlParams.get('equipe');
```

```

    if(teamSlug){ await renderTeamLogin(teamSlug); return; }
    const plansSlug = urlParams.get('planos');
    if(plansSlug){ await renderPublicPlans(plansSlug); return; }
    const clientSlug = urlParams.get('cliente');
    if(clientSlug){ await renderClientLogin(clientSlug); return; }
    const { data: { session } } = await supa.auth.getSession();
    if(!session){ renderAuthScreen('login'); return; }
    let shop = await ensureShop();
    if(!shop){
        // fallback: pode já existir mas a criação acima não rodou (ex: login direto)
        const { data } = await supa.from('shops').select('*').eq('owner_id',
session.user.id).maybeSingle();
        shop = data;
    }
    if(!shop){ renderAuthScreen('login', 'Não encontramos sua barbearia. Fale com o
suporte.');
```

```

    return; }
    currentShop = shop;
    if(!shopIsActive(shop)){ renderPaywallScreen(shop); return; }
    await loadState();
    await refreshNotifItems();
    subscribeBookingNotifications();
}

/* ===== INSTALAÇÃO NO CELULAR (PWA) ===== */
(function(){
    const manifest = {
        name: "Primer Barber", short_name: "Primer Barber",
        start_url: ".", display: "standalone",
        background_color: "#0a0b0d", theme_color: "#0a0b0d",
        icons: [
            { src: APP_LOGO_ICON, sizes: "256x256", type: "image/png", purpose: "any" },
            { src: APP_LOGO_ICON, sizes: "512x512", type: "image/png", purpose: "any" }
        ]
    };
    const blob = new Blob([JSON.stringify(manifest)], {type:'application/json'});
    const link = document.createElement('link');
    link.rel = 'manifest';
    link.href = URL.createObjectURL(blob);
    document.head.appendChild(link);
    const icon = document.createElement('link');
    icon.rel = 'apple-touch-icon'; icon.href = APP_LOGO_ICON;
    document.head.appendChild(icon);

    // Registra o Service Worker cedo, para as notificações funcionarem
    // assim que o dono ativar (não precisa esperar entrar no painel).
    if('serviceWorker' in navigator){
        navigator.serviceWorker.register('/sw.js').catch(()=>{});
    }

```

```
    }  
  }) ();
```

```
/* ===== INIT ===== */  
boot();  
</script>  
</body>  
</html>
```